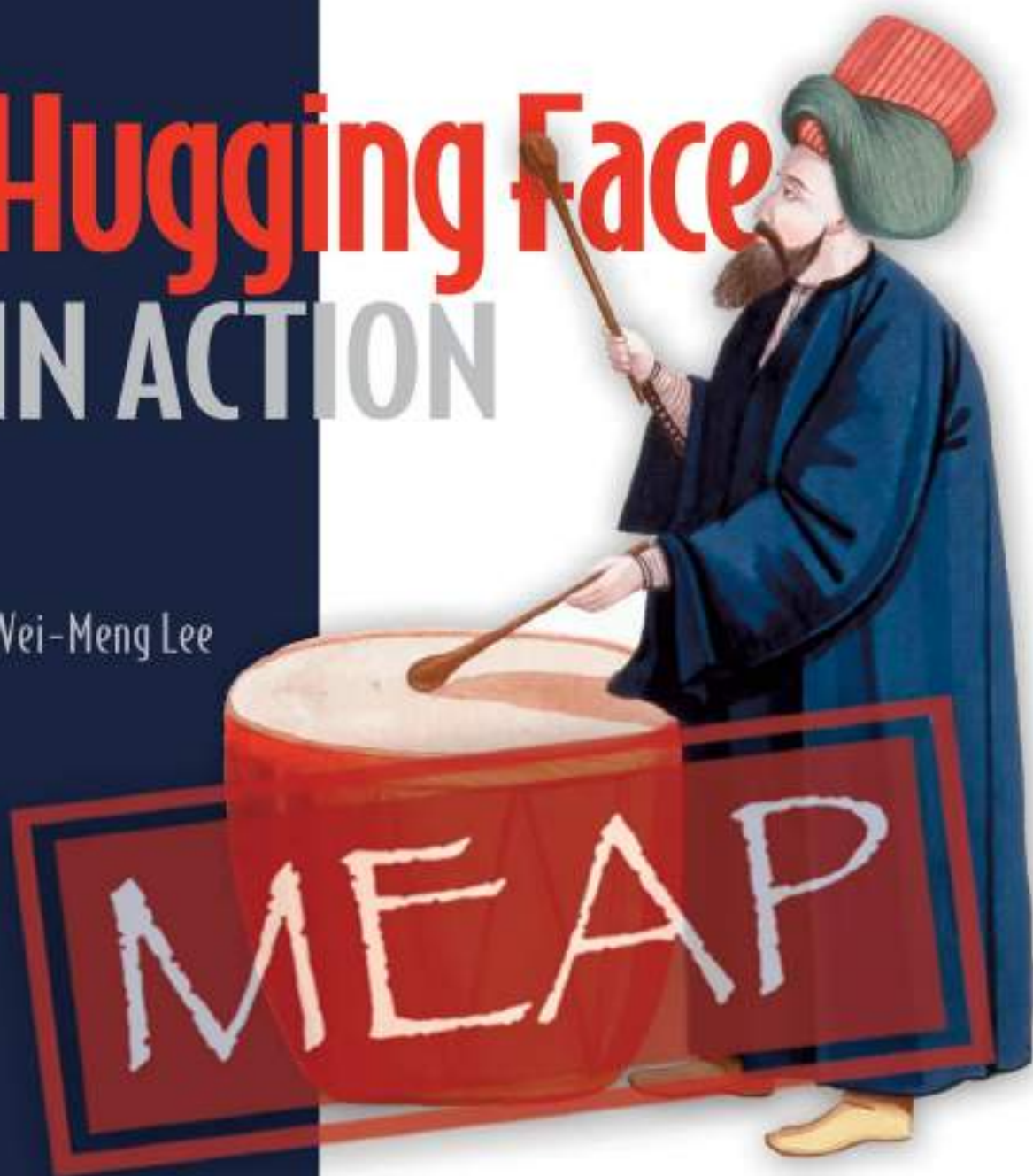


# Hugging Face

## IN ACTION

Wei-Meng Lee



MANNING



**MEAP Edition  
Manning Early Access Program**

**Hugging Face in Action  
Version 6**

**Copyright 2025 Manning Publications**

For more information on this and other Manning titles go to [manning.com](https://manning.com).

# welcome

---

Thank you for purchasing the MEAP version of *Hugging Face in Action*.

AI is incredibly significant today, and Hugging Face stands at the forefront of this transformative technology. By leveraging Hugging Face, you can quickly become productive with its extensive collection of pre-trained models and datasets. This book will guide you through building applications that:

- Understand natural language processing (NLP)
- Utilize cutting-edge Large Language Models (LLMs)
- Answer questions based on custom datasets (RAG apps)
- And more!

The book assumes you have a basic understanding of Python and are familiar with libraries such as NumPy and Pandas.

Here is an overview of the chapters:

- **Chapter 1** – Introduction to Hugging Face
- **Chapter 2** – Getting Started
- **Chapter 3** – Utilizing Hugging Face Transformers and Pipelines for NLP Tasks
- **Chapter 4** – Applying Hugging Face for Computer Vision
- **Chapter 5** – Working with Hugging Face Datasets
- **Chapter 6** – Training Machine Learning Models with AutoTrain
- **Chapter 7** – Implementing Agents
- **Chapter 8** – Developing LLM-based Applications with LangChain and LlamaIndex
- **Chapter 9** – Building LangChain Applications Visually with LangFlow
- **Chapter 10** – Creating Web-Based UIs with Gradio
- **Chapter 11** – Developing Locally-Running LLM-based Applications with GPT4ALL
- **Chapter 12** – Using LLMs to Query Your Local Data

Each chapter provides detailed, step-by-step instructions to guide you through the topics. As the APIs are continuously evolving, you might encounter some deprecated features. Your patience is appreciated as I work to keep the content updated.

Please be sure to post any questions, comments, or suggestions you have about the book in the [liveBook discussion forum](#).

Thank you once again for your interest and for purchasing the MEAP!

—*Wei-Meng Lee*

# *brief contents*

---

## CHAPTERS

- 1 Introduction to Hugging Face*
- 2 Getting Started*
- 3 Using Hugging Face Transformers and Pipelines for NLP Tasks*
- 4 Using Hugging Face for Computer Vision Tasks*
- 5 Exploring, Tokenizing, and Visualizing Hugging Face Datasets*
- 6 Fine-Tuning Pre-Trained Models and Working with Multimodal Models*
- 7 Creating LLM-based Applications using LangChain and LlamaIndex*
- 8 Building LangChain Applications Visually using LangFlow*
- 9 Programming Agents*
- 10 Building Web-Based UI using Gradio*
- 11 Building Locally-Running LLM-based Applications using GPT4All*
- 12 Using LLMs to Query Your Local Data*
- 13 Bridging LLMs to the Real World with the Model Context Protocol (MCP)*

# ***1 Introduction to Hugging Face***

## **This chapter covers**

- What Hugging Face is known for
- The Hugging Face Transformers library
- Exploring the various models hosted by Hugging Face
- The Gradio library

Hugging Face is an AI community that promotes the building, training, and deployment of open-source machine learning models. It has state-of-the-art models designed for different problem domains, such as: Natural Language Processing (NLP) tasks, Computer vision tasks, and Audio tasks. Besides providing tools for machine learning, Hugging Face also provides a platform for hosting pre-trained models and datasets. With AI at its peak now, Hugging Face is right at the epicenter of the whole AI revolution:

- It unleashes a new wave of applications that capitalize on the large amount of data available.
- A lot of complementary technologies are being developed, such as prototyping tools for LLM-based applications.
- Instead of focusing on the fundamentals (such as building neural networks from scratch or learning machine learning algorithms), developers can now focus on building AI-based apps to solve their problems immediately. AI is now a tool that developers can use directly, and not something that developers have to build themselves from scratch.

- Hugging Face’s philosophy is to promote open-source contributions and it is the hub of open-source models for NLP, computer vision, and other fields where AI plays vital roles.

Moving forward, I will highlight some of the key services and platforms provided by Hugging Face. You will also get a glimpse of the kinds of applications we will be building throughout this book. Hugging Face is most well-known for its Transformers library for developing NLP applications, its platform for sharing machine learning models and datasets, Hugging Face Spaces for hosting user-developed ML apps, and the Gradio Python library for rapid UI creation. In the next few sections, I will introduce you to some of these features available through the Hugging Face Hub.

Later in this book will also cover more advanced topics, such as:

- Building your own LLM-based applications using the LangChain framework
- Visually prototyping your LLM-based application with LangFlow
- Exploring alternatives to OpenAI’s GPT models, such as GPT4All
- Developing LLM-based applications without compromising the privacy of your data
- Creating agents that integrate tools like search engines or code interpreters
- Using the Model Context Protocol (MCP) to connect AI assistants to external data sources

## 1.1 Hugging Face Transformers Library

The *Transformers Library* is a Python package that contains open-source implementation of the Transformer architecture models for text, image, and audio tasks. It provides APIs for developers to download and make use of pre-trained models. Using pre-trained and state-of-the-art models, developers don’t have to spend time and resources to build their models from scratch.

As an example, consider the following code snippet (I will explain the code in more details and show you how to install the libraries in Chapters 2 and 3):

```
from transformers import pipeline

classifier = pipeline('text-classification',
                      model = 'distilbert-base-uncased-finetuned-sst-2-english',
                      revision = 'af0f99b')
```

In the above code snippet, I used the `pipeline()` function from the `transformers` package to perform a text-classification tasks. In particular, I want to create an application to detect the sentiments in a particular paragraph of text. In the above, I specified that I want to use this model - *'distilbert-base-uncased-finetuned-sst-2-english'*, and its version - *'af0f99b'*. That's it! There is no need for me to know how text classification works, nor do I need to have the knowledge to know how to train a model to perform this task.

In Hugging Face's Transformers library, a pipeline is a high-level, user-friendly API that simplifies the process of building and using complex natural language processing (NLP) workflows. It allows you to easily perform a sequence of NLP tasks, such as text classification, named entity recognition, translation, summarization, and more, with just a few lines of code.

You will learn more about transformers and pipelines later.

To put it to the test, I will just call the pipeline object and pass it a paragraph of text. It then returns me the result:

```
import pandas as pd

text = '''
I thought this was a wonderful way to spend time on a too hot summer
weekend, sitting in the air conditioned theater and watching a
light-hearted comedy. The plot is simplistic, but the dialogue is
witty and the characters are likable (even the well bread suspected
serial killer). While some may be disappointed when they realize
this is not Match Point 2: Risk Addiction, I thought it was proof
that Woody Allen is still fully in control of the style many of us
have grown to love.<br /><br />This was the most I'd laughed at one
of Woody's comedies in years (dare I say a decade?). While I've never
been impressed with Scarlet Johanson, in this she managed to tone
down her "sexy" image and jumped right into a average, but spirited
young woman.<br /><br />This may not be the crown jewel of his career
, but it was wittier than "Devil Wears Prada" and more interesting
than "Superman" a great comedy to go see with friends.
'''

result = classifier(text)
pd.DataFrame(result)
```

The paragraph of text above is obtained from the IMDB dataset (<https://huggingface.co/datasets/imdb>), a binary sentiment analysis dataset consisting of 50,000 reviews from the Internet Movie Database (IMDb) labeled as positive or negative.

The result returned by the pipeline object is formatted nicely as a Pandas DataFrame (see Figure 1.1), which shows that the sentiment is positive.



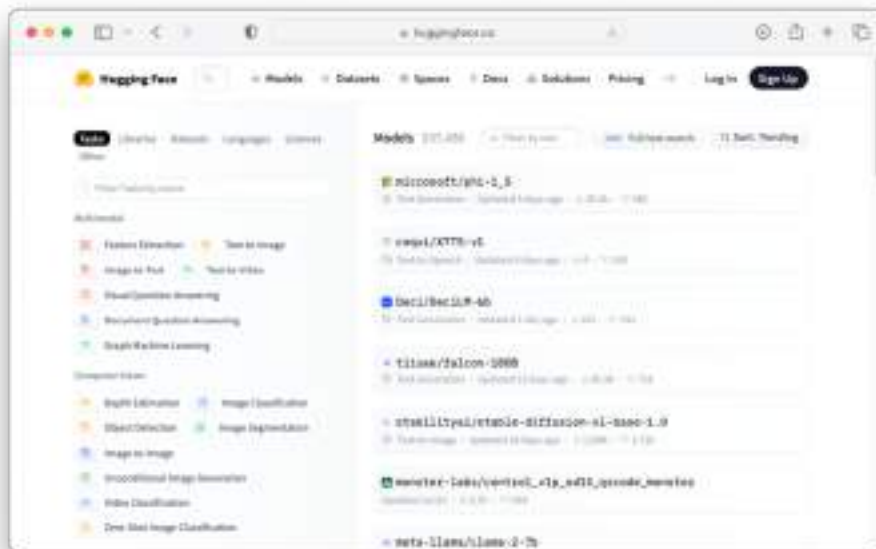
|   | label    | score   |
|---|----------|---------|
| 0 | POSITIVE | 0.99919 |

**Figure 1.1** The result of the sentiment analysis

As you can see from this very code snippet, you can perform a relatively complex task of sentiment analysis using a couple of statements by using the transformers library and the pipeline API from Hugging Face.

## 1.2 Hugging Face Models

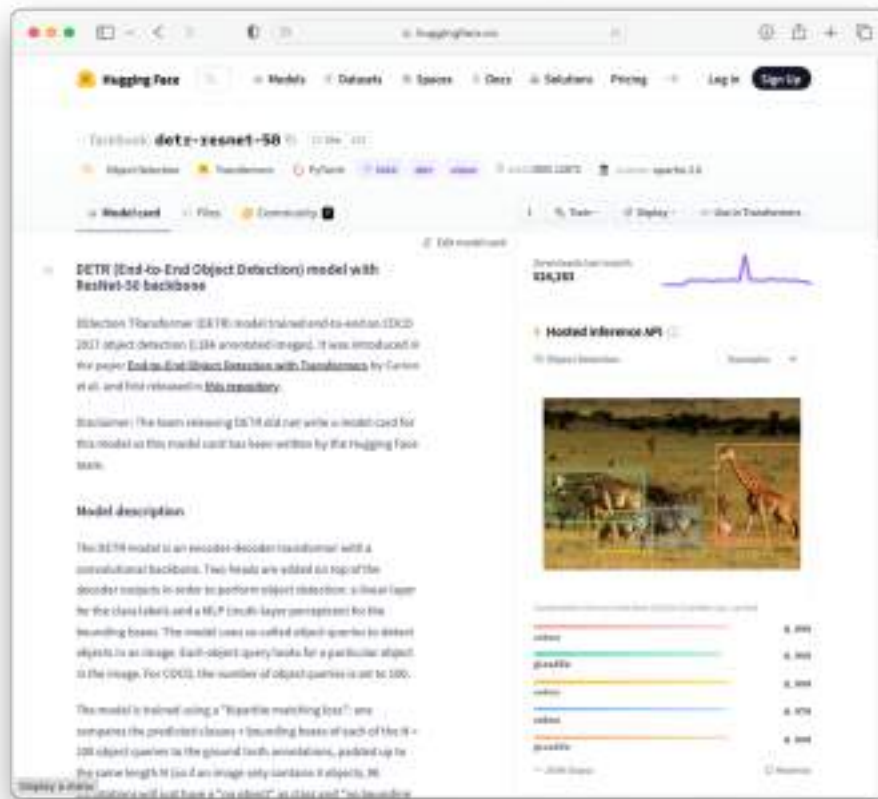
The Hugging Face Hub's Models page (<https://huggingface.co/models>; see Figure 1.2) hosts many pre-trained models for a wide variety of machine learning tasks. All the pre-trained models are stored in repositories, and Hugging Face makes exploring the details of the models very easy.



**Figure 1.2 Exploring the pre-trained models hosted on Hugging Face hub**

Many models have a widget that allows you to directly test them by running inferences directly in the web browser. To demonstrate this, let's search for a pre-trained model named "**facebook/detr-resnet-50**". This is a *DEtection TRansformer* (DETR) model trained end-to-end using the COCO 2017 object detection dataset (118k annotated images for training and 5k images for validation). COCO (Common Objects in Context) is a large-scale object detection, segmentation, and captioning dataset.

You can use the `facebook/detr-resnet-50` model to detect for objects in an image. You can read more about this model on Hugging Face's website at: <https://huggingface.co/facebook/detr-resnet-50> (see Figure 1.3). More importantly, Hugging Face provides the **Hosted inference API** where you can directly test the model using your web browser.



**Figure 1.3** You can test the model directly on Hugging Face hub using the Hosted inference API

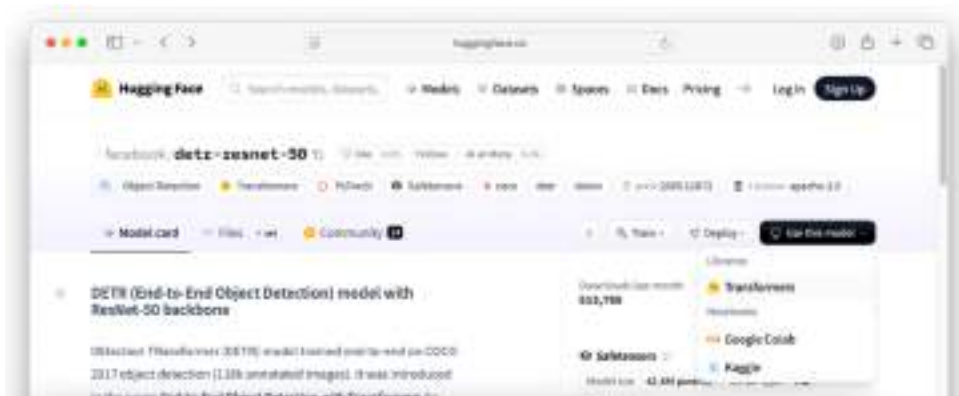
Hugging Face provide the Hosted Inference API so that developers can test and evaluate (for free) over 150,000 publicly accessible machine learning models, or your own private models, via simple HTTP requests, with the fast inference hosted on Hugging Face shared infrastructure.

As an example, I have dragged and dropped an image (see Figure 1.4; source of image: [https://en.wikipedia.org/wiki/Airport#/media/File:2013-02-22\\_10-06-11\\_South\\_Africa\\_Kwa\\_Zulu\\_Natal\\_Tongaat\\_King\\_Shaka\\_International\\_Airport.JPG](https://en.wikipedia.org/wiki/Airport#/media/File:2013-02-22_10-06-11_South_Africa_Kwa_Zulu_Natal_Tongaat_King_Shaka_International_Airport.JPG)) onto the **Hosted inference API** section of the page for the **facebook/detr-resnet-50** model on Hugging Face website. Using the model, the objects detected in the image are automatically shown, together with the level of confidence displayed next to each object name.



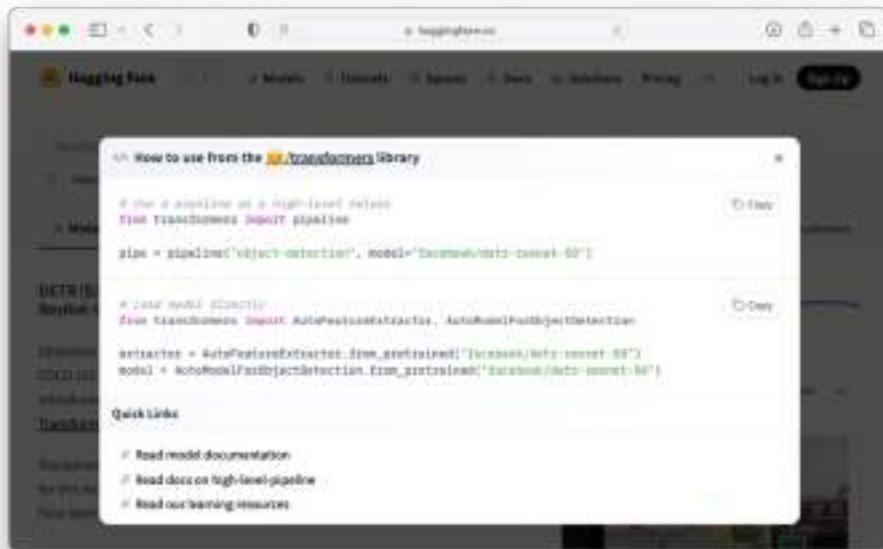
**Figure 1.4** Performing object detection using my uploaded image

In addition, you can also make use of this model in your own Python code. To do so, click on the “Use this model” button, followed by the Transformers button (see Figure 1.5).



**Figure 1.5** Locating the “</> Use in Transformers” button

You will see the code samples that show how to use the model (see Figure 1.6).



**Figure 1.6 Using the model using the transformers library**

You will learn more about using the transformers library with the various models for object detections in Chapter 4.

### 1.3 Hugging Face Gradio Python Library

As an AI developer, you spend a lot of time building and training your machine learning or deep learning models. Once your model is trained to your satisfaction, the next logical step would be to let your users try it out. Typically, this would mean building your dedicated UI (commonly web apps), or exposing your models using a REST API. However, this would mean you have to spend time building all these user interfaces.

Wouldn't it be nice if there is a package that can automatically expose your model so that the user can just try it out quickly? This is where Gradio comes in.

Gradio is an open-source Python library that makes it easy to create customizable user interfaces for machine learning models and data science workflows. With just a few lines of code, you can wrap your model in a simple web-based interface where users can upload inputs (such as text, images, or audio) and view the outputs in real time. It's widely used for demoing models, collecting user feedback, and building interactive machine learning applications. Gradio also integrates seamlessly with Hugging Face Spaces, allowing developers to share their apps publicly with the community.

Gradio was founded by Abubakar Abid, a PhD student at Stanford, focusing on deep learning applied to medical images and videos. During his PhD, he developed Gradio ([www.gradio.dev](http://www.gradio.dev)), an open-source Python library for creating GUIs for machine learning models. On December 21, 2021, Hugging Face announced its acquisition of Gradio.

To understand how Gradio works, consider the following example. Suppose you have a function named `transform_image()` that accepts an image and then returns the same image in grayscale format:

```
from skimage.color import rgb2gray

def transform_image(img):
    return rgb2gray(img)
```

To try it out this function, you must write your own user interface to accept an image from the user and then once the image is converted to grayscale you need to display the image back to the user. Using Gradio, things are much simplified – Gradio creates a web-based UI that allows users to drag and drop an image, and then displays the image that has been converted.

The following code snippet shows how Gradio binds to the `transform_image()` function:

```
import gradio as gr

demo = gr.Interface(fn = transform_image,
                    inputs = gr.Image(),
                    outputs = "image")

demo.launch()
```

When you run the above code, Gradio will now host your code on the local machine and create an UI as shown in Figure 1.7.



**Figure 1.7** Gradio provides a customizable UI for your ML projects

You can drop an image on the left side of the page, click the **Submit** button to send the image to the `transform_image()` function, and the result will now be displayed on the right side of the page. Figure 1.8 shows how my image is shown when it is converted to grayscale.



**Figure 1.8 Viewing the result of the converted image**

We will be talking more about Gradio throughout this book. And in the upcoming chapters you will learn about the various ways you can customize Gradio's look and feel.

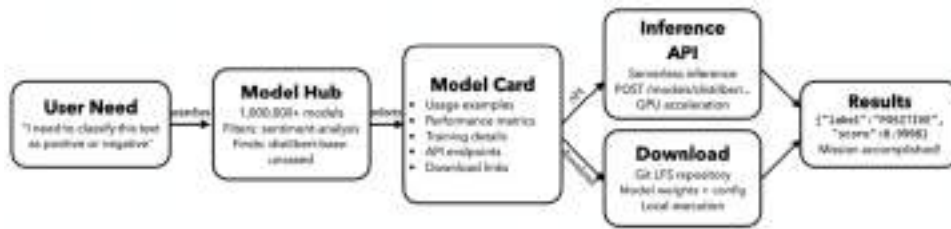
## 1.4 Understanding the Hugging Face Mental Model

As you have seen, Hugging Face isn't just a model repository. It's a complete AI problem-solving pipeline that systematically moves users from problems to solution.

Think of Hugging Face as the world's largest AI model library combined with an execution platform. Every day, millions of developers, researchers, and businesses follow this same basic pattern to go from having an AI problem to getting results.

Figure 1.9 shows a visual mental model showing Hugging Face's core process - how a user goes from needing an AI model to getting results. This happens millions of times daily across their platform.





**Figure 1.9 A visual mental model showing Hugging Face's core process**

In the following sections, I will explain the mental process a user goes through when faced with a need, to solving it using Hugging Face.

### 1.4.1 Step 1: The User Need

Everything starts with a specific problem. A developer sits at their computer thinking, "I need to classify this customer review as positive or negative" or "I need to translate this text from English to French." This isn't abstract - it's a concrete business need with real data waiting to be processed.

### 1.4.2 Step 2: Model Hub Discovery

The user heads to Hugging Face's Model Hub, which contains over one million pre-trained models. This isn't just a dumping ground - it's a sophisticated search and filtering system. Users can search by task (sentiment analysis, translation, image classification), by model architecture (BERT, GPT, ResNet), by language, or by performance metrics. The platform guides users from "I have a problem" to "Here's the specific model that solves it."

### 1.4.3 Step 3: The Model Card Bridge

This is where Hugging Face really shines. Every model comes with a detailed "Model Card" that serves as both documentation and gateway. These cards contain usage examples (copy-paste code snippets), performance benchmarks, training details, and crucially - information about how to actually use the model. The Model Card is the bridge between discovery and implementation.

### 1.4.4 Step 4: Two Execution Paths

Here's where users choose their adventure:

- Path A: Inference API - The fastest route. Users can send HTTP requests directly to Hugging Face's servers, which host the models on GPU clusters. No setup required - just send your text in a POST request and get JSON results back. This handles millions of API calls daily and auto-scales based on demand.

- Path B: Direct Download - For users who want to run models locally or integrate them into their own infrastructure. Behind the scenes, this uses Git LFS (Large File Storage) to handle multi-gigabyte model files. Users download the model weights and configuration files, then run them using the Transformers library.

#### 1.4.5 Step 5: Results Delivered

Both paths converge here - the user gets their answer. The sentiment classifier returns `{"label": "POSITIVE", "score": 0.9998}` and the user's original problem is solved. Mission accomplished.

### 1.5 Summary

- The *Transformers Library* is a Python package that contains open-source implementation of the Transformer architecture models for text, image, and audio tasks.
- In Hugging Face's Transformers library, a pipeline is a high-level, user-friendly API that simplifies the process of building and using complex natural language processing (NLP) workflows.
- The Hugging Face Hub's Models page hosts many pre-trained models for a wide variety of machine learning tasks.
- Gradio is a Python library that creates a Web UI that you can use to bind to your machine learning models, making it easy for you to test your models without spending time building the UI.
- Hugging Face isn't just a model repository. It's a complete AI problem-solving pipeline that systematically moves users from problems to solution.

## ***2 Getting Started***

### **This chapter covers**

- Using Anaconda
- Creating virtual environments using conda
- Using GPU in the `pipeline()` function
- Using the Hugging Face Hub package Setting up the Development Environment

Previously, you saw some of the exciting projects that you will be creating throughout this book using the Hugging Face pre-trained models as well as services such as AutoTrain. The main programming language you will be using is Python and personally my personal favorite IDE is Jupyter Notebook.

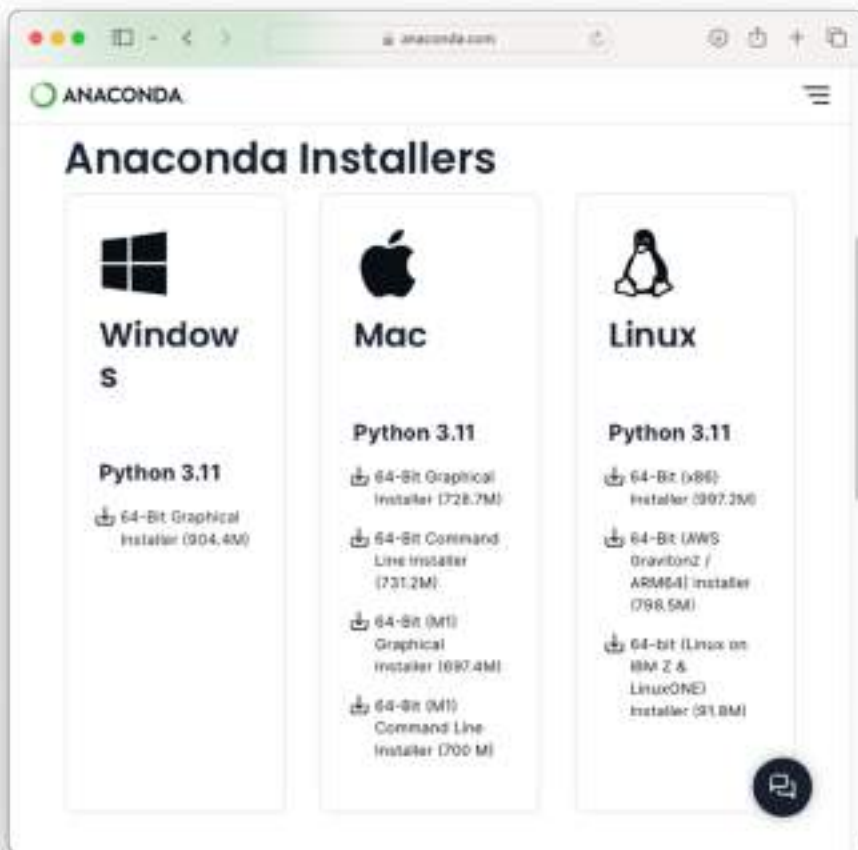
Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text. It's widely used in data science, scientific research, machine learning, and education due to its interactive and exploratory nature.

In the following sections, you will learn how to setup Jupyter Notebook and how to create a virtual environment to work with all the examples in this book.

## 2.1 Downloading Anaconda

The easiest way to install Jupyter Notebook is to download Anaconda. Anaconda is a distribution of Python and R programming languages that comes with a set of pre-installed libraries and tools commonly used in data science, machine learning, and scientific computing. It includes the `conda` package manager, which simplifies package management and environment creation. Jupyter Notebook, is one of the core components within Anaconda. Hence, installing Anaconda not only gives you access to Jupyter Notebook, but it also comes with many common used packages already installed.

To obtain Anaconda (free for personal use), go to <https://www.anaconda.com/download/success> (see Figure 2.1). Click on the respective download button for the operating systems you are using.



**Figure 2.1** Downloading Anaconda for the three major platforms – Windows, Mac, and Linux

When the installer is downloaded, double-click the installer, and follow the on-screen instructions to install Anaconda on your computer.

### 2.1.1 Creating Virtual Environments

With Anaconda downloaded and installed, you are now ready to start using it. But before you launch Jupyter Notebook and start writing code, it is recommended that you create a *virtual environment*. A virtual environment is a self-contained environment that allows you to install and manage Python packages separately from your system-wide Python installation. It's a useful tool for isolating dependencies and managing different project requirements.

To create a virtual environment, launch **Terminal** (macOS users) or **Anaconda Prompt** (for Windows users). Figure 2.2 shows how you can launch the Anaconda Prompt in Windows. In the Windows search box, searching for “anacon” will reveal the Anaconda Prompt in the search result.

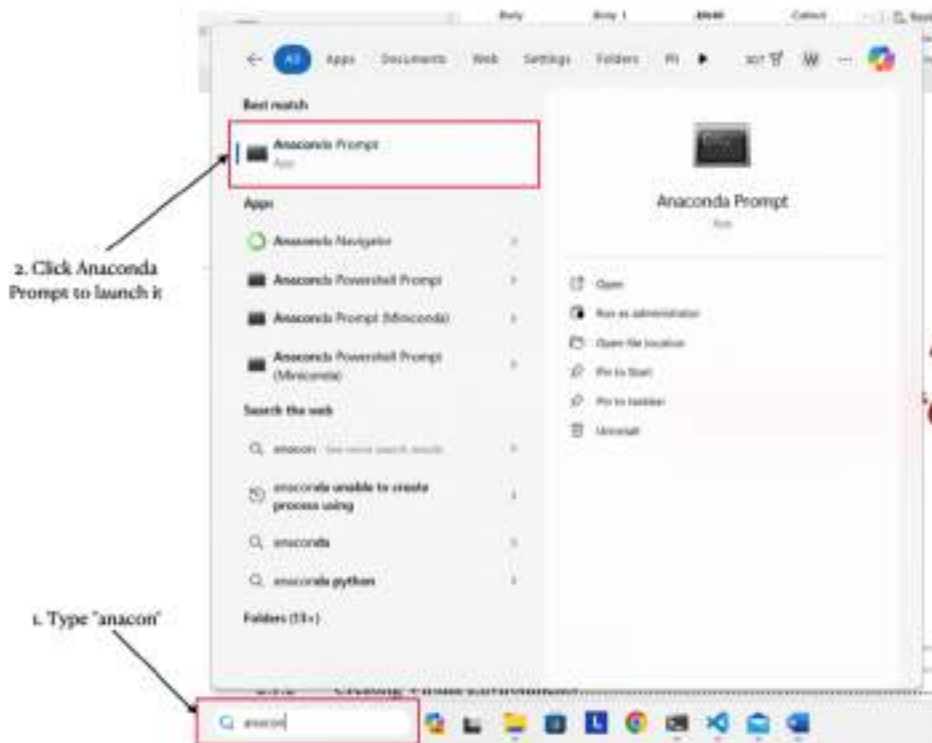


Figure 2.2 Launching the Anaconda Prompt in Windows

## ANACONDA PROMPT VS COMMAND PROMPT

For Windows users, the Anaconda Prompt looks just like the usual Command Prompt. In fact, they are the same, with one notable exception. The Anaconda Prompt has the Anaconda environment variables and paths already set up. This means that when you open the Anaconda Prompt, it's ready to use the Python interpreter, `conda` package manager, and other tools provided by Anaconda without requiring additional configuration.

Then, use the `conda` command to create a new virtual environment, like the following:

```
$ conda create -n HuggingFaceBook python=3.11 anaconda
```

The above command creates a virtual environment named **HuggingFaceBook** with Python 3.11. It also includes the Anaconda distribution.

You will be prompted to install the various packages (see Figure 2.3). Type “Y” and press Enter.

## ANACONDA DISTRIBUTION

The Anaconda distribution is a comprehensive package manager, environment manager, and Python distribution provided by Anaconda, Inc. It is designed to simplify package management and deployment of Python and R data science and machine learning applications. It includes popular libraries like NumPy, pandas, Matplotlib, scikit-learn, TensorFlow, PyTorch, Jupyter Notebook, and many others.

```

weimenglee -- python - conda create -n HuggingFaceBook python=3.11 anaconda -- 54x27
webencodings      pkgs/main/osx-arm64::webencodings-0.5.1-py311hce83da5_1
websocket-client   pkgs/main/osx-arm64::websocket-client-0.58.0-py311hce83da5_4
werkzeug           pkgs/main/osx-arm64::werkzeug-2.2.3-py311hce83da5_0
whatthepatch       pkgs/main/osx-arm64::whatthepatch-1.0.2-py311hce83da5_0
wheel              pkgs/main/osx-arm64::wheel-0.41.2-py311hce83da5_0
widgetsnbextension pkgs/main/osx-arm64::widgetsnbextension-3.6.2-py311hce83da5_1
wrapt              pkgs/main/osx-arm64::wrapt-1.14.1-py311h89987f9_0
wurlitzer          pkgs/main/osx-arm64::wurlitzer-3.0.2-py311hce83da5_0
xarray             pkgs/main/osx-arm64::xarray-2023.6.0-py311hce83da5_0
xlwings            pkgs/main/osx-arm64::xlwings-0.29.1-py311hce83da5_0
xyzservices        pkgs/main/osx-arm64::xyzservices-2022.9.0-py311hce83da5_1
xz                 pkgs/main/osx-arm64::xz-5.4.6-h99987f9_0
yaml               pkgs/main/osx-arm64::yaml-0.2.5-h1a28f6b_0
yapf               pkgs/main/osx-arm64::yapf-0.40.2-py311hce83da5_0
yarl               pkgs/main/osx-arm64::yarl-1.9.3-py311h89987f9_0
zeromq             pkgs/main/osx-arm64::zeromq-4.3.5-h313be8_0
zfp               pkgs/main/osx-arm64::zfp-1.0.0-h313be8_0
zict               pkgs/main/osx-arm64::zict-3.0.0-py311hce83da5_0
zipp               pkgs/main/osx-arm64::zipp-3.17.0-py311hce83da5_0
zlib               pkgs/main/osx-arm64::zlib-1.2.13-h5a8e001_0
zlib-ng            pkgs/main/osx-arm64::zlib-ng-2.0.7-n88987f9_0
zope               pkgs/main/osx-arm64::zope-1.0-py311hce83da5_1
zope.interface     pkgs/main/osx-arm64::zope.interface-5.4.0-py311h89987f9_0
zstd               pkgs/main/osx-arm64::zstd-1.5.5-h99987f9_0

Proceed [Y/n]? Y

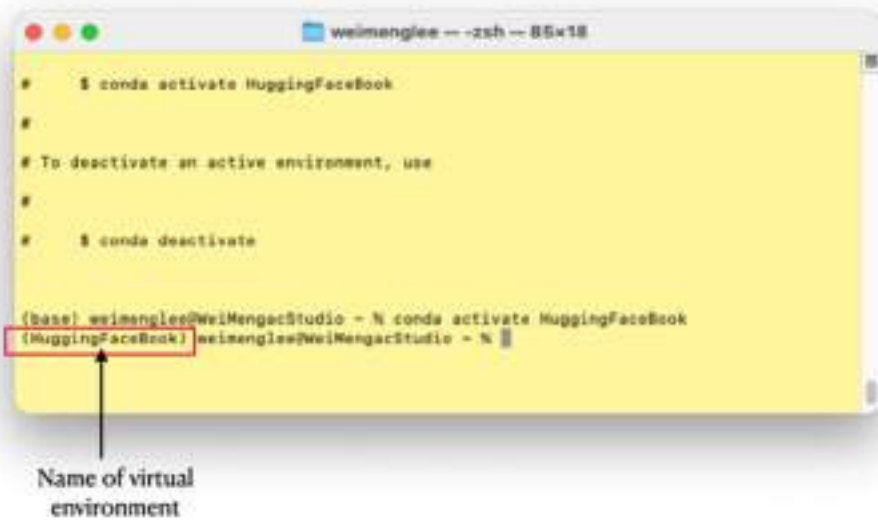
```

**Figure 2.3** Creating a new virtual environment and installing all the required packages

Once the installation is done, you need to “activate” (switch to) the virtual environment using the `conda activate` command:

```
$ conda activate HuggingFaceBook
```

The above statement activates the `HuggingFaceBook` virtual environment. When you have switched to a particular virtual environment, Terminal (or Anaconda Prompt) will prefix the prompt with the environment name (see Figure 2.4).



**Figure 2.4** The virtual environment name will prefix the prompt

### 2.1.2 Starting Jupyter Notebook

Once you have created a virtual environment, you are now ready to launch Jupyter Notebook. Personally, I would prefer to launch it via Terminal (or Anaconda Prompt). In Terminal, first create a folder that you want to save all your projects. Let's call this folder as **HF\_Projects**:

```
(HuggingFaceBook) weimenglee@WeiMengacStudio ~ % mkdir HF_Projects
```

Then, change the current directory to the newly created one:

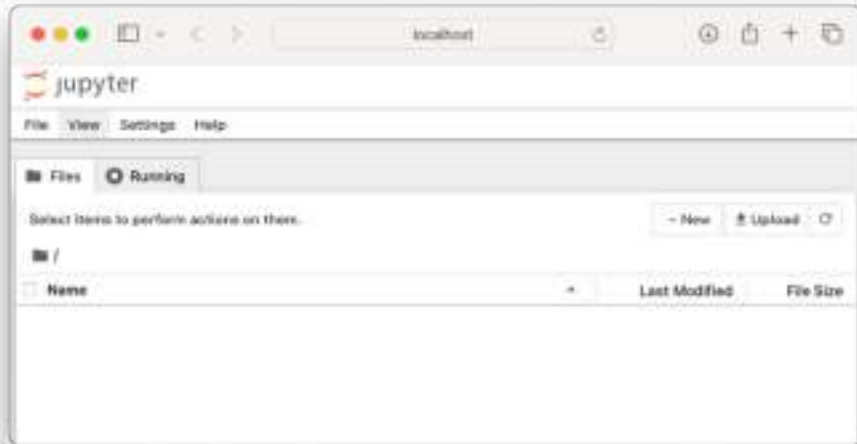
```
(HuggingFaceBook) weimenglee@WeiMengacStudio ~ % cd HF_Projects
```

To launch Jupyter Notebook, type `jupyter notebook`:

```
(HuggingFaceBook) weimenglee@WeiMengacStudio HF_Projects % jupyter notebook
```

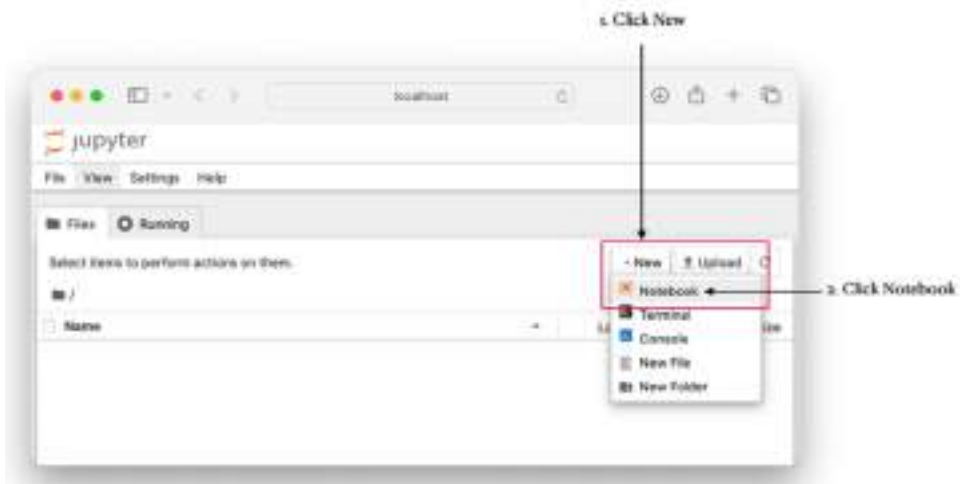
The web browser will now launch and display the Jupyter Notebook's main page as shown in Figure 2.5.





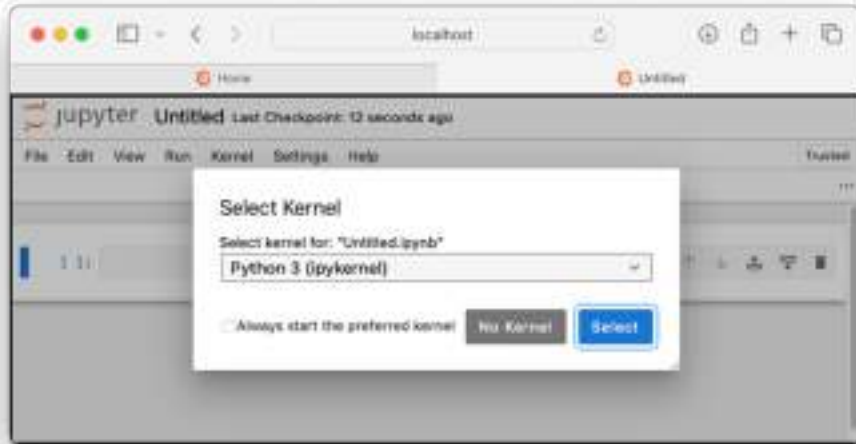
**Figure 2.5** The web browser will now display the Jupyter Notebook's main page

To create a new notebook, click the **New** button and select **Notebook** (see Figure 2.6).



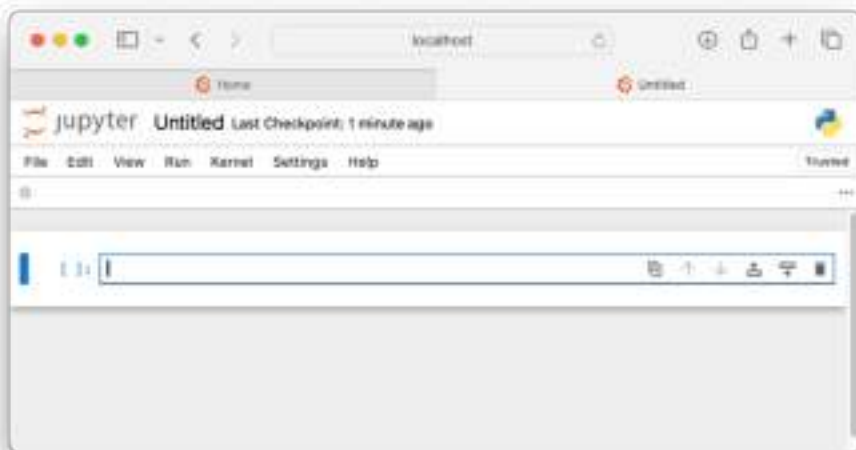
**Figure 2.6** Creating a new notebook

A new tab will appear (if you don't see it, your web browser might be blocking popped up window; clicking on the address bar should reveal the tab). Select the **Python 3 (ipykernel)** option (see Figure 2.7) and click **Select**.



**Figure 2.7 Selecting a kernel for your notebook**

You should now see the notebook ready for you to start coding (see Figure 2.8).

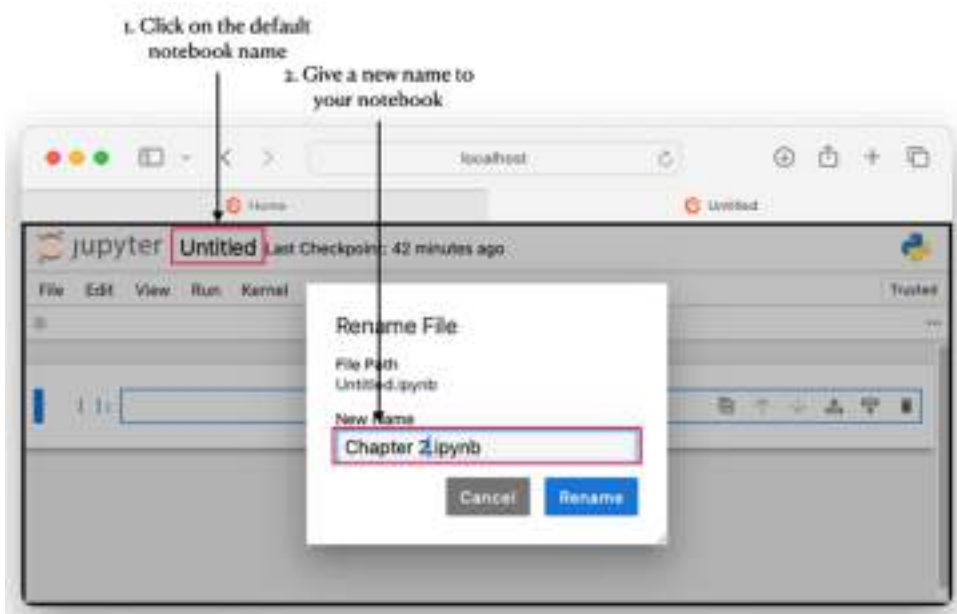


**Figure 2.8 The notebook is now ready to use**

## NEW JUPYTER NOTEBOOK USERS

If you are new to Jupyter Notebook, I suggest you check out the official online documentations of Jupyter Notebook at <https://jupyter-notebook.readthedocs.io/en/stable/notebook.html>.

Finally, rename the notebook by first clicking on the default file name and then entering a new name for your notebook (see Figure 2.9). Click Rename when you are done.



**Figure 2.9 Renaming your notebook**

The “**Chapter 2.ipynb**” file will now be saved in the **HF\_Projects** directory.

## 2.2 Installing the Transformers Library

For this book, you will be using the *Hugging Face Transformers* library extensively. The Hugging Face Transformers library, an open-source library developed by Hugging Face, provides an easy-to-use interface for working with state-of-the-art pre-trained models for various NLP tasks, such as text classification, named entity recognition (NER), text generation, question answering, and more. Later, we will discuss the Transformers library in more details, but for now, it is good idea to install it. You can install the **transformers** Python package directly in Jupyter Notebook using the following statement:

```
!pip install transformers
```

Or if you prefer, you can install it in Terminal (or Anaconda Prompt) using the following command:

```
$ pip install transformers
```

### 2.2.1 Support for GPU

In Hugging Face, you use the transformers library to perform various machine learning tasks such as Natural Language Processing (NLP), image recognition, and more. Behind the scene, the transformers library is primarily built on top of PyTorch, a popular deep learning framework primarily developed by FaceBook's AI Research Lab (FAIR). It leverages on PyTorch's capabilities for building neural network architectures, training models, and optimizing performance on tasks related to NLP.

#### TRANSFORMERS LIBRARY AND TENSORFLOW

The Transformers library also provides support for TensorFlow, another widely used deep learning framework developed by Google. This support for TensorFlow allows users to leverage the capabilities of the Transformers library within their TensorFlow-based workflows, enabling tasks related to natural language processing (NLP) and other machine learning tasks.

This book focuses on using the Transformers library with PyTorch.

PyTorch's notable feature includes its GPU support, enabling smooth integration with Nvidia's CUDA (Compute Unified Device Architecture), a parallel computing platform and programming model designed for GPUs. This support for GPU allows PyTorch to leverage CUDA to enable GPU acceleration for computations. PyTorch can offload tensor operations and computations to the GPU, significantly speeding up training and inferencing for deep learning models. Best of all, PyTorch's API can automatically handle the details of moving data between CPU and GPU memory as needed. Finally, PyTorch supports model parallelism, allowing you to split large models across multiple GPUs. This allows large models that are not able to fit into a single GPU to be distributed across GPUs.

That said, if you have a CUDA-supported GPU, you should make use of it to speed up your training and inferencing tasks.

To do so, you need to install PyTorch (plus a few other packages) from a location that contains the PyTorch wheels compatible with CUDA. The following command installs the **torch** (PyTorch), **torchvision** and **torchaudio** packages from <https://download.pytorch.org/whl/cu121>:

```
$ pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121 -U
```

Once the packages are installed, you can now test the packages to see if your GPU is supported. In Jupyter Notebook, you can use the following statements to check if CUDA is available on your system:

```
import torch
print(torch.cuda.is_available())
```

If you see a `True`, this means that CUDA is now supported on your system. The code snippet in Listing 2.1 prints out more detailed information about the GPU you have on your system:

#### Listing 2.1 Using the torch package to find out details of your GPU

```
import torch
use_cuda = torch.cuda.is_available()

if use_cuda:
    print('__CUDNN VERSION:', torch.backends.cudnn.version())
    print('__Number CUDA Devices:', torch.cuda.device_count())
    print('__CUDA Device Name:', torch.cuda.get_device_name(0))
    print('__CUDA Device Total Memory [GB]:',
          torch.cuda.get_device_properties(0).total_memory/1e9)
```

For example, on my laptop equipped with a Nvidia RTX 4060 GPU, this is the result printed by the above code snippet:

```
__CUDNN VERSION: 8801
__Number CUDA Devices: 1
__CUDA Device Name: NVIDIA GeForce RTX 4060 Laptop GPU
__CUDA Device Total Memory [GB]: 8.585216
```

You can also make use of the **GPUtil** package to find out details of your GPU, such as total number of GPUs, utilization load, temperature, memory used, etc. First, install **GPUtil** using the `pip` command:

```
!pip install GPUtil
```

The following statements gets the number of available GPUs on your system using the `getAvailable()` method:

```
import GPUtil

GPUtil.getAvailable()
```

For my system with one GPU, it returns the following result:

```
[0]
```

You can retrieve information about each GPU in your system by utilizing the `getGPUs()` method. Iterate through each GPU to fetch details like its name, utilization, memory usage, temperature, and total memory (see Listing 2.2).

#### Listing 2.2 Using the GPUtil package to get details of each GPU

```
import GPUtil

gpus = GPUtil.getGPUs()

for gpu in gpus:
    print("GPU ID:", gpu.id)
    print("GPU Name:", gpu.name)
    print("GPU Utilization:", gpu.load * 100, "%")
    print("GPU Memory Utilization:", gpu.memoryUtil * 100, "%")
    print("GPU Temperature:", gpu.temperature, "C")
    print("GPU Total Memory:", gpu.memoryTotal, "MB")
```

### 2.2.2 Using GPU in the Pipeline object

For the `pipeline` object to make use of the GPU, you need to explicitly specify the `device` parameter when calling the `pipeline()` function. For example, the following code snippet shows the `pipeline()` function making use of the first (or single) GPU in your system:

```
from transformers import pipeline
question_classifier = pipeline("text-classification",
                              model="huaen/question_detection",
                              device = 0)
```

## TRANSFORMERS PIPELINE

In Hugging Face's Transformers library, a pipeline is a high-level, user-friendly API that simplifies the process of building and using complex natural language processing (NLP) workflows. It allows you to easily perform a sequence of NLP tasks, such as text classification, named entity recognition, translation, summarization, and more, with just a few lines of code.

The above sample creates a `pipeline` object that performs question classification – it takes in a string and returns a result indicating if the string presented is a question, or not.

Besides specifying a number for the `device` parameter to specify that you want to use the GPU for processing, you can also set it to a string. For example, the above statement can be rewritten as:

```
question_classifier = pipeline("text-classification",
                              model="huaen/question_detection",
                              device = "cuda:0") #A
```

#A `cuda:0` refers to the first GPU on your system

On Mac, you can accelerate Hugging Face pipelines using Metal Performance Shaders (MPS) for faster inference on Apple Silicon. Just set `device` to `"mps:0"`:

```
question_classifier = pipeline("text-classification",
                              model="huaen/question_detection",
                              device = "mps:0")
```

Table 2.1 shows the various values you can use to set the `device` parameter and their uses.

**Table 2.1** The values to set for the device parameter in the `pipeline()` function

| Numeric value | String value       | Description                                                                                                                                                              |
|---------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -1            | "cpu"              | Make use of the CPU for processing. This is the default device to use for the <code>pipeline()</code> function.                                                          |
| 0             | "cuda"<br>"cuda:0" | Make use of the single/first GPU on your system.                                                                                                                         |
| 1             | "cuda:1"           | Make use of the second GPU on your system.                                                                                                                               |
| n             | "cuda: n"          | Make use of the (n+1) <sup>th</sup> GPU on your system                                                                                                                   |
| 0             | "mps :0"           | Refers to the Metal Performance Shaders (MPS) backend in PyTorch, which allows machine learning models to run on Apple's built-in GPU (found in M1, M2, M3 chips, etc.). |

If you are not sure if your `pipeline` object is using the CPU or GPU, you can print it out using the `device.type` attribute, like this:

```
print(question_classifier.device.type)
```

A reliable approach to selecting the optimal inference device is to check for CUDA or MPS support. If neither is available, the system defaults to the CPU. Listing 2.3 demonstrates how to implement this.



**Listing 2.3 Auto-Detecting CUDA, MPS, or CPU for PyTorch Inference**

```
from transformers import pipeline
import torch

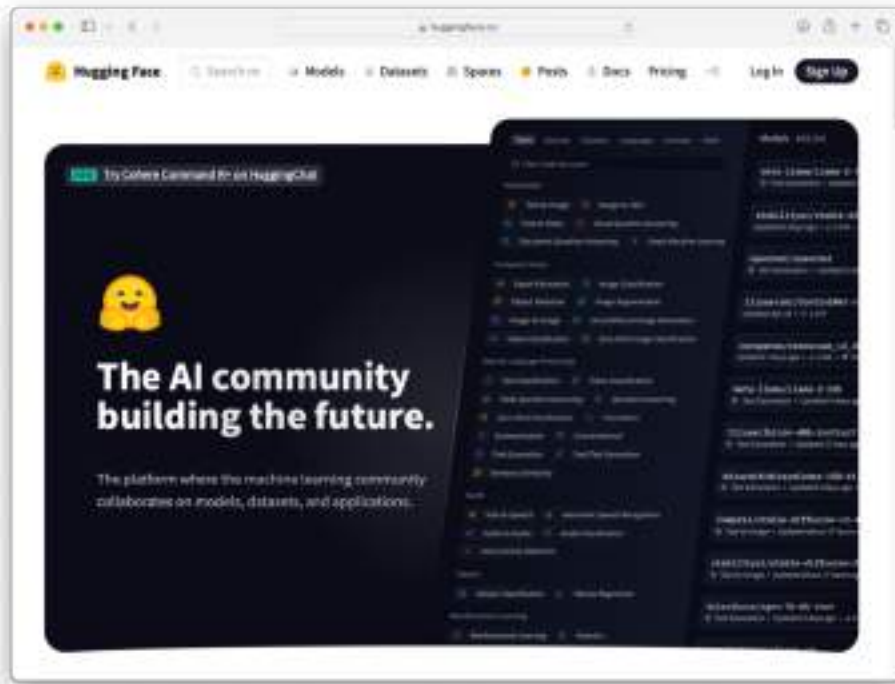
if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"

question_classifier = pipeline("text-classification",
                              model="huan/question_detection",
                              device=device)

print(f"Using device: {device}")
```

## 2.3 Installing the Hugging Face Hub Package

The Hugging Face Hub (<https://huggingface.co>; see Figure 2.10) is the main go-to place for all things related to Hugging Face – using pre-trained models, demos, datasets, and more.



**Figure 2.10 The Hugging Face Hub**

While you can use a web browser to visit the Hugging Face Hub, you can directly interact with the hub using the **huggingface\_hub** (a command line tool) package. Using this package, you can now perform tasks such as:

- Managing project repositories
- Uploading and downloading files
- Fetching models

To install the **huggingface\_hub** package, use the `pip` command:

```
!pip install huggingface_hub
```

### 2.3.1 Downloading Files

On the Hugging Face Hub, you can find many pre-trained models that you can freely use. Often, when you use a model for the first time, the transformers library will automatically download the files associated with the model for you and store it locally on your computer. However, there are times where it is more usefully to be able to manually download the files you need so that you can run your code offline.

To download a file from the Hugging Face hub, you can go directly to the model's page and click the download button. For example, consider the model named "google/pegasus-xsum" (<https://huggingface.co/google/pegasus-xsum/tree/main>). On the page, if you want to download the config.json file, you can click on the download button (see Figure 2.11).



**Figure 2.11** Downloading a file directly from a model's page

Using the **huggingface\_hub** package, you can programmatically download the file using the `hf_hub_download()` function, like this:

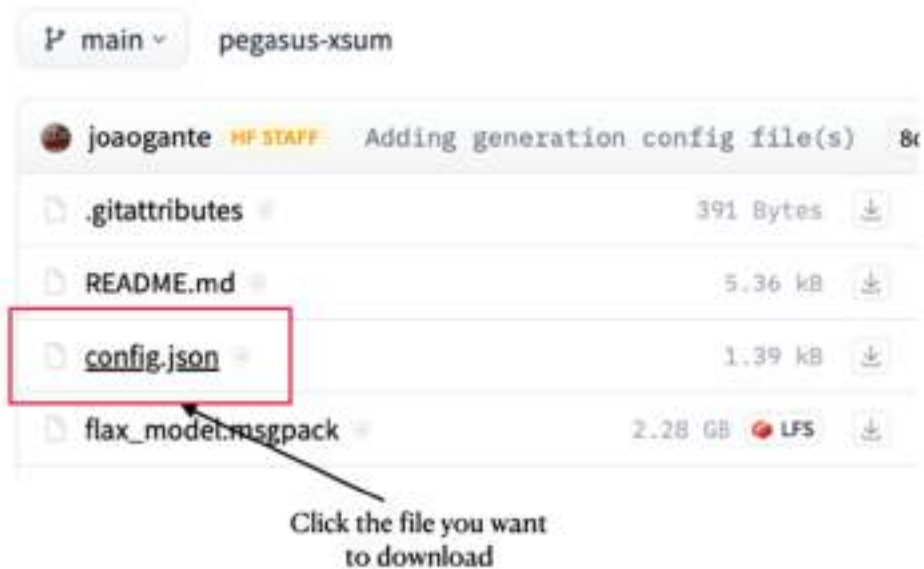
```
from huggingface_hub import hf_hub_download

hf_hub_download(repo_id="google/pegasus-xsum",
                filename = "config.json")
```

The **config.json** file will now be downloaded to the following directory:

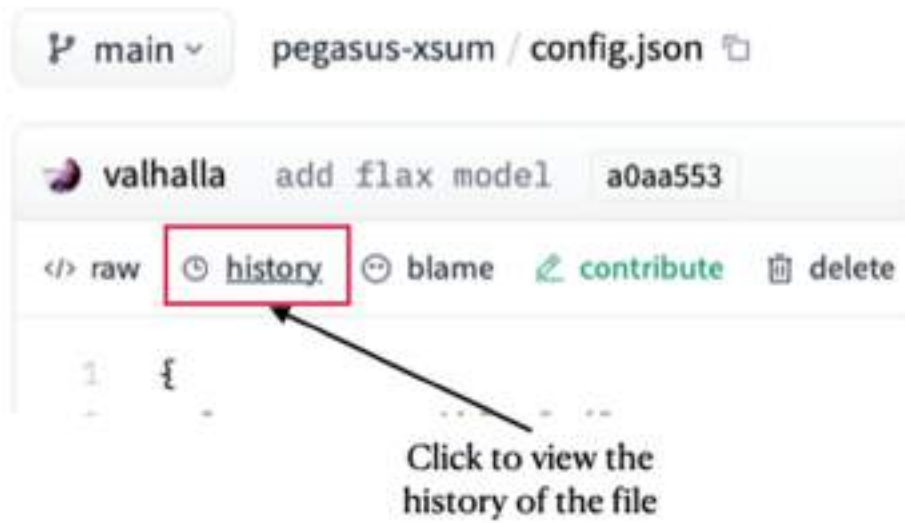
```
<home_directory>/.cache/huggingface/hub/models--google--pegasus-xsum/snapshots/8d8ffc158a3bee9fbb03afacdfc347c823c5ec8b/
```

By default, the latest version of the file from the `main` branch is downloaded. However, in some cases you want to download a particular version of the file (e.g. from a specific branch, a PR, a tag or a commit hash). To do so, first click on the file that you want to download (see Figure 2.12).



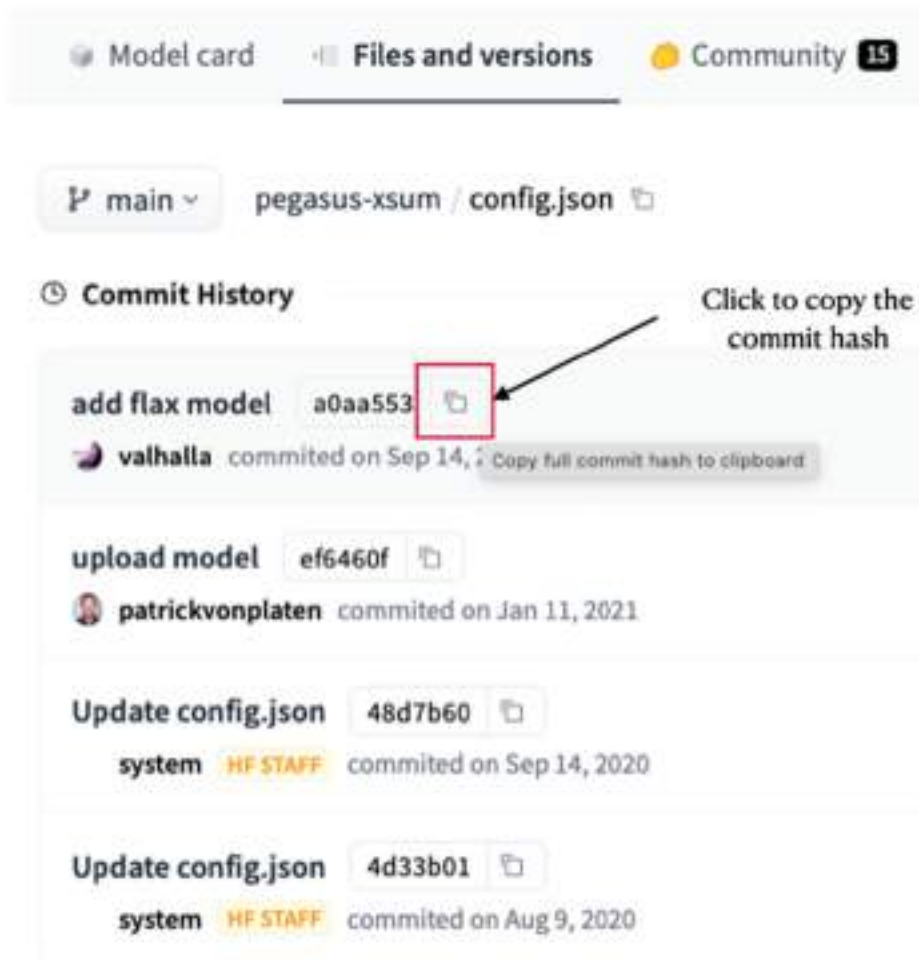
**Figure 2.12** Selecting the file you want to download

Then, click the history link (see Figure 2.13).



**Figure 2.13 Viewing the historical commits for a project**

Finally, copy the commit hash of the specific version of the file that you want to download (see Figure 2.14).



**Figure 2.14 Copying the commit hash for a file**

With the commit hash of the file that you want to download, you can now set it as the value for the `revision` parameter in the `hf_hub_download()` function:

```
hf_hub_download(
    repo_id="google/pegasus-xsum",
    filename="config.json",
    revision="a0aa5531c00f59a32a167b75130805098b046f9c"
)
```

### 2.3.2 Using the Hugging Face CLI

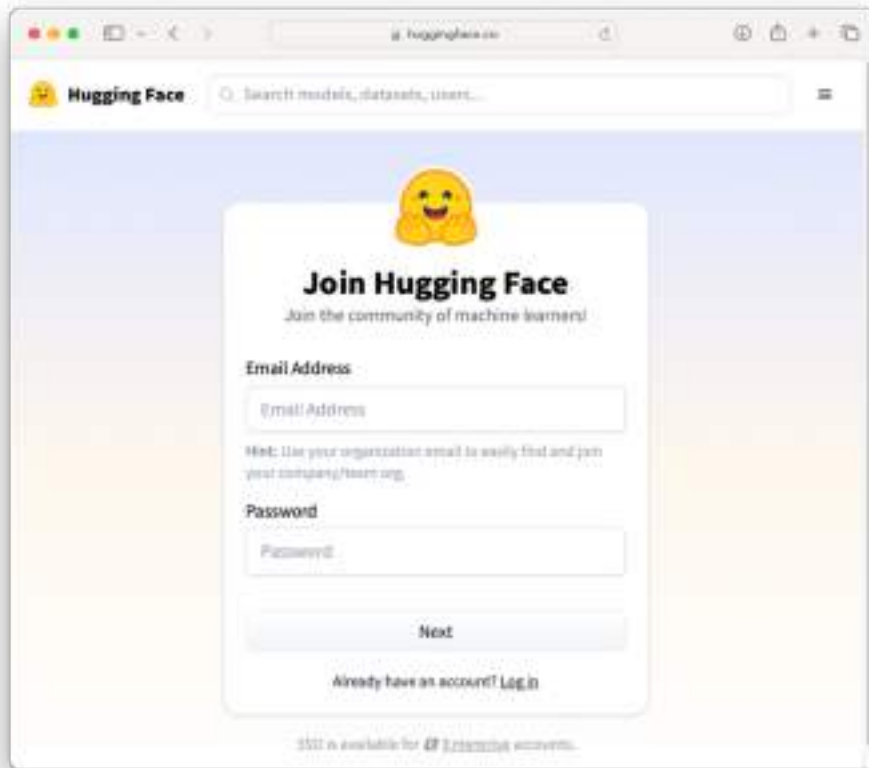
The **huggingface\_hub** package also includes the Hugging Face CLI, a command line interface tool that allows you to authenticate your applications using tokens. In Terminal (or Anaconda Prompt), type `huggingface-cli` to view the various options you can use:

```
$ huggingface-cli
usage: huggingface-cli <command> [<args>]

positional arguments:
  {env,login,whoami,logout,repo,upload,download,lfs-enable-largefiles,lfs-
multipart-upload,scan-cache,delete-cache}
                                huggingface-cli command helpers
  env                        Print information about the environment.
  login                      Log in using a token from huggingface.co/settings/tokens
  whoami                     Find out which huggingface.co account you are logged in
as.
  logout                     Log out
  repo                       {create} Commands to interact with your huggingface.co
repos.
  upload                     Upload a file or a folder to a repo on the Hub
  download                   Download files from the Hub
  lfs-enable-largefiles      Configure your repository to enable upload of files >
5GB.
  scan-cache                 Scan cache directory.
  delete-cache               Delete revisions from the cache directory.

options:
  -h, --help                 show this help message and exit
```

Using the CLI, you can login to Hugging Face Hub programmatically. Before you can do that, you need to create an account at <https://huggingface.co/join> (see Figure 2.15).



**Figure 2.15** Signing up for a new Hugging Face account

Hugging Face uses access tokens to authenticate users. This is used when users need to download private repositories, upload files, create PRs, etc. Once you have signed up as a Hugging Face user, you should create an access token for yourself at <https://huggingface.co/settings/tokens>. Once you have created a token, you can log in to Hugging Face Hub using the following command:







Copy a token from your Hugging Face tokens page and paste it below.  
Immediately click login after copying your token or it might be stored in  
plain text in this notebook file.

Token:

☒ Add token as git credential?

Login

**Pro Tip:** If you don't already have one, you can create a dedicated  
'notebooks' token with 'write' access, that you can then easily reuse for all  
notebooks.

**Figure 2.16 Logging in to Hugging Face Hub from Jupyter Notebook**

Note that if you encountered an error related to ipywidgets in Jupyter Notebook, you should be able to fix it by updating the version of ipywidgets to the latest version:

```
!pip install -U ipywidgets
```

## 2.4 Summary

- The Anaconda package comes with the conda package manager which simplifies package management and environment creation. It also comes with Jupyter Notebook.
- Creating virtual environments allow you to install and manage Python packages separately from your system-wide Python installation. It's a useful tool for isolating dependencies and managing different project requirements.
- The easiest way to start Jupyter Notebook is to launch it from Terminal (or Anaconda Prompt).

- The Transformers library is primarily built on top of PyTorch, a popular deep learning framework primarily developed by FaceBook's AI Research Lab (FAIR).
- PyTorch supports GPU, enabling smooth integration with Nvidia's CUDA (Compute Unified Device Architecture), a parallel computing platform and programming model designed for GPUs.
- The Hugging Face Hub package allows you to download files, upload files, as well as perform authentication using the CLI.

## ***3 Using Hugging Face Transformers and Pipelines for NLP Tasks***

### **This chapter covers**

- The Transformer architecture
- Using the Hugging Face Transformers library
- Using the pipeline function in the Transformers library
- Performing NLP tasks using the Transformers library

You have had a glimpse of the Hugging Face Transformers library and how to use it to perform object detection using one of the pre-trained models hosted by Hugging Face. Now, we will first go behind the scenes and learn about the *Transformers* package – the *Transformer* architecture and the various components that make it work. The aim of this book is not to dive into the detailed workings of the Transformer model, but we want to briefly discuss it so that you have some basic understanding of how things work.

Next, we will make use of the `pipeline()` function that ships with the *transformers* package to perform the various NLP tasks, such as text classifications, text generation, text summarization and more.

## TRANSFORMERS VS TRANSFORMER

When we talk about *Transformers*, we are referring to the open-source library created by Hugging Face that provides pre-trained *transformer* models and tools for NLP tasks.

The *Transformer*, on the other hand, refers to the neural network architecture introduced in the paper "Attention is All You Need" by Vaswani et al. in 2017.

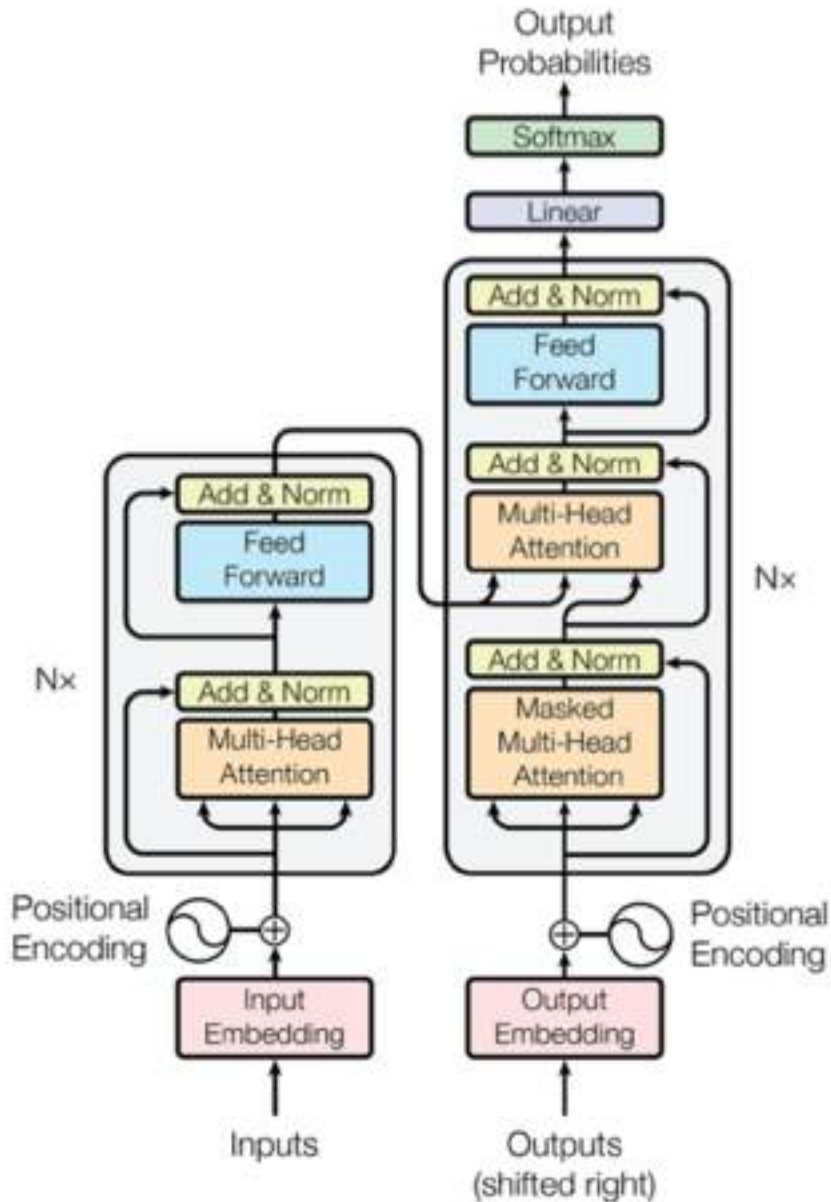
## 3.1 Introduction to the Transformer Architecture

The Transformer architecture, first introduced in the paper "Attention is All You Need" by Vaswani et al. in 2017, has become the foundation for many state-of-the-art models in natural language processing (NLP). It relies heavily on the *self-attention mechanism* to process input data in parallel, making it more efficient and effective for many tasks compared to previous neural network architectures like recurrent neural networks (RNNs) and long short-term memory networks (LSTMs).

Figure 3.1 shows the transformer architecture taken from the *Attention is All You Need* paper.

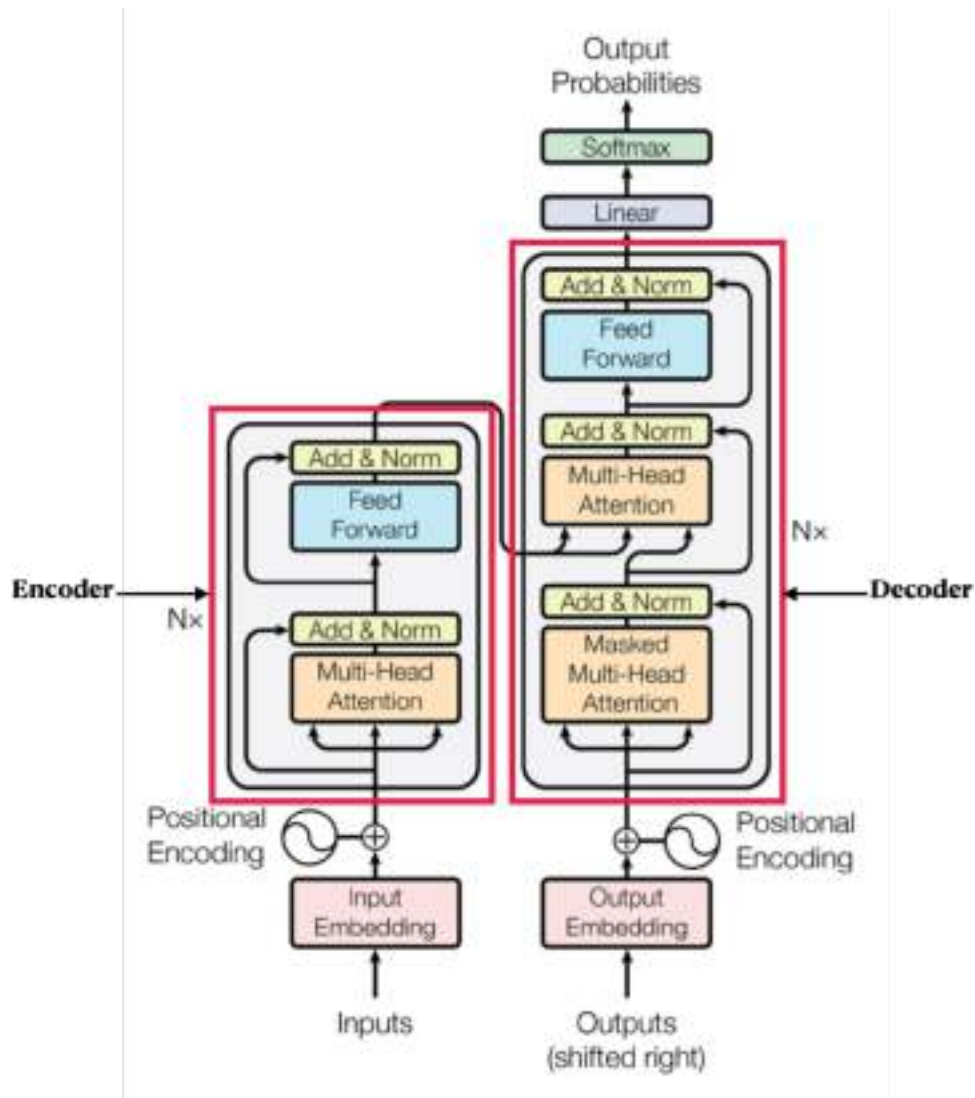
## SELF-ATTENTION MECHANISM

The self-attention mechanism is a crucial component of the transformer architecture. It enables the model to weigh the importance of different words in a sequence when encoding a particular word, allowing the model to capture dependencies and relationships between words irrespective of their distance in the sequence. This mechanism is what allows transformers to handle long-range dependencies and parallelize the processing of input sequences effectively.



**Figure 3.1 The Transformer Architecture** (source: Attention Is All You Need <https://arxiv.org/pdf/1706.03762.pdf>)

At a very high level, the transformer architecture consists of two main blocks – *Encoder* and *Decoder* (see figure 3.2).



**Figure 3.2 A transformer model contains of an Encoder and a Decoder**

The Encoder gets the inputs and builds a representation of it. The Decoder uses the encoder's representation, along with other inputs, to generate a target sequence (the outputs probabilities). One intriguing aspect of this model is that each component can function independently. For instance, you can utilize a model solely featuring the Encoder component, which proves beneficial for tasks like sentence classification or named entity recognition (NER), where comprehension of the input is paramount. Conversely, models employing only the Decoder are suitable for tasks such as text generation. Furthermore, models incorporating both the Encoder and Decoder are well-suited for endeavors like text summarization or text translation.

From Figure 3.2, some of the key components of the transformer architecture are as follows:

- **Input Embeddings**
  - Converts input tokens into dense vectors of fixed size.
  - Adds positional encodings to retain information about the order of the tokens, as the model processes input in parallel and lacks inherent sequential information.
- **Encoder** (comprises multiple identical layers, each containing two main components)
  - **Multi-Head Attention:** Computes the attention scores between each pair of input tokens to capture dependencies regardless of their distance.
  - **Feed-Forward Neural Network (FFN):** Comprises two linear transformations with a ReLU activation in between.
- **Decoder** (also consists of multiple identical layers with additional components to handle the target sequence)
  - **Masked Multi-Head Attention:** Similar to the encoder's multi-head attention but prevents attending to future tokens in the target sequence (to ensure auto-regressive properties during training).
  - **Multi-Head Attention:** Allows the decoder to attend to the Encoder's output.
- **Positional Encoding**
  - Adds information about the position of each token in the sequence, as self-attention operates without considering token order.
  - Typically implemented using sine and cosine functions of different frequencies.
- **Layer Normalization and Residual Connections**
  - **Layer Normalization:** Stabilizes and accelerates training by normalizing the input across the features.
  - **Residual Connections:** Adds the input of each sub-layer to its output to help with gradient flow and prevent vanishing/exploding gradients.
- **Final Linear and Softmax Layers**



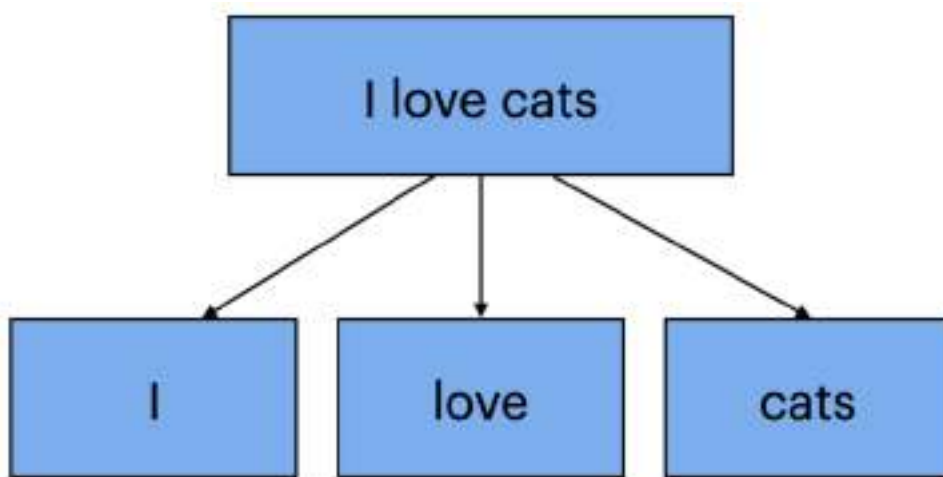
- In the **Decoder**, after processing the sequence through multiple layers, a final linear transformation followed by a **Softmax** layer is used to produce probabilities for the next token.

### 3.1.1 Tokenization

In the context of NLP and machine learning, a "token" is a chunk of text that a model processes as a single unit. Tokens can represent an individual words, punctuation marks, or other linguistic elements, depending on the specific tokenization strategy employed. Tokenization is the process of converting a text document or sentence down into smaller units.

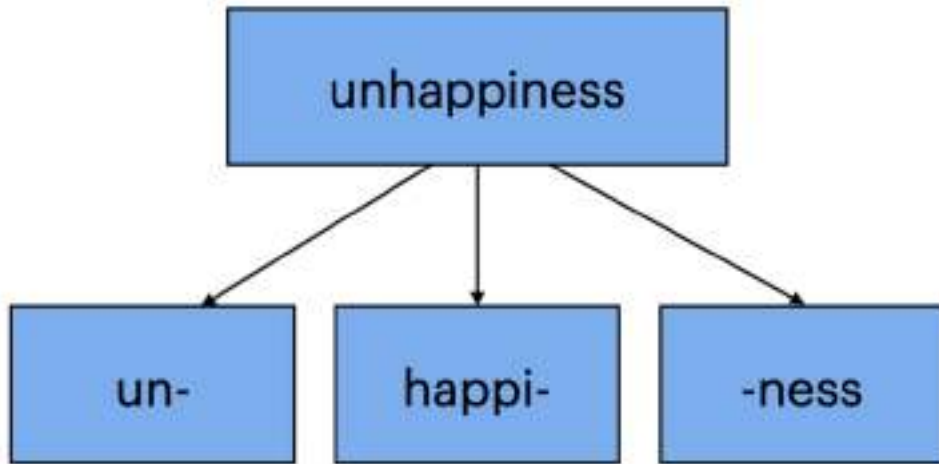
There are generally a few types of tokenization strategies:

- **Word Tokenization** – splitting text into individual word based on whitespace or punctuation characters. Figure 3.3 shows how the sentence "I love cats" is tokenized into three tokens.



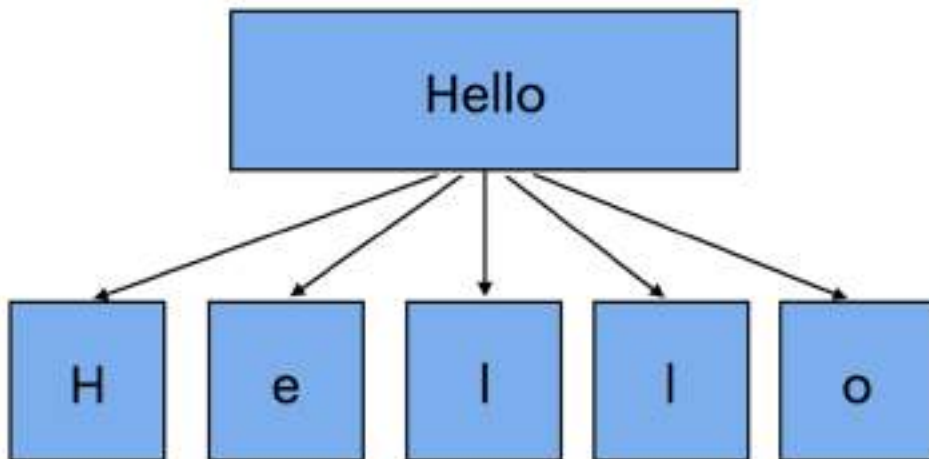
**Figure 3.3 How a sentence is tokenized into three tokens using the word tokenization technique**

- **Subword Tokenization** – breaks text into smaller linguistic units, such as prefixes, suffixes, or root words. It's commonly used for languages with complex morphology or in tasks like machine translation. Figure 3.4 shows how the word "unhappiness" is tokenized into three tokens. Subword tokenization enables the model to encompass a broad and varied vocabulary while circumventing the constraints imposed by a rigid word-level vocabulary. This technique is especially useful for comprehending and generating text across diverse contexts, languages, and domains, enhancing the model's capability to accommodate linguistic variations and diminish vocabulary dimensions.



**Figure 3.4** How a sentence is tokenized into three tokens using the subword tokenization technique

- **Character-Level tokenization** - segmenting text into individual characters, including letters, digits, and punctuation marks. Figure 3.5 shows how the word "Hello" is tokenized into five tokens. Character-level tokenization is ideal for tasks requiring meticulous analysis at the character level, particularly when handling languages with intricate word structures like Chinese. Nonetheless, it entails a larger vocabulary size and may result in reduced interpretability at more advanced linguistic levels.



**Figure 3.5** How a sentence is tokenized into three tokens using the character-level tokenization technique

Here is an example of how to perform sub-word tokenization using the BERT model:

```

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")    #A
input_text = "What is unhappiness?"
tokens = tokenizer.tokenize(input_text, return_tensors="pt")      #B

print(f"{tokens = }")

#A load a pre-trained tokenizer
#B tokenize input text

```

The above code snippet prints out the following tokens:

```
tokens = ['what', 'is', 'un', '##ha', '##pp', '##iness', '?']
```

### WHAT IS BERT?

BERT stands for **Bidirectional Encoder Representations from Transformers**. It is a transformer-based machine learning model designed for natural language processing (NLP) tasks.

It is commonly used in natural language processing tasks such as question answering, text classification, named entity recognition, part-of-speech tagging, text summarization, sentiment analysis, language translation, text generation, coreference resolution, paraphrase detection, semantic search, textual entailment, and dialog systems.

The ## prefix attached to some tokens in the output indicates that the token is a continuation of the previous one in the original word. It is used to signify that they are part of a larger token. When decoding these tokens back into the original text, the ## prefixes are typically removed, and the tokens are combined to reconstruct the original words.

### 3.1.2 Token Embeddings

Once the text is tokenized, the next step is to perform *token embeddings*. Token embeddings converts tokens into numerical vectors. These embeddings capture semantic and syntactic information about the tokens, enabling machine learning models to understand the underlying meaning and relationships between words in natural language text.

The embeddings are learned based on the co-occurrence and contextual relationships between words in the training corpus. As a result, words that have similar meanings or appear in similar contexts tend to have similar representations in the embedding space.

The code snippet in Listing 3.1 shows how you can perform token embedding on a paragraph of text.

### Listing 3.1 Performing token embeddings on a paragraph of text

```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")    #A
model = BertModel.from_pretrained("bert-base-uncased")          #A

input_text = '''
After a long day at work, Sarah decided to relax by taking her
dog for a walk in the park. As they strolled along the
tree-lined paths, Sarah's dog, Max, eagerly sniffed around,
chasing after squirrels and birds. Sarah smiled as she watched
Max enjoy himself, feeling grateful for the companionship and
joy that her furry friend brought into her life.'''

tokens = tokenizer(input_text, return_tensors="pt")

with torch.no_grad():
    outputs = model(**tokens)                                    #B

last_hidden_states = outputs.last_hidden_state                  #C
print("Token embeddings:")                                       #D
for token, embedding in zip(tokens["input_ids"][0],
                             last_hidden_states[0]):
    word = tokenizer.decode(int(token))
    print(f"{word}: {embedding}")
```

**#A Load pre-trained BERT tokenizer and model**

**#B Generate token embeddings**

**#C Extract token embeddings from the last layer**

**#D Print the token embedding for each word**

In this example, we tokenized the input text using the BERT model and then extract the token embeddings from the last layer of the model's outputs. Finally, we print out the embedding for each word, which looks like that shown in Listing 3.2.

**Listing 3.2 The embeddings for each word in the paragraph**

```

Token embeddings:
[ C L S ]: tensor(
  [ 5.1886e-03, -1.3432e-01, -6.8117e-01, -5.0901e-02, -1.3148e-01,
    -2.2708e-01,  4.2620e-01,  7.9117e-01, -3.0209e-01, -6.5137e-02,
    ...
    -9.4111e-02, -4.7972e-01,  9.1932e-02, -3.9814e-01,  4.3560e-02,
    1.8024e-01,  7.4798e-01,  2.8064e-01])
a f t e r: tensor(
  [-3.1720e-01, -3.1491e-01,  1.3892e-01,  3.9379e-01,  1.3412e-01,
    4.2373e-01,  4.9870e-01,  7.1422e-01,  1.0452e-01, -7.0356e-01,
    ...

```

It would be useful to plot the token embeddings on a graph so that you can visualize how the tokens are related to one another. However, since the embeddings are in very high dimensions, we first need to reduce their dimensionality to two dimensions so that we can visualize them effectively. One common technique for dimensionality reduction and visualization is **t-SNE** (t-Distributed Stochastic Neighbor Embedding). We can use t-SNE to project the high-dimensional embeddings into a 2D space. Here's how you can do it (see Listing 3.3).

**Listing 3.3 Using the t-SNE technique to project high-dimensional embeddings into 2D space**

```

from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

tsne = TSNE(n_components=2, perplexity=5, random_state=42)      #A
embeddings_tsne = tsne.fit_transform(last_hidden_states[0])    #A

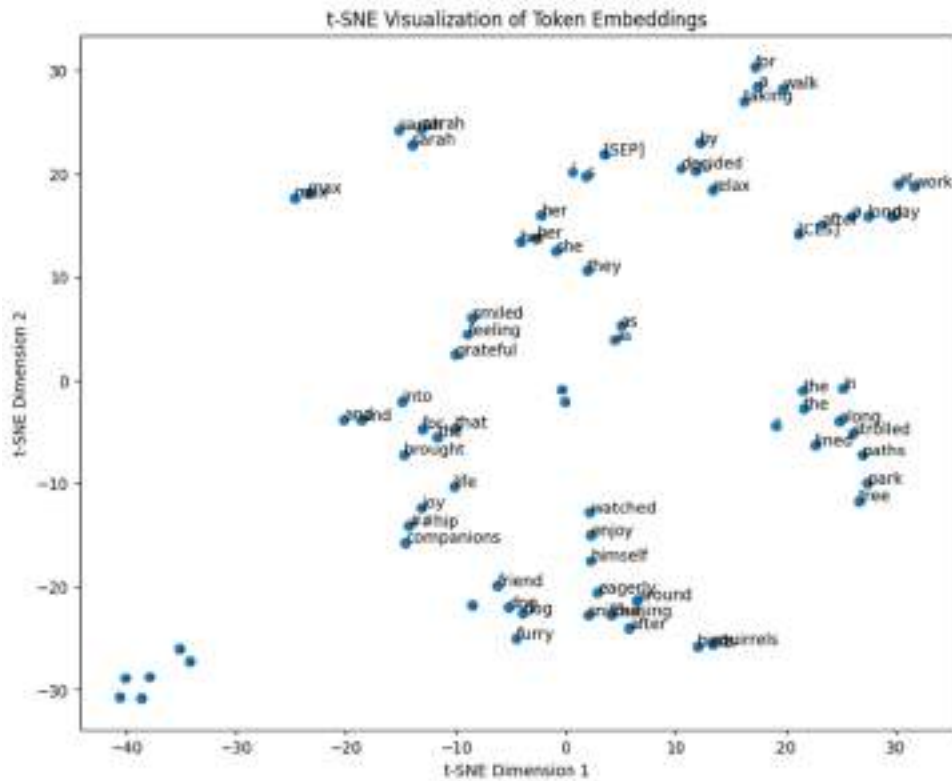
plt.figure(figsize=(10, 8))                                    #B
plt.scatter(embeddings_tsne[:, 0],                               #B
            embeddings_tsne[:, 1], marker='o')                  #B
for i, word in enumerate(tokenizer.convert_ids_to_tokens(      #B
    tokens["input_ids"][0])):                                   #B
    plt.annotate(word, xy=(embeddings_tsne[i, 0],              #B
                           embeddings_tsne[i, 1]),             #B
                 fontsize=10)                                   #B
plt.xlabel('t-SNE Dimension 1')
plt.ylabel('t-SNE Dimension 2')
plt.title('t-SNE Visualization of Token Embeddings')
plt.show()

```

**#A** Reduce dimensionality using t-SNE with lower perplexity, a parameter controlling nearest neighbors—higher for larger datasets but always less than the sample count.

**#B** Plot the embeddings in a 2D graph

Figure 3.6 shows the graph showing how the various tokens are grouped according to their embeddings.



**Figure 3.6** The visualization of token embeddings for the various words in a paragraph in two-dimensional space

In short, word embeddings allow you to see which words are often used together. It captures semantic relationships between words based on their usage patterns in large text corpora. For example:

- The vectors for “king” and “queen” are closer to each other than unrelated words like “train” or “buildings”.
- The vectors for words such as “bread” and “butter” are closer to each other as they often co-occur in text.

### 3.1.3 Positional Encoding

Positional encoding plays a crucial role in transformer-based models by imparting essential positional information about the order of tokens within a sequence. This positional context is vital for the model to grasp the meaning and context of the input accurately. By incorporating positional encoding into token embeddings, the model gains the ability to discern between tokens based on their positions in the sequence. Without this encoding, the model would struggle to differentiate between tokens solely based on their sequential positions, significantly impairing its performance on tasks requiring a nuanced understanding of sequences, such as language modeling, machine translation, and text generation. Positional encoding is typically added to the token embeddings before they are input to the transformer model.

The code snippet in Listing 3.4 shows how you can extract the positional embeddings on a paragraph of text.



**Listing 3.4 Printing out the positional encoding for each token**

```

from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

input_text = '''
After a long day at work, Sarah decided to relax by taking her
dog for a walk in the park. As they strolled along the
tree-lined paths, Sarah's dog, Max, eagerly sniffed around,
chasing after squirrels and birds. Sarah smiled as she watched
Max enjoy himself, feeling grateful for the companionship and
joy that her furry friend brought into her life.'''

tokens = tokenizer(input_text, return_tensors="pt")
embeddings = model.embeddings #A
positional_embeddings = embeddings.position_embeddings.weight #B
position_ids = torch.arange(tokens['input_ids'].size(1),
                             dtype=torch.long).unsqueeze(0) #C
input_positional_embeddings = positional_embeddings[position_ids] #D

print("Positional embeddings shape:", input_positional_embeddings.shape)
print("Positional embeddings for each token:")

for token_id, pos_embedding in zip(tokens['input_ids'][0],
                                   input_positional_embeddings[0]):
    token = tokenizer.decode([token_id])
    print(f"{token}: {pos_embedding}")

```

**#A** access the embeddings layer directly

**#B** extract the positional embeddings

**#C** extract the position IDs from the input tokens

**#D** get the positional encodings for the input tokens

In the BERT model, positional encodings are already integrated into the model architecture and are automatically added to the input embeddings. So, in this example we are simply extracting the positional embeddings for each token. Listing 3.5 shows the output of the code.

**Listing 3.5 The output for the positional encoding**

```

Positional embeddings shape: torch.Size([1, 77, 768])
Positional embeddings for each token:
[CLS]: tensor([ 1.7505e-02, -2.5631e-02, -3.6642e-02, -2.5286e-02,  7.9709e-03,
               -2.0358e-02, -3.7631e-03, -4.6880e-03,  6.2253e-03, -3.8342e-02,
               1.3103e-02, -3.7083e-03, -2.1014e-02,  1.1626e-02, -3.9546e-02,
               ...
               4.0483e-03, -3.4331e-02,  1.0333e-02, -1.0450e-02, -1.4161e-02,
               3.3437e-05,  6.8312e-04,  1.5441e-02], grad_fn=<UnbindBackward0>)
after: tensor([ 7.7580e-03,  2.2613e-03, -1.9444e-02, -1.7131e-02, -1.3234e-02,
               1.4102e-02, -3.7121e-03, -1.0888e-02,  6.2255e-03, -3.4778e-02,
               -7.7945e-03, -1.4488e-02, -1.1725e-02,  1.0181e-02, -5.9442e-03,
               ...

```

In short, positional encodings provide information about the positions of words within a sequence. It allows models to understand the order of words in a sentence. For example:

- The sentence "The cat sat on the sofa" has a different meaning than "The sofa sat on the cat".
- Positional embeddings allow you to differentiate between "Tom loves Susan" and "Susan loves Tom".

### 3.1.4 Transformer Block

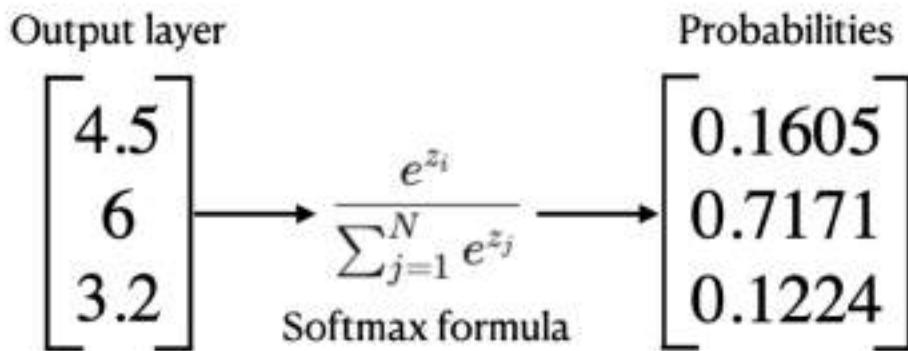
At the heart of the Transformer architecture is the Transformer block, which is the key component responsible for encoding and decoding information across multiple layers. The transformer block contains the following components:

- **Self-Attention Mechanism** - The Transformer block utilizes self-attention, also known as scaled dot-product attention. This mechanism allows the model to weigh the importance of different words (tokens) in a sequence based on their relationships with each other. It computes attention scores for each pair of words in the sequence and uses these scores to construct context-aware representations of each word.
- **Feedforward Neural Networks** - After the self-attention mechanism, the Transformer block applies feedforward neural networks (FFNs) to process each word's representation independently and in parallel. These FFNs typically consist of two linear transformations separated by a non-linear activation function like ReLU.
- **Residual Connections and Layer Normalization** - To facilitate effective gradient flow and ease training, residual connections are employed around each sub-layer (self-attention and FFNs) of the Transformer block. Additionally, layer normalization is applied to stabilize training and improve the speed and convergence of the model.

### 3.1.5 Softmax

The last component in the transformer architecture is the Softmax. Softmax is a mathematical function that converts a vector of numbers into a probability distribution, where the probability of each element is proportional to the exponentiation of that element's value relative to the sum of all the exponentiated values in the vector. In the context of neural networks, Softmax is often used as the final activation function in classification tasks.

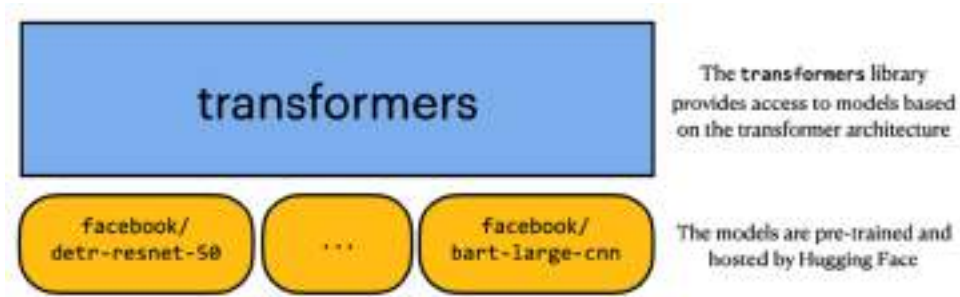
Figure 3.7 shows an example of the use of the Softmax function. On the left are the values for the output layer of a neural network. To transform the values into probabilities that the model can interpret, you use the Softmax function to generate the probability distribution. The probabilities generated all sum up to 1. Each value in the probabilities represents the probability of a class or category.



**Figure 3.7 Using Softmax function to generate a set of probabilities based on the values of the output layer of a neural network**

## 3.2 What are Hugging Face Transformers?

Now that you have a clearer idea of how the transformer architecture work, it is time to focus our attention on the key subject of this book – the *Hugging Face Transformers*. As the name implies, the Hugging Face Transformers is an open-source library and platform developed by Hugging Face. The aim of this library is to provide an easy-to-access interface for working with state-of-the-art transformer-based models. Figure 3.8 summarizes the use of the Hugging Face Transformers library.



**Figure 3.8 The role of the Hugging Face Transformers library**

Next, you will learn about how you can make use of transformers library from Hugging Face, the various pre-trained models that you can use, and how you can simplify the process using *pipelines*.

### 3.2.1 What are Pre-trained Transformers Models?

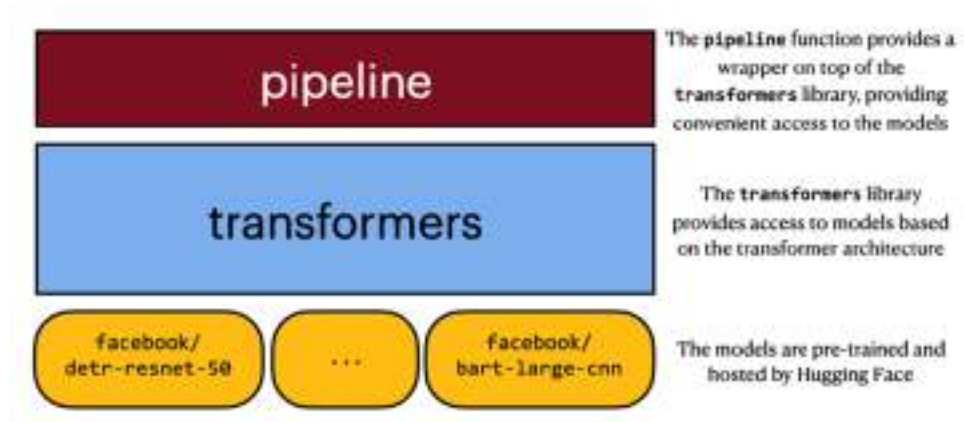
Pre-trained transformers models refer to transformer-based neural network models that have been pre-trained on vast amounts of text data. These models are trained using unsupervised learning techniques, such as language modeling or masked language modeling, on large text corpora to learn the statistical properties of natural language.

Examples of pre-trained transformer models include BERT, GPT, RoBERTa, T5, and many others. These models serve as powerful building blocks for various NLP applications and have significantly advanced the state-of-the-art in the field:

- **BERT** (Bidirectional Encoder Representations from Transformers) – Introduced bidirectional training for transformers, capturing context from both directions of a word.
- **GPT** (Generative Pre-trained Transformer) – Focuses on language generation tasks by predicting the next word in a sequence.
- **RoBERTa** (Robustly optimized BERT approach) – Optimized version of BERT with improved training strategies and larger datasets.
- **DistilBERT** – A smaller and faster variant of BERT, suitable for deployment in resource-constrained environments.
- **T5** (Text-To-Text Transfer Transformer) – Trained on a unified framework where all tasks are treated as text-to-text transformations.

### 3.2.2 What are Transformers Pipelines?

To make the Hugging Face Transformers library easier to use, Hugging Face provides the convenient and user-friendly `pipeline()` function (or simply known as a *pipeline*) that shields the lower level details from the developer. Figure 3.9 shows the role played by the `pipeline()` function.



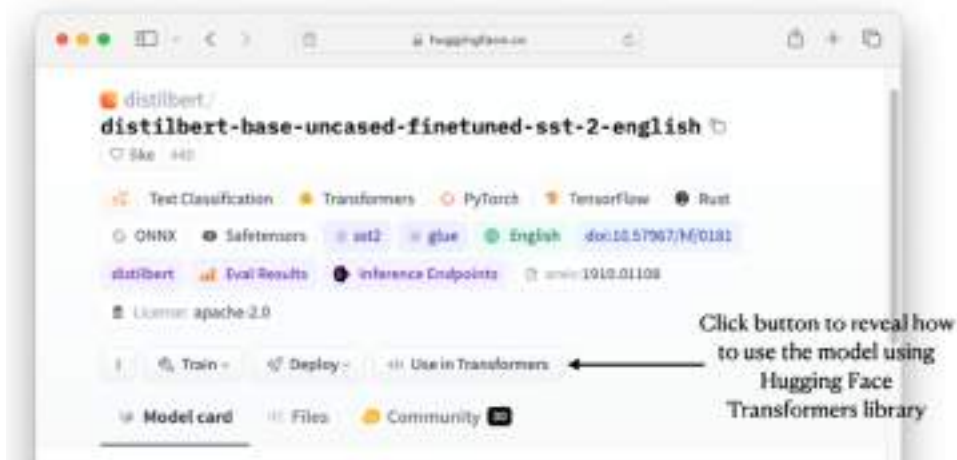
**Figure 3.9 Pipelines are high-level APIs that utilize the transformer models provided by the transformers library**

Consider the `distilbert/distilbert-base-uncased-finetuned-sst-2-english` model, which is a pre-trained DistilBERT model that has been fine-tuned on the SST-2 (Stanford Sentiment Treebank) dataset for sentiment analysis in English.

#### WHAT IS DISTILBERT?

DistilBERT is a lighter, smaller, and faster version of BERT (Bidirectional Encoder Representations from Transformers), a popular transformer-based model for natural language processing (NLP). It was introduced by researchers at Hugging Face in a paper titled "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter" by Sanh et al. (2019).

On Hugging Face web page for this model (<https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>; see figure 3.10), you can click on the `</>` **Use in Transformers** button to reveal how you can use this model.



**Figure 3.10 Learning how to use the model using the Transformers library**

Figure 3.11 shows there are two ways to use the model:

- Through the transformers pipeline, or
- Use the model directly



**Figure 3.11 Two ways to use the pre-trained model – through the pipeline, or use the model directly**

Personally, I find using the model through the pipeline much easier. Table 3.1 shows the various reasons for using either approach.

**Table 3.1 Reasons for using a model directly, or through a pipeline**

| Reasons for using a model directly                                                                                                  | Reasons for using a model through a pipeline                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fine-grained control. You have more control over the input preprocessing, tokenization, model inference, and post-processing steps. | Simplicity. Pipelines provide a high-level and user-friendly interface for using the model without understanding the underlying complexities of model loading, input processing, and inferencing.                                                            |
| Flexibility. You can experiment with different model architectures, hyperparameters, and input formats easily.                      | Rapid prototyping. With pipelines, you can work with models with just a few lines of code.                                                                                                                                                                   |
| Better understanding. Working directly with the model allows you to better understand how and model works.                          | Pre-configure settings. Pipelines come with pre-configured settings and default parameters optimized for common use cases. This can save time and effort in selecting the appropriate model, fine-tuning hyperparameters, and handling input/output formats. |

Now, you will learn how to use the model directly and we will use the much more streamlined approach using pipelines.

### 3.2.3 Using the Models Directly

Our next goal will be to learn how to use a pre-trained Hugging Face model directly. For demonstration, you will use the `distilbert/distilbert-base-uncased-finetuned-sst-2-english` model that was mentioned previously. Using this model, you will use it to perform sentiment analysis on a piece of text.

#### WHAT IS SENTIMENT ANALYSIS?

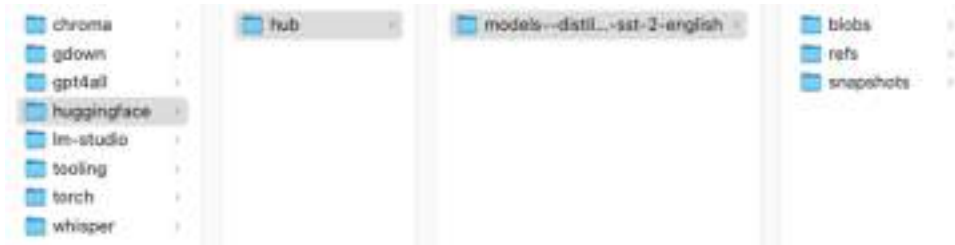
Sentiment analysis is a natural language processing (NLP) technique used to determine the sentiment expressed in a piece of text. It involves analyzing textual data to categorize the sentiment as positive, negative, or neutral, indicating the overall emotional tone or polarity of the text.

For the first step, you need to load the tokenizer from the model using the `AutoTokenizer.from_pretrained()` method:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "distilbert/distilbert-base-uncased-finetuned-sst-2-english")
```

Performing this step will download the pre-trained model onto the `~/.cache/huggingface/hub/` directory of your machine (for computers running the macOS). For Windows machines, the directory will be `C:\users\<user_name>\.cache\huggingface\hub\. The model will be saved in a directory named after the model's name. For the model we are using in this section, the directory is named models--distilbert--distilbert-base-uncased-finetuned-sst-2-english (see Figure 3.12).`



**Figure 3.12** The directory containing the downloaded model

Next, load the model using the `AutoModelForSequenceClassification.from_pretrained()` method:

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert/distilbert-base-uncased-finetuned-sst-2-english")
```

Before you can use the model to perform sentiment analysis on a paragraph of text, you need to tokenize it using the `tokenizer` object:



```
import torch

text = "I loved the movie, it was fantastic!"

inputs = tokenizer(text, return_tensors = "pt")           #A
print(inputs)
```

**#A Tokenize the text**

The `tokenizer` object returns a dictionary containing the tokenized representation of the input text, suitable for consumption by a PyTorch model (indicated by the `"pt"` value):

```
{'input_ids': tensor([[ 101, 1045, 3866, 1996, 3185, 1010,
2009, 2001, 10392, 999, 102]]), 'attention_mask': tensor([[1,
1, 1, 1, 1, 1, 1, 1, 1, 1]])}
```

The result of the `tokenizer` object is then passed into `model` for inferencing:

```
outputs = model(**inputs)                               #A
print(outputs)
```

**#A perform inference**

In the above statements, you are providing the model with the token IDs (stored in `input_ids` key) and optionally other tensors like the attention mask (stored in the `attention_mask` key). The model then performs inference on these inputs, generating predictions or other relevant outputs depending on the specific model and task. You will see the following output:

```
SequenceClassifierOutput(loss=None, logits=tensor([[ -4.3428,  4.6955]]),
grad_fn=<AddmmBackward0>), hidden_states=None, attentions=None)
```

In particular, take note of the values in the `logits` key: `[[ -4.3428, 4.6955]]`. The value is of shape (1,2), where the first dimension corresponds to the batch size (1 in this case) and the second dimension corresponds to the number of classes (2 in this case – 0 for “negative” and 1 for “positive”). Each element in the tensor represents the model's confidence score for a particular class. In this example, the model outputs `[-4.3428, 4.6955]`, which suggests that the model assigns a higher confidence score to the second class compared to the first class (see Figure 3.13).

| Class 0<br>(Negative) | Class 1<br>(Positive) |
|-----------------------|-----------------------|
| -4.3428               | 4.6955                |

**Figure 3.13 The confidence score for each class**

Based on the interpretation of the result, you can now extract the class with the highest confidence score and determine the class of the result:

```
predicted_label = torch.argmax(outputs.logits) #A
sentiment = "positive" if predicted_label == 1 else "negative"

print("Predicted sentiment:", sentiment)

#A get the predicted label (0 for negative, 1 for positive)
```

You will get the following output:

```
Predicted sentiment: positive
```

### 3.2.4 Using Transformers Pipelines

Now that you have seen how to use a transformers model directly, it is now time to learn how to use the transformers pipeline. The simplest way to use a pipeline is to specify the task you want to perform. But what are the tasks supported in the first place? An easy way is to specify a task that is *not* supported and view the error message printed out (which will then show a list of supported tasks):

```
from transformers import pipeline
try:
    dummy_pipeline = pipeline(task="dummy")
except Exception as e:
    print(e)
```

The above code snippet prints out the following error message:

```
"Unknown task dummy, available tasks are ['audio-classification', 'automatic-speech-recognition', 'conversational', 'depth-estimation', 'document-question-answering', 'feature-extraction', 'fill-mask', 'image-classification', 'image-feature-extraction', 'image-segmentation', 'image-to-image', 'image-to-text', 'mask-generation', 'ner', 'object-detection', 'question-answering', 'sentiment-analysis', 'summarization', 'table-question-answering', 'text-classification', 'text-generation', 'text-to-audio', 'text-to-speech', 'text2text-generation', 'token-classification', 'translation', 'video-classification', 'visual-question-answering', 'vqa', 'zero-shot-audio-classification', 'zero-shot-classification', 'zero-shot-image-classification', 'zero-shot-object-detection', 'translation_XX_to_YY']"
```

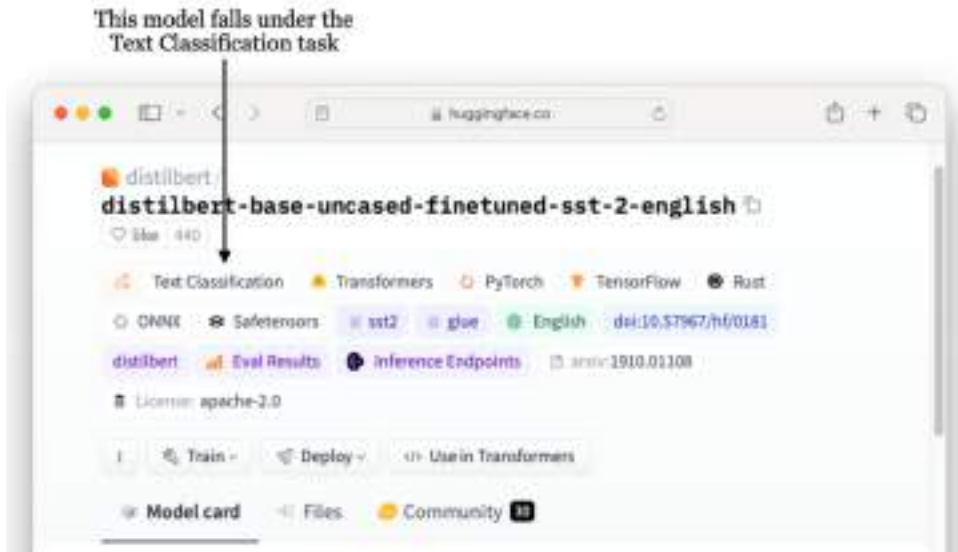
You can now see the list of tasks supported.

Let's now use the `distilbert/distilbert-base-uncased-finetuned-sst-2-english` model that we have used in the previous section. This model falls under the `text-classification` task. How would you know? Well, earlier in Figure 3.11, the code sample already specified the task:

```
from transformers import pipeline

pipe = pipeline("text-classification",
                model="distilbert/distilbert-base-uncased-finetuned-sst-2-english")
```

Alternatively, you can also view from the model's page on Hugging Face website and find the task that this model is trained for (see Figure 3.14).



**Figure 3.14 Finding the type of task a model falls under**

Note that you can simply specify the tasks in the `pipeline` method and leave out the model that you want to use. The `pipeline` method will then use the default model and revision (version) for that task:

```
pipe = pipeline("text-classification")
```

However, this is not recommended as the default model to use might change in the next release of the `pipeline` method. Hence, it is better to specify the `model` parameter explicitly:

```
from transformers import pipeline

classifier = pipeline(task = 'text-classification',
                      model = 'distilbert/distilbert-base-uncased-finetuned-sst-2-english')
```

If you have already identified the model you want to use, you do not have to specify the `task` parameter – just specify the model, like this:

```
classifier = pipeline(
    model = 'distilbert/distilbert-base-uncased-finetuned-sst-2-english')
```

In addition, if you have a CUDA-compliant GPU on your computer, you can specify that the pipeline be allocated to run on the GPU:

```
classifier = pipeline(  
    model = 'distilbert/distilbert-base-uncased-finetuned-sst-2-english',  
    device = "cuda")
```

By default, `device` is set to `"cpu"`, which allows you to run a model using just the CPU of your computer.

### WHAT IS CUDA (COMPUTE UNIFIED DEVICE ARCHITECTURE)?

CUDA is a parallel computing platform and application programming interface (API) model created by NVIDIA. CUDA allows developers to use NVIDIA GPUs (Graphics Processing Units) for general-purpose processing, in addition to their traditional graphics rendering tasks.

Let's now define two blocks of text containing reviews of two restaurants:

```
review1 = '''From the warm welcome to the exquisite dishes and impeccable  
service, dining at Gourmet Haven is an unforgettable experience that  
leaves you eager to return.'''  
  
review2 = '''Despite high expectations, our experience at Savor Bistro  
fell short; the food was bland, service was slow, and the overall  
atmosphere lacked charm, leaving us disappointed and unlikely to  
revisit.'''
```

We can now use the pipeline object (`classifier`) to help us perform a sentiment analysis on the first review:

```
print(classifier(review1))
```

You will see the following output, indicating that the review contains positive sentiment:

```
[{'label': 'POSITIVE', 'score': 0.9998437166213989}]
```

You can also pass in multiple blocks of text using a list, like this:

```
print(classifier([review1, review2]))
```

The above statement returns the following output:

```
[{'label': 'POSITIVE', 'score': 0.9998437166213989},  
 {'label': 'NEGATIVE', 'score': 0.9997773766517639}]
```

When contrasting using the model directly versus employing the pipeline, the simplicity and straightforwardness of pipelines become apparent. By simply initializing a pipeline object with the desired task and model, data can seamlessly flow through for inference, yielding immediate outputs. However, there will be instances where direct model usage is essential for finer control. In this book, I will prefer pipelines whenever feasible and turn to using the model directly when precise control is required.

### 3.3 Using Transformers for Natural Language Processing (NLP) Tasks

One of the primary tasks that the Transformers library has been created for was to perform NLP tasks, apart from other tasks such as computer vision, audio, and reinforcement learning. Now, we will discuss the various NLP tasks you can perform using the transformers library:

- Text Classification
- Text Generation
- Text Summarization
- Text Translation
- Zero-shot classification
- Question Answering

#### 3.3.1 Text Classification

Earlier on, you saw how you can perform sentiment analysis using a text classification model. Another form of text classification task is `question-detection`. Using this task, you can detect if a sentence contains a question or not. To do that, you can use the `huan/question_detection` model:

```
from transformers import pipeline

question_classifier = pipeline("text-classification",
                              model="huan/question_detection")
```

Using the pipeline created, you can now pass in a string to determine if it contains a question:

```
response = question_classifier(
    "'Have you ever pondered the mysteries that lie beneath  
the surface of everyday life?'"')
print(response)
```

The above statement prints out the following output, indicating that it is very likely (with 99.76% confidence) a question:

```
[{'label': 'question', 'score': 0.9975988268852234}]
```

Let's try another one:

```
response = question_classifier(
    '"Life is a journey that must be traveled, no matter  
how bad the roads and accommodations." - Oliver Goldsmith"')
print(response)
```

The response now is that this is likely not a question:

```
[{'label': 'non_question', 'score': 0.9996856451034546}]
```

Another text classification task you can perform is **language detection**. Using a model such as `papluca/xlm-roberta-base-language-detection`, you can pass it a string so that it can try to detect the language of a given sentence:

```
language_classifier = pipeline("text-classification",
                               model="papluca/xlm-roberta-base-language-detection")

response = language_classifier("日本の桜は美しいです。")
print(response)
```

The above statements printed out the following result, indicating that it is most likely Japanese:

```
[{'label': 'ja', 'score': 0.9913387298583984}]
```

One more example of text classification is a **spam classifier**, where you can identify incoming messages (such as emails, text messages, comments, etc.) as either "spam" (unwanted or unsolicited messages) or "ham" (legitimate messages). For this, you can use the `Delphia/twitter-spam-classifier` model:

```
spam_classifier = pipeline("text-classification",
                           model="Delphia/twitter-spam-classifier")

response = spam_classifier(
    '''Congratulations! You've been selected as the winner of our
    exclusive prize draw. Claim your reward now by clicking on
    the link below!''')

print(response)
```

The above statements printed the following result, indicating that the string provided is likely to be spam:

```
[{'label': 1, 'score': 0.7446919679641724}]
```

On the other hand, the following string is not likely to be spam:



```
response = spam_classifier(
    '''Hi Jimmy, I hope you're doing well. I just wanted to remind
    you about our meeting tomorrow at 10 AM in conference room A.
    Please let me know if you have any questions or need any
    further information. Looking forward to seeing you there!''')

print(response)
```

The following output confirms that:

```
[{'label': 0, 'score': 0.7776529788970947}]
```

### 3.3.2 Text Generation

Another common NLP task is next generation. The text generation task involves creating new, coherent, and contextually relevant text based on a given prompt or input. This task leverages on machine learning models, particularly those based on deep learning and neural networks, to produce human-like text. The following code snippet shows how to use the `openai-community/gpt2` model to generate a paragraph of text based on an initial start sentence:

```
from transformers import pipeline

generator = pipeline("text-generation",
                    model="openai-community/gpt2")
generator("In this course, we will teach you how to")
```

It generates the following output (note that the output will be different each time the code snippet is run):

```
[{'generated_text': 'In this course, we will teach you how to build the best
online games or use it to build your own. After this, this course covers: 1) how
to make awesome games in Google Play and 2) how to develop a game based on'}]
```

You can control the output using the `max_length` (the maximum number of tokens in the generated text) and `num_return_sequences` (number of paragraphs generated) parameters:

```
generator("In this course, we will teach you how to",
        max_length = 50,
        num_return_sequences = 3)
```

Here is the output generated:

```
[{'generated_text': 'In this course, we will teach you how to build and
customize a modern React based project. We will show you ways to simplify
your development as well as make your app run with no effort. We will give
you many practical tips and tricks. After'},
 {'generated_text': 'In this course, we will teach you how to use the
Raspberry Pi in a production computer to help you connect the Raspberry Pi
to the internet. We will learn about connecting other IoT networks and how
to connect to them within the Raspberry Pi.\n\n'},
 {'generated_text': "In this course, we will teach you how to use PHP's
built-in filters, and how to use PHP's PHP-based classes. We will also
explain how to perform PHP operations such as create an array of
attributes, create an array of"}]
```

### 3.3.3 Text Summarization

Summarizing text is another widely recognized task in NLP. The key goal of text summarization is to condense large amounts of text into shorter, coherent summaries while preserving key information and main ideas. It is very useful in various applications such as news aggregation, document summarization, and content generation.

There are two main approaches to text summarization:

- **Extractive Summarization** – involves selecting and extracting important sentences or phrases directly from the original text.
- **Abstractive Summarization** – generates summaries by paraphrasing and rephrasing the original text in a more concise form.

To try out the text summarization task, you can use the `facebook/bart-large-cnn` model:

```
from transformers import pipeline

summarizer = pipeline("summarization",
                      model="facebook/bart-large-cnn")
```

Let's try to summarize the following block of text on quantum computers (see Listing 3.6).

**Listing 3.6 Summarizing a long paragraph of text**

```
article = ""
```

```
A quantum computer is a computer that exploits quantum mechanical
phenomena. At small scales, physical matter exhibits properties of
both particles and waves, and quantum computing leverages this
behavior using specialized hardware. Classical physics cannot
explain the operation of these quantum devices, and a scalable
quantum computer could perform some calculations exponentially
faster than any modern "classical" computer. In particular, a
large-scale quantum computer could break widely used encryption
schemes and aid physicists in performing physical simulations;
however, the current state of the art is still largely
experimental and impractical.
```

```
The basic unit of information in quantum computing is the qubit,
similar to the bit in traditional digital electronics. Unlike a
classical bit, a qubit can exist in a superposition of its two
"basis" states, which loosely means that it is in both states
simultaneously. When measuring a qubit, the result is a
probabilistic output of a classical bit. If a quantum computer
manipulates the qubit in a particular way, wave interference
effects can amplify the desired measurement results. The design
of quantum algorithms involves creating procedures that allow a
quantum computer to perform calculations efficiently.
```

```
Physically engineering high-quality qubits has proven challenging.
If a physical qubit is not sufficiently isolated from its
environment, it suffers from quantum decoherence, introducing noise
into calculations. National governments have invested heavily in
experimental research that aims to develop scalable qubits with
longer coherence times and lower error rates. Two of the most
promising technologies are superconductors (which isolate an
electrical current by eliminating electrical resistance) and ion
traps (which confine a single atomic particle using electromagnetic
fields).
```

```
Any computational problem that can be solved by a classical computer
can also be solved by a quantum computer.[2] Conversely, any problem
that can be solved by a quantum computer can also be solved by a
classical computer, at least in principle given enough time. In other
words, quantum computers obey the Church-Turing thesis. This means
that while quantum computers provide no additional advantages over
classical computers in terms of computability, quantum algorithms
```

for certain problems have significantly lower time complexities than corresponding known classical algorithms. Notably, quantum computers are believed to be able to solve certain problems quickly that no classical computer could solve in any feasible amount of time—a feat known as "quantum supremacy." The study of the computational complexity of problems with respect to quantum computers is known as quantum complexity theory.

```
print(summarizer(article,
                 min_length = 100,
                 max_length = 250,
                 do_sample = False))
```

The `summarizer` object by default returns a `max_length` of 142 tokens. In the above statement, you indicate that you want the summary to have at least 100 tokens and that it should also not exceed 250 tokens. The `do_sample=False` indicates that sampling should not be used during generation. Instead, the model will deterministically select the most likely tokens at each step of the generation process. This is essentially performing extractive summarization they we have discussed earlier. The above statement generated the following summary:

```
[{'summary_text': 'A quantum computer is a computer that exploits quantum mechanical phenomena. Classical physics cannot explain the operation of these quantum devices. A scalable quantum computer could perform some calculations exponentially faster than any modern "classical" computer. The basic unit of information in quantum computing is the qubit, similar to the bit in traditional digital electronics. The design of quantum algorithms involves creating procedures that allow a quantum computer to perform calculations efficiently. The study of the computational complexity of problems with respect to quantum computers is known as quantum complexity theory.'}]
```

Let's try again, but this time with `do_sample = True`:

```
print(summarizer(article,
                 min_length = 100,
                 max_length = 250,
                 do_sample = True))
```

This means that during the generation of the summary, the model will use sampling to select the next token probabilistically based on the predicted distribution of tokens. This allows for more diverse and creative generation, as the model can explore different possibilities at each step rather than selecting the most likely token deterministically. The above statement generated the following output:

```
[{'summary_text': 'Quantum computers exploit quantum mechanical phenomena. The basic unit of information in quantum computing is the qubit, similar to the bit in traditional digital electronics. A large-scale quantum computer could break widely used encryption and aid physicists in performing physical simulations. National governments have invested heavily in research that aims to develop scalable qubits with longer coherence times and lower error rates.quantum computers obey the Church-Turing thesis, which means that any computational problem that can be solved by a classical computer can also be solve by a quantum computer.'}]
```

### 3.3.4 Text Translation

Text translation stands as one of the earliest foundational tasks in NLP. Its evolution spans from the initial days of rule-based methods and bilingual dictionaries to the groundbreaking Transformer architecture, marking a journey of substantial advancements in quality enhancement and fluency.

On Hugging Face, there are several text translation models you can use to translate text from one language to another. Let's start with the `google-t5/t5-base` model, a variant of the T5 (Text-To-Text Transfer Transformer) model developed by Google AI:

```
from transformers import pipeline

translator = pipeline("translation",
                      model = "google-t5/t5-base")
```

When you run the above code, you will see a warning like this:

```
UserWarning: "translation" task was used, instead of "translation_XX_to_YY",
defaulting to "translation_en_to_de"
```

This means that the `translator` object will default to translating your text from English to German. Let's give it a try:

```
translator("How are you?")
```

You will see the following result containing the translated text:

```
[{'translation_text': 'Wie sind Sie?'}]
```

The recommended way to correctly perform translation is to specify the translation task using the format - `translation_XX_to_YY`, where `XX` is the language to translate from and `YY` is the language to translate into. Here's an example of a request to translate from English to French:

```
translator = pipeline(task = 'translation_en_to_fr',
                      model = "google-t5/t5-base")
translator('Wikipedia is hosted by the Wikimedia Foundation, a non-profit
organization that also hosts a range of other projects.')
```

Here is the result containing the translated text in French:

```
[{'translation_text': "Wikipedia est hébergée par la Wikimedia Foundation,
un organisme sans but lucratif qui héberge également une série d'autres
projets."}]
```

You can also translate from English to German:

```
translator = pipeline(task = 'translation_en_to_de',
                      model = "google-t5/t5-base")
translator('Wikipedia is hosted by the Wikimedia Foundation, a non-profit
organization that also hosts a range of other projects.')
```

And the text is now translated to German:

```
[{'translation_text': 'Wikipedia wird von der Wikimedia Foundation
gehostet, einer gemeinnützigen Organisation, die auch eine Reihe anderer
Projekte beherbergt.'}]
```

What happens if you want to translate from English to Chinese? Unfortunately, the `google-t5/t5-base` model does not support translation to Chinese. In general, to know the languages supported by a model, you can check out the model's page on Hugging Face Hub, or refer to its source code in Github. A more pragmatic way is to try out different tasks and see if it works. For example, you can try `translation_en_to_zh` to see if it can translate from English to Chinese (`zh` is the ISO 639-1 language code for Chinese).

To translate from English to Chinese, you can use the `facebook/m2m100_418M` model. But before you can use this model, you need to install the `sentencepiece` package, a widely-used library for tokenization:

```
!pip install sentencepiece
```

You can now try to translate the following text in English to Chinese:

```
translator = pipeline('translation_en_to_zh',
                      model = 'facebook/m2m100_418M')

translator('Wikipedia is hosted by the Wikimedia Foundation, a non-profit
organization that also hosts a range of other projects.')
```

The translated output looks like this:

```
[{'translation_text':
  '维基百科是维基百科基金会主办的,是一家非营利组织,还主办了许多其他项目。'}]
```

You can also use the model to translate Chinese to English. The following code snippet translates the above output in Chinese back to English:

```
translator = pipeline(task = 'translation_zh_to_en',
                      model = "facebook/m2m100_418M",
                      max_length = 400)

translator("维基百科是维基百科基金会主办的,是一家非营利组织,还主办了许多其他项目。")
```

Here is the output in English:

```
[{'translation_text': 'Wikipedia is hosted by the Wikipedia Foundation, a non-
profit organization and hosts many other projects.'}]
```

### 3.3.5 Zero-Shot Classification

Earlier on, you saw an example on sentiment analysis, where the model is trained on a set of labelled examples (classifying them as either positive or negative). While this is useful in helping you analyze the sentiment in a new paragraph of text, it has its limitations. For example, suppose you have a description of a new gadget and want to automatically classify it into categories such as Home Appliances, Electronics, etc. Using the model that has been pre-trained on a fixed set of labels is not going to be useful in this case. This is where *Zero-shot classification* comes in.

There are various types of zero-shot classification, such as:

- Zero-shot text classification
- Zero-shot image classification
- Zero-shot audio classification
- Zero-shot video classification
- Zero-shot graph classification

#### ZERO-SHOT AND ONE-SHOT CLASSIFICATION

When discussing zero-shot classification, another term often comes up – *one-shot classification*. While zero-shot classification is the task of classifying previously unseen classes during training of a model, one-shot classification refers to the technique where a model is trained to recognize classes with only one example (or just a few examples) per class during training. For example, for a one-shot classification model trained to recognize cats and dogs, only one image of a cat and one image for a dog is provided during the training. Its aim is to generalize from a small number of examples. This technique is useful when collecting large number of samples is difficult, or prohibitively expensive.

For our next task, we will try out zero-shot text and image classification. *Zero-shot text classification* refers to the task of classifying text into predefined categories or labels without having access to any labeled examples for training. Zero-shot text classification is trained on NLI (Natural Language Inference) tasks.

To try out how zero-shot text classification works, let's use the `joeddav/xlm-roberta-large-xnli` model located at <https://huggingface.co/joeddav/xlm-roberta-large-xnli>. As this is a private repository, you need to apply for a free Hugging Face token (of type READ) from <https://huggingface.co/settings/tokens>. Once you have obtained the token, you can login to Hugging Face using the **huggingface-cli** tool in Terminal (or Command Prompt):







Copy a token from your Hugging Face tokens page and paste it below.  
Immediately click login after copying your token or it might be stored in plain text in this notebook file.

Token:

☒ Add token as git credential?

Login

**Pro Tip:** If you don't already have one, you can create a dedicated 'notebooks' token with 'write' access, that you can then easily reuse for all notebooks.

**Figure 3.15 Finding the type of task a model falls under**

In addition, you need to install two packages – `sentencepiece` and `protobuf`. You can do so in Jupyter notebook:

```
!pip install sentencepiece
!pip install protobuf
```

Let's now create a pipeline object that uses the `joeddav/xlm-roberta-large-xnli` model:

```
from transformers import pipeline

zero_shot_classifier = pipeline("zero-shot-classification",
                                model='joeddav/xlm-roberta-large-xnli')
```

This model is fine-tuned on the **XLM-RoBERTa** model (which was pre-trained on 2.5TB of filtered CommonCrawl data containing 100 languages) on a combination of NLI data in 15 languages. You can use the model with any of the following 15 languages – English, French, Spanish, German, Greek, Bulgarian, Russian, Turkish, Arabic, Vietnamese, Thai, Chinese, Hindi, Swahili, and Urdu.

Let's define two paragraphs of text and then use them to let the model predict the possible labels for the text:

```
text1 = '''
In the intricate realm of global affairs, the interplay of power,
diplomacy, and governance stands as a defining force in the
trajectory of nations. Amidst fervent debates in legislative
chambers and pivotal dialogues among world leaders, ideologies
clash and policies take shape, shaping the course of societies.
Issues such as economic disparity, environmental stewardship, and
human rights take precedence, driving conversations and shaping
public sentiment. In an age of digital interconnectedness, social
media platforms have emerged as influential channels for discourse
and activism, amplifying voices and reshaping narratives with
remarkable speed and breadth. As citizens grapple with the
complexities of contemporary governance, the pursuit of accountable
and transparent leadership remains paramount, reflecting an
enduring quest for fairness and inclusivity in societal governance."
'''

text2 = '''
In the tender tapestry of human connection, romance weaves its
delicate threads, binding hearts in a dance of passion and longing.
From the flutter of a first glance to the warmth of an intimate
embrace, love blooms in the most unexpected places, transcending
barriers of time and circumstance. In the gentle caress of a hand
and the whispered promises of affection, two souls find solace in
each other's embrace, navigating the complexities of intimacy with
tender care. As the sun sets and stars illuminate the night sky,
lovers share stolen moments of intimacy, lost in the intoxicating
rhythm of each other's presence. In the symphony of love, every
glance, every touch, speaks volumes of a shared bond that defies
explanation, leaving hearts entwined in an eternal embrace.
'''
```

We can now make use of the `zero_shot_classifier` object to help us determine if `text1` contains text related to technology, politics, business, or romance:

```
candidate_labels = ["technology", "politics", "business", "romance"]
prediction = zero_shot_classifier(text1,
                                candidate_labels,
                                multi_label = True)
```

The result is then converted into a Pandas DataFrame for easy viewing:

```
import pandas as pd
display(pd.DataFrame(prediction).drop(["sequence"], axis=1))
```

Figure 3.16 shows that the result indicated that the text is most probably related to politics.

|   | labels     | scores   |
|---|------------|----------|
| 0 | politics   | 0.990988 |
| 1 | technology | 0.828358 |
| 2 | romance    | 0.465753 |
| 3 | business   | 0.278939 |

**Figure 3.16 The result of the zero-shot classification**

You can also pass in multiple paragraphs (enclosed within a list) to the `zero_shot_classifier` object:

```
prediction = zero_shot_classifier([text1, text2],
                                candidate_labels,
                                multi_label = True)
display(pd.DataFrame(prediction).drop(["sequence"], axis=1))
```

Figure 3.17 shows the results for the two paragraphs (with the most probable label at the front of the list of labels):

|   | labels                                    | scores                                             |
|---|-------------------------------------------|----------------------------------------------------|
| 0 | [politics, technology, romance, business] | [0.99098884929656982, 0.8283584713935852, 0.465... |
| 1 | [romance, business, politics, technology] | [0.9982308149337769, 0.11879944801330566, 0.01...  |

**Figure 3.17 Results of the one-shot classification for two paragraphs**

How about zero-shot image classification? For this, you can use the `openai/clip-vit-large-patch14-336` model:

```
from transformers import pipeline

classifier = pipeline("zero-shot-image-classification",
                     model = "openai/clip-vit-large-patch14-336")
```



**Figure 3.18 An image of an airplane (source: [https://en.wikipedia.org/wiki/Aircraft#/media/File:Emirates\\_Airbus\\_A380-861\\_A6-EER\\_MUC\\_2015\\_04.jpg](https://en.wikipedia.org/wiki/Aircraft#/media/File:Emirates_Airbus_A380-861_A6-EER_MUC_2015_04.jpg))**

Let's use the model to detect if the image shown in Figure 3.18 is an airplane, car, or train:

```
labels_for_classification = ["airplane", "car", "train"]
scores = classifier("Emirates_Airbus_A380-861_A6-EER_MUC_2015_04.jpg",
                   candidate_labels = labels_for_classification)
pd.DataFrame(scores)
```

Figure 3.19 shows that the image most probably is an airplane.

|   | score    | label    |
|---|----------|----------|
| 0 | 0.997296 | airplane |
| 1 | 0.002082 | car      |
| 2 | 0.000623 | train    |

**Figure 3.19** Result of the zero-shot image classification

### 3.3.1 Question Answering (QA) Tasks

Another task that the pipeline object can perform is *question answering (QA)*. I don't think you need an example of that – if you ever used Google to search for answers to your questions, you already know what a QA task is. Hugging Face hosts many QA models ([https://huggingface.co/models?pipeline\\_tag=question-answering](https://huggingface.co/models?pipeline_tag=question-answering)) that you can experiment with.

Question answering (QA) models are valuable for several reasons:

- **Efficient information retrieval** – QA models are able to quickly retrieve information of interest from a large collection of text.

- **Natural language understanding** – QA models are able to understand natural language inputs and generate context relevant answers.

Let's use the `deepset/roberta-base-squad2` model to understand how QA model works. This is a **roberta-base** model (pretrained model on English language using a masked language modelling (MLM) objective), fine-tuned using the SQuAD 2.0 dataset.

### SQUAD 2.0 DATASET

The Stanford Question Answering Dataset (SQuAD) is a reading comprehension dataset, consisting of questions posed by crowd workers on a set of Wikipedia articles, where the answer to every question is a segment of text, or span, from the corresponding reading passage, or the question might be unanswerable.

First, create an instance of the model using the `pipeline()` function:

```
from transformers import pipeline

QA_model = pipeline(task = "question-answering",
                    model = "deepset/roberta-base-squad2")
```

We will have a paragraph of text discussing the origin of the Singapore name:

```

text = '''
The English name of "Singapore" is an anglicisation of the native
Malay name for the country, Singapura (pronounced [sinapura]),
which was in turn derived from the Sanskrit word for 'lion city'
(Sanskrit: सिंहपुर; romanised: Siṃhapura; Brahmi: 𑀲𑀸𑀭𑀸𑀓𑀾𑀢𑀺; literally
"lion city"; siṃha means 'lion', pura means 'city' or 'fortress' ).
Pulau Ujong was one of the earliest references to Singapore Island,
which corresponds to a Chinese account from the third century
referred to a place as Pú Luó Zhōng (Chinese: 蒲羅中), a
transcription of the Malay name for 'island at the end of a
peninsula'. Early references to the name Temasek (or Tumasik) are
found in the Nagarakretagama, a Javanese eulogy written in 1365,
and a Vietnamese source from the same time period. The name possibly
means Sea Town, being derived from the Malay tasek, meaning 'sea' or
'lake'. The Chinese traveller Wang Dayuan visited a place around 1330
named Danmaxi (Chinese: 淡馬錫; pinyin: Dànmǎxí; Wade-Giles: Tan Ma Hsi)
or Tam ma siak, depending on pronunciation; this may be a transcription
of Temasek, alternatively, it may be a combination of the Malay Tanah
meaning 'land' and Chinese xi meaning 'tin', which was traded on the
island
'''

```

You can use the model to ask a question based on the text. For example, you would like to know the meaning of the name "Singapura":

```

question = {
    'question': 'What is the meaning of Singapura?',
    'context': text
}

model_response = QA_model(question)
pd.DataFrame([model_response])

```

Figure 3.20 shows the answer provided by the model.



|   | score    | start | end | answer    |
|---|----------|-------|-----|-----------|
| 0 | 0.096135 | 186   | 195 | lion city |

**Figure 3.20** The result showing that the meaning of Singapura is lion city

### 3.4 Summary

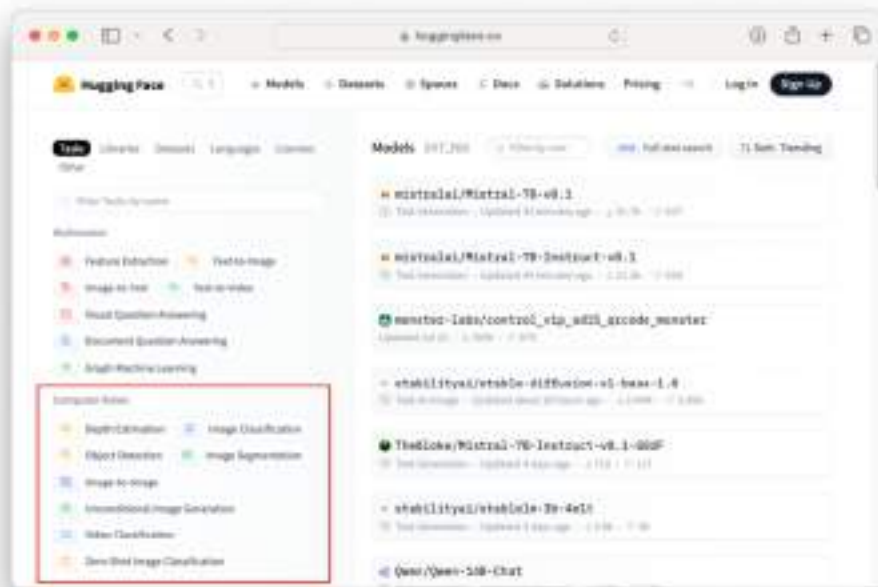
- At a very high level, the transformer architecture consists of two main blocks – Encoder and Decoder.
- The self-attention mechanism is a crucial component of the transformer architecture. It enables the model to weigh the importance of different words in a sequence when encoding a particular word, allowing the model to capture dependencies and relationships between words irrespective of their distance in the sequence.
- A "token" is a chunk of text that a model processes as a single unit.
- Token embeddings convert tokens into numerical vectors. These embeddings capture semantic and syntactic information about the tokens, enabling machine learning models to understand the underlying meaning and relationships between words in natural language text.
- Positional encodings provide information about the positions of words within a sequence. It allows models to understand the order of words in a sentence.
- Softmax is a mathematical function that converts a vector of numbers into a probability distribution, where the probability of each element is proportional to the exponentiation of that element's value relative to the sum of all the exponentiated values in the vector.
- The Hugging Face Transformers is an open-source library and platform developed by Hugging Face. The aim of this library is to provide an easy-to-access interface for working with state-of-the-art transformer-based models.
- The pipeline function provides a wrapper on top of the transformers library, providing convenient access to the various pre-trained models.
- Zero-shot classification is the task of classifying previously unseen classes during training of a model, while one-shot classification refers to the technique where a model is trained to recognize classes with only one example (or just a few examples) per class during training.

## ***4 Using Hugging Face for Computer Vision Tasks***

### **This chapter covers**

- The different types of computer vision models on Hugging Face
- Various ways to use the models for object detection
- Performing video content and image classification
- Performing image segmentation

Previously, you learned about Hugging Face transformers and pipelines. You also learned how to make use some of the pre-trained models for NLP tasks, such as sentiment analysis and text translation, to name a few. Besides NLP tasks, Hugging Face also provides a vast collection of pre-trained models for computer vision tasks (see Figure 4.1; <https://huggingface.co/models>).



**Figure 4.1 Computer Vision-related models on Hugging Face**

Using all these hosted pre-trained models, you can create interesting applications such as detecting objects in images, detecting the age of a person, and more. Now, you will make use of a number of all these models for computer vision tasks.

## 4.1 Types of Computer Vision Models on Hugging Face

The computer vision models hosted on Hugging Face are grouped into the following tasks:

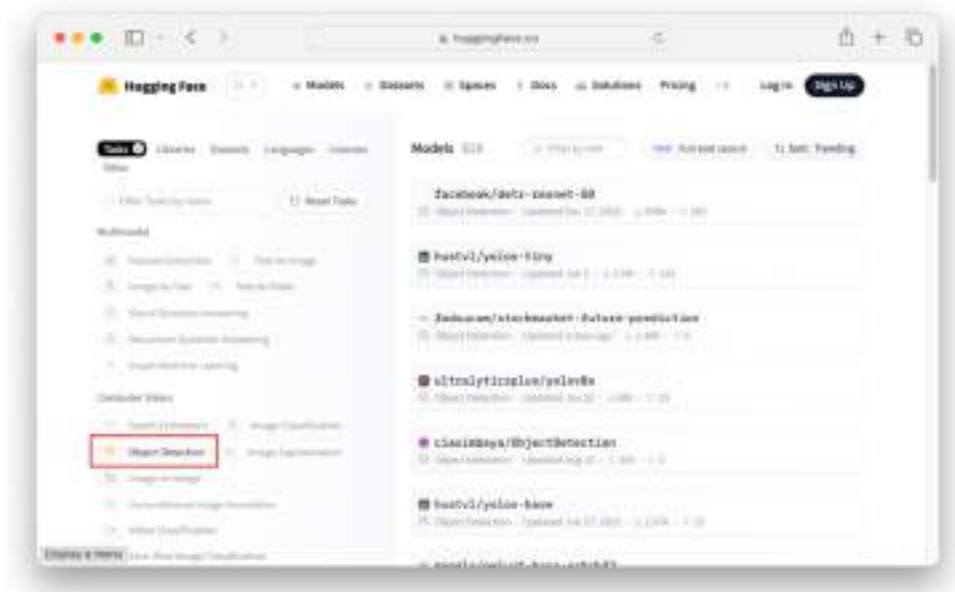
- Object Detection
- Image Classification
- Image Segmentation
- Video Classification
- Depth Estimation
- Image-to-Image
- Unconditional Image Generation
- Zero-Shot Image Classification

Now, you will learn how to perform some of these tasks using the models hosted by Hugging Face. Specifically, you will learn the first four tasks listed above.

## 4.2 Object Detection

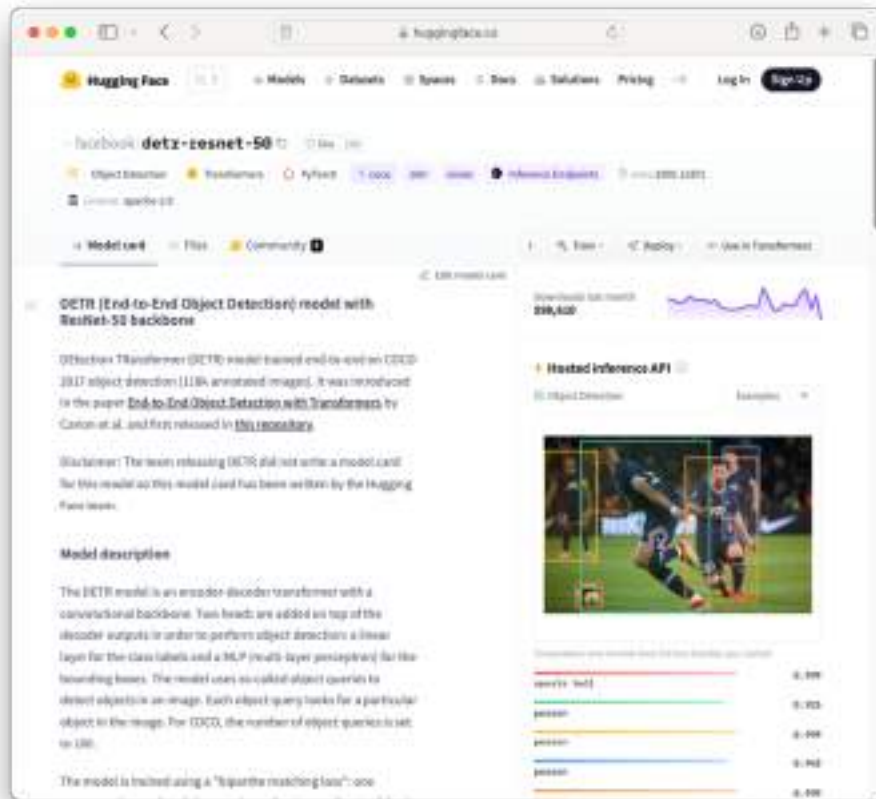
*Object detection* is a computer vision technique that involves identifying and locating objects of interest within an image or video. The primary goal of object detection is to not only classify the objects in the image or video but also to determine their precise positions by drawing bounding boxes around them.

Hugging Face hosts several models that have been pre-trained to detect objects in images. You can see a list of models at [https://huggingface.co/models?pipeline\\_tag=object-detection&sort=trending](https://huggingface.co/models?pipeline_tag=object-detection&sort=trending) (see Figure 4.2).



**Figure 4.2** List of pre-trained object detection models at Hugging Face

We'll be taking a look at one specific model – facebook/detr-resnet-50 (<https://huggingface.co/facebook/detr-resnet-50>; see Figure 4.3).



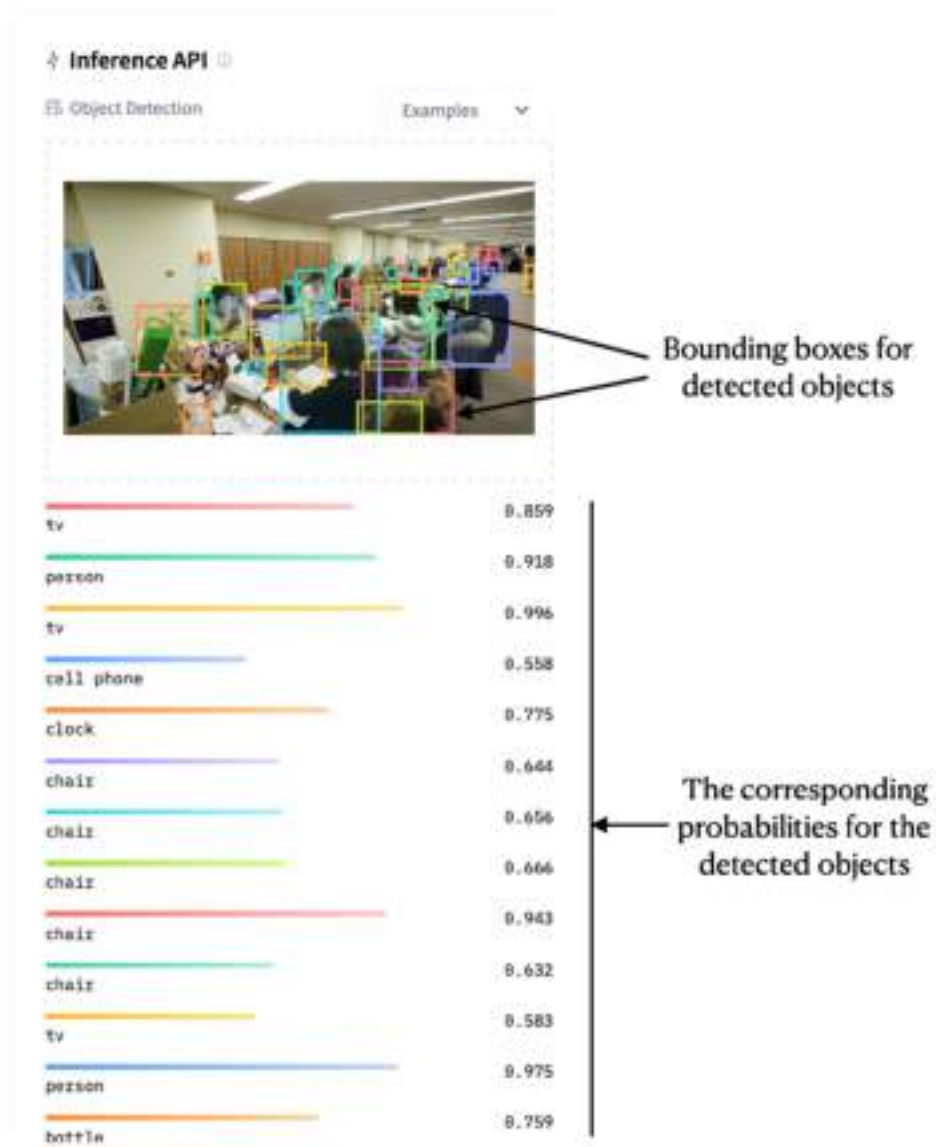
**Figure 4.3** The facebook/detr-resnet-50 model for object detection

You can test the model directly on Hugging Face using the Hosted inference API feature (you need to login to Hugging Face first by creating a free account). For this, let's try to use an image of an office with a few ladies ([https://en.wikipedia.org/wiki/Office#/media/File:Good\\_Smile\\_Company\\_offices\\_ladies.jpg](https://en.wikipedia.org/wiki/Office#/media/File:Good_Smile_Company_offices_ladies.jpg); see Figure 4.4).



**Figure 4.4** Image by Danny Choo - Flickr: Good Smile Company Offices, CC BY-SA 2.0, <https://commons.wikimedia.org/w/index.php?curid=14609862>

When you drag and drop the image onto the Hosted inference API section on the model's page on Hugging Face, you will see the list of objects detected as well as their corresponding probabilities (see Figure 4.5).

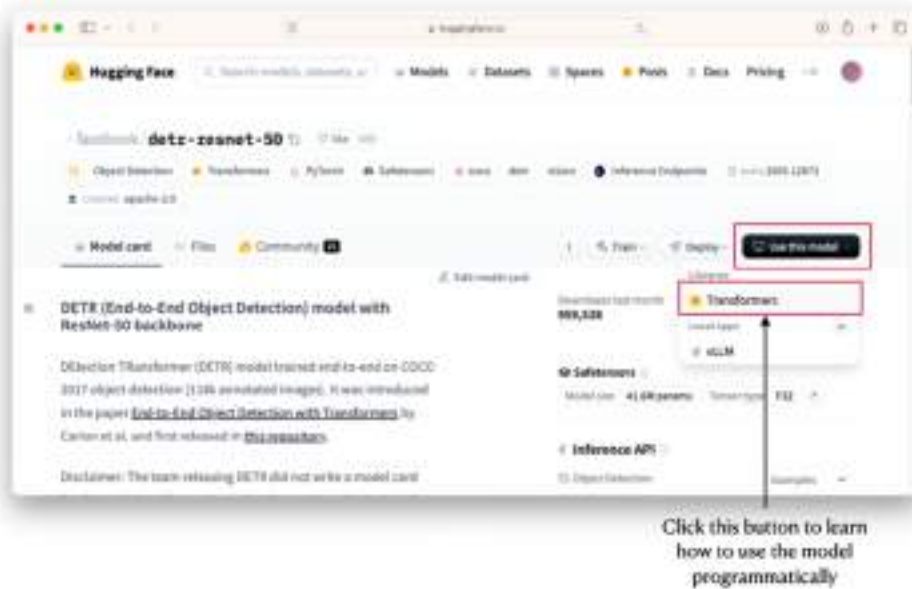


**Figure 4.5 The detected objects in the image and their corresponding probabilities**

When you use your mouse to hover the name of a detected object, the image will highlight the bounding box for the selected object.

### 4.2.1 Using the Model Directly

Obviously, the model would be more useful if you are able to make use of it programmatically. Hugging Face provides some useful tips for making use of their hosted models. On the page of the model, click the Use this model button and then Transformers (see Figure 4.6).

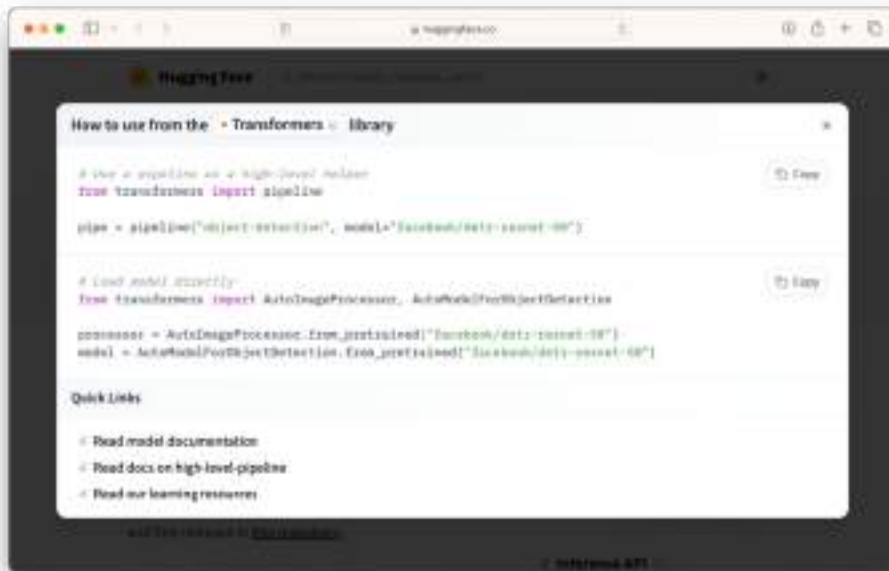


**Figure 4.6** Hugging Face provides tips on how to use the models using transformers

You will see that there are two ways to use the model programmatically (see Figure 4.7):

- Use a transformer pipeline, or
- Load the model directly





**Figure 4.7 Two ways to use the models on Hugging Face**

Before you use the model, you need to install two packages:

- transformers
- timm

### TIMM PACKAGE

`timm` is a deep-learning library created by Ross Wightman and is a collection of SOTA computer vision models, layers, utilities, optimizers, schedulers, data-loaders, augmentations and also training/validating scripts with ability to reproduce ImageNet training results. See <https://timm.fast.ai> for more details.

Type the following commands in Terminal to install the two packages:

```
$ pip install transformers
$ pip install timm
```

Let's now load the model directly:

```

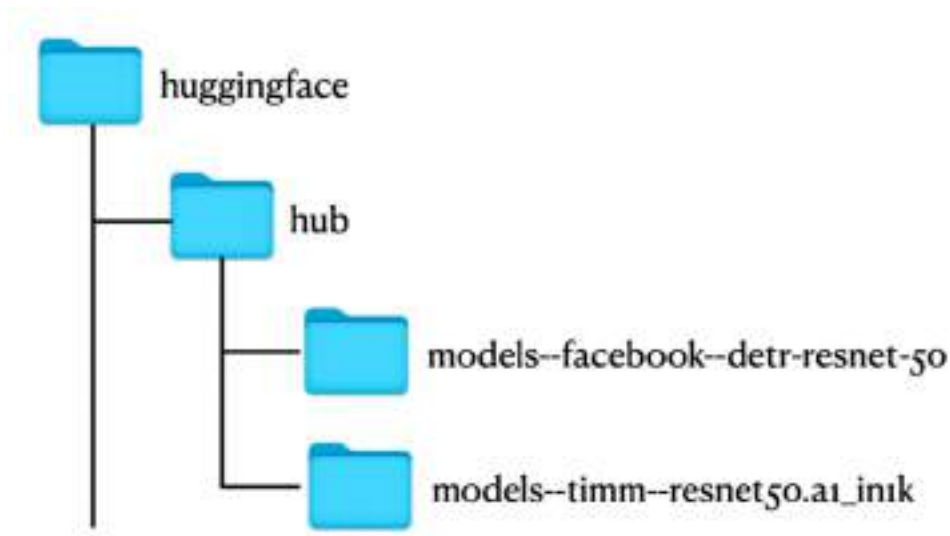
from transformers import DetrImageProcessor, DetrForObjectDetection

image_processor = DetrImageProcessor.from_pretrained(
    "facebook/detr-resnet-50")
model = DetrForObjectDetection.from_pretrained(
    "facebook/detr-resnet-50")

```

`DetrImageProcessor` is a class (from the `transformers` package) that is used for processing images to be used as input to the DETR (DEtection TRansformer) algorithm. The `DetrForObjectDetection` module provides access to pre-trained DETR models. The pre-trained model we are using in this case is `facebook/detr-resnet-50`.

The above code snippet downloads the weights of the model onto the `.cache/huggingface/hub` folder of your home directory. There will be two folders created in the `~/.cache/huggingface/hub` folder as shown in Figure 4.8:



**Figure 4.8** The weights of the models need to be downloaded onto your computer before you can use them

It would be very useful to know what objects the models were trained to detect. To do so, use the `config.id2label` attribute of the model:

```
model.config.id2label
```

The model can detect a total of 90 objects. You will see a printout like the following (for brevity only the first and last five object names are shown here):

```
{0: 'N/A',
 1: 'person',
10: 'traffic light',
11: 'fire hydrant',
12: 'street sign',
...
87: 'scissors',
88: 'teddy bear',
89: 'hair drier',
9: 'boat',
90: 'toothbrush'}
```

Before you load the image to be used for detection, it would be useful to be able to display it out for inspection. So here's a helper function called `loadImage()` (see Listing 4.1):

#### Listing 4.1 Displaying the image used for object detection

```
from PIL import Image, ImageDraw
import requests
import torch

def loadImage(url):
    if url.startswith('http'):
        image = Image.open(requests.get(url, stream=True).raw)
    else:
        image = Image.open(url)
    return image

image = loadImage('http://bit.ly/46xv3sL')
display(image)
```

**#A** if the image is from the Web

**#B** the image is local

## PIL (PYTHON IMAGING LIBRARY)

You may need to install the PIL package if you do not already have it on your system. PIL is the Python Imaging Library by Fredrik Lundh and Contributors (<https://pypi.org/project/Pillow/>). You can install the PIL package using:

```
pip install Pillow
```

To perform object detection, first you need to prepare the input image:

```
inputs = image_processor(images = image,
                           return_tensors = "pt")
```

You use the `image_processor` object to preprocess the input image before feeding it into the neural network. This function then returns you a PyTorch tensor that looks like this:

```
{'pixel_values':
  tensor(
    [
      [
        [-1.1075, -1.0904, -1.0733, ..., -0.4397, -0.4226, -0.4226],
        [-1.1418, -1.1247, -1.1075, ..., -0.4397, -0.4226, -0.4226],
        [-1.1932, -1.1932, -1.1760, ..., -0.4397, -0.4226, -0.4054],
        ...,
        [-0.3712, -0.4054, -0.4739, ..., -0.5253, -0.4911, -0.4739],
        [-0.2856, -0.3198, -0.3883, ..., -0.5082, -0.5082, -0.5082],
        [-0.2342, -0.2684, -0.3369, ..., -0.5082, -0.5253, -0.5253]],
      [
        [-1.0903, -1.0728, -1.0553, ..., -0.4076, -0.3901, -0.3901],
        [-1.1253, -1.1078, -1.0903, ..., -0.4076, -0.3901, -0.3901],
        [-1.1779, -1.1779, -1.1604, ..., -0.4076, -0.3901, -0.3725],
        ...,
        [-0.5476, -0.5651, -0.6001, ..., -0.5476, -0.5126, -0.4951],
        [-0.4776, -0.4951, -0.5476, ..., -0.5301, -0.5301, -0.5301],
        [-0.4251, -0.4601, -0.5126, ..., -0.5301, -0.5476, -0.5476]],
      [
        [-1.2467, -1.2293, -1.2119, ..., -0.5670, -0.5495, -0.5495],
        [-1.2816, -1.2641, -1.2467, ..., -0.5495, -0.5321, -0.5321],
        [-1.3339, -1.3339, -1.3164, ..., -0.5321, -0.5147, -0.4973],
        ...,
        [-1.0898, -1.1247, -1.1596, ..., -0.5147, -0.4798, -0.4624],
        [-1.0376, -1.0724, -1.1247, ..., -0.4973, -0.4973, -0.4973],
```

```

        [-1.0027, -1.0376, -1.0898, ..., -0.4973, -0.5147, -0.5147]]]]),
'pixel_mask': tensor([[[1, 1, 1, ..., 1, 1, 1],
                        [1, 1, 1, ..., 1, 1, 1],
                        [1, 1, 1, ..., 1, 1, 1],
                        ...,
                        [1, 1, 1, ..., 1, 1, 1],
                        [1, 1, 1, ..., 1, 1, 1],
                        [1, 1, 1, ..., 1, 1, 1]]]])}

```

The `PyTorch` tensor is then unpacked as a keyworded argument to be used as the input for the `model` object:

```
outputs = model(**inputs)
```

The `model` object represents the pre-trained network. When you pass in the preprocessed `PyTorch` tensor to this object, it performs a forward pass through the network and returns the output of the model. The output of the model is then used for object detection:

```

target_sizes = torch.tensor([image.size[:-1]])

results = image_processor.post_process_object_detection(
    outputs,
    target_sizes = target_sizes,
    threshold = 0.9)[0]
results

```

The `post_process_object_detection()` function takes in the following arguments:

- the output of the model that was created earlier
- the target size of the image
- the threshold value for filtering out predictions — in this case return only those with confidence greater than 90%

The `post_process_object_detection()` function returns a dictionary containing the objects detected in the image:

```
{'scores': tensor([0.9180, 0.9961, 0.9426, 0.9753, 0.9622,
                  0.9882, 0.9872, 0.9372, 0.9976, 0.9987,
                  0.9174, 0.9896, 0.9997, 0.9822, 0.9970],
                  grad_fn=<IndexBackward0>),
 'labels': tensor([ 1, 72, 62,  1,  1,  1, 76, 72,
                   1,  1,  1, 64,  1, 72,  1]),
 'boxes': tensor([[549.6678, 145.2847, 564.6752, 165.3628],
                  [317.9362, 212.8838, 416.3602, 299.5169],
                  [508.3657, 306.7018, 661.2424, 429.7788],
                  [673.2169, 135.5043, 705.7635, 174.4243],
                  [703.4085, 115.4306, 722.6825, 140.0320],
                  [454.9466, 142.5465, 497.3364, 202.9241],
                  [344.1364, 276.7948, 445.4079, 346.2501],
                  [309.7489, 194.9189, 374.5581, 237.4787],
                  [395.9247, 152.1988, 446.4625, 216.5487],
                  [237.3090, 174.7686, 308.3646, 264.4060],
                  [720.7039, 112.1340, 737.7415, 131.0087],
                  [124.8140, 211.3712, 230.1096, 330.4417],
                  [369.2618, 226.4449, 535.6130, 427.6963],
                  [491.1188, 181.2496, 530.7089, 223.4560],
                  [516.3748, 177.5891, 628.3662, 318.1332]],
                  grad_fn=<IndexBackward0>)}
}
```

There are three key/value pairs in the dictionary:

- scores — the confidence of each detected object
- labels — the index of the detected object in `model.config.id2label`
- boxes — the bounding boxes of each detected object

The best way to visualize the detected objects is to draw bounding boxes around the objects (see Listing 4.2):

**Listing 4.2 Drawing bounding boxes around the detected objects**

```

import random

draw = ImageDraw.Draw(image)

for score, label, box in zip(results["scores"], results["labels"],
results["boxes"]):
    box = [round(i, 2) for i in box.tolist()]
    print(
        f"Detected {model.config.id2label[label.item()]} with confidence "
        f"{(score.item() * 100):.2f}% at {box}"
    )

    r = random.randint(0, 255)
    g = random.randint(0, 255)
    b = random.randint(0, 255)
    color = (r, g, b)

    draw.rectangle(box,                                     #A
                   outline=color,                           #A
                   width=2)                                  #A

    draw.text((box[0], box[1]-10),                          #B
              model.config.id2label[label.item()],          #B
              fill='white')                                  #B

display(image)

```

**#A** draw bounding box around object

**#B** display the object label

You will see the following output: each of the detected objects and its associated confidence:

```

Detected person with confidence 91.80% at [549.67, 145.28, 564.68, 165.36]
Detected tv with confidence 99.61% at [317.94, 212.88, 416.36, 299.52]
Detected chair with confidence 94.26% at [508.37, 306.7, 661.24, 429.78]
Detected person with confidence 97.53% at [673.22, 135.5, 705.76, 174.42]
Detected person with confidence 96.22% at [703.41, 115.43, 722.68, 140.03]
Detected person with confidence 98.82% at [454.95, 142.55, 497.34, 202.92]
Detected keyboard with confidence 98.72% at [344.14, 276.79, 445.41, 346.25]
Detected tv with confidence 93.72% at [309.75, 194.92, 374.56, 237.48]
Detected person with confidence 99.76% at [395.92, 152.2, 446.46, 216.55]
Detected person with confidence 99.87% at [237.31, 174.77, 308.36, 264.41]
Detected person with confidence 91.74% at [720.7, 112.13, 737.74, 131.01]
Detected potted plant with confidence 98.96% at [124.81, 211.37, 230.11, 330.44]
Detected person with confidence 99.97% at [369.26, 226.44, 535.61, 427.7]
Detected tv with confidence 98.22% at [491.12, 181.25, 530.71, 223.46]
Detected person with confidence 99.70% at [516.37, 177.59, 628.37, 318.13]

```

At the same time, the bounding box for each object is drawn on the original image (see Figure 4.9).



**Figure 4.9** The bounding box for each detected object is drawn on the original image

### 4.2.2 Using Transformers Pipeline

The second approach to using the model is to use the Hugging Face transformers pipeline, which was discussed in Chapter 3. Here is how you load the `facebook/detr-resnet-50` model:



```
from transformers import pipeline

detection = pipeline("object-detection", model="facebook/detr-resnet-50")
```

Once you have created a pipeline object (`detection` in this case), you can directly pass the image (in PIL format) to the pipeline and obtain the result:

```
results = detection(image)
results
```

Note that the pipeline object (`detection`) can also take in an image URL, not just a PIL image object. That is, you can also call the pipeline object like this:

```
results = detection('http://bit.ly/46xv3sL')
```

The result printed would look like this:

```
[{'score': 0.9179903864860535,
  'label': 'person',
  'box': {'xmin': 549, 'ymin': 145, 'xmax': 564, 'ymax': 165}},
 {'score': 0.9960624575614929,
  'label': 'tv',
  'box': {'xmin': 317, 'ymin': 212, 'xmax': 416, 'ymax': 299}},
 {'score': 0.9425505995750427,
  'label': 'chair',
  'box': {'xmin': 508, 'ymin': 306, 'xmax': 661, 'ymax': 429}},
 {'score': 0.9753392338752747,
  'label': 'person',
  'box': {'xmin': 673, 'ymin': 135, 'xmax': 705, 'ymax': 174}},
 {'score': 0.962176501750946,
  'label': 'person',
  'box': {'xmin': 703, 'ymin': 115, 'xmax': 722, 'ymax': 140}},
 {'score': 0.9881888628005981,
  'label': 'person',
  'box': {'xmin': 454, 'ymin': 142, 'xmax': 497, 'ymax': 202}},
 {'score': 0.9871691465377808,
  'label': 'keyboard',
  'box': {'xmin': 344, 'ymin': 276, 'xmax': 445, 'ymax': 346}},
 {'score': 0.9371852874755859,
```

```

    'label': 'tv',
    'box': {'xmin': 309, 'ymin': 194, 'xmax': 374, 'ymax': 237}},
{'score': 0.9975801706314087,
 'label': 'person',
 'box': {'xmin': 395, 'ymin': 152, 'xmax': 446, 'ymax': 216}},
{'score': 0.9986708164215088,
 'label': 'person',
 'box': {'xmin': 237, 'ymin': 174, 'xmax': 308, 'ymax': 264}},
{'score': 0.9173707365989685,
 'label': 'person',
 'box': {'xmin': 720, 'ymin': 112, 'xmax': 737, 'ymax': 131}},
{'score': 0.9895991086959839,
 'label': 'potted plant',
 'box': {'xmin': 124, 'ymin': 211, 'xmax': 230, 'ymax': 330}},
{'score': 0.9996592998504639,
 'label': 'person',
 'box': {'xmin': 369, 'ymin': 226, 'xmax': 535, 'ymax': 427}},
{'score': 0.9821581840515137,
 'label': 'tv',
 'box': {'xmin': 491, 'ymin': 181, 'xmax': 530, 'ymax': 223}},
{'score': 0.9970135688781738,
 'label': 'person',
 'box': {'xmin': 516, 'ymin': 177, 'xmax': 628, 'ymax': 318}}]

```

The result is a list of dictionaries for each detected object. To draw the label and bounding box for each object, use the following code snippet as shown in Listing 4.3:

**Listing 4.3 Drawing bounding boxes around the detected objects (using the pipeline object)**

```

import random

draw = ImageDraw.Draw(image)

for object in results:
    box = [i for i in object['box'].values()]
    print(
        f"Detected {object['label']} with confidence "
        f"{(object['score'] * 100):.2f}% at {box}"
    )

    r = random.randint(0, 255)
    g = random.randint(0, 255)
    b = random.randint(0, 255)
    color = (r, g, b)

    draw.rectangle(box,                                #A
                   outline=color,                       #A
                   width=2)                             #A

    draw.text((box[0], box[1]-10),                     #B
              object['label'],                         #B
              fill='white')                             #B

display(image)

```

#A draw bounding box around object

#B display the object label

The image would be identical to that as shown earlier in Figure x. Using the pipeline object, you can also get a list of labels directly using the `model.config.id2label` attribute:

```
detection.model.config.id2label
```

### 4.2.3 Binding to Webcam

Instead of detecting objects in still images, you can go one step further and make use of your webcam to capture videos and detect the objects in it. Using Python to display your webcam content is easy using OpenCV. To install OpenCV, use the pip command:

```
$ pip install opencv-python
```

With OpenCV installed, let's first write the code to connect the webcam using OpenCV and then display the videos on the screen. Create a text file named `object_detection.py` and populate it with the statements as shown in Listing 4.4:

#### Listing 4.4 Displaying webcam image in Python

```
import cv2

stream = cv2.VideoCapture(0) #A

while(True):
    (grabbed, frame) = stream.read() #B

    cv2.imshow("Image", frame) #C

    key = cv2.waitKey(1) & 0xFF
    if key == ord("q"): #D
        break

stream.release() #E
cv2.waitKey(1) #E
cv2.destroyAllWindows() #E
cv2.waitKey(1) #E
```

**#A default webcam**

**#B capture frame-by-frame**

**#C show the frame**

**#D press q to break out of the loop**

**#E cleanup**

If your computer/laptop has multiple webcams, change the number in the `VideoCapture` class accordingly:

```
stream = cv2.VideoCapture(1)
stream = cv2.VideoCapture(2)
```

In Terminal, type the following command to run the program:

```
$ python object_detection.py
```

You should now see a window displaying the video captured by your webcam (see Figure 4.10).



**Figure 4.10** The webcam capturing the image of the author

To detect the objects in the webcam, add the statements to the `object_detection.py` file as shown in Listing 4.5:

**Listing 4.5** Detecting objects in the webcam

```
from transformers import pipeline
from PIL import Image
import cv2

font = cv2.FONT_HERSHEY_SIMPLEX #A
color = (0, 255, 255) #A
stroke = 2 #A

detection = pipeline("object-detection", #B
                      model="facebook/detr-resnet-50") #B

stream = cv2.VideoCapture(0) #C

while(True):
```

```

(grabbed, frame) = stream.read() #D

image = Image.fromarray(frame) #E

results = detection(image) #F

for object in results:
    box = [i for i in object['box'].values()] #G

    cv2.rectangle(frame, #H
                  (box[0],box[1]), #H
                  (box[2],box[3]), #H
                  color, stroke) #H

    cv2.putText(frame, f'({object["label"]})', #I
                (box[0],box[1]-8), #I
                font, 1, color, #I
                stroke, cv2.LINE_AA) #I

cv2.imshow("Image", frame) #J
key = cv2.waitKey(1) & 0xFF
if key == ord("q"): #K
    break

stream.release() #L
cv2.waitKey(1) #L
cv2.destroyAllWindows() #L
cv2.waitKey(1) #L

```

**#A** font, color and line thickness for drawing rectangle and text

**#B** load the object detection model

**#C** default webcam

**#D** capture frame-by-frame

**#E** convert the video image from NumPy array to a Pil image

**#F** detect objects in the image

**#G** get the coordinates for the object detected

**#H** draw bounding box around object detected

**#I** draw the label for the object

**#J** show the frame

**#K** press q to break out of the loop

**#L** cleanup

You need to convert the image captured by the webcam from NumPy array to a PIL image before you can send it to the model for object detection. Figure 4.11 shows the webcam successfully detecting some of the objects in the webcam.

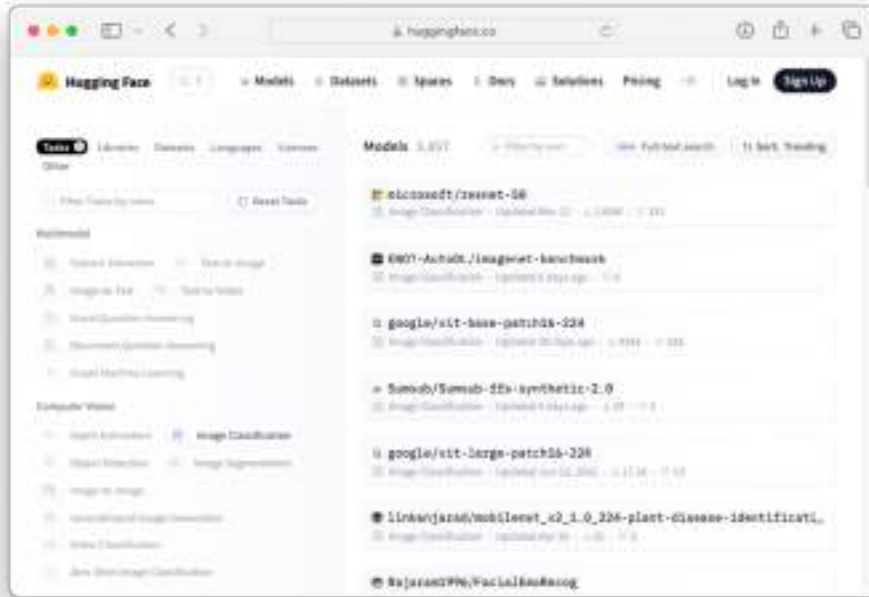


**Figure 4.11** Detecting objects in the webcam

## 4.3 Image Classification

*Image classification* is a computer vision task that involves categorizing or labeling an image into one or more predefined classes or categories. The goal of image classification is to recognize and assign the most appropriate label to a given image based on its content.

Hugging Face hosts a series of models for image classification (see Figure 4.12; [https://huggingface.co/models?pipeline\\_tag=image-classification&sort=trending](https://huggingface.co/models?pipeline_tag=image-classification&sort=trending)).



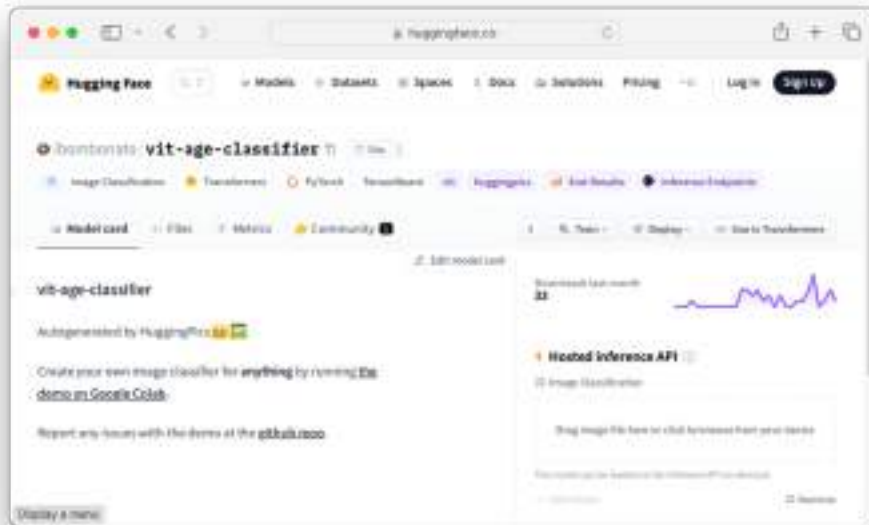
**Figure 4.12 Image classification models hosted by Hugging Face**

Here are two sample models for image classification – age classification:

- <https://huggingface.co/ibombonato/vit-age-classifier> (see Figure 4.13)
- <https://huggingface.co/nateraw/vit-age-classifier>

Both models can predict the age of a person in an image.





**Figure 4.13** The `ibombonato/vit-age-classifier` model for age classification

To test out this model, let's use an official portrait of Barack Obama taken in 2009. Since he was born in 1961, that would make him 48 years old (2009-1961) at the time the picture was taken (see Figure 4.14).



**Figure 4.14** Official portrait of Barack Obama 2009 (source: [https://upload.wikimedia.org/wikipedia/commons/thumb/e/e9/Official\\_portrait\\_of\\_Barack\\_Obama.jpg/800px-Official\\_portrait\\_of\\_Barack\\_Obama.jpg](https://upload.wikimedia.org/wikipedia/commons/thumb/e/e9/Official_portrait_of_Barack_Obama.jpg/800px-Official_portrait_of_Barack_Obama.jpg))

You can test out the model using the Hosted inference API at the Hugging Face model's page (see Figure 4.15). The result shows that there is a very high probability that the person in the image is between 40 to 50 years old, which is correct.



Figure 4.15 Testing Barack Obama's photo on the model

To use the model programmatically, the easiest way is to use the Hugging Face transformers pipeline:

```
from transformers import pipeline

classifier = pipeline("image-classification",
                      model="ibombonato/vit-age-classifier")

classifier('https://bit.ly/3PET3TP')
classifier
```

The result is printed as follows:

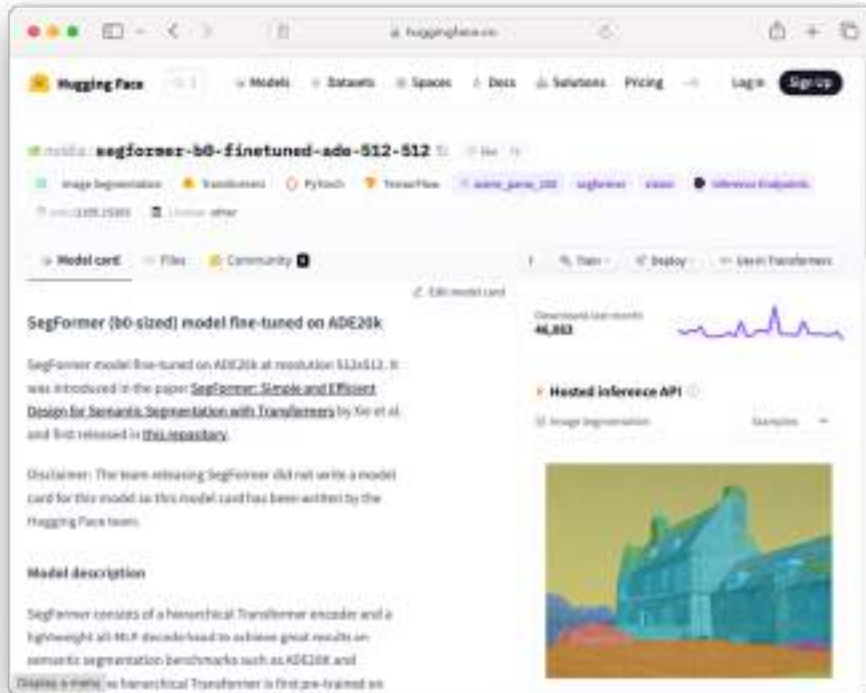
```
[{'score': 0.9248570799827576, 'label': '40-50'},
 {'score': 0.058465585112571716, 'label': '30-40'},
 {'score': 0.003433708567172289, 'label': '50-60'},
 {'score': 0.0028221411630511284, 'label': '20-30'},
 {'score': 0.0023411917500197887, 'label': '0-10'}]
```

## 4.4 Image Segmentation

Another commonly used computer vision technique is *image segmentation*. Image segmentation is a technique that involves separating an image into multiple segments, or regions. Each segment corresponds to a particular object of interest. Using image segmentation, you can analyze an image and extract valuable information from it. Some of its uses are:

- Medical imaging – used to identify and segment tumors in MRI or CT scans
- Object detection and recognition – like object detection we have seen earlier, you can also use image segmentation to identify and locate objects in an image
- Document processing – used to segment text regions in scanned documents
- Biometrics – used to identify and localize faces in images or video frames

Hugging Face contains several image segmentation models that you can use. One of them is the SegFormer model fine-tuned on ADE20k (<https://huggingface.co/nvidia/segformer-b0-finetuned-ade-512-512>). Figure 4.16 shows the SegFormer model fine-tuned on ADE20k model on Hugging Face website (see Figure 4.16).



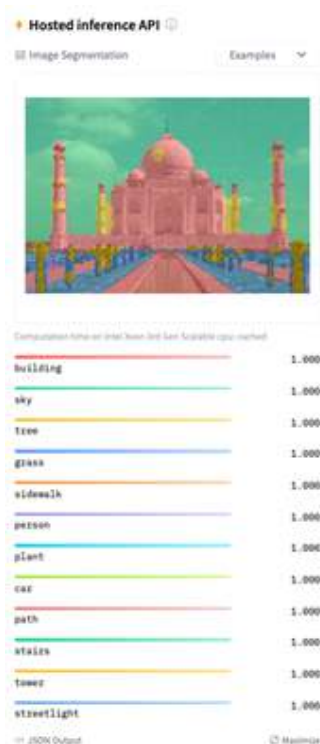
**Figure 4.16** The SegFormer model fine-tuned on ADE20k model page at Hugging Face

To test the segmentation model, you can use an image of the Taj Mahal (see Figure 4.17) and drag and drop it onto the Hosted inference API section of the page.



**Figure 4.17** Picture of the Taj Mahal (Source: [https://commons.wikimedia.org/wiki/File:Taj\\_Mahal,\\_Agra,\\_India\\_edit3.jpg](https://commons.wikimedia.org/wiki/File:Taj_Mahal,_Agra,_India_edit3.jpg))

Figure 4.18 shows the result of the inferencing.



**Figure 4.18** The result of the image segmentation using an image of Taj Mahal

As you can see from the result, the model is able to detect the various objects (such as building, sky, tree, etc.) in the image and highlight the various segments in the image. You can actually over your mouse over the various segment labels and the image will highlight the selected label.

#### 4.4.1 Using the Model Programmatically

As always, we want to be able to use the model programmatically. First, let's load the model and then check how many objects the model can detect. The easiest way to use the model is to use a transformers pipeline:

```
from transformers import pipeline

segmentation = pipeline("image-segmentation",
                        model="nvidia/segformer-b0-finetuned-ade-512-512")

segmentation.model.config.id2label
```

Here are the first and last five objects it can detect (the model can detect a total of 150 objects):

```
{0: 'wall',  
 1: 'building',  
 2: 'sky',  
 3: 'floor',  
 4: 'tree',  
 ...  
145: 'shower',  
146: 'radiator',  
147: 'glass',  
148: 'clock',  
149: 'flag'}
```

For this example, let's use an image from [https://unsplash.com/photos/EC\\_GhFRGTAY](https://unsplash.com/photos/EC_GhFRGTAY) (see Figure 4.19) to discover the various segments in the image.



**Figure 4.19** A picture of a man and an aircraft flying overhead (source: [https://unsplash.com/photos/EC\\_GhFRGTAY](https://unsplash.com/photos/EC_GhFRGTAY))

To detect the various segments in the image, pass the URL of the image to the pipeline object:

```

from PIL import Image
import requests

url = 'https://bit.ly/46iDeJQ'
results = segmentation(url)
results

```

The output of the `results` variable is a list of dictionaries containing details of each of the segments detected in the picture:

```

[{'score': None,
  'label': 'wall',
  'mask': <PIL.Image.Image image mode=L size=1587x2381>},
 {'score': None,
  'label': 'building',
  'mask': <PIL.Image.Image image mode=L size=1587x2381>},
 {'score': None,
  'label': 'sky',
  'mask': <PIL.Image.Image image mode=L size=1587x2381>},
 {'score': None,
  'label': 'person',
  'mask': <PIL.Image.Image image mode=L size=1587x2381>},
 {'score': None,
  'label': 'airplane',
  'mask': <PIL.Image.Image image mode=L size=1587x2381>}]

```

In particular, the `mask` element contains the mask of the segment detected. To view each of the detected masks, loop through the `results` variable:

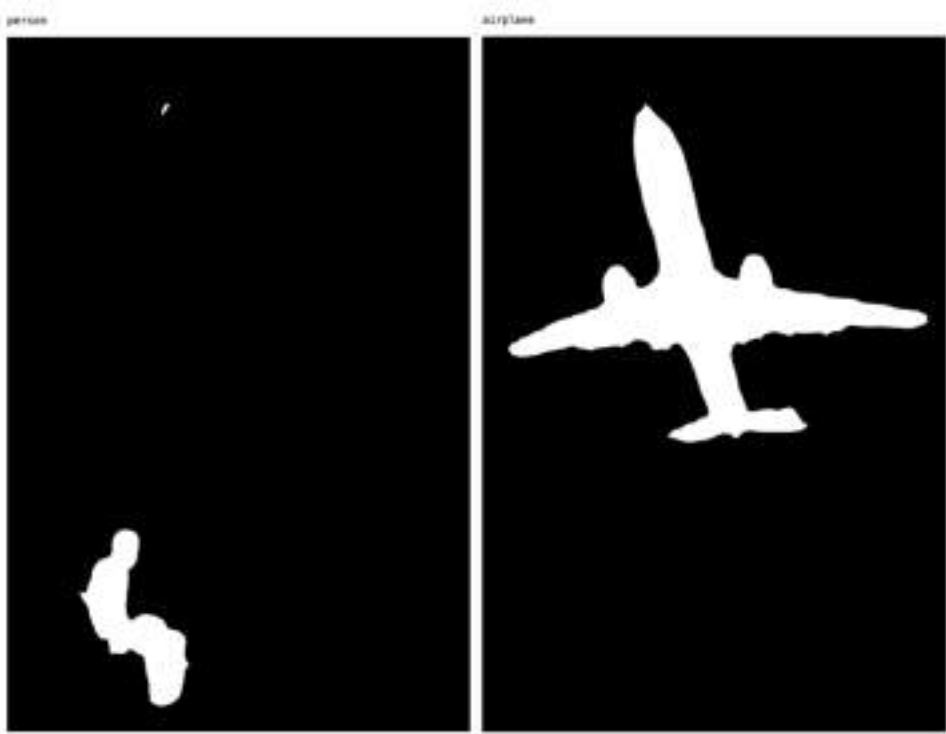
```

for result in results:
    print(result['label'])
    display(result['mask'])

```

Figure 4.20 shows the masks for `person` and `airplane`.





**Figure 4.20** The masks for the person and airplane segments

The white portion of the mask represents the part of the picture containing the segment of interest. You can apply the mask over the original image using the following code snippet:

```
image = Image.open(requests.get(url, stream=True).raw)

for result in results:
    base_image = image.copy()
    mask_image = result['mask']

    base_image.paste(mask_image, mask=mask_image)           #A
    print(result['label'])                                   #B
    display(base_image)
```

#A apply the mask over the original image

#B print out the label of the segment

Figure 4.21 shows the person and airplane masks applied over the original image.



**Figure 4.21** The original image with the person and airplane masks applied

When you apply the mask over the image, notice that the segment of interest is in white. It would be more natural to invert this – that is, the segment of interest should be shown while the rest should be in white. To do this, you can invert the mask using the `invert()` function from the `ImageOps` class in the `PIL` package. The following changes invert the mask and then apply it over the original image:

```

from PIL import ImageOps

for result in results:
    base_image = image.copy()
    mask_image = result['mask']

    mask_image = ImageOps.invert(mask_image)           #A
    base_image.paste(mask_image, mask=mask_image)       #B
    print(result['label'])                             #C
    display(base_image)

```

**#A invert the mask**

**#B apply the mask over the original image**

**#C print out the label of the segment**

Figure 4.22 shows the inverted masks for the person and airplane applied on the original image.



**Figure 4.22 The inverted masks applied on the original image**

### 4.4.2 Binding to Gradio

Instead of manually specifying the URL of the image that we want to use on the model, it would be more convenient to create an UI for the user to try out the segmentation model. Here, we are going to make use of the `Gradio` package to create an UI and then bind it to the function that performs the segmentation. To install the `gradio` package, use the following command:

```
$ pip install gradio
```

#### WHAT IS GRADIO?

Gradio is an open-source Python library that simplifies the process of creating user interfaces for machine learning models and other applications. It is designed to make it easy for developers to build interactive web interfaces for their machine learning models without requiring extensive knowledge of web development.

Let's first create a function called `segmentation` that uses the `SegFormer` model fine-tuned on ADE20k model for segmentation (see Listing 4.6):

**Listing 4.6 Creating a function that uses a model for segmentation**

```

from transformers import SegformerForSemanticSegmentation

model = pipeline("image-segmentation",                                #A
                 model="nvidia/segformer-b0-finetuned-ade-512-512") #A

def segmentation(image, label):
    image = Image.fromarray(image)                                     #B
    results = model(image)                                           #C
    for result in results:
        if result['label'] == label:
            base_image = image.copy()
            mask_image = result['mask']
            mask_image = ImageOps.invert(mask_image)                 #D
            base_image.paste(mask_image, mask=mask_image)             #E
            return(base_image)

```

**#A create a segmentation model**

**#B convert image from NumPy array to PIL format**

**#C use the model for inferencing**

**#D invert the mask**

**#E apply the mask over the original image**

One important point to note here is that when the user passes in an image through the Gradio UI, the image will be sent to the `segmentation()` function as a NumPy array. Hence it is essential to convert it to a Pillow (PIL) image using the `Image.fromarray()` function. When the model returns the result, you will iterate through the result and look for the label that is specified by the user (in the `label` parameter). The function then inverts the corresponding mask, applies it to the image and return it to the caller.

To bind the `segmentation()` function to Gradio, use the `Interface()` class, like this:

```

import gradio as gr

image_input = gr.Image(label = "Image to segmentize")

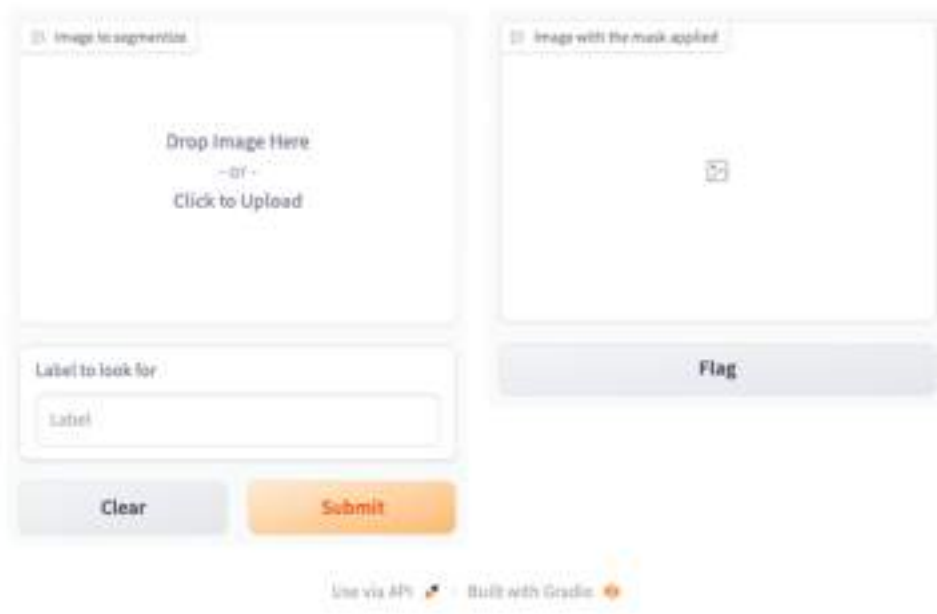
label = gr.Textbox(label = "Label to look for", placeholder = "Label")

image_output = gr.Image(label = "Image with the mask applied")

gr.Interface(segmentation,
             [image_input, label],
             image_output).launch()

```

Figure 4.23 shows how Gradio will look like.



The image shows a Gradio web interface for image segmentation. It consists of two main panels. The left panel, titled "Image to segment", contains a large white area with the text "Drop Image Here" and "Click to Upload". Below this is a text input field labeled "Label to look for" with the placeholder text "Label". At the bottom of the left panel are two buttons: "Clear" and "Submit". The right panel, titled "Image with the mask applied", contains a large white area with a small icon in the center. Below this panel is a button labeled "Flag". At the bottom of the interface, there are two links: "Use via API" and "Built with Gradio".

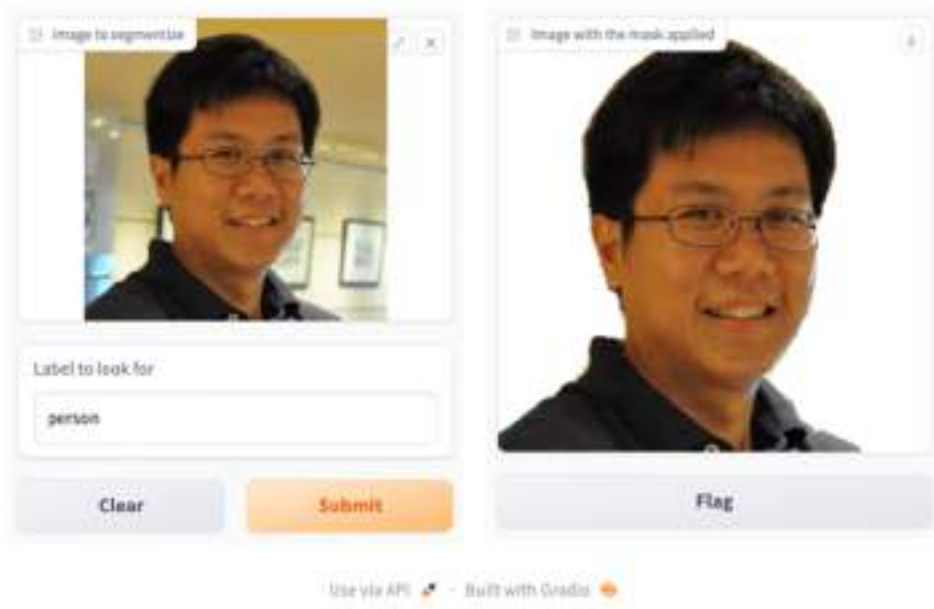
**Figure 4.23 Gradio expects an image and text for input**

Drag and drop an image containing a person onto Gradio. Figure 4.24 shows an image of the author. Also, type in the label that you want to search for. In this example, say you want to search for person in the image, and so you type "person" as the label. Click Submit.



**Figure 4.24 The Gradio UI with the image populated and the label entered**

When the function returns the result, it is shown on the right side of Gradio (see Figure 4.25).

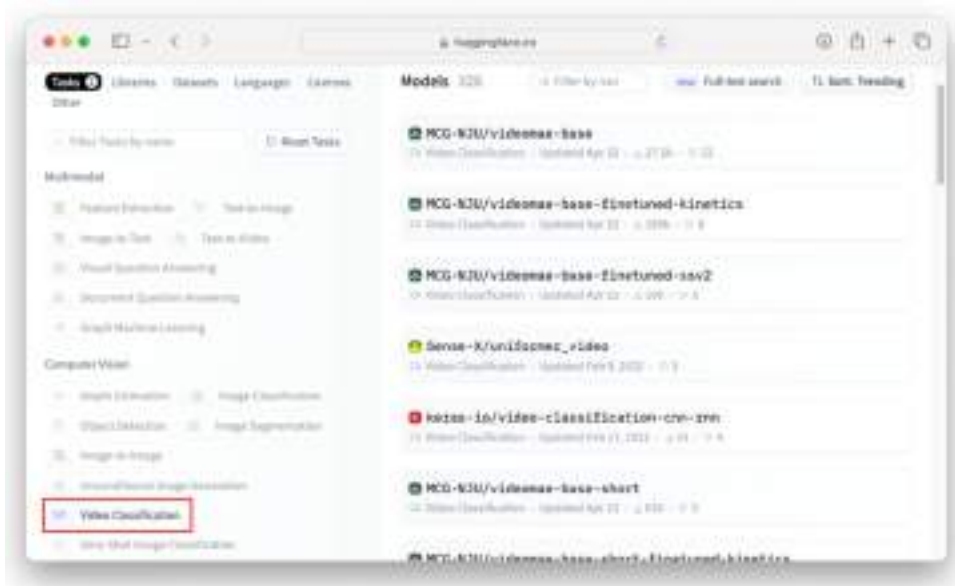


**Figure 4.25 Gradio returns the image with the inverted mask applied to the original image**

## 4.5 Video Content Classification

So far all our examples revolves around detecting objects in still images / webcam inputs. It would be fun to be able to classify objects in video streams. Let's investigate how we can do that.

On Hugging Face Hub's Model page, there is a category on Video Classification ([https://huggingface.co/models?pipeline\\_tag=video-classification&sort=trending;](https://huggingface.co/models?pipeline_tag=video-classification&sort=trending;) see Figure 4.26).



**Figure 4.26** The video classification models page on Hugging Face

For this example, we will be using the VideoMae model (MCG-NJU/videomae-base-short-finetuned-kinetics). VideoMAE performs the task of masked video modeling for video pre-training.

### VIDEOMAE

VideoMae stands for **V**ideo **M**asked **A**uto**E**ncoders

The details for the VideoMae model we are using can be found at: <https://huggingface.co/MCG-NJU/videomae-base-short-finetuned-kinetics> (see Figure 4.27).



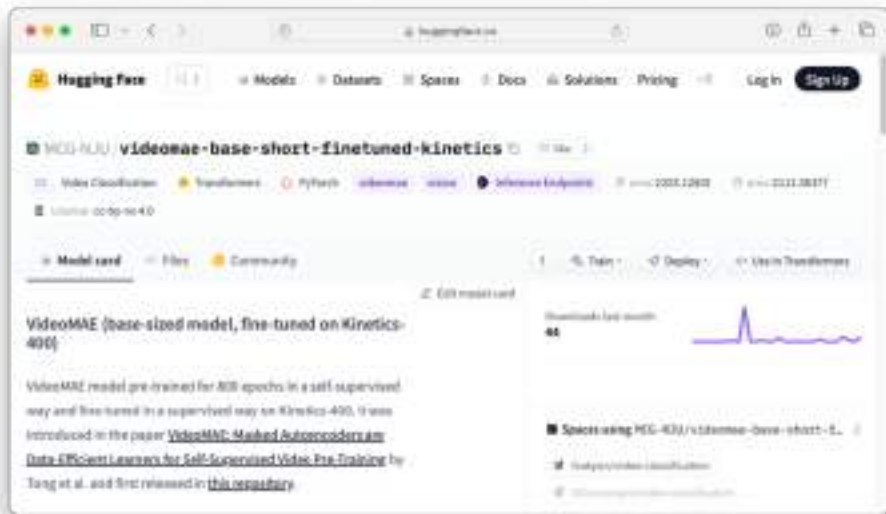


Figure 4.27 The page for the MCG-NJU/videomae-base-short-finetuned-kinetics model

#### 4.5.1 Installing the prerequisites

To use the MCG-NJU/videomae-base-short-finetuned-kinetics model, you need to install the `decord` Python package.

##### WHAT IS DECORD?

Decord is a Python package that provides efficient video decoding capabilities. It is designed to handle video data and extract frames from videos with a focus on performance and speed.

Decord is particularly useful for applications that require video analysis, computer vision, machine learning, and deep learning, where fast video frame extraction is crucial.

For Windows and Intel Mac users, you can install `decord` using the following command:

```
$ pip install decord
```

At the time of writing, an ARM version of the decord package does not exist, and so Apple Silicon Mac users will not be able to install the decord library directly using the above pip command. Fortunately, the folks at EVA (<https://github.com/georgia-tech-db/evadb>) have created a fork of the decord library at <https://pypi.org/project/eva-decord/> that makes it possible for Apple Silicon Mac users to install the decord library. Apple Silicon Mac users should install the decord library using the following command:

```
$ pip install eva-decord
```

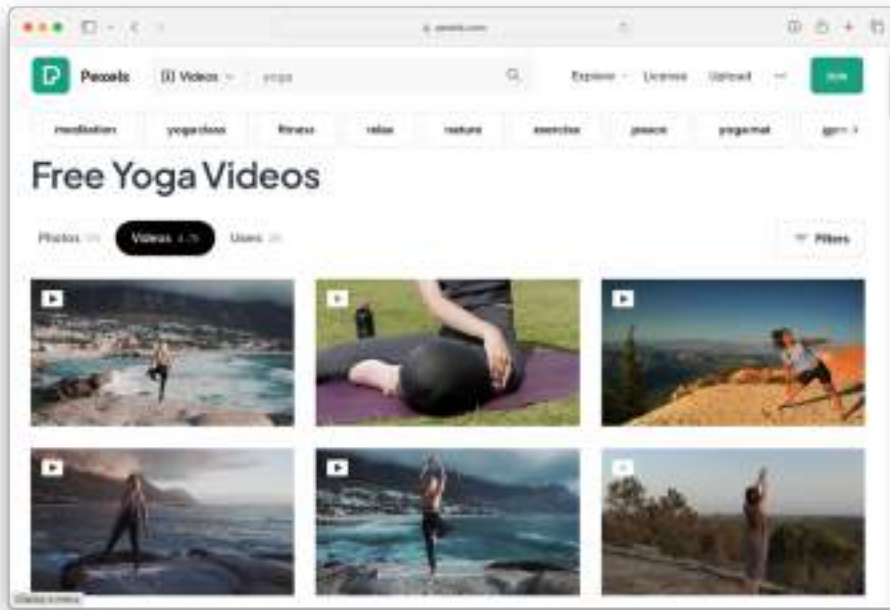
#### 4.5.2 Downloading the videos for testing

For testing, you would need some videos. If you already have videos that you can use for testing that are directly accessible using a URL, you can skip to the next section. If not, you need to download some videos so that you can access them using an URL.

To download the videos locally, first create a folder named `webserver` in your home directory (or any directory you prefer):

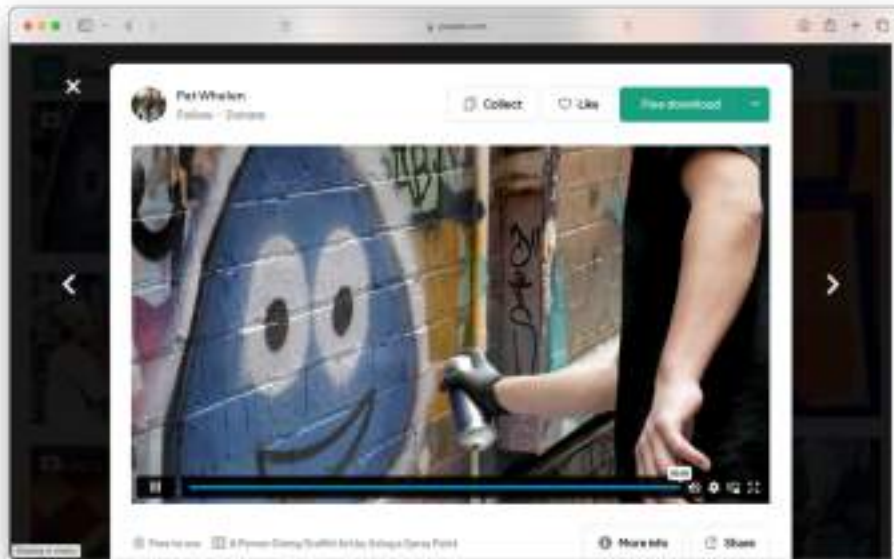
```
$ cd ~  
$ mkdir webserver
```

Next, we need some sample videos to test out the model. You can use `Pexels` (<https://www.pexels.com/search/videos/sample/>), a repository of royalty-free stock photos and videos (See Figure 4.28).



**Figure 4.28 Pexels is a repository of royalty-free stocks photos and videos**

For testing, download a video of person spray-painting a wall (see Figure 4.29; <https://www.pexels.com/video/a-person-doing-graffiti-art-by-using-a-spray-paint-5621707/>). Once the download is done, move it into the `webserver` folder that you have just created. The name of the video in this example is `pexels-pat-whelen-5621707` (1080p).



**Figure 4.29 A video of a person spray-painting a wall**

Once the video is located in the `webserver` folder, you will now use the `"python -m http.server"` command to start a simple HTTP server from the local file system:

```
$ cd ~/webserver
$ python -m http.server
```

If the HTTP web server is started correctly, you will see the following printout indicating that the web server is listening at port 8000:

```
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

### 4.5.3 Using the transformers pipeline object

With the `decord` library installed and the video ready, you can now write the code to make use of the model using the `transformers` pipeline:

```
from transformers import pipeline

video_classifier = pipeline("video-classification",
                             model="MCG-NJU/videomae-base-short-finetuned-kinetics")
```

As usual, the first thing you want to know when using a model is to know the type of object that the model is capable of recognizing:

```
video_classifier.model.config.id2label
```

You will see a list of 400 objects. The following are the first and last five of them:

```
{0: 'abseiling',
 1: 'air drumming',
 2: 'answering questions',
 3: 'applauding',
 4: 'applying cream',
 ...
 395: 'wrestling',
 396: 'writing',
 397: 'yawning',
 398: 'yoga',
 399: 'zumba'}
```

To detect the kind of objects that are present in the video, call the pipeline object with the URL of the video:

```
video_classifier(
    'http://localhost:8000/pexels-pat-whelen-5621707 (1080p).mp4')
```

The results returned looks like this:

```
[{'score': 0.6775813102722168, 'label': 'spray painting'},
 {'score': 0.05606633797287941, 'label': 'throwing axe'},
 {'score': 0.032091718167066574, 'label': 'blasting sand'},
 {'score': 0.01113071758300066, 'label': 'spraying'},
 {'score': 0.007230783812701702, 'label': 'plastering'}]
```

As you can see, it accurately detected that the main activity in the video is spray painting.

## 4.6 Summary

- Object detection is a computer vision technique that involves identifying and locating objects of interest within an image or video.
- *There are two ways to use a model from Hugging Face programmatically:*
  - Through a transformer pipeline, or
  - Load the model directly
- `DetrImageProcessor` is a class (from the transformers package) that is used for processing images to be used as input to the DETR (DEtection TRansformer) algorithm.
- The `DetrForObjectDetection` module provides access to pre-trained DETR models.
- Image classification is a computer vision task that involves categorizing or labeling an image into one or more predefined classes or categories
- Image segmentation is a technique that involves separating an image into multiple segments, or regions.

## ***5 Exploring, Tokenizing, and Visualizing Hugging Face Datasets***

### **This chapter covers**

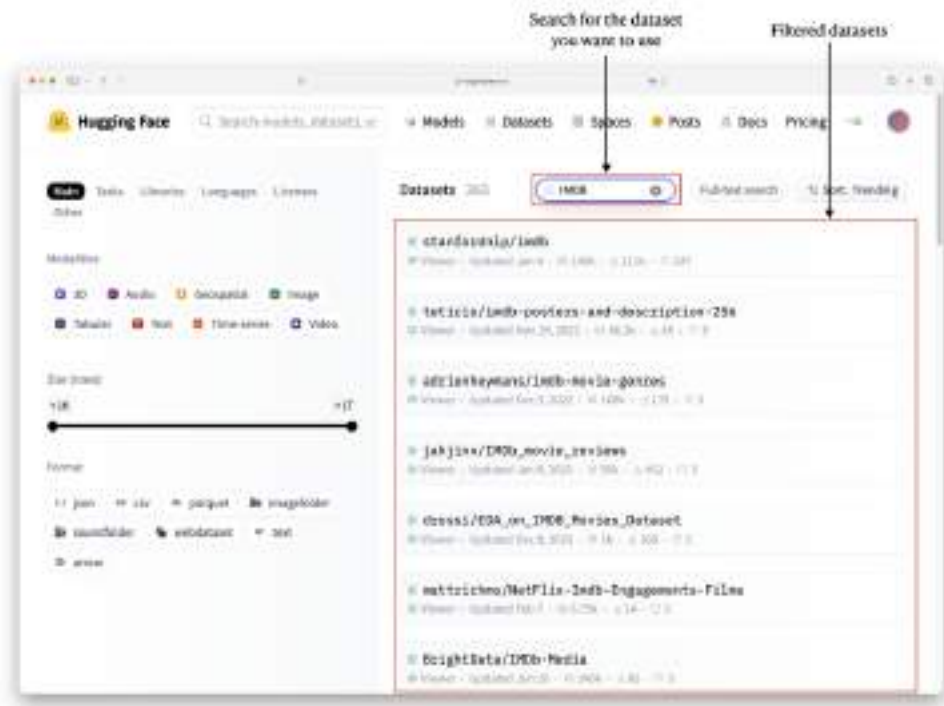
- What is Hugging Face Datasets?
- How to programmatically download Hugging Face Datasets
- What is tokenization and how you can apply it on Hugging Face datasets
- How to perform data visualization on Hugging Face datasets

Hugging Face is an AI platform that develops, trains, and deploys cutting-edge open-source machine learning models. Alongside providing a hub for these trained models, Hugging Face also hosts a wide array of datasets (available at <https://huggingface.co/datasets>), which you can leverage for your own projects.

We will now guide you through accessing datasets from Hugging Face and show you how to programmatically download them to your local machine. You will also gain a deeper understanding of tokenization, including how to tokenize datasets and prepare your data for fine-tuning, which we will cover in chapter 6 we will explore how to visualize various datasets with Hugging Face.

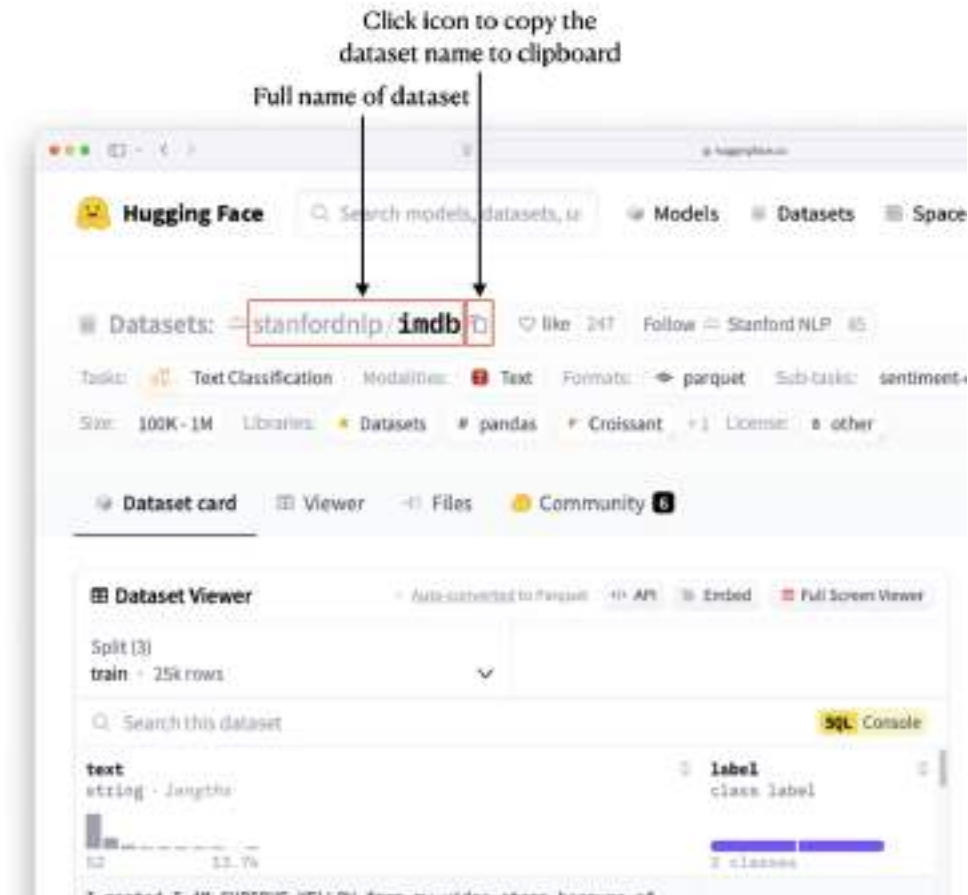






**Figure 5.2 Searching for the desired dataset**

Click on the dataset you want to use in the filtered dataset and you will be directed to the page showing details of the dataset. The full name of the dataset is as shown in figure 5.3. The easiest way to copy the full name of the dataset is to click on the Copy icon.



**Figure 5.3** Locating the full name of the dataset

### 5.1.1 Getting the list of datasets available

Now that you know where to look for the datasets you want to use on the Hugging Face Datasets page, let's investigate how to work with the datasets programmatically in Python. To do so, you first need to install the following two libraries using the pip command:

```
!pip install huggingface_hub
!pip install datasets
```

Let's first start by using the `list_datasets()` function to show a list of available datasets that are available:

```
from huggingface_hub import list_datasets

datasets = list_datasets() #A

#A Get the generator of datasets
```

The `list_datasets()` function returns a generator that points to a list of datasets. You cannot directly access the datasets using an index, nor should you convert the list of datasets using the `list()` function (doing so will likely cause memory overflow as the list of datasets is very large). Instead, you should use the `next()` function to iterate through the list of datasets:

```
dataset = next(datasets) #A
print(dataset)

#A Get the next dataset using next()
```

The dataset is encapsulated in the `DatasetInfo` object, with various fields containing the details of the dataset as shown in Listing 5.1.

#### Listing 5.1 The content of the `DatasetInfo` object

```
DatasetInfo(
  id='fka/awesome-chatgpt-prompts',
  author='fka',
  sha='459a66186f8f83020117b8acc5ff5af69fc95b45',
  created_at=datetime.datetime(2022, 12, 13, 23, 47, 45,
                               tzinfo=datetime.timezone.utc),
  last_modified=datetime.datetime(2024, 9, 3, 21, 28, 41,
                                   tzinfo=datetime.timezone.utc),
  private=False,
  gated=False,
  disabled=False,
  downloads=9522,
  downloads_all_time=None,
  likes=6218,
  paperswithcode_id=None,
  tags=['task_categories:question-answering',
        'license:cc0-1.0',
        'size_categories:n<1K',
        'format:csv',
        'modality:text',
        'library:datasets',
        'library:pandas',
```

```

        'library:mlcroissant',
        'library:polars',
        'region:us',
        'ChatGPT'],
    trending_score=90,
    card_data=None,
    siblings=None
)

```

To print the first five datasets, you can use the following code snippet, which prints the ID of each dataset:

```

for i in range(5):
    dataset = next(datasets)
    print(dataset.id)

```

#A

**#A print the id of the dataset**

You should see something like this:

```

fka/awesome-chatgpt-prompts
qq8933/OpenLongCoT-Pretrain
Spawning/PD12M
wyu1/Leopard-Instruct
OpenCoder-LLM/opc-sft-stage1

```

The ID of each dataset is what you have seen earlier in figure 5.3 – the full name of the dataset.

### 5.1.2 Validating the availability of a dataset

Before you download a dataset from Hugging Face, it is useful to verify that it is available for download. You can use the code snippet shown in listing 5.2 to verify its availability.

**Listing 5.2 Code snippet to verify a dataset's availability**

```
import requests

token = 'Hugging_Face_Token' #A
dataset_id = 'fka/awesome-chatgpt-prompts' #B

headers = {"Authorization": f"Bearer {token}"}
API_URL = f"https://datasets-server.huggingface.co/is-valid?dataset={dataset_id}"

def query():
    response = requests.get(API_URL, headers=headers)
    return response.json()

data = query()
data
```

**#A** Replace with your own hugging face token

**#B** Replace with the name of the dataset you want to verify

For this example, you need to apply for a Hugging Face access token (<https://huggingface.co/settings/tokens>). The access token will allow you to authenticate and access models, datasets, and other resources on the Hugging Face platform.

In the above example, the code snippet returns the following result for the 'fka/awesome-chatgpt-prompts' dataset:

```
{'preview': True,
 'viewer': True,
 'search': True,
 'filter': True,
 'statistics': True}
```

Alternatively, you can also use the curl command in Terminal (or Command Prompt) to validate if a dataset is available:

```
$ curl -X GET "https://datasets-server.huggingface.co/is-
valid?dataset=fka/awesome-chatgpt-prompts"
```

Figure 5.4 shows the syntax for using curl to check the validity of a dataset.

```
curl -X GET "https://datasets-server.huggingface.co/is-valid?dataset=<dataset_name>"
```

**Figure 5.4** The syntax for using the curl command to check the validity of a dataset

You can replace the *"fka/awesome-chatgpt-prompts"* dataset name with the full name of the dataset you want to use. If the dataset is available, you will see the following output (formatted for clarity):

```
{
  "preview":true,
  "viewer":true,
  "search":true,
  "filter":true,
  "statistics":true
}
```

### 5.1.3 Downloading a dataset

Once you have verified that a dataset is available for download, you can download it using the `load_dataset()` function. The following code snippet downloads the *'stanfordnlp/imdb'* dataset:

```
from datasets import load_dataset

dataset_id = 'stanfordnlp/imdb'

dataset = load_dataset(dataset_id)           #A
print(dataset)                             #B

#A load the IMDB dataset
#B view the dataset structure
```

#### IMDB DATASET

The IMDB dataset is a popular dataset used for natural language processing (NLP) tasks, specifically for sentiment analysis. It consists of movie reviews from the Internet Movie Database (IMDB) and includes both positive and negative reviews.

You should see the following output:

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

The result is represented as a `DatasetDict` object, separated into the following subsets (known as *splits*):

- **train:** The training dataset, used to train models.
- **test:** The testing dataset, used to evaluate the model's performance.
- **unsupervised:** This subset often contains unlabeled data, which can be used for unsupervised or semi-supervised learning tasks.

The `num_rows` attribute represents the number of rows in each split of the data.

You can also view the different splits of the datasets using the curl command, like this:

```
$ curl -X GET "https://datasets-server.huggingface.co/
splits?dataset=stanfordnlp/imdb"
```

Figure 5.5 shows the syntax for using curl to check the splits of a dataset:

```
curl -X GET "https://datasets-server.huggingface.co/splits?dataset=<dataset_name>"
```

**Figure 5.5** The syntax for using the curl command to check the splits of a dataset

You should see the following output as shown in listing 5.3.

**Listing 5.3 Output containing the splits of the dataset**

```
{
  "splits":[
    {
      "dataset":"stanfordnlp/imdb",
      "config":"plain_text",
      "split":"train"
    },
    {
      "dataset":"stanfordnlp/imdb",
      "config":"plain_text",
      "split":"test"
    },
    {
      "dataset":"stanfordnlp/imdb",
      "config":"plain_text",
      "split":"unsupervised"
    }
  ],
  "pending":[],
  "failed":[]
}
```

Continuing with the Python code snippet, let's view the first row in the *train* subset:

```
dataset['train'][0]
```

You should see the output (formatted for clarity) as shown in listing 5.4.

**Listing 5.4 The content of the first row of the IMDB dataset**

```
{
  'text': 'I rented I AM CURIOUS-YELLOW from my video store
because of all the controversy that surrounded it when it
was first released in 1967. I also heard that at first it
was seized by U.S. customs if it ever tried to enter this
country, therefore being a fan of films considered
"controversial" I really had to see this for myself.<br />
<br />The plot is centered around a young Swedish drama
student named Lena who wants to learn everything she can
about life. In particular she wants to focus her
```



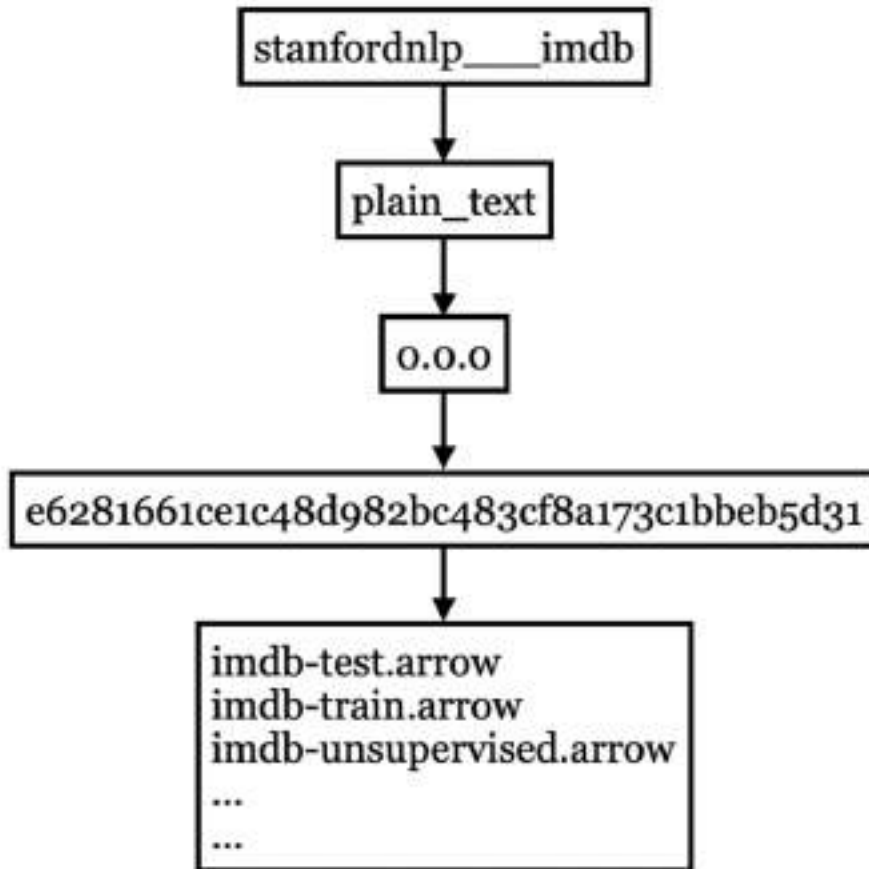
attentions to making some sort of documentary on what the average Swede thought about certain political issues such as the Vietnam War and race issues in the United States. In between asking politicians and ordinary denizens of Stockholm about their opinions on politics, she has sex with her drama teacher, classmates, and married men.

<br /><br />What kills me about I AM CURIOUS-YELLOW is that 40 years ago, this was considered pornographic. Really, the sex and nudity scenes are few and far between, even then it's not shot like some cheaply made porno. While my countrymen mind find it shocking, in reality sex and nudity are a major staple in Swedish cinema. Even Ingmar Bergman, arguably their answer to good old boy John Ford, had sex scenes in his films.<br /><br />I do commend the filmmakers for the fact that any sex shown in the film is shown for artistic purposes rather than just to shock people and make money to be shown in pornographic theaters in America. I AM CURIOUS-YELLOW is a good film for anyone wanting to study the meat and potatoes (no pun intended) of Swedish cinema. But really, this film doesn't have much of a plot.',

```
'label': 0
}
```

### WHERE IS THE DATASET STORED LOCALLY?

When you download a dataset from Hugging Face, it is stored in the `~/.cache/huggingface/datasets` folder on your Mac (on Windows it would be in `C:\users\<username>\.cache\huggingface\datasets`). Each dataset would be stored in its respective folder. For example, the `'stanfordnlp/imdb'` dataset would be stored in its own set of folders and subfolders as shown in Figure 5.6. Observe that there are three arrow (Parquet) files corresponding to the three splits of the dataset – train, test, and unsupervised.



**Figure 5.6** The folder structure of the '*stanfordnlp/imdb*' dataset

Do that note that each dataset has its own folder structure. So, the best way to understand the structure of a particular dataset is by exploring the files and folders created within its specific cache directory. Each dataset may have different organizational layouts depending on the nature of the data, the splits provided (e.g., train, validation, test), and how the dataset is pre-processed by Hugging Face.

If you only want to download a particular split of the dataset, you can specify the split name in the `split` parameter, like this:

```
dataset = load_dataset(dataset_id,
                       split='train')
print(dataset)
```

The above statement only downloads the *train* split of the dataset. This approach is useful if you plan to work with a specific split, as it saves both time and disk space by avoiding the need to download the entire dataset.

The downloaded dataset will now look like this:

```
Dataset({
  features: ['text', 'label'],
  num_rows: 25000
})
```

To access the first row you now just need to use this:

```
dataset[0]
```

There is now no need to specify the `train` key as what you did earlier:

```
dataset['train'][0]
```

### 5.1.4 Shuffling the Dataset

If you want to shuffle the dataset you have downloaded, you can use the `shuffle()` method to randomize the order of the data:

```
dataset_id = 'stanfordnlp/imdb'
dataset = load_dataset(dataset_id)

shuffled_dataset = dataset.shuffle(seed = 42)
```

The `seed` parameter in the `shuffle()` method is used to ensure reproducibility of the shuffling process. By setting a specific seed value, you can guarantee that the dataset is shuffled in the same way each time you run the code. This is useful for debugging, experimentation, or when you want to ensure consistent results across different runs.

### 5.1.5 Streaming the Dataset

Sometimes, the dataset you are trying to download may be too large to fit into memory all at once. This can cause issues if your system lacks enough memory to hold the entire dataset, and it can also increase processing time if you don't need to have the entire dataset in memory at once. To prevent downloading the entire dataset in one shot, you can set the `streaming` parameter to `True` in the `load_dataset()` function:

```
from datasets import load_dataset

dataset_id = 'stanfordnlp/imdb'
dataset = load_dataset(dataset_id,
                       streaming = True) #A
print(dataset)
```

**#A Streaming the IMDB dataset**

What this does is that the `load_dataset()` function will now return an `IterableDatasetDict` object instead of downloading the entire dataset (see Listing 5.5).

#### Listing 5.5 The content of the `IterableDatasetDict` object

```
IterableDatasetDict({
  train: IterableDataset({
    features: ['text', 'label'],
    num_shards: 1
  })
  test: IterableDataset({
    features: ['text', 'label'],
    num_shards: 1
  })
  unsupervised: IterableDataset({
    features: ['text', 'label'],
    num_shards: 1
  })
})
```

To fetch the dataset, you need to enumerate through the dataset and fetch them one row at a time. The following code snippet shows how you can obtain and print the first five rows in the training dataset:

```

for i, example in enumerate(dataset["train"]):           #A
    if i < 5:                                           #B
        print(example)
    else:
        break

```

**#A Stream through the data and print the first few examples**

**#B Show the first 5 examples**

By fetching the data one row at a time, you can efficiently process large datasets without loading them entirely into memory.

### 5.1.6 Getting the Parquet files of a dataset

While you can use the `load_dataset()` function to download the dataset onto your computer, there may be times when you prefer to directly download the dataset in Parquet format instead.

#### PARQUET FILE FORMAT

Parquet is a columnar storage file format designed for efficient data storage and processing, particularly in big data environments. It is optimized for querying and analyzing large datasets, offering significant compression and performance improvements over row-based formats like CSV.

Parquet is schema-based, meaning it stores both the data and its schema, enabling better data organization and faster access. Its columnar structure allows for efficient read and write operations, especially when only a subset of columns is needed, and it supports complex nested data structures. Parquet is widely used with data processing frameworks like Apache Spark, Hive, and Hadoop due to its compatibility with various big data tools and systems.

The Hugging Face datasets service automatically converts all public datasets to the *Parquet* format. The Parquet format offers significant performance improvements, especially for large datasets. As an example, let's use the following curl command to get the Parquet files associated with the *"stanfordnlp/imdb"* dataset:

```

$ curl -X GET "https://datasets-server.huggingface.co/parquet?
dataset=stanfordnlp/imdb"

```

The above command returns the result (formatted for clarity) as shown in listing 5.6.

**Listing 5.6 The result containing the URLs for the various Parquet files**

```
{
  "parquet_files":[
    {
      "dataset":"stanfordnlp/imdb",
      "config":"plain_text",
      "split":"test",
      "url":"https://huggingface.co/datasets/stanfordnlp/
        imdb/resolve/refs%2Fconvert%2Fparquet/
        plain_text/test/0000.parquet",
      "filename":"0000.parquet",
      "size":20470363
    },
    {
      "dataset":"stanfordnlp/imdb",
      "config":"plain_text",
      "split":"train",
      "url":"https://huggingface.co/datasets/stanfordnlp/
        imdb/resolve/refs%2Fconvert%2Fparquet/
        plain_text/train/0000.parquet",
      "filename":"0000.parquet",
      "size":20979968
    },
    {
      "dataset":"stanfordnlp/imdb",
      "config":"plain_text",
      "split":"unsupervised",
      "url":"https://huggingface.co/datasets/stanfordnlp/
        imdb/resolve/refs%2Fconvert%2Fparquet/
        plain_text/unsupervised/0000.parquet",
      "filename":"0000.parquet",
      "size":41996509
    }
  ],
  "pending":[],
  "failed":[],
  "partial":false
}
```

As you can see, the result contains the URLs of the Parquet file for each of the splits. Using these URLs, you can use Python to access the Parquet file of the split(s) directly, as the following example shows:

```

!pip install pyarrow
import pandas as pd

url = "https://huggingface.co/datasets/stanfordnlp/" + \
      "imdb/resolve/refs%2Fconvert%2Fparquet/" + \
      "plain_text/unsupervised/0000.parquet"
df = pd.read_parquet(url, engine='pyarrow')

display(df.head())

```

Figure 5.7 shows the first five rows of the unsupervised split of the dataset.

|   | text                                              | label |
|---|---------------------------------------------------|-------|
| 0 | This is just a precious little diamond. The pl... | -1    |
| 1 | When I say this is my favourite film of all ti... | -1    |
| 2 | I saw this movie because I am a huge fan of th... | -1    |
| 3 | Being that the only foreign films I usually li... | -1    |
| 4 | After seeing Point of No Return (a great movie... | -1    |

**Figure 5.7** The first five rows of the unsupervised split

## 5.2 Tokenization in NLP

Tokenization is a foundational process in Natural Language Processing (NLP) that breaks down text into manageable units or tokens, allowing models to interpret and process language more effectively. Tokenization has the following key uses:

- **Text Preprocessing:** Tokenization helps preprocess text data, simplifying tasks like filtering out punctuation, converting to lowercase, and handling special characters.
- **Representation for Machine Learning Models:** Most NLP models, including transformers and LLMs, require text to be in a numerical format. Tokenization transforms text into numerical IDs that the models can work with.

- **Efficiency and Memory Optimization:** Smaller tokens, like subwords, allow models to handle larger vocabulary sizes with fewer parameters. This also makes it easier for models to capture nuances, such as suffixes, prefixes, and infixes, which is especially useful for inflective languages.
- **Foundation for Further NLP Tasks:** Tokenization provides a foundation for more advanced NLP tasks like named entity recognition (NER), part-of-speech tagging, machine translation, and text summarization.

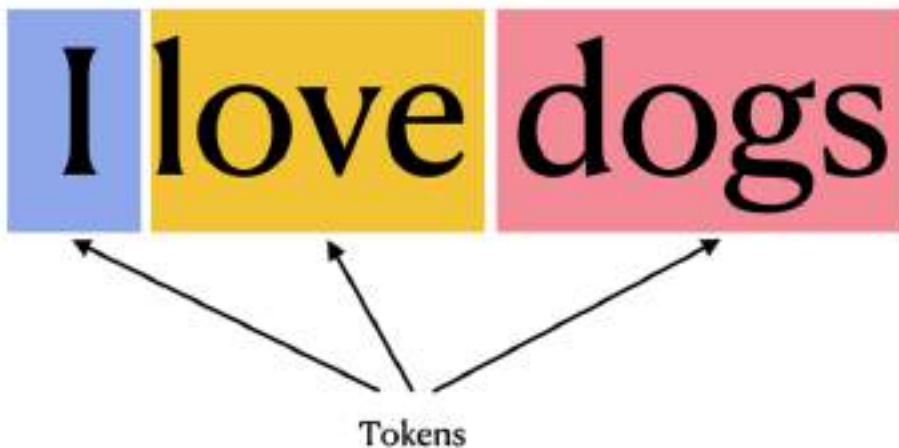
In the following sections, we will discuss the different methods of tokenization, how they work, and how we can tokenize a dataset from Hugging Face so that we can prepare it for fine-tuning.

### 5.2.1 Types of Tokenization Methods

There are several types of tokenization methods, each suited to different tasks and languages. Here are some of the main types of tokenization:

- **Word-level tokenization** - Involves splitting text into individual words.
- **Subword-level tokenization** - Splits words into smaller meaningful units or subwords.
- **Character-level tokenization** - Breaks text into individual characters. Commonly used for languages like Chinese or Japanese, where word boundaries are less obvious

An example of word-level tokenization is shown in figure 5.8.



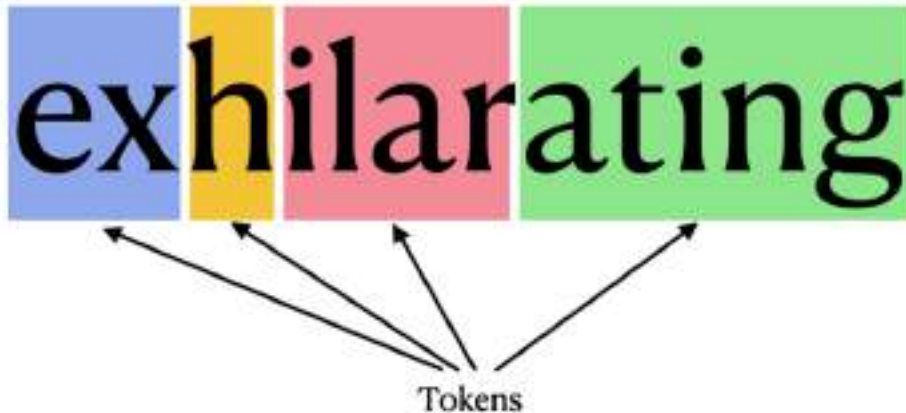
**Figure 5.8 Example of word-level tokenization - the sentence is tokenized into three tokens**

In this example, the string *"I love dogs"* has been tokenized into three different words — "I", "love", and "dogs". This type of tokenization is called *Word-level tokenization*.



Word-level tokenization, while straightforward, struggles with out-of-vocabulary words, requires a large vocabulary for diverse languages, and doesn't capture the internal structure of words, which limits a model's generalization abilities. Some models that use this method are **Word2Vec** and **GloVe**.

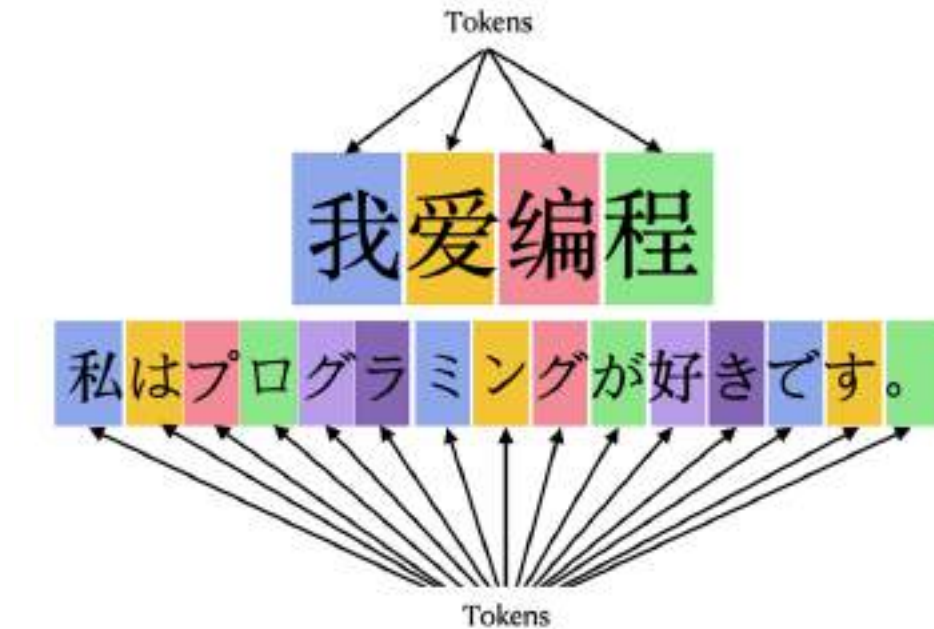
Most newer models, especially transformer-based models like **BERT** and **GPT**, prefer subword or byte-pair encoding (BPE) tokenization to overcome these issues, providing better flexibility and generalization across languages and word forms. An example of *subword-level tokenization* is shown in Figure 5.9.



**Figure 5.9 Example of subword-level tokenization - the single word “exhilarating” is tokenized into four tokens**

In this example, the word “*exhilarating*” is tokenized into four tokens - “ex”, “h”, “ilar”, and “ating”. Subword-level tokenization offers several advantages in NLP, especially for models working with diverse language data. One primary benefit is that it handles out-of-vocabulary (OOV) words effectively by breaking them into smaller, recognizable sub-units. This ability helps avoid the need to discard or ignore unfamiliar words. Additionally, subword-level tokenization preserves more information than word-level tokenization, especially with languages that have complex morphology or compound words, making it ideal for handling misspellings, rare words, and different grammatical forms.

Character-level tokenization is usually used on languages such as Chinese or Japanese. Figure 5.10 shows an example of a sentence “I love programming” in both Simplified Chinese and Japanese tokenized into character-level tokens.



**Figure 5.10 Examples of character-level tokenization using Simplified Chinese and Japanese**

For the Simplified Chinese example, each character represents a meaningful unit in the sentence, making character tokenization particularly useful for languages like Chinese, Japanese, and Korean where words are often composed of multiple characters and spaces are not typically used to delimit them. For Japanese, each individual character, including Kanji, Hiragana, Katakana, and punctuation, is treated as a token. This type of tokenization works well for handling Japanese text when you're interested in the most granular form of the data.

### 5.2.2 Tokenizing Hugging Face Datasets

Hugging Face datasets are compatible with built-in tokenizers and data loaders, making it easy to preprocess and tokenize large datasets efficiently before feeding them into models.

Using the IMDB dataset that we have loaded in earlier, let's use the `'bert-base-uncased'` model as the tokenizer and use it to tokenize the IMDB dataset:

```
from transformers import AutoTokenizer

dataset = load_dataset(dataset_id)
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
tokenized_dataset = dataset.map(
    lambda examples:
        tokenizer(examples['text'],
                  truncation = True,
                  padding = 'max_length'),
    batched = True)
```

Here we used the `AutoTokenizer` class from Hugging Face that automatically loads the appropriate tokenizer class for the specified pre-trained model, in this case, *bert-base-uncased*, a version of the BERT (Bidirectional Encoder Representations from Transformers) model from Hugging Face's transformers library that is trained on uncased text data. The `map()` method of the dataset is used to apply a function to each element or batch of elements in the dataset. You can now print out the tokenized dataset:

```
print(tokenized_dataset)
```

You will see the output as shown in listing 5.7.

**Listing 5.7 The content of the tokenized dataset**

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label', 'input_ids', 'token_type_ids',
              'attention_mask'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label', 'input_ids', 'token_type_ids',
              'attention_mask'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label', 'input_ids', 'token_type_ids',
              'attention_mask'],
    num_rows: 50000
  })
})
```

Observe that each of the splits of the dataset contains three new attributes – `input_ids`, `token_type_ids`, and `attention_mask`. Let's examine each of them one by one. Let's start with the `input_ids`:

```
print(tokenized_dataset['train'][0]['input_ids'])
```

You will see the following:

```
[101, 1045, 12524, 1045, 2572, 8025, 1011, 3756, 2013, 2026,
2678, 3573, 2138, 1997, 2035, 1996, 6704, 2008, 5129, 2009,
2043, 2009, 2001, 2034, 2207, 1999, 3476, 1012, 1045, 2036,
...
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Each number above represents the token ID of a corresponding token. The trailing 0s at the end of the list act as padding tokens, ensuring that all sequences in the batch have the same length for consistent processing by the model.

## TOKEN ID

A token ID is a unique numerical identifier assigned to a token (a word, subword, punctuation mark, or even whitespace, depending on the tokenizer used) in the context of natural language processing (NLP). Token IDs are generated during the tokenization process when text data is processed into input that a language model (such as BERT, GPT, or other transformer models) can understand and use.

To convert the token IDs back to the tokens, you can use the `convert_ids_to_tokens()` method of the tokenizer:

```
tokens = tokenizer.convert_ids_to_tokens(
    tokenized_dataset['train'][0]['input_ids'])
print(tokens)
```

You should now see the following:

```
['[CLS]', 'i', 'rented', 'i', 'am', 'curious', '-',
 'yellow', 'from', 'my', 'video', 'store', 'because',
 'of', 'all', 'the', 'controversy', 'that', 'surrounded',
 ...
 ...
 '##men', 'mind', 'find', 'it', 'shocking'
 ...
 '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]',
 '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]',
 '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]']
```

The first token is `[CLS]`, which signifies the start of the string. The `##` symbol in front of some tokens indicates that the token is a subword unit that is a continuation or suffix of a larger word. In simpler terms, it signifies that the token is not a standalone word, but rather a fragment that combines with the preceding tokens to form a complete word. The `[PAD]` token indicates padding in tokenized sequences. It is used to ensure that all input sequences to a model have the same length by filling shorter sequences with padding tokens. This padding process is necessary because many transformer-based models, such as BERT, expect input tensors of uniform size to allow for efficient batch processing during training or inference.

The second attribute, `token_type_ids`, is used to differentiate between multiple segments within a single input. This is particularly useful for models like BERT, which can process inputs consisting of two separate segments (e.g., a sentence pair in tasks like next-sentence prediction or question answering). `token_type_ids` help the model distinguish which tokens belong to which segment: typically, tokens from the first segment are assigned 0, and tokens from the second segment are assigned 1. In the case of a single text input, such as an IMDB movie review, there is only one segment, so all tokens will have the same `token_type_id`, which is 0. Therefore, when printing `token_type_ids` for such an input, you will see an array of zeros, like this:

```
[0, 0, 0, ..., 0, 0, 0]
```

This indicates that the entire sequence is treated as a single segment, and no distinction is made between multiple segments.

The third attribute, `attention_mask`, is used to inform the model which tokens should be *attended to* (processed) and which should not. It is especially important when padding tokens are present in the input, as the model should ignore the padding during its computations.

Value of 1: Indicates that the token should be attended to (i.e., it's a real token and not padding).

Value of 0: Indicates that the token is a padding token and should not contribute to the model's attention mechanism.

You can print the attention mask for the first row of training data like this:

```
print(tokenized_dataset['train'][0]['attention_mask'])
```

You will see the following output:

```
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 ...
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Figure 5.11 shows the relationships between token IDs, tokens, and attention mask for the first row in the *train* split in the dataset.



**Figure 5.11** The token IDs, tokens, and attention mask for the first row of the training data in the dataset

Tokenizing datasets is a crucial step, as it converts raw text into a format that can be effectively processed by machine learning models. This process is especially important when preparing data for fine-tuning models, as it ensures that the model can understand and work with the input text. Tokenization breaks down the text into smaller units, such as words or subwords, which the model can then use to learn patterns and make predictions during training.

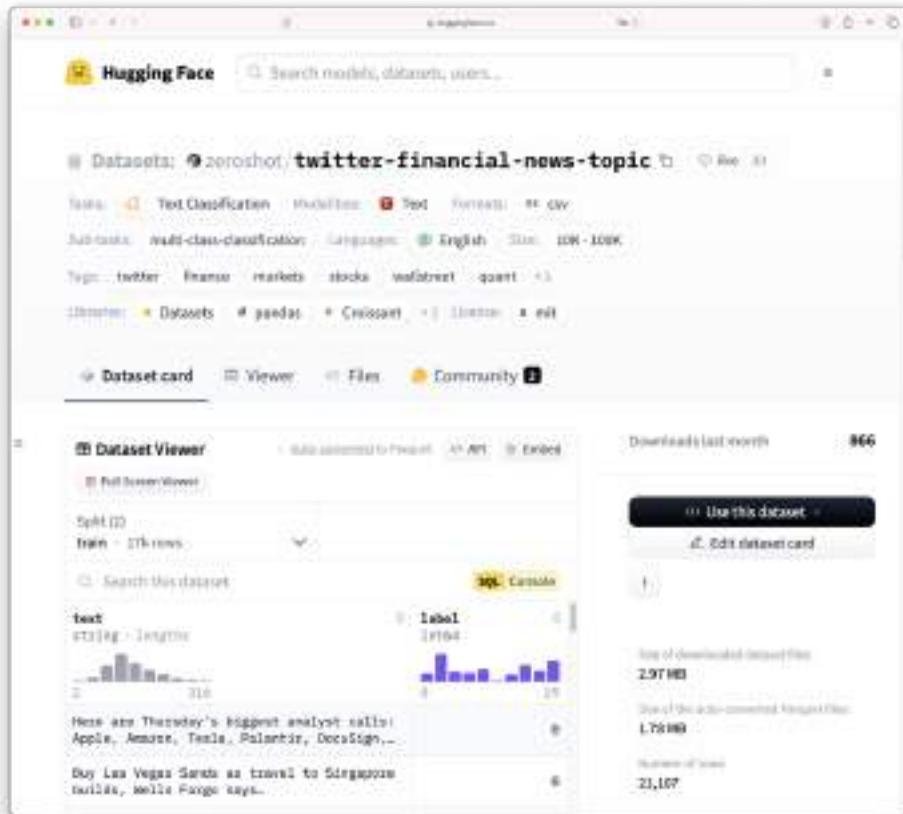
## 5.3 Visualizing the Datasets

The Hugging Face datasets provide a great opportunity to practice data visualization techniques, which allows you to glean additional insights from your data and generate actionable conclusions that can inform decision-making or model optimization.

Now, we will make use of two particular datasets from Hugging Face for data visualization.

### 5.3.1 Using the Twitter Financial News Topic dataset

Let's first start off with the *Twitter Financial News* Topic dataset. The Twitter Financial News Topic dataset is an English-language dataset containing an annotated corpus of finance-related tweets. This dataset is used to classify finance-related tweets for their topic. The dataset holds 21,107 documents annotated with 20 labels. You can find out more about this dataset by searching for the name of this dataset on Hugging Face's website, or by going directly to this URL: <https://huggingface.co/datasets/zeroshot/twitter-financial-news-topic> (see Figure 5.12).



**Figure 5.12 The Twitter Financial News Topic dataset hosted by Hugging Face**

Let's first load the dataset and extract the `train` split:

```
from datasets import load_dataset

dataset = load_dataset('zeroshot/twitter-financial-news-topic') #A

train_data = dataset['train'] #B
```

**#A Step 1: Load the dataset**

**#B Step 2: Extract the topic labels from the training set**

Let's print out the first row of the data:

```
print(train_data[0])
```



You should now see the text as well as the label of the first row:

```
{'text': "Here are Thursday's biggest analyst calls: Apple, Amazon, Tesla, Palantir, DocuSign, Exxon & more https://t.co/QPN8GwL7Uh", 'label': 0}
```

Likewise, let's print out the last row in the `train` split:

```
print(train_data[-1])
```

You should now see the following:

```
{'text': "Brazil's Petrobras says it signed a $1.25 billion sustainability loan https://t.co/X9iTvKtKtj https://t.co/hCKnxYi8AA", 'label': 3}
```

The label is a number referencing the various topics. You can get this list from the dataset's page on Hugging Face's web site. You can define the list of topics using a dictionary as shown in Listing 5.8.

**Listing 5.8 Defining the list of topics for the Twitter Financial News Topic dataset**

```

topics = {
    "LABEL_0": "Analyst Update",
    "LABEL_1": "Fed | Central Banks",
    "LABEL_2": "Company | Product News",
    "LABEL_3": "Treasuries | Corporate Debt",
    "LABEL_4": "Dividend",
    "LABEL_5": "Earnings",
    "LABEL_6": "Energy | Oil",
    "LABEL_7": "Financials",
    "LABEL_8": "Currencies",
    "LABEL_9": "General News | Opinion",
    "LABEL_10": "Gold | Metals | Materials",
    "LABEL_11": "IPO",
    "LABEL_12": "Legal | Regulation",
    "LABEL_13": "M&A | Investments",
    "LABEL_14": "Macro",
    "LABEL_15": "Markets",
    "LABEL_16": "Politics",
    "LABEL_17": "Personnel Change",
    "LABEL_18": "Stock Commentary",
    "LABEL_19": "Stock Movement",
}

```

Using this dictionary, you can now create a mapping for all the labels in the train subset:

```

mapped_labels = [topics[f"LABEL_{label}"]]
                  for label in train_data['label']]

```

Using this dataset, we want to see the distribution of all the topics and examine which topic has the most data and which has the least. You can now plot a bar chart showing the distribution as shown in Listing 5.9.

**Listing 5.9 Plotting the dataset using a bar chart**

```

import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 6))

bins = np.arange(len(topics) + 1) - 0.5                                #A
plt.hist(mapped_labels,
         bins = bins,
         edgecolor = 'black',
         color = 'skyblue',
         alpha = 0.7)

plt.xticks(np.arange(len(topics)),                                     #B
          list(topics.values()),
          rotation = 90,
          ha = 'center')

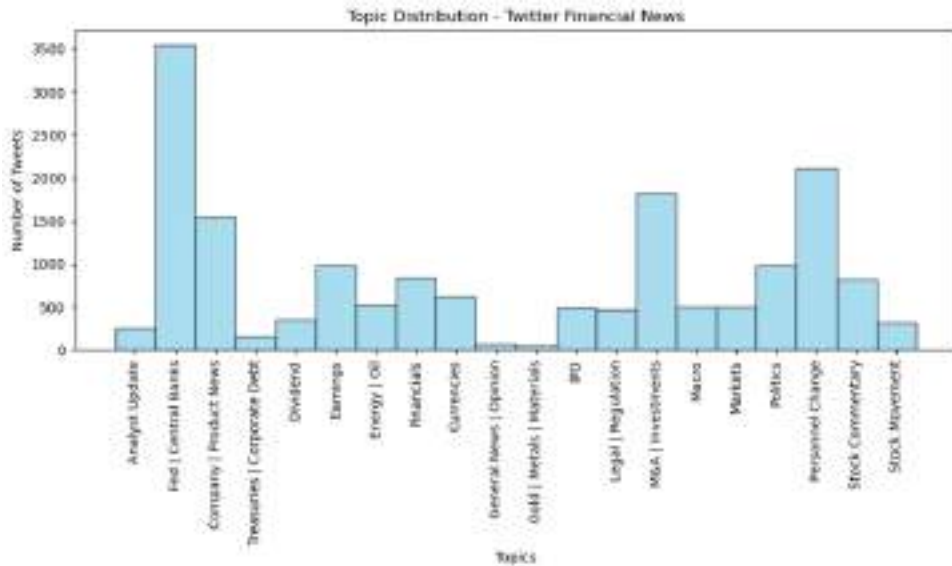
plt.title("Topic Distribution - Twitter Financial News")
plt.xlabel("Topics")
plt.ylabel("Number of Tweets")
plt.tight_layout()
plt.show()

```

**#A** Create bin edges to center the labels

**#B** Set the ticks at the center of each bin

Figure 5.13 shows the bar chart showing the distribution of the topics.

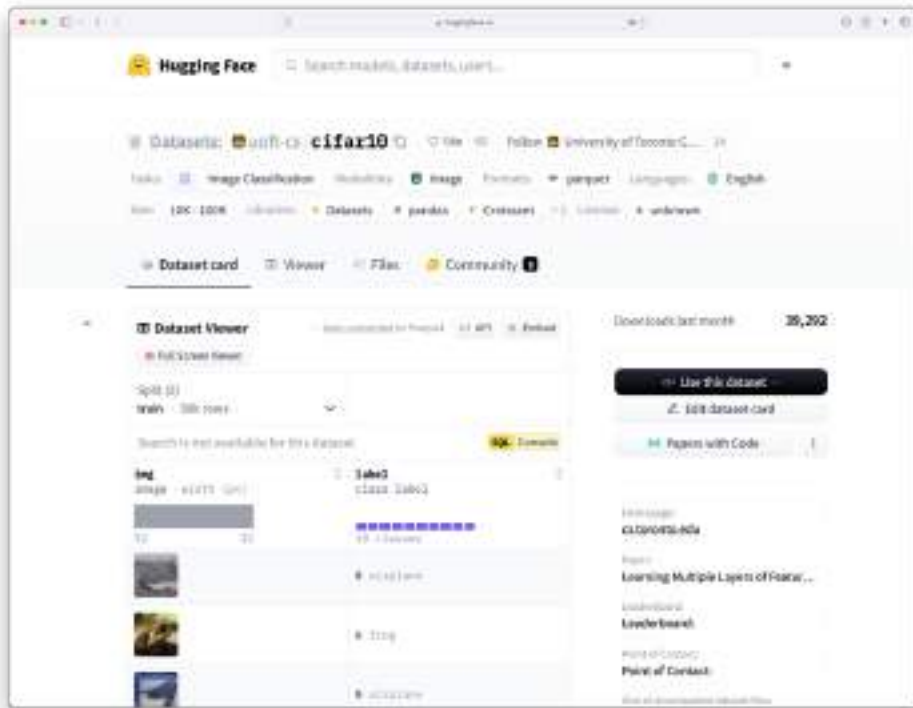


**Figure 5.13** You can visualize the various topics and the number of news related to each

### 5.3.2 Using the CIFAR-10 dataset

CIFAR-10 is a widely used dataset in machine learning and computer vision, containing 60,000 labelled 32x32 color images divided into 10 classes, such as airplanes, automobiles, and animals, with 50,000 for training and 10,000 for testing. Its simplicity and size make it a popular benchmark for image classification models, especially convolutional neural networks (CNNs), and an excellent resource for educational projects.

You can use Hugging Face's datasets library to experiment with CIFAR-10, and visualizations using matplotlib can help in better understanding the data. To illustrate this, we will use the 'uoft-cs/cifar10' dataset (<https://huggingface.co/datasets/uoft-cs/cifar10>; see also figure 5.14).



**Figure 5.14** The dataset page for CIFAR-10 on Hugging Face

As usual, let's start by downloading the CIFAR-10 dataset:

```
from datasets import load_dataset
import matplotlib.pyplot as plt
import numpy as np

dataset = load_dataset('uoft-cs/cifar10')
print(dataset)
```

**#A** Load the CIFAR-10 dataset

Printing the dataset downloaded will show the following:

```
DatasetDict({
  train: Dataset({
    features: ['img', 'label'],
    num_rows: 50000
  })
  test: Dataset({
    features: ['img', 'label'],
    num_rows: 10000
  })
})
```

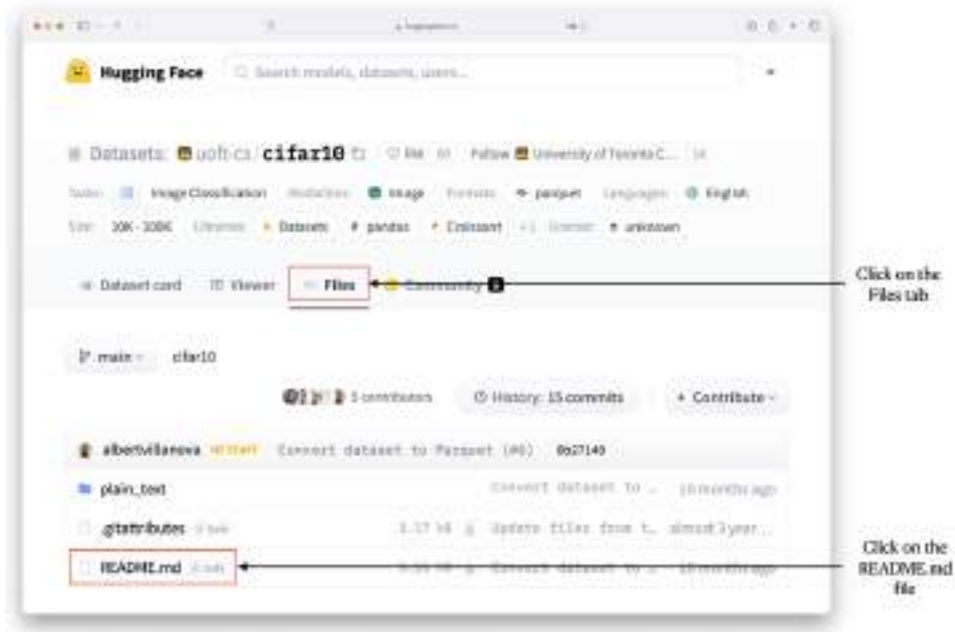
All the 60,000 images are stored in the dataset. For example, the first image in the *train* split can be accessed as follows:

```
dataset['train'][0]
```

You will see the following output (formatted for clarity):

```
{
  'img': <PIL.PngImagePlugin.PngImageFile image mode=RGB size=32x32>,
  'label': 0
}
```

Unfortunately, the page for the `uoft-cs/cifar10` dataset does not contain the labels for the dataset. Instead, you need to click on the Files tab and then the README.md file (see figure 5.15) to view its content. You can also view this page directly using this URL: <https://huggingface.co/datasets/uoft-cs/cifar10/blob/main/README.md>.



**Figure 5.15** The page for the CIFAR-10 dataset on Hugging Face

Listing 5.10 shows the content of the README.md file, with the key labelled `names` and their corresponding values, which map numbers to labels.

#### Listing 5.10 Content of the README.md file

```

annotations_creators:
  - crowdsourced
language_creators:
  - found
language:
  - en
license:
  - unknown
multilinguality:
  - monolingual
size_categories:
  - 10K<n<100K
source_datasets:
  - extended|other-80-Million-Tiny-Images
task_categories:
  - image-classification
task_ids: []
paperswithcode_id: cifar-10

```

```

pretty_name: Cifar10
dataset_info:
  config_name: plain_text
  features:
    - name: img
      dtype: image
    - name: label
      dtype:
        class_label:
          names:
            '0': airplane
            '1': automobile
            '2': bird
            '3': cat
            '4': deer
            '5': dog
            '6': frog
            '7': horse
            '8': ship
            '9': truck
  splits:
    - name: train
      num_bytes: 113648310
      num_examples: 50000
    - name: test
      num_bytes: 22731580
      num_examples: 10000
  download_size: 143646105
  dataset_size: 136379890
configs:
  - config_name: plain_text
    data_files:
      - split: train
        path: plain_text/train-*
      - split: test
        path: plain_text/test-*
    default: true

```

With these labels, you can define a dictionary and create a function called `show_images()` to display a 5x5 grid of images, each accompanied by its corresponding label (see Listing 5.11).



**Listing 5.11 Displaying a 5x5 grid of images**

```

labels = { #A
    0: "airplane",
    1: "automobile",
    2: "bird",
    3: "cat",
    4: "deer",
    5: "dog",
    6: "frog",
    7: "horse",
    8: "ship",
    9: "truck"
}

def show_images(images, labels, labels_dict): #B
    plt.figure(figsize=(5, 5))
    for i in range(25):
        plt.subplot(5, 5, i + 1)
        plt.imshow(images[i])
        plt.title(labels_dict[labels[i]]) #C
        plt.axis('off')
    plt.tight_layout()
    plt.show()

#D
train_samples = dataset['train'].shuffle(seed=42).select(range(25)) #E

#F
images = [sample['img'] for sample in train_samples]
class_labels = [sample['label'] for sample in train_samples]

#G
show_images(images, class_labels, labels)

```

**#A Define labels for the CIFAR-10 classes**

**#B Visualize a few images from the training set**

**#C Use the labels dictionary to get class names**

**#D Get some samples from the training dataset**

**#E Randomly select 25 samples**

**#F Extract images and labels**

**#G Display the images with class names**

Figure 5.16 shows the grid of 5x5 random images from the CIFAR-10 dataset.



**Figure 5.16** 25 random images from the CIFAR-10 dataset

## 5.4 Summary

- Hugging Face Datasets is an open-source library designed for accessing and processing large datasets commonly used in machine learning and data science.
- Use the `list_datasets()` function to show a list of available datasets.
- Use the `load_dataset()` function to download a dataset.
- Datasets are usually split into subsets, such as train, test, unsupervised, and more.
- For large datasets, you can stream the data and download one row at a time.
- You can also get the Parquet version of the dataset from Hugging Face.

- Tokenization is a foundational process in Natural Language Processing (NLP) that breaks down text into manageable units or tokens.
- Subword-level tokenization is the most common tokenization method used by newer models such as BERT and GPT.
- Tokenizing a dataset allows it to be used for applications such as fine-tuning a model.
- Visualizing a dataset allows you to gain a better understanding of its structure, distribution, and underlying patterns.

## ***6 Fine-Tuning Pre-Trained Models and Working with Multimodal Models***

### **This chapter covers**

- Using the *yelp\_polarity* dataset to fine-tune a pre-trained model
- Using a fine-tuned model to perform classification tasks
- Fine-tuning a pre-trained model to perform multiclass classification tasks
- Working with multimodal models

Up until this chapter, you have seen how to work with pre-trained models from Hugging Face to tackle a variety of tasks, leveraging their general capabilities for tasks such as text classification, object detection, and language generation. Now, we will delve into the process of fine-tuning these models to adapt them for more specialized tasks, enhancing their performance by training them on domain-specific data.

We will also explore multimodal models, which combine multiple types of data—such as images and text—to address more complex tasks (like identifying the type of animals in an image based on both visual features and descriptive text) that require the integration of different information sources. By the end of this chapter, you will have a solid understanding of how to fine-tune models for better task-specific accuracy and how to work with models that handle multimodal inputs for richer, more comprehensive solutions.

## 6.1 Fine-Tuning Pre-Trained Models

Fine-tuning is a machine learning technique where a pre-trained model, which has already learned general patterns from a large dataset, is further trained on a smaller, domain-specific dataset to adapt it for a particular task. This process leverages the knowledge the model has gained from the initial training, enabling it to perform well with less data (fewer examples or a smaller dataset specific to the new task) and computational resources. Fine-tuning typically involves adjusting only the later layers of the model or applying a lower learning rate to avoid losing the valuable features learned during the pre-training phase. It is commonly used in natural language processing and computer vision tasks, allowing models like transformers and convolutional neural networks (CNNs) to be customized for specific applications, such as sentiment analysis or object detection, without needing to train from scratch.

Let's learn how to fine-tune a pre-trained model from Hugging Face to perform sentiment analysis of restaurant reviews.

### 6.1.1 Loading the yelp\_polarity Dataset

To illustrate the process of fine-tuning a model, we will use the *yelp\_polarity* Dataset, available through Hugging Face's datasets library. This labeled dataset is derived from the larger Yelp Dataset Challenge and is specifically tailored for text classification tasks such as sentiment analysis. It consists of Yelp reviews that express detailed opinions about businesses like restaurants, hotels, and services. The dataset includes two labels: 0 for negative sentiment and 1 for positive sentiment, making it well-suited for binary classification tasks.

Let's first load the dataset and examine its content:

```
from datasets import load_dataset

dataset = load_dataset("yelp_polarity")           #A
print(dataset)                                   #B
```

#A load the Yelp dataset (full review dataset with train and test splits)

#B inspect the dataset to understand the structure

The content of the dataset is as follows:

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 560000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 38000
  })
})
```

Observe that the train split dataset has 560,000 rows and the test split has 38,000 rows. It would be useful to look at the first row in the train split:

```
train_dataset = dataset['train']          #A
print(train_dataset[0])                  #B
```

#A Access the train split  
#B Print the first example

Listing 6.1 shows the text of the first row together with the label of 0, which indicates that this is a negative review.

#### Listing 6.1 The content of the first row of the train split of the *yelp\_polarity* dataset

```
{
  'text': "Unfortunately, the frustration of being Dr.
    Goldberg's patient is a repeat of the experience
    I've had with so many other doctors in NYC -
    good doctor, terrible staff. It seems that his
    staff simply never answers the phone. It usually
    takes 2 hours of repeated calling to get an answer.
    Who has time for that or wants to deal with it? I
    have run into this problem with many other doctors
    and I just don't get it. You have office workers,
    you have patients with medical needs, why isn't
    anyone answering the phone? It's incomprehensible
    and not work the aggravation. It's with regret
    that I feel that I have to give Dr. Goldberg 2
    stars.",
  'label': 0
}
```

### 6.1.2 Filtering the yelp\_polarity Dataset

Before using the *yelp\_polarity* dataset to fine-tune your model, consider the following points:

1. **Topic Variety:** The dataset includes reviews on a wide range of topics, not just restaurants. If your goal is to fine-tune a model specifically for restaurant reviews, it's recommended to filter the dataset to include only those reviews related to restaurants.
2. **Dataset Size:** The dataset can be quite large, which might pose challenges during fine-tuning. While having more data is generally beneficial, it's often practical to limit the dataset size. For example, using a subset of around 5,000 reviews can make training more manageable without compromising performance significantly.

Listing 6.2 demonstrates how to filter the dataset to include only rows containing the word "restaurant" and then extract a subset of 5,000 rows.

**Listing 6.2 Filtering the *yelp\_polarity* dataset**

```

train_dataset = dataset["train"] #A
test_dataset = dataset["test"] #A

restaurant_train_reviews = train_dataset.filter( #B
    lambda x: "restaurant" in x["text"].lower()
)

restaurant_test_reviews = test_dataset.filter( #B
    lambda x: "restaurant" in x["text"].lower()
)

number_of_reviews = 5000
subset_train_reviews = restaurant_train_reviews.shuffle( #C
    seed = 42).select(range(number_of_reviews))
subset_test_reviews = restaurant_test_reviews.shuffle( #C
    seed = 42).select(range(number_of_reviews))

subset_dataset = { #D
    "train": subset_train_reviews,
    "test": subset_test_reviews
}

from datasets import DatasetDict
yelp_restaurant_dataset = DatasetDict(subset_dataset) #E

print(yelp_restaurant_dataset) #F

```

**#A** Select the "train" and "test" splits

**#B** Filter for restaurant-related reviews in the train and test datasets

**#C** Shuffle and get 5000 rows

**#D** Create a DatasetDict to return both train and test datasets

**#E** Display the structure to match the requested format

**#F** Print the dataset structure

The reduced dataset now contains 5,000 rows each for the training and test splits:



```
DatasetDict({
  train: Dataset({
    features: ['label', 'text'],
    num_rows: 5000
  })
  test: Dataset({
    features: ['label', 'text'],
    num_rows: 5000
  })
})
```

Let's verify the first row of the reduced dataset by viewing its content:

```
yelp_restaurant_dataset['train'][0]
```

You will see a review related to a restaurant that expresses a negative sentiment:

```
{
  'text': 'My girlfriend and I have been wanting to come
          here for awhile, we finally came & we had the
          worst experience ever. We asked our server for
          a few minutes to look over the menu & he never
          came back. 15 minutes later, someone finally
          came and took our order. We waited awhile and
          when they brought our food, they got the whole
          order wrong. My girlfriend ordered soup and it
          never came out. Worst service ever. Would not
          recommend this restaurant to anyone.',
  'label': 0
}
```

### 6.1.3 Tokenizing the Reduced Dataset

With the reduced dataset, you can now perform tokenization on it using the *'distilbert-base-uncased'* model (see listing 6.3).

**Listing 6.3 Performing tokenization on the dataset**

```

from transformers import AutoTokenizer

model_checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)      #A

def tokenize_function(examples):                                #B
    return tokenizer(examples["text"],
                      padding = "max_length",
                      truncation = True,
                      max_length = 512)

tokenized_datasets = yelp_restaurant_dataset.map(              #C
    tokenize_function,
    batched=True)

tokenized_datasets

```

**#A** load a pre-trained model and tokenizer - DistilBERT for sentiment classification

**#B** function to tokenize the dataset

**#C** apply the tokenization function to the dataset

After the tokenization, you should see the following result:

```

DatasetDict({
  train: Dataset({
    features: ['text', 'label', 'input_ids', 'attention_mask'],
    num_rows: 5000
  })
  test: Dataset({
    features: ['text', 'label', 'input_ids', 'attention_mask'],
    num_rows: 5000
  })
})

```

### 6.1.4 Setting Up a Pre-trained Model for Sequence Classification

Let's now load a pre-trained model (*distilbert-base-uncased*) so that you can fine-tune it to perform sentiment analysis on restaurant reviews (see listing 6.4).

**Listing 6.4 Loading a pre-trained model for sequence classification**

```

from transformers import AutoModelForSequenceClassification
import torch

model = AutoModelForSequenceClassification.from_pretrained(      #A
    model_checkpoint, num_labels = 2)

if torch.backends.mps.is_available():                          #B
    device = torch.device("mps")
else:
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu")

model.to(device)                                              #C

```

**#A** load pre-trained model for sequence classification

**#B** determine the device

**#C** move the model to the selected device

The `AutoModelForSequenceClassification` class in Hugging Face's transformers library is a versatile model loader specifically tailored for sequence classification tasks. It enables you to load any pre-trained model architecture compatible with sequence classification using a provided model checkpoint. This class automatically appends the necessary classification layers to the pre-trained model, making it ready for tasks such as sentiment analysis, spam detection, or topic classification.

**WHAT IS A SEQUENCE CLASSIFICATION TASK?**

Sequence classification tasks involve assigning a single label or category to an entire sequence of data, such as a sentence, paragraph, or even a longer sequence of tokens.

In the above code listing 6.3, we also implement logic to detect the runtime environment. This includes checking whether the code is running on a macOS system with MPS enabled or on a Windows machine with CUDA support. If a GPU is detected, the model is transferred to the GPU to leverage accelerated computation.

## MPS AND CUDA

Metal Performance Shaders (MPS) is Apple's framework for GPU-accelerated computations on macOS devices with Apple Silicon (M1/M2/M3/M4). It is optimized for machine learning tasks and leverages the Metal API to provide efficient, high-performance computing for deep learning models. MPS enables native support for PyTorch and TensorFlow, allowing developers to train and infer models on Apple hardware.

Compute Unified Device Architecture (CUDA) is NVIDIA's parallel computing platform and API for leveraging NVIDIA GPUs for general-purpose processing. Widely adopted in the machine learning community, CUDA enables frameworks like PyTorch and TensorFlow to run deep learning models with unparalleled speed and efficiency. It supports an extensive ecosystem of tools for training, optimization, and deployment.

Both MPS and CUDA are essential for accelerating deep learning workflows, tailored to their respective hardware ecosystems.

When you run the code snippet, you will see the architecture of the `DistilBertForSequenceClassification` model as shown in listing 6.5.

### Listing 6.5 Architecture of the `DistilBertForSequenceClassification` model

```
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSdpaAttention(
            (dropout): Dropout(p=0.1, inplace=False)
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12,
            elementwise_affine=True)
          (ffn): FFN(
```

```

        (dropout): Dropout(p=0.1, inplace=False)
        (lin1): Linear(in_features=768, out_features=3072, bias=True)
        (lin2): Linear(in_features=3072, out_features=768, bias=True)
        (activation): GELUActivation()
    )
    (output_layer_norm): LayerNorm((768,), eps=1e-12,
        elementwise_affine=True)
)
)
)
)
(pre_classifier): Linear(in_features=768, out_features=768, bias=True)
(classifier): Linear(in_features=768, out_features=2, bias=True)
(dropout): Dropout(p=0.2, inplace=False)
)

```

### 6.1.5 Configuring and Initializing a Trainer for Fine-Tuning a Pre-Trained Model

To fine-tune a pre-trained model on a dataset, you can utilize the `Trainer` and `TrainingArguments` classes from Hugging Face. The code snippet in listing 6.6 demonstrates how to set up the `Trainer` and `TrainingArguments` classes to train the model using the training split of the tokenized dataset and evaluate the model using the test split of the tokenized dataset.

**Listing 6.6 Setting up the TrainingArguments and Trainer classes for fine-tuning**

```

from transformers import Trainer, TrainingArguments

training_args = TrainingArguments(                                #A
    output_dir = "./results",                                     #B
    eval_strategy = "epoch",                                     #C
    save_strategy = "epoch",                                     #D
    learning_rate = 2e-5,                                        #E
    per_device_train_batch_size = 16,                             #F
    per_device_eval_batch_size = 16,                             #G
    num_train_epochs = 3,                                         #H
    weight_decay = 0.01,                                         #I
    logging_dir = "./logs",                                       #J
    logging_steps = 10,                                           #K
    save_steps = 500,                                             #L
    load_best_model_at_end = True,                                #M
)

trainer = Trainer(                                              #N
    model = model,
    args = training_args,
    train_dataset = tokenized_datasets["train"],
    eval_dataset = tokenized_datasets["test"],
)

trainer.train()                                                #O

```

#A Set up training arguments

#B Directory to save results

#C Evaluate model after each epoch

#D Save the model after each epoch

#E Learning rate

#F Batch size for training

#G Batch size for evaluation

#H Number of training epochs

#I Weight decay for regularization

#J Directory for logs

#K Log every 10 steps

#L Save the model every 500 steps

#M Load the best model at the end of training

#N set up the Trainer

#O fine-tune the model

The training process duration will vary based on your machine's configuration and resources. It can take anywhere from 10 minutes to an hour, so patience is key (you will know roughly how much time is needed during the training). If the training is taking too long, consider reducing the dataset size from 5,000 rows to a smaller number, such as 1,000, to speed up the process. Once the training is complete, you will see the training loss and validation loss for each epoch (iteration) of the process (note that your results may vary; see figure 6.1). These values provide insights into how well the model is learning and generalizing to unseen data, with the training loss indicating the model's performance on the training data, and the validation loss showing its performance on the validation set.

| Epoch | Training Loss | Validation Loss |
|-------|---------------|-----------------|
| 1     | 0.167500      | 0.171618        |
| 2     | 0.039400      | 0.219457        |
| 3     | 0.048500      | 0.218590        |

**Figure 6.1 The result printed at the end of the training**

With the model trained, it would be useful to save it to disk so that you can use it at a later time without going through the same training process again:

```
model.save_pretrained("./results/final_model")           #A
tokenizer.save_pretrained("./results/final_tokenizer")   #A
```

**#A save the fine-tuned model and tokenizer**

Also, you can evaluate the model and print its result:

```
eval_results = trainer.evaluate()                         #A
print(f"Evaluation results: {eval_results}")
```

**#A evaluate the model**

Here is the evaluation report printed for the training:

```
Evaluation results: {'eval_loss': 0.1825547218322754,
                    'eval_runtime': 63.1454,
                    'eval_samples_per_second': 79.182,
                    'eval_steps_per_second': 4.957,
                    'epoch': 3.0}
```

Here's what to make of the result:

- Evaluation Loss: 0.18255 – This represents the model's loss during evaluation, with lower values indicating better performance. This is a relatively low loss value, which generally indicates good performance, especially if you're working on a classification task like sentiment analysis.
- Evaluation Runtime: 63.1454 seconds – The total time taken to run the evaluation.
- Evaluation Samples per Second: 79.182 – The number of samples processed per second during evaluation. This value reflects the model's processing speed during evaluation, which seems decent, but it mainly inform you about the efficiency rather than quality.
- Evaluation Steps per Second: 4.957 – The number of evaluation steps completed per second. Just like evaluation samples per second, this value informs you about the efficiency rather than quality.
- Epoch: 3.0 – The results are from the third epoch of training. You can try varying this value to monitor how the loss evolves with more training.

The evaluation results suggest that the model is performing reasonably well.

### 6.1.6 Using the Fine-Tuned Model

With the fine-tuned model trained and saved, you can now use it to perform a sentiment analysis on a new restaurant review. Listing 6.7 shows how to load the fine-tuned model, move the model and the inputs to the GPU if available, and then derive the result.

**Listing 6.7 Using the fine-tuned model**

```
from transformers import AutoTokenizer, \
                        AutoModelForSequenceClassification
import torch

new_model = AutoModelForSequenceClassification.from_pretrained(  #A
    "./results/final_model")
new_tokenizer = AutoTokenizer.from_pretrained(  #A
    "./results/final_tokenizer")

new_model.to(device)  #B
```



```

sentence = '''
I had an amazing experience dining at this restaurant last night.
From the moment we walked in, the staff made us feel welcomed and
were incredibly attentive. Our server was friendly, knowledgeable,
and made great recommendations from the menu.

The food was absolutely delicious. I had the grilled salmon, and
it was cooked to perfection—tender, flavorful, and served with a
lovely citrus glaze that complemented it beautifully. The roasted
vegetables on the side were fresh and perfectly seasoned. My
partner had the pasta, which was creamy and rich in flavor, with
just the right amount of spice.

The ambiance was warm and inviting, with cozy lighting and tasteful
decor. It was the perfect place to relax and enjoy a nice meal. The
dessert, a decadent chocolate lava cake, was the perfect way to end
the meal.

Overall, this restaurant exceeded my expectations in every way.
Excellent food, exceptional service, and a wonderful atmosphere.
I'll definitely be back and highly recommend it to anyone looking
for a great dining experience.
'''

inputs = new_tokenizer(sentence,                                     #C
                        return_tensors = "pt",
                        padding = True,
                        truncation = True,
                        max_length = 512)

inputs = {key: value.to(device) for key, value in inputs.items()} #D

new_model.eval()                                                  #E

with torch.no_grad():
    outputs = new_model(**inputs)                                  #F
logits = outputs.logits                                           #G
probabilities = torch.nn.functional.softmax(logits, dim=-1)      #H
predicted_class = torch.argmax(probabilities, dim=-1).item()      #I

if predicted_class == 1:                                          #J
    print(f"Sentiment: Positive (Confidence: \
          {probabilities[0][1].item():.2f})")

```

```
else:
    print(f"Sentiment: Negative (Confidence: \
          {probabilities[0][0].item():.2f})")
```

```
#A Reload the model and tokenizer
#B Move the inference to GPU
#C Tokenize the input sentence
#D Move inputs to GPU/MPS
#E Put the model in evaluation mode
#F Run the model to get predictions
#G Get the logits (raw scores) from the model output
#H Convert logits to probabilities using softmax
#I Get the predicted class (index of the maximum probability)
#J Output the predicted sentiment
```

The sample review provided in the code listing yields a positive sentiment with a confidence of 0.99:

```
Sentiment: Positive (Confidence: 0.99)
```

Let's modify the content of the sentence variable with a negative review and examine the result:

```

sentence = '''
I visited this place last night with high expectations after
hearing some good things, but it was honestly one of the worst
dining experiences I've had in a while. The service was
incredibly slow, even though the restaurant wasn't crowded.
Our waiter seemed disinterested and forgot half of our order.

When the food finally came, it was cold and tasted bland. The
pasta was overcooked, and my steak was underseasoned and chewy.
The side of vegetables looked like they had been reheated from
a previous meal.

To make things worse, the ambiance was far too noisy, and we
had to wait an extra 20 minutes for the check. I tried to
address my concerns with the manager, but they seemed
uninterested in hearing feedback. Overall, I felt like I had
wasted both my time and money.

I will definitely not be coming back, and I would not recommend
this place to anyone.
'''

```

This time, it returns a negative sentiment:

```
Sentiment: Negative (Confidence: 0.99)
```

### 6.1.7 Fine-Tuning Models for Multiclass Text Classification

Unlike the *yelp\_polarity* dataset, which uses binary labels (0 or 1) for sentiment classification, the *yelp\_review\_full* dataset contains labels corresponding to a star rating system ranging from 1 to 5. Each review is assigned a star rating that reflects the user's sentiment about the business:

1. Very negative sentiment
2. Negative sentiment
3. Neutral sentiment
4. Positive sentiment
5. Very positive sentiment

These ratings provide a more granular view of sentiment compared to the binary labels in the *yelp\_polarity* dataset. This makes the *yelp\_review\_full* dataset suitable for tasks such as multi-class sentiment classification or regression, where the goal is to predict one of five sentiment categories based on the review text.

Let's load the *yelp\_review\_full* dataset to examine its content:

```
from datasets import load_dataset

dataset = load_dataset("yelp_review_full")           #A
print(dataset)                                     #B
```

**#A Load Yelp Reviews dataset with ratings from 1 to 5**

**#B Display the structure of the dataset**

You will see the following output:

```
DatasetDict({
  train: Dataset({
    features: ['label', 'text'],
    num_rows: 650000
  })
  test: Dataset({
    features: ['label', 'text'],
    num_rows: 50000
  })
})
```

There are 650,000 rows in the train split and 50,000 in the test split. As usual, let's filter the dataset to only contain reviews that are related to restaurants (see listing 6.8).

**Listing 6.8 Filtering the *yelp\_review\_full* dataset to contain only rows related to restaurants**

```

from datasets import DatasetDict

train_dataset = dataset["train"]           #A
test_dataset = dataset["test"]             #A

restaurant_train_reviews = train_dataset.filter(           #B
    lambda x: "restaurant" in x["text"].lower()
)

restaurant_test_reviews = test_dataset.filter(
    lambda x: "restaurant" in x["text"].lower()
)

number_of_reviews = 5000                      #C
subset_train_reviews = restaurant_train_reviews.shuffle(
    seed=42).select(range(number_of_reviews))
subset_test_reviews = restaurant_test_reviews.shuffle(
    seed=42).select(range(number_of_reviews))

subset_dataset = {                               #D
    "train": subset_train_reviews,
    "test": subset_test_reviews
}

yelp_restaurant_dataset = DatasetDict(subset_dataset)      #E

print(yelp_restaurant_dataset)                          #F

```

**#A** select the "train" and "test" splits

**#B** filter for restaurant-related reviews in the train and test datasets

**#C** only use 5000 reviews for training

**#D** create a DatasetDict to return both train and test datasets

**#E** display the structure to match the requested format

**#F** Print the dataset structure

The reduced dataset now looks like this:

```
DatasetDict({
  train: Dataset({
    features: ['label', 'text'],
    num_rows: 5000
  })
  test: Dataset({
    features: ['label', 'text'],
    num_rows: 5000
  })
})
```

Let's take a look at the first row of the train split:

```
yelp_restaurant_dataset['train'][0]
```

The output looks like this:

```
{'label': 2,
 'text': "This place is good, but I think I just ordered
         the wrong thing. The Hibiscus Enchiladas were
         just way too sweet for me. I think they should
         be on the dessert menu and not dinner menu. I'd
         like to give it another chance and order
         something else next time, but other than that
         a good vibe. Seemed like the typical American
         Mexican restaurant."}
```

As explained, the label is a number from 1 to 5 representing the sentiment level.

Just like what we have done earlier, you can now tokenize the reduced dataset like what we have done earlier in listing 6.3. With the dataset tokenized, the next step would be to use a pre-trained model and fine-tune it to perform sentiment analysis, just like in listing 6.4. However, since there are now five different labels (1 to 5) instead of 0 and 1, you need to set the `num_labels` parameter to 5 instead of 2:

```
model = AutoModelForSequenceClassification.from_pretrained( #A
    model_checkpoint,
    num_labels = 5)
```

**#A load pre-trained model for sequence classification**

You now proceed with fine-tuning the model (the code is same as listing 6.6).

When the model is fine-tuned, save the model to a new folder named `final_model_multiclass` and the tokenizer to `final_tokenizer_multiclass`:

```
model.save_pretrained("./results/final_model_multiclass")      #A
tokenizer.save_pretrained("./results/final_tokenizer_multiclass") #A
eval_results = trainer.evaluate()                               #B
print(eval_results)                                           #C
```

#A Save the fine-tuned model and tokenizer

#B Evaluate the model on the test set

#C Print the evaluation results

You can refer to the source code that accompanies this book for the complete source code.

Once the training is complete, you will see the training loss and validation loss for each epoch (iteration) of the process (see figure 6.2).

| Epoch | Training Loss | Validation Loss |
|-------|---------------|-----------------|
| 1     | 1.066800      | 0.962330        |
| 2     | 0.799900      | 0.917825        |
| 3     | 0.773900      | 0.927565        |

**Figure 6.2 The result printed at the end of the training**

The results of the evaluation is as follows:

```
{
  'eval_loss': 0.9178248047828674,
  'eval_runtime': 63.0299,
  'eval_samples_per_second': 79.327,
  'eval_steps_per_second': 4.966,
  'epoch': 3.0
}
```

You can now load the fine-tuned model and use it to perform multiclass sentiment analysis (see listing 6.9).

**Listing 6.9 Using the fine-tuned model to perform multi-class sentiment analysis**

```
from transformers import AutoModelForSequenceClassification
from transformers import AutoTokenizer
import torch

new_reviews = [
    "The food was amazing and the service was excellent!",
    "The restaurant was dirty and the food was cold.",
    "Decent experience, but nothing special."
]

if torch.backends.mps.is_available(): #A
    device = torch.device("mps")
else:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

new_tokenizer = AutoTokenizer.from_pretrained( #B
    "./results/final_tokenizer_multiclass")

inputs = new_tokenizer(new_reviews, #C
    padding = "max_length",
    truncation = True,
    return_tensors = "pt")

inputs = {key: value.to(device) for key, value in inputs.items()} #D

new_model = AutoModelForSequenceClassification.from_pretrained( #E
    "./results/final_model_multiclass")
new_model.to(device)

new_model.eval()

with torch.no_grad(): #F
    outputs = new_model(**inputs)
    logits = outputs.logits
    predictions = torch.argmax(logits, dim=-1)

star_ratings = predictions + 1 #G
for review, rating in zip(new_reviews, star_ratings):
```



```
print(f"Review: {review}\nPredicted Star Rating: \
      {rating.item()}\n")
```

```
#A Determine the device
#B Load the tokenizer
#C Tokenize the reviews
#D Move inputs to GPU / MPS
#E Load the fine-tuned model
#F Perform inference
#G Assuming classes are 0-4, map to 1-5
```

In code listing 6.9, you had three sample restaurant reviews:

```
new_reviews = [
    "The food was amazing and the service was excellent!",
    "The restaurant was dirty and the food was cold.",
    "Decent experience, but nothing special."
]
```

Using the fine-tuned model, these are the predicted ratings:

```
Review: The food was amazing and the service was excellent!
Predicted Star Rating:          5

Review: The restaurant was dirty and the food was cold.
Predicted Star Rating:          1

Review: Decent experience, but nothing special.
Predicted Star Rating:          3
```

Looks like they are pretty accurate!

## 6.2 Working with Multimodal Models

So far, all the models that we have worked right till this point has been what we called *single modal models*. A single modal model is a machine learning model designed to work with data from a single modality, such as text, images, audio, or numerical data. For example:

- An NLP model such as GPT or BERT, is trained to work exclusively with textual data.
- A Convolutional Neural Network (CNN), such as ResNet, is trained to process and analyze visual data like images.

A multimodal model, on the other hand, is a machine learning model designed to process and integrate data from multiple modalities—different types of information such as text, images, audio, video, or structured data. By combining these modalities, multimodal models aim to learn richer and more comprehensive representations, allowing them to perform tasks that involve multiple types of inputs. Here are some example uses of multimodal models:

- Multimodal models can be used in image captioning, where they combine visual data (images) with natural language processing (text) to generate descriptive captions.
- Another example is visual question answering (VQA), where the model takes an image and a question as inputs and provides an answer by reasoning across both modalities.
- Similarly, in speech-to-text systems, audio data (speech) is processed and converted into textual information.
- Other real-world applications include multimodal sentiment analysis, where text, audio tone, and visual cues (like facial expressions) are combined to assess sentiment, and autonomous driving systems, where data from cameras (images), lidar (structured data), and GPS are integrated to understand the environment and make driving decisions.

Multimodal models have several advantages over single-modal models. They combine different types of data, like text, images, and audio, for better understanding and can handle noise or missing information in one type by relying on others. This improves accuracy, generalization, and performance across tasks like image captioning and visual question answering. By using multiple data sources together, they reduce biases, mimic human decision-making, and handle complex tasks more effectively.

We will start with a quick recap of using a single-modal model and then explore how multimodal models operate.

### 6.2.1 Single-Modal Models

A good example of a single-modal model is the “facebook/detr-resnet-50” model. It is a DEtection TRansformer (DETR) model developed by Facebook AI and used for object detection and image segmentation, combining the strengths of Convolutional Neural Networks (CNNs) and Transformers. In the following example, we will use it to detect objects in an image. First, let’s load an image from the web:

```

from PIL import Image, ImageDraw
import requests

url = 'https://images.unsplash.com/' + \
      'photo-1563460716037-460a3ad24ba9'           #A
if url.startswith('http'):                         #B
    image = Image.open(requests.get(url, stream=True).raw)
else:
    image = Image.open(url)                         #C
image

#A Url of the image
#B if the image is from the web
#C the image is local

```

Figure 6.3 shows the picture containing a dog and a cat.



**Figure 6.3.** An image containing a dog and a cat (image source: <https://unsplash.com/photos/short-coated-white-and-brown-puppy-lvzo69e18nk>)

We will use the “facebook/detr-resnet-50” model to try to detect the dog and cat in the picture. Listing 6.10 shows how to do this.

**Listing 6.10 Using the "facebook/detr-resnet-50" model to detect objects in images**

```

from transformers import DetrImageProcessor, DetrForObjectDetection
import torch

image_processor = DetrImageProcessor.from_pretrained(
    "facebook/detr-resnet-50")
model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")

inputs = image_processor(images = image,                                #A
                        return_tensors = "pt")

model.eval()

with torch.no_grad():
    outputs = model(**inputs)                                           #B
target_sizes = torch.tensor([image.size[:-1]])                        #C

results = image_processor.post_process_object_detection(                #D
    outputs,
    target_sizes = target_sizes,
    threshold = 0.9)[0]
print(results)

```

**#A** Process the image so that the preprocessed data ((e.g., resizing, normalization, and converting to tensors)) can be used as inputs by the model

**#B** The outputs variable contains the raw predictions from the model, which include detected object information

**#C** Create the target size in the format of (height,width)

**#D** Converts the raw model outputs into human-readable object detection results

The model returned the following result (formatted for clarity):

```

{
  'scores': tensor([0.9924, 0.9989], grad_fn=<IndexBackward0>),
  'labels': tensor([17, 18]),
  'boxes': tensor([[4.5560e+00, 2.8760e+03, 2.4111e+03, 4.9051e+03],
                  [2.2206e+02, 1.4505e+03, 3.6523e+03, 4.6681e+03]],
                  grad_fn=<IndexBackward0>)
}

```

Using the result, you can plot bounding boxes around the detected objects using the code as shown in Listing 6.11.

**Listing 6.11 Plotting bounding boxes around detected object in an image**

```

draw = ImageDraw.Draw(image)

for score, label, box in zip(results["scores"],
                             results["labels"],
                             results["boxes"]):
    print(
        f"Detected {model.config.id2label[label.item()]} with confidence "
        f"{(score.item() * 100):.2f}% at {box}"
    )
    box = [round(i, 2) for i in box.tolist()]
    draw.rectangle(box,
                  outline = 'green',
                  width = 10)
    draw.text((box[0], box[1]-10),
              model.config.id2label[label.item()],
              fill = 'green')
display(image)

```

**#A** Print the object detected and the confidence

**#B** Draw bounding box around object

**#C** Display the object label

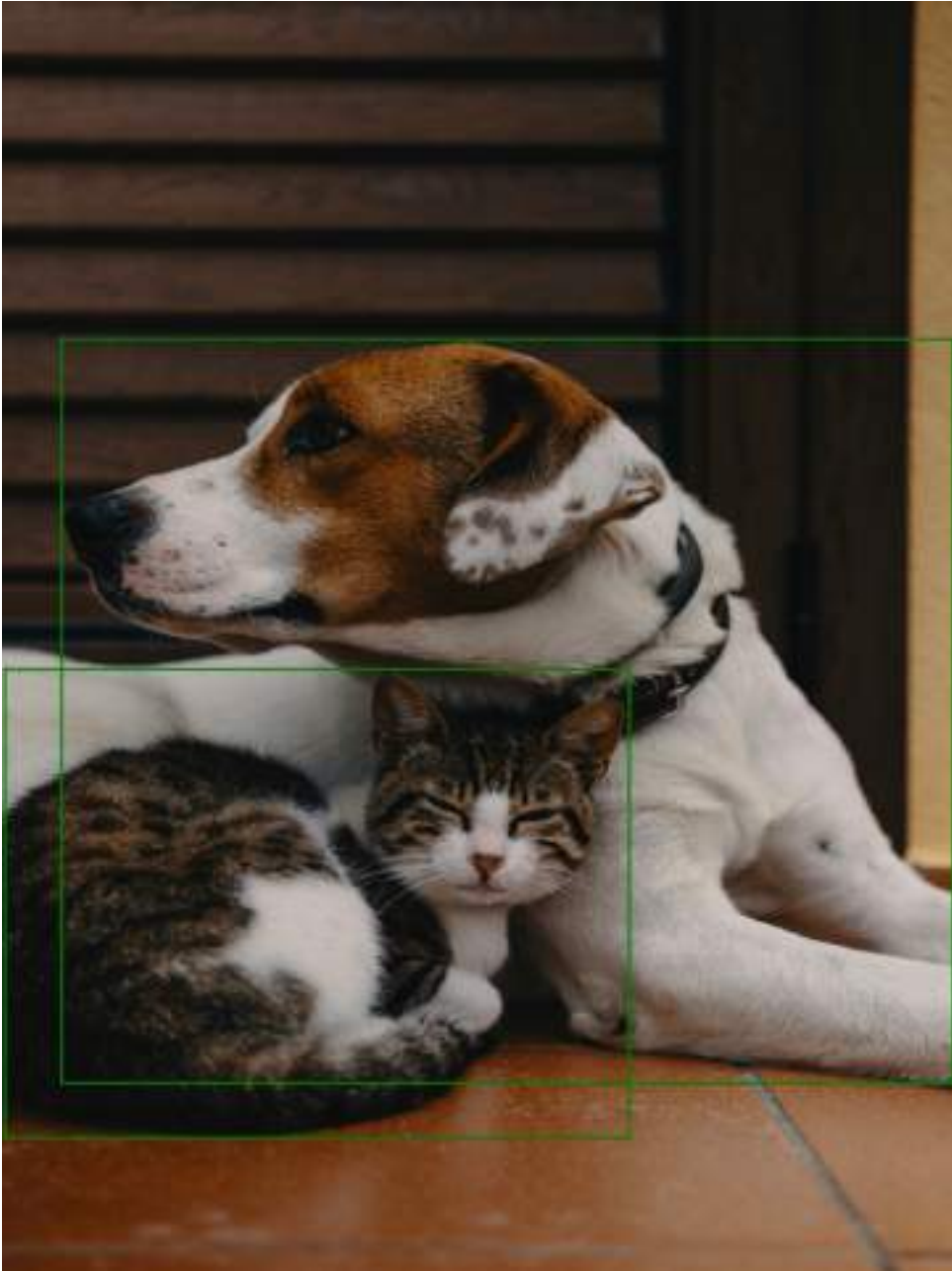
The code in Listing 6.11 prints out the following result:

```

Detected cat with confidence 99.47% at tensor(
  [1.4345e+00, 1.2385e+03,
   1.0472e+03, 2.1296e+03],
  grad_fn=<UnbindBackward0>)
Detected dog with confidence 99.92% at tensor(
  [96.8109, 630.3585,
   1587.2705, 2028.8726],
  grad_fn=<UnbindBackward0>)

```

Figure 6.4 displays the bounding boxes around the dog and cat, with the label for each detected object shown in the top-left corner of the respective box.



**Figure 6.4** The detected objects in the image with the bounding boxes drawn

This example demonstrates the use of a single-modal model, specifically the DETection TRansformer (DETR) model, for object detection. The model processes only visual data (an input image) to identify objects and their bounding boxes. You will soon see how a multimodal model can enhance this by integrating additional data types, such as text, to improve contextual understanding and provide more comprehensive insights for tasks like image captioning, visual question answering, and other complex applications that require the fusion of multiple modalities.

## 6.2.2 Multimodal Models

Now that you've seen how a single-modal model works to identify objects in an image, let's explore how a multimodal model like CLIP differs. In particular, let's use the Contrastive Language–Image Pre-training (CLIP) model to demonstrate how it can process both images and text together. Unlike a single-modal model, CLIP understands the relationship between images and textual descriptions, allowing it to perform tasks like image classification based on natural language labels. By leveraging both visual and textual data, CLIP can make more accurate predictions, handle more complex queries, and generalize across various tasks without needing task-specific training. Let's walk through how CLIP works and see how it can classify images based on a set of textual descriptions.

### WHAT IS CLIP

Contrastive Language–Image Pre-training (CLIP) is a multimodal model developed by OpenAI that connects text and image understanding. It can process and relate textual descriptions to visual content without requiring explicit fine-tuning for specific tasks. It can understand both text and images and establish meaningful relationships between them.

During training, CLIP aligns text descriptions with corresponding images while distinguishing them from unrelated pairs. For example, the caption "a dog playing in the park" is linked to an image of a dog but separated from images of cats or cars.

CLIP can perform tasks it wasn't specifically trained for by leveraging its broad understanding of text and images. For example, it can classify images based on user-provided text labels without requiring task-specific data.

CLIP uses two separate neural networks:

- A text encoder (transformer-based) to process text.
- An image encoder (based on models like ResNet or Vision Transformer) to process images.

First, let's load an image that we want to work. For this, we will load an image of a dog:



```
import torch
import requests
from PIL import Image

url = 'https://images.unsplash.com/' + \
      'photo-1491604612772-6853927639ef' #A

image = Image.open(requests.get(url, stream=True).raw)
display(image)

#A Url of the image
```

Figure 6.5 shows the image of a dog loaded from the web.



**Figure 6.5** An image of a dog (image source: <https://unsplash.com/photos/close-up-photo-of-black-and-white-siberian-husky-dog-NKN25UfGfkQ>)

For this example, you will use a multimodal model to simultaneously process both visual and textual data (see listing 6.12). The CLIP model is designed to learn the relationship between images and their corresponding textual descriptions. By using both an image and a list of labels (text), the model would be able to determine which label best matches the content of the image. This approach allows the model to not only analyze visual features but also to leverage contextual information from the text, providing a more powerful and versatile solution compared to single-modal models that only process one type of data (e.g., images).

#### Listing 6.12 Using a multimodal model to simultaneously process both visual and textual data

```
from transformers import CLIPProcessor, CLIPModel

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32") #A
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

labels = ["cat", "dog", "tiger", "train"]
inputs = processor(text = labels,                                #B
                  images = image,
                  return_tensors = "pt",
                  padding=True)

model.eval()

with torch.no_grad():
    outputs = model(**inputs)                                    #C

logits_per_image = outputs.logits_per_image                      #D

probs = logits_per_image.softmax(dim=1)                          #E
most_likely_index = torch.argmax(probs, dim=1).item()            #F
most_likely_object = labels[most_likely_index]

print(f"The most likely object is: {most_likely_object}")
```

**#A Download the CLIP model**

**#B** The inputs variable contains the data that is passed to the CLIP model for processing. It is the processed representation of both the text and image data, formatted in a way that the model can understand.

**#C** The outputs variable contains the result of processing the inputs through the CLIP model. The model processes the text and image data, and the outputs object contains the computed features and similarity scores.

**#D** This is the image-text similarity score

**#E** We can take the softmax to get the label probabilities

**#F** Get and print the most likely object

The above example is an example of a multimodal model in action. It involves processing and combining data from two different modalities: text and images. Specifically, the code uses the CLIP (Contrastive Language–Image Pretraining) model from OpenAI, which is designed to understand and compute the similarity between textual descriptions and images.

Here's how the code works:

- Step 1: You download the "*openai/clip-vit-base-patch32*" model and its corresponding processor using the `CLIPModel` and `CLIPProcessor` classes.
- Step 2: The input image (loaded using libraries like PIL) and a list of possible labels (`["cat", "dog", "tiger", "train"]`) are processed by the `CLIPProcessor`, which converts them into a format suitable for the model (token embeddings for text and normalized tensors for the image).
- Step 3: The model processes both the image and the text inputs.
  - It extracts visual features from the image.
  - It extracts textual features from the labels.
- Step 4: The model computes similarity scores between the image features and each label's text features, producing a set of logits (`logits_per_image`).
- Step 5: The logits are converted into probabilities using the `softmax()` function, and the label with the highest probability is selected as the most likely match for the image.

The example returns the following output:

The most likely object is: dog

## 6.3 Summary

- Fine-tuning is a machine learning technique where a pre-trained model, which has already learned general patterns from a large dataset, is further trained on a smaller, domain-specific dataset to adapt it for a particular task.
- The *yelp\_polarity* dataset is derived from the larger Yelp Dataset Challenge and is specifically tailored for text classification tasks such as sentiment analysis.
- Sequence classification tasks involve assigning a single label or category to an entire sequence of data, such as a sentence, paragraph, or even a longer sequence of tokens
- Metal Performance Shaders (MPS) is Apple's framework for GPU-accelerated computations on macOS devices with Apple Silicon (M1/M2/M3/M4).

- Compute Unified Device Architecture (CUDA) is NVIDIA's parallel computing platform and API for leveraging NVIDIA GPUs for general-purpose processing.
- To fine-tune a pre-trained model on a dataset, you utilize the `Trainer` and `TrainingArguments` classes from Hugging Face.
- A single modal model is a machine learning model designed to work with data from a single modality, such as text, images, audio, or numerical data.
- A multimodal model, on the other hand, is a machine learning model designed to process and integrate data from multiple modalities—different types of information such as text, images, audio, video, or structured data.
- Contrastive Language–Image Pre-training (CLIP) is a multimodal model developed by OpenAI that connects text and image understanding.

## ***7 Creating LLM-based Applications using LangChain and LlamaIndex***

### **This chapter covers**

- **Introducing Large Language Models (LLM)**
- **Creating LLM Applications using LangChain**
- **Connecting LLMs to Your Private Data using LlamaIndex**
- **Running Local Models for Vector Embedding**

In chapter 3, you learned how to employ transformers pipeline to access diverse pre-trained models for various Natural Language Processing (NLP) tasks, including sentiment classification, named entity extraction, and text summarization. However, in practical scenarios, the goal is to seamlessly integrate various models, encompassing those from Hugging Face and OpenAI, into custom applications. Enter LangChain, a solution that facilitates the customization of NLP applications by linking different components based on specific requirements.

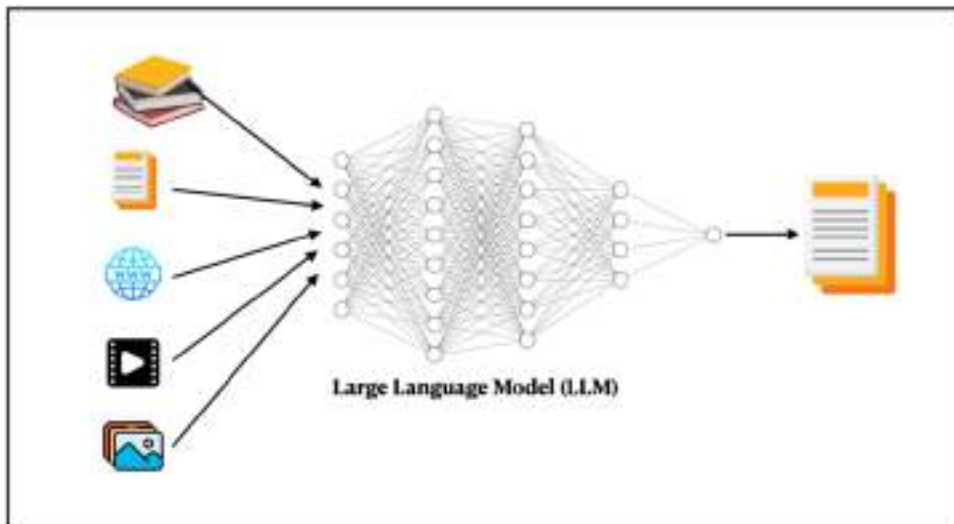
While pre-trained models prove beneficial, it's important to note that they were trained on external data, not your own. Often, there arises a requirement to utilize a model for answering questions pertinent to your unique dataset. For instance, imagine possessing a dataset with numerous receipts and invoices. Leveraging a pre-trained model, you would want it to summarize your purchases or identify vendors associated with specific items. This is where LlamaIndex becomes indispensable. With LlamaIndex, you gain the ability to connect an LLM to your proprietary data, empowering it to address queries tailored to your dataset.

Now, you will delve into the initiation of LangChain for constructing NLP applications using Large Language Models (LLMs). Furthermore, you'll explore the utilization of LlamaIndex to develop NLP applications specifically trained to respond to inquiries related to your private data.

## 7.1 Introduction to Large Language Models (LLM)

A Large Language Model (LLM) is a type of AI model that is designed to understand and generate human-like text based on the patterns and structures it has learned from massive amount of training data. Specifically, LLM refers to a class of advanced Natural Language Processing (NLP) models. LLMs have the following features:

- **Size and scale** – LLMs are trained on massive amount of data, including books, articles, web sites, videos, and pictures (see figure 7.1).
- **Pre-training** – LLMs go through a pre-training phase where they make use of the large amount of training data to learn the statistical relationships between words and sentences. This allows them to acquire a general understanding of grammar, syntax, and semantics.
- **Fine-tuning** – After the pre-training, LLMs are fine-tuned on specific tasks or datasets. This process makes the LLM perform specific tasks, such as text classification, sentiment analysis, language translation, etc.



**Figure 7.1 Large Language Models are trained using large amount of data – books, articles, web pages, videos, and images.**

To have a perspective of how “large” LLMs are, consider the numbers shown in Table 1-1 (source: A Beginner’s Guide to Large Language Models Part 1 from Nvidia).

**Table 7.1 Some LLMs and the size of their training data**

| Large Language Models                                                     | Number of trainable parameters | Number of tokens in the training data |
|---------------------------------------------------------------------------|--------------------------------|---------------------------------------|
| NVIDIA Model (Megatron-Turing Natural Language Generation Model (MT-NLG)) | 530 billion                    | 270 billion                           |
| OpenAI (GPT-3 Davinci Model)                                              | 175 billion                    | 499 billion                           |

Specifically, for the OpenAI's GPT-3 Davinci Model, it has 175 billion trainable parameters and uses 499 billion tokens in the training data.

### TRAINABLE PARAMETERS

Trainable parameters are the variables within a machine learning or deep learning model that are adjusted during the training process to enable the model to make accurate predictions or perform a specific task. These parameters are learned from the training data and fine-tuned to optimize the model's performance on a given task.

In neural networks, trainable parameters typically include:

**Weights** – these are the values associated with the connections between neurons in different layers of the network. During training, the weights are repeatedly adjusted to allow the network to better learn the relationships between input data and the desired output.

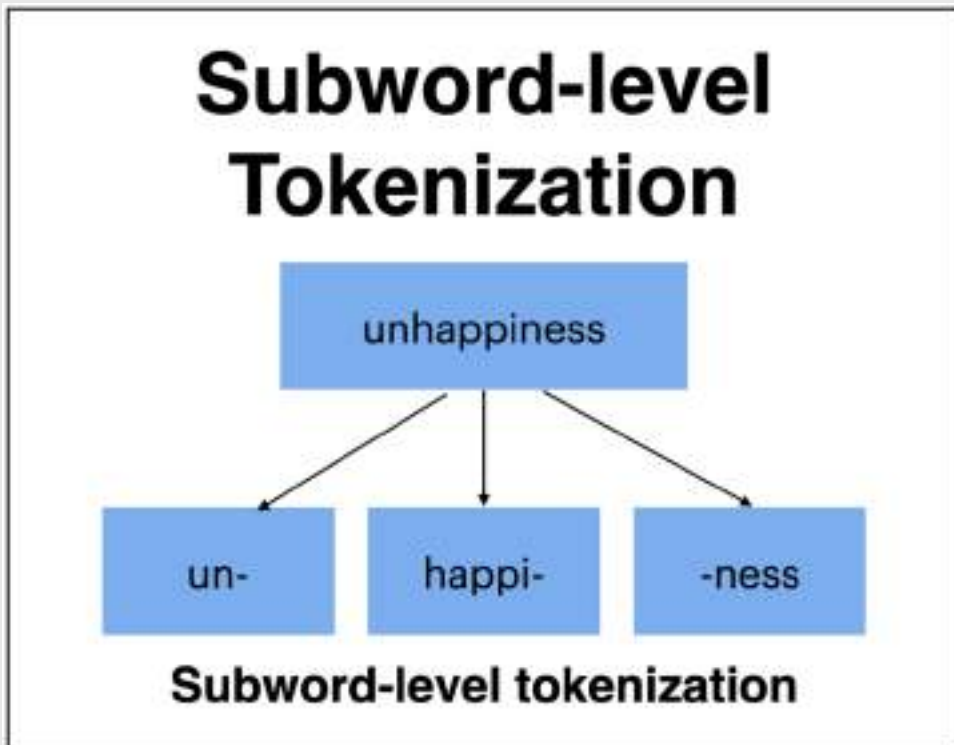
**Biases** – Biases are the values associated with each neuron in a neural network layer. They help shift the activation function's output and allow the network to learn complex patterns.

In short, the more trainable parameters you have in a neural network, the longer it takes to train the system. Training a model with many trainable parameters requires immense computing power and memory. Hence, it is very expensive to train an LLM.



## WHAT ARE TOKENS?

In the context of NLP and machine learning, a "token" is a chunk of text that a model processes as a single unit. Let's use the example of the word "unhappiness". Instead of processing the word as a single unit, a GPT (see the next section on what a GPT is) such as OpenAI's GPT-3.5 uses a technique known as *subword-level tokenization*, called Byte-Pair Encoding (BPE). This technique allows the model to have a large and diverse vocabulary without running into the limitations of a fixed word-level vocabulary. This is particularly useful for understanding and generating text in various contexts, languages, and domains. The figure below shows the word broken down into three tokens.



**Figure 7.2** How the word "unhappiness" is tokenized into three tokens using the subword-level tokenization technique.

Note that besides *subword-level tokenization*, there are other techniques that you can use, such as *word-level tokenization* and *character-level tokenization*.

## 7.2 Introduction to LangChain

LangChain is a framework built around LLMs , designed to simplify the creation of applications using LLMs. You can think of it as a “chain” that connects the various components that are required to create advanced use cases around LLMs. A chain may contain the following components:

- Prompt Templates — templates for different types of conversations with LLMs
- LLMs — Large Language Models such as GPT-3, GPT-4, etc
- Agents — Agents use LLM to decide what actions to be taken
- Memory — Short or Long term memory

### 7.2.1 Installing LangChain

To use Langchain, you first need to install the `langchain` package using the `pip` command:

```
!pip install langchain
```

All of the following code is tested against version 0.3.4 of LangChain. Let’s learn how to use LangChain to connect a LLM together with a prompt template to create a simple LLM application.

### 7.2.2 Creating a Prompt Template

To create a LLM application, the first component you will create is a `PromptTemplate`.

#### PROMPT TEMPLATE

*A prompt template* is used to structure the instruction or query given to the model to obtain the desired outputs. It provide a flexible way to guide the model’s behavior by incorporating special tokens and instructions.

A `PromptTemplate` consists of a string template, and it accepts a list of parameters from the users that can be used to generate a prompt for an LLM. The following code snippet creates a `PromptTemplate` containing a single parameter called `question`:

**Listing 7.1 Creating a simple PromptTemplate component**

```

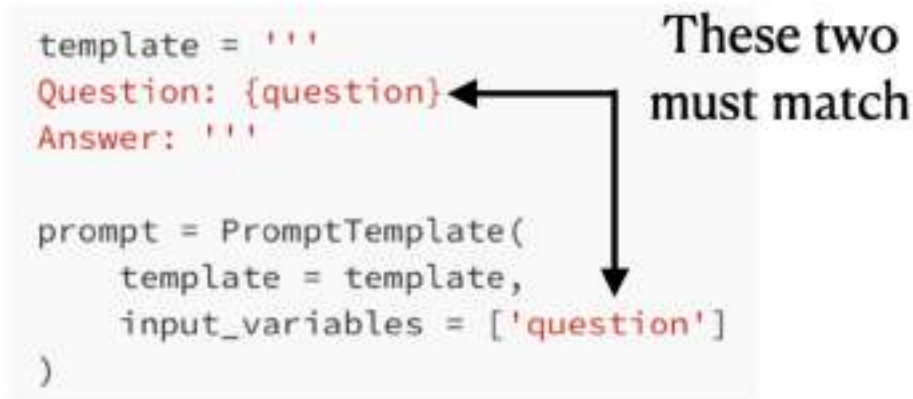
from langchain import PromptTemplate

template = '''
Question: {question}
Answer:
'''

prompt = PromptTemplate(
    template = template,
    input_variables = ['question']
)
prompt

```

The `template` parameter in the `PromptTemplate` class accepts a string template. The `input_variables` parameter accepts a list of parameters from the user that will be used to generate the prompt. Figure 7.3 shows the relationship between the value of the `input_variables` parameter and the variable in the string template:



**Figure 7.3 The input variable is linked to the variable in the string template**

Here is the output of the `PromptTemplate` when printed:

```

PromptTemplate(input_variables=['question'],
               template='\nQuestion: {question}\nAnswer: ')

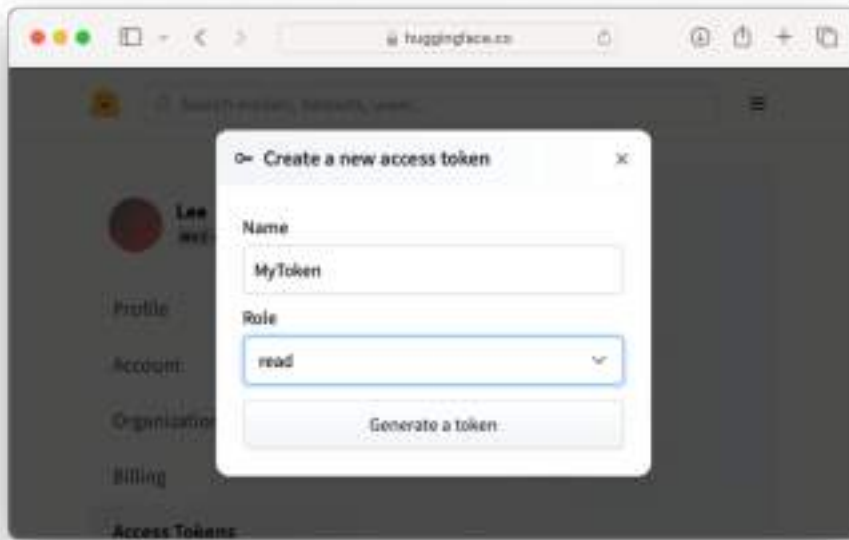
```

In the prompt template created above, you want to ask the LLM a question and expects an answer back from it.

### 7.2.3 Specifying an LLM

Now that the prompt template is created, the next step would be to select and use an LLM. Let's start off with an LLM that is hosted by Hugging Face. But before you can use the LLM from Hugging Face, you need to sign up for a free account at Hugging Face (<https://huggingface.co/join>) and create a token at <https://huggingface.co/settings/tokens>.

Create a *read* token and give it a name (see figure 7.4).



**Figure 7.4** Creating a Hugging Face token

Then, copy the access token and save it in the environment variable:

```
import os
os.environ['HUGGINGFACEHUB_API_TOKEN'] = 'Your_HuggingFace_Token'
```

To make use of an LLM from HuggingFace, you need to install the `langchain-huggingface` package:

```
!pip install langchain-huggingface
```

You can now make use of the `HuggingFaceEndPoint` class to access an LLM from HuggingFace:

```

from langchain_huggingface import HuggingFaceEndpoint

hub_llm = HuggingFaceEndpoint(
    endpoint_url="https://api-
➡inference.huggingface.co/models/HuggingFaceH4/zephyr-7b-alpha",
    temperature = 1
)

```

The above code snippet uses the 'HuggingFaceH4/zephyr-7b-alpha' model.

### PERFORMING INFERENCING USING THE HUGGINGFACEENDPOINT CLASS

Note that when you use the `HuggingFaceEndPoint` class to specify an LLM, the inferencing is performed on Hugging Face's end and not locally on your computer.

Zephyr is a series of language models that are trained to act as helpful assistants. Zephyr-7B-a is the first model in the series, and is a fine-tuned version of [mistralai/Mistral-7B-v0.1](https://huggingface.co/mistralai/Mistral-7B-v0.1) that was trained on a mix of publicly available, synthetic datasets using [Direct Preference Optimization \(DPO\)](https://huggingface.co/HuggingFaceH4/zephyr-7b-alpha). You can find more information about this model from <https://huggingface.co/HuggingFaceH4/zephyr-7b-alpha>.

As shown in Figure 7.5, the 'HuggingFaceH4/zephyr-7b-alpha' is a Text Generation model.

The 'HuggingFaceH4/  
zephyr-7b-alpha' is a  
Text Generation model

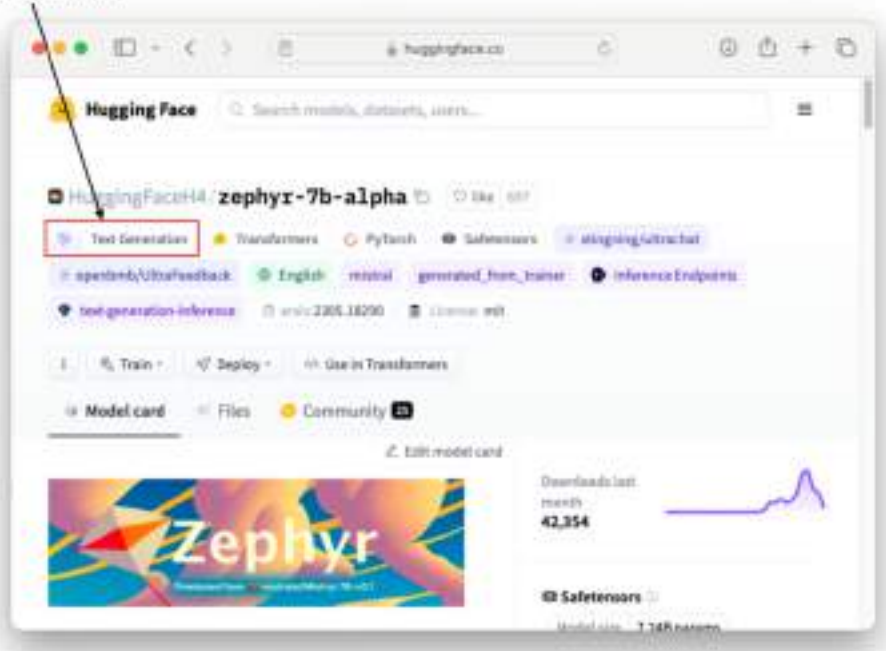


Figure 7.5 The 'HuggingFaceH4/zephyr-7b-alpha' is a Text Generation model.

## 7.2.4 Creating an LLM Chain

The next step would be to create an LLM chain. You to combine your prompt template and model (LLM) to create a chain so that you can run queries against the LLM:

```
from langchain_core.output_parsers import StrOutputParser

llm_chain = prompt | hub_llm | StrOutputParser()
```

Essentially, you are chaining all the various components (PromptTemplate, hub\_llm, and the StrOutputParser). The StrOutputParser object is used to handle the output and parse it into a string. The | operator in llm\_chain = prompt | hub\_llm | StrOutputParser() represents a **pipeline-style chaining** in LangChain. This syntax makes it easy to link together different components—like prompts, language models, and output parsers—in a sequence to process data from start to finish. Each component takes the output of the previous step, processes it, and passes it along to the next.

You are now ready to use the chain to answer your questions!

## 7.2.5 Running the Chain

You can now test the application by calling the `run()` method of the LLM Chain:

```
qn = "Who is Elon Musk?"
print(llm_chain.invoke(qn))
```

Here is the response from the LLM:

```
1. Elon Musk is the CEO of SpaceX, Tesla, and Neuralink
```

Remember that all the inferencing is done on Hugging Face's server. Observe what happens if you ask a follow-up question:

```
qn = "Is he married?"
print(llm_chain.invoke(qn))
```

You will get a response like the following because the LLM does not maintain a context between questions:

```
I do not have personal information about individuals. however, based on the
information available to me
```

So how do you solve this? Let's take a look.

## 7.2.6 Maintaining a Conversation

To maintain context between questions, the LLM needs to be able to know what the response questions and answers were. One way to do this would be to alter the prompt template and provide a history of the conversation:

```
template = '''
Current conversation: {history}
Human: {question}
AI:
'''
```

The above code snippet shows that there are now two variables in the template string:

- `history`
- `question`

Correspondingly, the `input_variables` parameter must also specify the two variables in the list:

```
prompt = PromptTemplate(
    template = template,
    input_variables = ['question', 'history']
)
```

Next, create a new chain using the updated `PromptTemplate` object:

```
llm_chain = prompt | hub_llm | StrOutputParser()
```

You can now ask a question by passing in the question and history to the `invoke()` method:

```
qn = "Who is Elon Musk?"
response = llm_chain.invoke({'question':qn, 'history':''})
print(response)
```

The above code generated the following response:

```
Elon Musk is a South African-born American entrepreneur, business magnate, and engineer.
```

If you ask a follow-up question (note that you need to pass the previous response from the LLM into the `invoke()` method):

```
qn = "Is he married?"
response = llm_chain.invoke({'question':qn, 'history':response})
print(response)
```

You will get a response that understands the context of the question:

```
Elon Musk is currently not married. He was previously married to Canadian author and journalist, Tal
```

You can now rewrite the above code snippet using a while-loop so that you can prompt the user to ask a question and get the LLM to answer it:

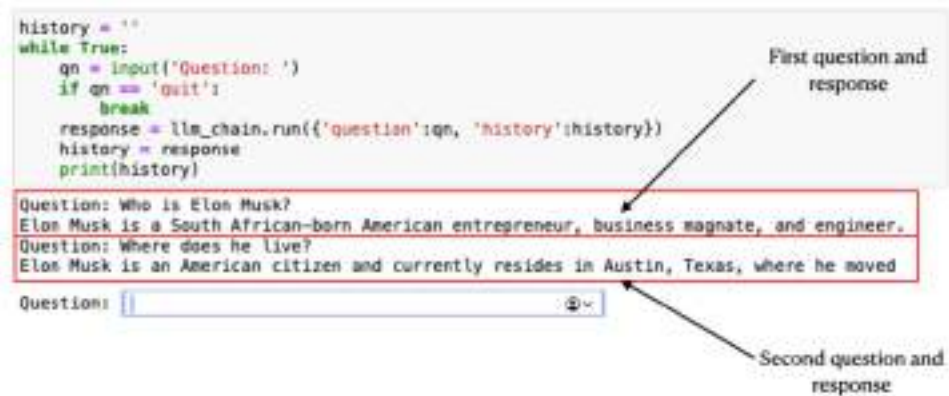


```

history = ''
while True:
    qn = input('Question: ')
    if qn == 'quit':
        break
    response = llm_chain.invoke({'question':qn, 'history':history})
    history = response
    print(history)

```

Figure 7.6 shows a sample flow of the question-and-answer session:



**Figure 7.6** User can ask questions and get a response from the LLM

### 7.2.7 Using the RunnableWithMessageHistory class

The previous section showed how to modify the prompt to maintain a conversation with the LLM. An alternative approach to maintaining a chat conversation is to use the `RunnableWithMessageHistory` class. The `RunnableWithMessageHistory` is a class in LangChain that allows you to manage the history of interactions (messages) during a conversation between the user and an AI model. It is part of LangChain's conversational framework, used to build systems that keep track of conversation history across multiple exchanges. The following code snippet shows how to use the `RunnableWithMessageHistory` class:

#### Listing 7.2 Using the `RunnableWithMessageHistory` class to maintain conversation

```

import os
from langchain import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_huggingface import HuggingFaceEndpoint
from langchain_core.runnables.history import RunnableWithMessageHistory

```

```

os.environ['HUGGINGFACEHUB_API_TOKEN'] = 'your_hugging_face_token'

template = '''
Question: {question}
Answer:
'''

prompt = PromptTemplate(
    template = template,
    input_variables = ['question']
)

hub_llm = HuggingFaceEndpoint(
    endpoint_url="https://api-inference.huggingface.co/models
    ➡/HuggingFaceH4/zephyr-7b-alpha",
    temperature = 1
)

class SessionHistory: #A
    def __init__(self):
        self.messages = []

    def add_messages(self, messages):
        self.messages.extend(messages) #B

    def get_messages(self):
        return self.messages

session_history = SessionHistory() #C

def get_session_history(): #D
    return session_history

llm_chain = RunnableWithMessageHistory( #E
    prompt | hub_llm | StrOutputParser(),
    get_session_history = get_session_history
)

while True: #F
    #G
    user_question = input("Ask a question (type 'exit' to stop): ")

    #H
    if user_question.lower() == "quit":

```

```

    print("Ending conversation.")
    break

#I
input_data = {"question": user_question}

response = llm_chain.invoke(input_data) #J

session_history.add_messages([ #K
    {"role": "user", "content": user_question},
    {"role": "assistant", "content": response}
])

#L
print(f"AI: {response}")

```

**#A** a class to store session history that includes the 'messages' attribute

**#B** add a list of messages to the history

**#C** initialize the session history

**#D** a function to retrieve the current session history

**#E** create the chain with message history

**#F** start a loop for multiple questions

**#G** get the user input

**#H** exit condition

**#I** prepare the input data

**#J** get the model's response

**#K** add the user question and assistant response to the session history

**#L** display the response

You can now ask follow-up questions after the initial question. The main addition to this code snippet is the `SessionHistory` class that allows the history of the exchanges between the user and the model to be saved. An instance of the `SessionHistory` class is then passed to the `RunnableWithMessageHistory` class through the `get_session_history` parameter:

```

llm_chain = RunnableWithMessageHistory(
    prompt | hub_llm | StrOutputParser(),
    get_session_history = get_session_history
)

```

The LLM can now retain the context of the conversation and is able to answer the follow-up question correctly. How do you retrieve the chat history? Well, you can do so via the `get_messages()` method from the `session_history` object:

```
print(session_history.get_messages())
```

The following shows an example conversation history (formatted for clarity):

### Listing 7.3 An example of a chat history

```
[
  HumanMessage(
    content='Who is Bill Gates?',
    additional_kwargs={},
    response_metadata={}
  ),

  AIMessage(
    content="Bill Gates is an American business magnate, software developer,
    entrepreneur, and philanthropist. He is best known as the co-founder of Microsoft
    Corporation, the world's largest personal computer software company. Gates is
    also the co-chair of the Bill & Melinda Gates Foundation, a private charitable
    organization that is dedicated to improving health and reducing poverty
    worldwide. As of 2021, he is one of the wealthiest individuals in the world.",
    additional_kwargs={}, response_metadata={}
  ),

  {'role': 'user', 'content': 'Who is Bill Gates?'},

  {'role': 'assistant', 'content': "Bill Gates is an American business magnate,
  software developer, entrepreneur, and philanthropist. He is best known as the co-
  founder of Microsoft Corporation, the world's largest personal computer software
  company. Gates is also the co-chair of the Bill & Melinda Gates Foundation, a
  private charitable organization that is dedicated to improving health and
  reducing poverty worldwide. As of 2021, he is one of the wealthiest individuals
  in the world."},

  HumanMessage(
    content='Is he rich?',
    additional_kwargs={},
    response_metadata={}
  ),

  AIMessage(
    content='<|>\nYes, Bill Gates is one of the wealthiest individuals in the world
    according to the information provided in the AIMessage.',
    additional_kwargs={},
    response_metadata={}
  )
]
```

```

),

{'role': 'user', 'content': 'Is he rich?'},

{'role': 'assistant', 'content': '<|>\nYes, Bill Gates is one of the wealthiest
individuals in the world according to the information provided in the
AIMessage.'}
]
Text Completion

```

Besides asking questions, the 'HuggingFaceH4/zephyr-7b-alpha', being a text generation model, can also perform text completion. The trick to making it perform text completion, is to modify the prompt template.

The following code snippet shows the entire program, with the prompt string changed to text completion:

**Listing 7.4 Performing text completion**

```

import os
from langchain import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_huggingface import HuggingFaceEndpoint

os.environ['HUGGINGFACEHUB_API_TOKEN'] = 'Your_HuggingFace_Token'

template = '''
Complete this: {question}
'''

prompt = PromptTemplate(
    template = template,
    input_variables = ['question']
)
prompt

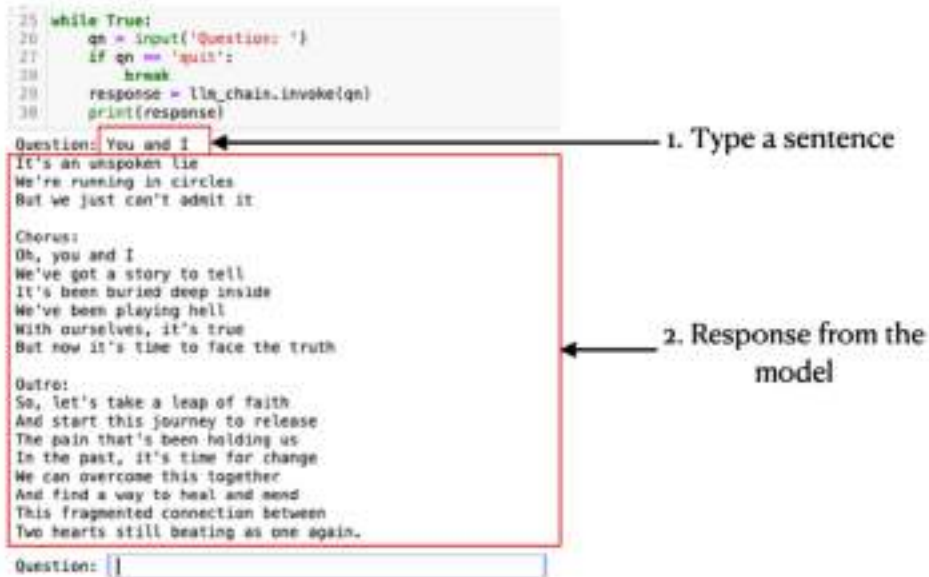
hub_llm = HuggingFaceEndpoint(
    endpoint_url="https://api-inference.huggingface.co/models
➡/HuggingFaceH4/zephyr-7b-alpha",
    temperature = 1
)

llm_chain = prompt | hub_llm | StrOutputParser()

while True:
    qn = input('Question: ')
    if qn == 'quit':
        break
    response = llm_chain.invoke(qn)
    print(response)

```

Run the program and ask a question. Figure 7.7 shows the response from the model.



**Figure 7.7** You can get the model to complete your sentence

## 7.2.8 Using Other LLMs

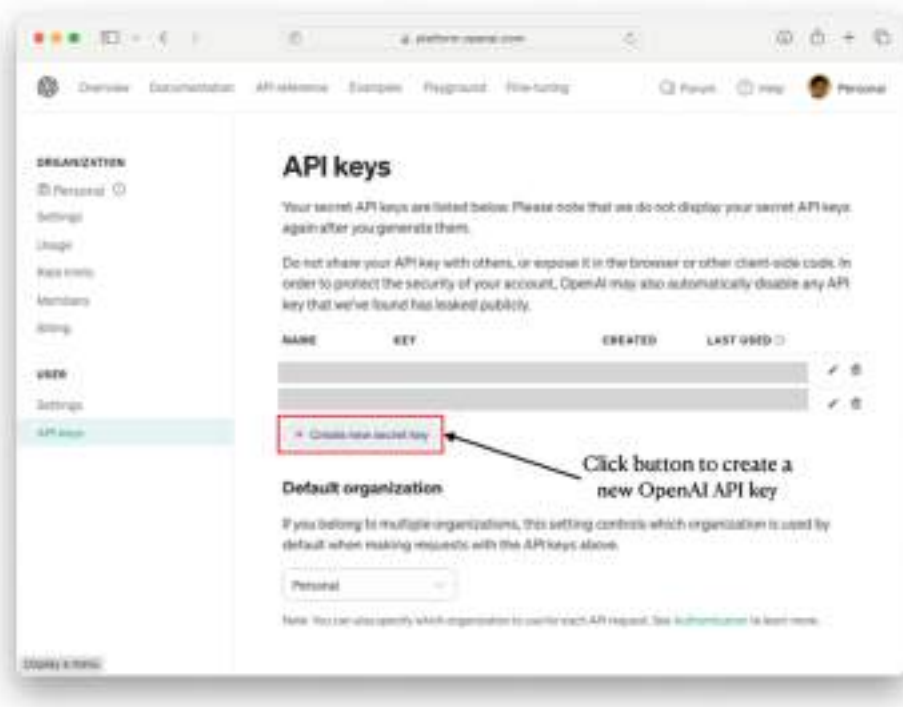
Apart from using the LLMs from Hugging Face, you can also use LLMs from other providers. Moving forward, you will have the chance to try out two other LLMs – one from OpenAI and one from Hugging Face.

### USING OPENAI

Let's use an LLM from OpenAI. Before you can use the LLM from OpenAI, you need to install the `langchain_openai` package using the `pip` command:

```
!pip install langchain_openai
```

Also, to make use of OpenAI's LLM, you need to apply for an API key (which is *pay-per-use*). You can apply for one at: <https://platform.openai.com/account/api-keys>. Figure 7.8 shows how you can apply for an OpenAI API key.



**Figure 7.8 Applying for an OpenAI API key**

Once you have obtained your OpenAI API key, add the following statements to add an environment variable named `OPENAI_API_KEY` and set it to your OpenAI API key:

```
import os
os.environ['OPENAI_API_KEY'] = "OPENAI_API_KEY"
```

To view the list of models you can use from OpenAI, check out the page at <https://platform.openai.com/docs/models/overview>. Here are some models that you can use:

- `gpt-3.5-turbo` - Most capable GPT-3.5 model and optimized for chat at 1/10th the cost of an older model - `text-davinci-003`.
- `gpt-4o-mini` - More capable than any GPT-3.5 model, able to do more complex tasks, and optimized for chat.

To make use of a model (say, `gpt-4o-mini`) from OpenAI, use the `ChatOpenAI` class and pass it the name of the model:



```
from langchain.chat_models import ChatOpenAI

openai_model = ChatOpenAI(model_name = 'gpt-4o-mini')
```

Then, as we have previously done, create a `PromptTemplate` and an LLM and chain them together:

#### Listing 7.5 Creating a chain using OpenAI LLM

```
from langchain import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

template = '''
Question: {question}
Answer: '''

prompt = PromptTemplate(
    template = template,
    input_variables = ['question']
)

llm_chain = prompt | openai_model | StrOutputParser()
```

You can now pose questions to the OpenAI model:

```
question = "Who is Steve Jobs"
print(llm_chain.invoke(question))
```

You will get a reply like this:

```
Steve Jobs was an American entrepreneur, businessman, inventor, and co-founder of Apple Inc. He is widely recognized as a pioneer of the personal computer revolution of the 1970s and 1980s, along with his business partner and Apple co-founder Steve Wozniak. Jobs also served as the CEO of Pixar Animation Studios and was a member of the board of directors of The Walt Disney Company. He passed away in 2011 from complications related to pancreatic cancer.
```

## USING TIUAE/FALCON-7B-INSTRUCT MODEL

Apart from OpenAI, there are several other models from Hugging Face that you can use. One such model is 'tiiuae/falcon-7b-instruct' (<https://huggingface.co/tiiuae/falcon-7b-instruct>). The 'tiiuae/falcon-7b-instruct' model is a large language model (LLM) developed by the Technology Innovation Institute (TII) in Abu Dhabi, UAE. It is an instruction-tuned version of Falcon 7B, a transformer-based model designed to understand and follow specific instructions provided by users.

As usual, create a `PromptTemplate` and `HuggingFaceEndPoint` and then chain them together:

### Listing 7.6 Using the tiiuae/falcon-7b-instruct model

```
import os
from langchain_core.output_parsers import StrOutputParser
from langchain_huggingface import HuggingFaceEndpoint
from langchain import PromptTemplate

os.environ['HUGGINGFACEHUB_API_TOKEN'] = 'Your_HuggingFace_Token'

template = '''
Question: {question}
Answer: '''

prompt = PromptTemplate(
    template = template,
    input_variables = ['question']
)

hub_llm = HuggingFaceEndpoint(endpoint_url=
    "https://api-inference.huggingface.co
    ➔/models/tiiuae/falcon-7b-instruct",
    temperature = 1
)

llm_chain = prompt | hub_llm | StrOutputParser()
```

You can now ask question using the `llm_chain` object:

```
question = "Translate this to Spanish: Which is the way to the train station?"
print(llm_chain.invoke(question))
```

You should see the following response:

```
¿A dónde está la estación de tren?
```

Here's another question:

```
question = "What is the capital of France?"  
print(llm_chain.invoke(question))
```

And the response looks like this:

```
The capital of France is Paris.
```

## 7.3 Connecting LLMs to Your Private Data using LlamaIndex

So far you have seen how interesting it is to use LangChain to connect to LLMs provided by the various companies (such as OpenAI and Hugging Face) to build your own chat-based applications where you can ask questions using natural language. However, while LLMs are trained using a huge amount of data, they are not trained using your own data. It would be more useful if an LLM can be used to answer specific questions pertaining to your data, rather than the data that it has been trained on.

LlamaIndex solves this problem by connecting your data and adding them to an existing LLM. It does this by using a technique known as *Retrieval-Augmented Generation* (RAG).

RAG is a technique used to enhance the performance of large language models (LLMs) by integrating them with an external retrieval system. The main idea is to supplement the language model's response generation with relevant information from a knowledge base or document store, enabling it to generate more accurate, context-aware responses without needing to know everything ahead of time.

### LLAMAINDEX

LlamaIndex is a data framework for LLM-based applications to ingest, structure, and access private or domain-specific data. It's available in Python and TypeScript.

### 7.3.1 Installing the Packages

For this example, use the `pip` command to install the following packages:

```
!pip install llama_index
!pip install llama-index-embeddings-huggingface
!pip install llama-index-llms-huggingface
```

Let's learn how to use LlamaIndex with a Hugging Face model to index your private data so that an LLM can answer questions pertaining to your data.

### 7.3.2 Preparing the Documents

For this example, we will make use of LlamaIndex to answer questions based on a number of receipts saved in PDF format. Besides PDF, LlamaIndex supports a number of common file formats, such as:

- Text Files: .txt
- Microsoft Word Documents: .doc and .docx
- Markdown Files: .md
- HTML Files: .html or .htm
- CSV Files: .csv
- JSON Files: .json

Create a folder named "Training Documents" in the same directory as your Jupyter notebook. Populate the folder with the receipts. For this example, the folder contains four PDF documents (receipts of some purchases; see Figure 7.9).

## Content of the "Training Documents" Folder



**PDF documents**

**Figure 7.9** The "Training Documents" folder contains four PDF files

### 7.3.3 Loading the Documents

To load the PDFs into memory for indexing, use the `SimpleDirectoryReader` class, a component of LlamaIndex that helps facilitate the reading and indexing of documents from a specified directory:

```

from llama_index.core import SimpleDirectoryReader

loader = SimpleDirectoryReader(
    input_dir="./Training Documents",
    recursive=True,
    required_exts=[".pdf"],
)

#A
documents = loader.load_data()

#A loads the documents

```

In the above code snippet, you specify the

- **Directory Input:** It allows you to specify a directory from which to load documents.
- **Recursive Loading:** With the `recursive` parameter set to `True`, it can search through subdirectories and load documents from those as well.
- **File Type Filtering:** The `required_exts` parameter enables you to filter which file types to load, such as PDFs, text files, Word documents, etc.

### 7.3.4 Using an Embedding Model

Once the documents are loaded, the next step is to perform *vector embedding* to convert the text data into vector representations. This allows for more efficient querying, similarity search, and further processing of the documents.

#### VECTOR EMBEDDING

A vector embedding is a numerical representation of objects (such as words, sentences, images, or any data point) in a continuous vector space. Each object is mapped to a vector of numbers in such a way that the spatial relationships between these vectors reflect similarities or semantic relationships between the objects themselves.

You can use models such as `HuggingFaceEmbedding` to generate these embeddings for each document, enabling you to make use of machine learning models in your application. For this example, we will use the 'BAAI/bge-small-en-v1.5' model:

```
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embedding_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
```

The 'BAAI/bge-small-en-v1.5' model is a specific pre-trained model available on the Hugging Face Model Hub. This model is part of the BGE (Bag of Graph Embeddings) series, which is designed for generating embeddings for English text. Models in this series often focus on producing embeddings that capture semantic information effectively, making them useful for various NLP tasks.

### DATA PRIVACY FOR VECTOR EMBEDDING

By using a model such as 'BAAI/bge-small-en-v1.5', the embedding process takes place locally on your computer. This ensures that the privacy of your data.

### 7.3.5 Indexing the Document

You can now start to index the document using the embedding model via the `VectorStoreIndex` class, which you will use to create an index and then save the vector embeddings on disk:

```
from llama_index.core import VectorStoreIndex

index = VectorStoreIndex.from_documents(
    documents,
    embed_model = embedding_model,
)

index.storage_context.persist(persist_dir=".")
```

#A

**#A save the index in the current directory**

The vector embeddings are now stored in the same directory as your Jupyter notebook. There are five files created:

- `image__vector_store.json` — contains the vector embeddings associated with documents that are categorized as images. If your index includes image documents or embeddings generated from image data, this file will store that specific information.

- `default__vector_store.json` — stores the main vector embeddings for the default category of documents in your index. It contains the embeddings of text documents that are not specifically categorized as images or other types.
- `graph_store.json` — contain information related to any graph structures or relationships that exist between the indexed documents. This could include links, relationships, or any other metadata that captures the connectivity or hierarchy of the documents.
- `index_store.json` — holds metadata and configurations related to the index itself. It may include information about how the index was constructed, parameters used during creation, and other relevant settings.
- `docstore.json` — contains the actual document data or references to the documents that were indexed. It serves as a storage for the documents themselves, allowing the index to retrieve and reference them when necessary.

Saving the vector embeddings to disk is beneficial because it allows you to perform the embedding process only once — unless the content of your documents change. Once the embeddings are saved, you can simply load them from disk in the future, avoiding the need to re-embed the documents. In the next step, you will learn how to load these vector embeddings directly from disk.

### 7.3.6 Loading the Embeddings

Once the index is persisted to disk, you can load it into memory using the `StorageContext` class and the `load_index_from_storage()` function:

```
from llama_index.core import StorageContext, load_index_from_storage
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embedding_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")

storage_context = StorageContext.from_defaults(persist_dir=".")
index = load_index_from_storage(storage_context,
                               embed_model = embedding_model)
```

Note that you need to use the same embedding model as you have used earlier for indexing.

### 7.3.7 Using an LLM for Querying

Now that the documents are indexed, you can utilize a Hugging Face model to ask questions based on your local documents:



**Listing 7.7 Querying the local documents using an LLM**

```

from transformers import AutoModelForCausalLM, AutoTokenizer
from llama_index.llms.huggingface import HuggingFaceLLM
import torch

#A
if torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#B
tokenizer = AutoTokenizer.from_pretrained(
    "meta-llama/Llama-3.2-3B-Instruct")
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-3B-Instruct").to(device)

#C
huggingface_llm = HuggingFaceLLM(
    model=model,
    tokenizer=tokenizer,
)

#D
query_engine = index.as_query_engine(llm=huggingface_llm)

```

**#A** determine the device

**#B** load a model and tokenizer from Hugging Face

**#C** initialize HuggingFaceLLM

**#D** set the LLM to use

In this example, I will use the `AutoModelForCausalLM` class to load the 'meta-llama/Llama-3.2-3B-Instruct' model, which will be set as the query engine for the index.

**DATA PRIVACY FOR INFERENCE**

Again, using a model such as 'meta-llama/Llama-3.2-3B-Instruct' ensures that the inference of the model is done locally on your computer. This ensures that the privacy of your data.

### SHIFTING THE WORKLOAD TO THE GPU

The `to()` method in PyTorch is used to move a model or tensor to a specified device, typically a CPU or GPU. If you have an Apple Silicon Mac (with a M1, M2, M3, or later processor), processing is moved to the GPU (`mps`, or Metal Performance Shaders). If you have a NVIDIA GPU on your Windows PC, processing is moved to the GPU (`cuda`, or Compute Unified Device Architecture). If you have none of the above, then the processing is done on the CPU.

### 7.3.8 Asking Questions

You can now ask questions related to the receipts using the 'meta-llama/Llama-3.2-3B-Instruct' model:

```
while True:
    question = input("Question: ")
    if question.lower() == "quit": break
    print(query_engine.query(question).response)
```

The following dialog shows the question asked and the response by the model:

```
Question: How much did I pay for the keyboard?
162.90 SGD.
```

### 7.3.9 Using LlamaIndex with OpenAI

You may have noticed that running a LLM locally is slow. This is because local models often require significant computational resources, including high memory and processing power, to handle the large number of parameters and complex calculations involved in generating responses. Additionally, the performance can be further affected by the hardware limitations of the machine being used, such as CPU speed and GPU availability. This can lead to longer inference times compared to accessing models hosted on cloud platforms, where powerful infrastructure is optimized for fast processing. More significantly, the quality of the answer may not be as good as what you expected.

An alternative to running a local LLM is to use OpenAI's models, such as the 'gpt-4o-mini'. To do that, you need to install the following packages:

```
!pip install langchain_community
!pip install langchain_openai
```

The following code snippet loads the 'gpt-4o-mini' model from OpenAI and sets it as the query engine for the index:

```
from langchain_openai import ChatOpenAI
import os

os.environ["OPENAI_API_KEY"] = "OpenAI_API_Key"
openai_llm = ChatOpenAI(temperature = 0.7,
                        model_name = "gpt-4o-mini")

query_engine = index.as_query_engine(llm = openai_llm)
```

Note that you will need to supply an OpenAI API key to use the service. Keep in mind that all API calls to OpenAI are billed according to their usage rates.

You can now ask questions just like what you did earlier:

```
while True:
    question = input("Question: ")
    if question.lower() == "quit": break
    print(query_engine.query(question).response)
```

Here is a sample dialog:

**Question: How much did I pay for the keyboard?**  
 You paid a total of 169.90 for the Razer Blackwidow V3 Gaming Keyboard.

You will notice that the responses are much faster and the quality of the responses is better than that of the local model. This is because OpenAI's cloud-based models leverage powerful, optimized infrastructure capable of processing large volumes of data and performing complex computations more efficiently. Additionally, these models benefit from continual updates and improvements made by OpenAI, which allows them to deliver more accurate and contextually relevant responses compared to locally run models that may be constrained by hardware limitations and lack access to the latest training data.

What about the privacy of your data? When you call `index.as_query_engine(llm=openai_llm)`, the query engine is set up to handle queries using the OpenAI model. This means that when you submit a query to the index, the query will be processed by the OpenAI model. Since the model is hosted by OpenAI, any data you send through this query engine (including the questions and potentially the document contents) will be transmitted to OpenAI's servers for processing. This means that both the queries you make and any relevant context from your indexed documents will be sent to OpenAI.

If you are concerned with the privacy of your data, you should use a local model, such as 'meta-llama/Llama-3.2-3B-Instruct' as illustrated in the earlier example.

### 7.3.10 Creating a Web Frontend for the App

To make the query engine easy to use, you will bind it to the Gradio library. Gradio is an open-source Python library that is used to build machine learning and data science demos and web applications. To use Gradio in your Python application, you need to install it using the `pip` command:

```
!pip install gradio
```

Next, create a function named `my_chat_bot` that calls the `query()` method of the query engine:

```
def my_chat_bot(input_text):
    response = query_engine.query(input_text)
    return response.response
```

Finally, bind the `my_chat_bot()` function with Gradio:

```
import gradio as gr

gr.Interface(fn = my_chat_bot,
             title = "Enquiry",
             inputs = "text",
             outputs = "text").launch()
```

#A bind it to gradio

When you run the above code snippet, you will see the UI as shown in Figure 7.10.

```
Running on local URL: http://127.0.0.1:7952
To create a public link, set 'share=True' in 'launch()'.
```

The figure shows a web interface titled "Enquiry". It has two main columns. The left column contains an "input\_text" field with a text input area, a "Clear" button below it, and a "Submit" button to the right of the "Clear" button. The right column contains an "output" field with a text area, and a "Flag" button below it.

**Figure 7.10 The Gradio interface is bound to the query engine**

You can ask a question pertaining to the receipts in PDF and then click the Submit button. Figure 7.11 shows the response returned by the query engine.

The figure shows the same "Enquiry" web interface as Figure 7.10, but with content. The "input\_text" field now contains the text "How many cables did I buy?". The "output" field contains the text "You bought a total of 3 cables.". The "Submit" button is highlighted in orange, indicating it was recently clicked.

**Figure 7.11 Asking a question regarding the purchases stored in the PDF receipts**

### 7.3.11 Holding a conversation

While you can ask questions using the query engine, it is not able to hold a conversation with the user. For example, suppose you asked the following questions in succession:

- How many cables did I buy?
- How much did I pay for them?

The query engine would not be able to answer the second follow-up question as it holds no memory of the previous conversation. If you want to hold a conversation with the LLM, use the `as_chat_engine()` method instead of `as_query_engine()`:

```
query_engine = index.as_chat_engine(llm=openai_llm) #A
```

#A use this for holding a conversation

Then, use the `chat()` method to chat with the user:

**Listing 7.8 Binding Gradio to the my\_chat\_bot() function**

```
def my_chat_bot(input_text):
    response = query_engine.chat(input_text) #A
    return response.response

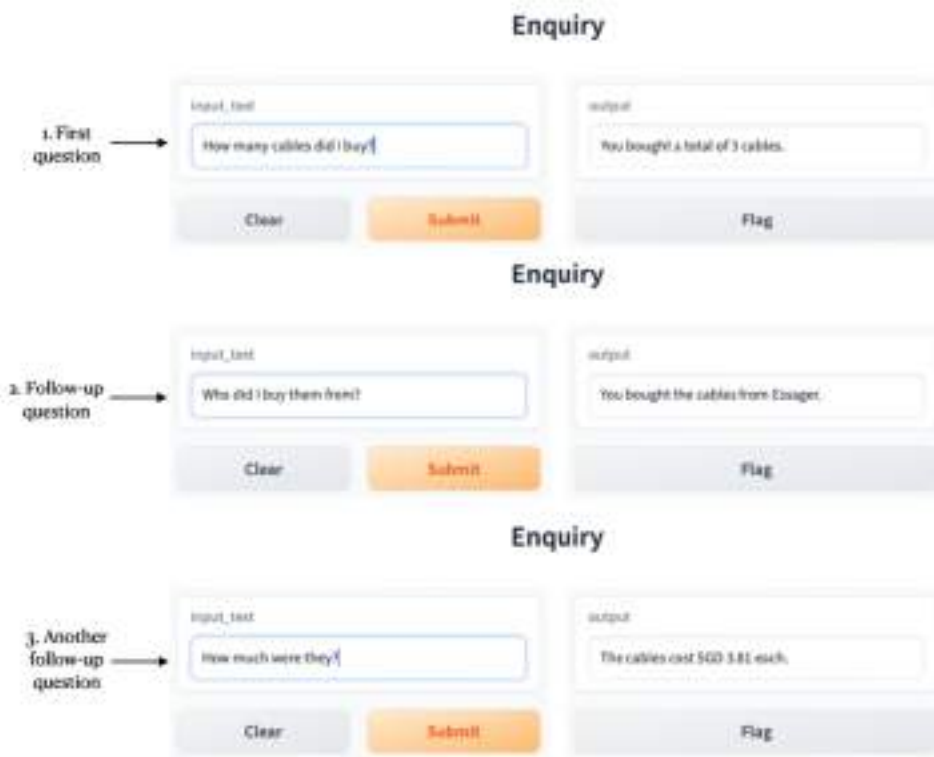
import gradio as gr

gr.Interface(fn = my_chat_bot, #B
            title = "Enquiry",
            inputs = "text",
            outputs = "text").launch()
```

#A use the chat() function

#B bind it to gradio

You can now ask a question, followed by other related questions (see Figure 7.12 for an example chat session).



**Figure 7.12** Asking a series of follow-up questions and the chatbot can maintain the conversation

### 7.3.12 Creating a Chatbot UI

The previous example showed how you can ask follow-up questions with the query engine and maintain a conversation. However, that UI is not very user friendly for engaging a chat with the LLM – every time you want to ask the next question you must clear the input textbox, type in your new question, and then click the Submit button. For chatting, it would be better to have the UI redesigned for that purpose. Fortunately, Gradio allows you to customize its look-and-feel so that it makes it easy for you to chat with the LLM. You can use the following template:

**Listing 7.9 Creating a Chatbot UI Template**

```
import gradio as gr

with gr.Blocks() as mychatbot:
    chatbot = gr.Chatbot() #A
    question = gr.Textbox() #B

    def chat(message, chat_history):
        content = "Responses from chatbot..." #C
        chat_history.append((message, content))
        return "", chat_history

    question.submit(fn = chat, #D
                   inputs = [question, chatbot],
                   outputs = [question, chatbot])

mychatbot.launch()
```

#A displays a chatbot

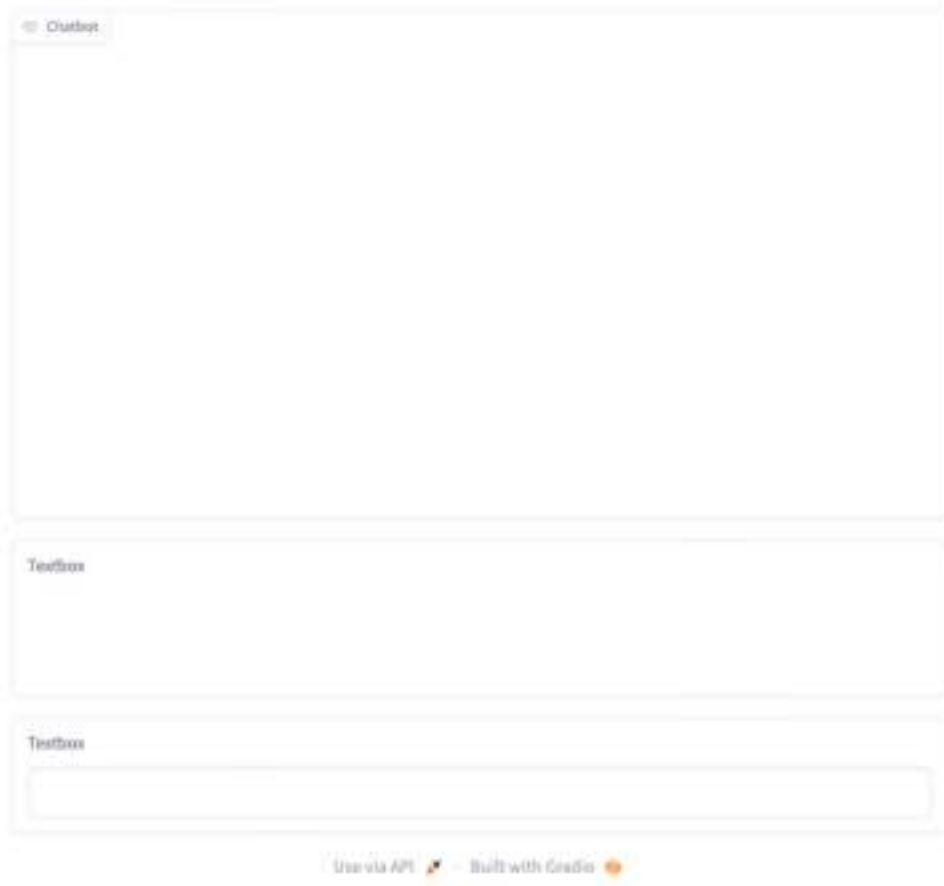
#B for user to ask a question

#C replace content with the actual responses from a chatbot

#D wire up the event handler for Submit button (when user press Enter)

The `Blocks` class is a low-level API that allows you to create custom web applications. The `Chatbot` class displays a chatbot UI while the `Textbox` class creates a textbox to allow the user to enter a question. The textbox has a `submit()` function which will be triggered when the user presses the Enter key after typing in the question. In the code snippet above, you should replace the statements in bold with the code that interfaces with your chat engine. Whatever responses returned by your chat engine will then be appended to the `chat_history` parameter.

Figure 7.13 shows how the UI looks like now.



**Figure 7.13 The Gradio chat user interface**

To interface with the chat engine you have created earlier, replace the bolded statement in the previous code snippet with the call to the `my_chat_bot()` function:



**Listing 7.10 Replacing the placeholder with the my\_chat\_bot() function**

```

import gradio as gr

with gr.Blocks() as mychatbot:
    chatbot = gr.Chatbot() #A
    question = gr.Textbox() #B

    def chat(message, chat_history):
        content = my_chat_bot(message)
        chat_history.append((message, content))
        return "", chat_history

    #C
    question.submit(fn = chat,
                    inputs = [question, chatbot],
                    outputs = [question, chatbot])

mychatbot.launch()

```

**#A displays a chatbot**

**#B for user to ask a question**

**#C wire up the event handler for Submit button (when user press Enter)**

You can now chat with the query engine. Figure 7.14 shows how a typical conversation looks like.



**Figure 7.14 Using the chat user interface to chat with the chat engine**

## 7.4 Summary

- A Large Language Model (LLM) is a type of AI model that is designed to understand and generate human-like text based on the patterns and structures it has learned from massive amount of training data.
- A "token" is a chunk of text that a model processes as a single unit.
- LangChain is a framework built around LLMs , designed to simplify the creation of applications using LLMs.
- A LangChain application is made up of components chained together.
- You can run an LLM locally or use one that is cloud-based, such as those from OpenAI.
- LlamaIndex is a data framework for LLM-based applications to ingest, structure, and access private or domain-specific data.
- RAG is a technique used to enhance the performance of large language models (LLMs) by integrating them with an external retrieval system.
- Running a model locally for vector embedding ensures your data stays within your computer.
- You can create a web frontend for your LLM applications using Gradio

## ***8 Building LangChain Applications Visually using LangFlow***

### **This chapter covers**

- What is LangFlow?
- Creating a LangChain project using LangFlow
- Using and configuring the various components in LangFlow
- Using LangFlow to query your own data

Previously, you learned how to build LLM-based applications by chaining various components such as Prompt Template, LLMs, Memory, and more. You also learned how to use LlamaIndex to connect an LLM to answer questions pertaining to your own data. To use LangChain, you must download the `langchain` package and then use the various APIs in the framework.

Now, you will learn an easier approach to building LLM-based applications using LangChain – instead of writing code, you will build LangChain apps using a drag-and-drop tool known as LangFlow. Using LangFlow, you can now get started with LangChain without getting bogged down with the details of coding and can instantly preview your applications without any complicated setup process.

## 8.1 What is LangFlow

LangFlow is an open-source library that allows you to build LLM-based applications using LangChain through a drop-and-drop visual interface. LangFlow is built on top of LangChain so that you can develop AI applications faster and easier through a No Code Low Code experience.

The source code for LangFlow can be downloaded from <https://github.com/logspace-ai/langflow>. For this chapter, we will focus on how to install LangFlow on your computer so that you can get productive immediately.

There are several ways to install LangFlow on your computer. You will learn:

- How to install LangFlow locally on your computer using the `pip` command
- How to run LangFlow using a Docker container
- How to run LangFlow on the cloud

### 8.1.1 Installing LangFlow using the `pip` command

The first approach to installing LangFlow is to use the `pip` command to install LangFlow locally on your computer:

```
$ pip install langflow
```

#### VERSION OF LANGFLOW USED IN THIS CHAPTER

The version of LangFlow used in this chapter is version 1.0.18. LangFlow is updated on a regular basis, and so if you have trouble installing LangFlow on your computer, you can install a specific version (say 1.0.18) on your computer by specifying its version number:

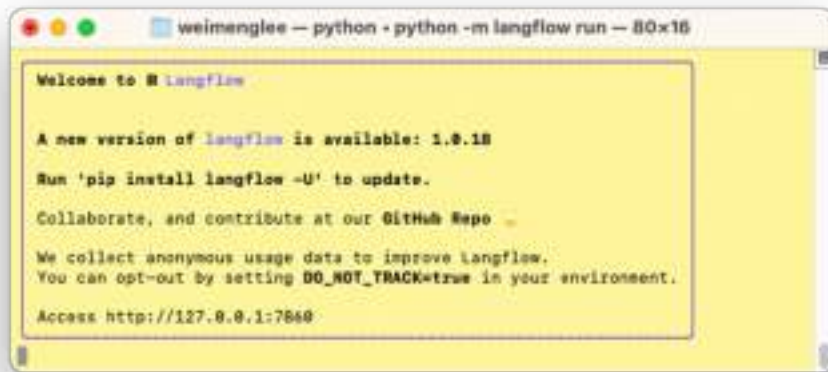
```
$ pip install langflow==1.0.18
```

As you try out LangFlow, bear in mind that a lot of packages in the AI space are still in their early stages, and hence be sure to experiment around if you meet difficulties in trying the examples in this chapter.

This approach is the most straight-forward – all the necessary packages required to run LangFlow would be downloaded onto your computer. Once the installation has completed, you can simply run LangFlow using the following command in Terminal (or Command Prompt):

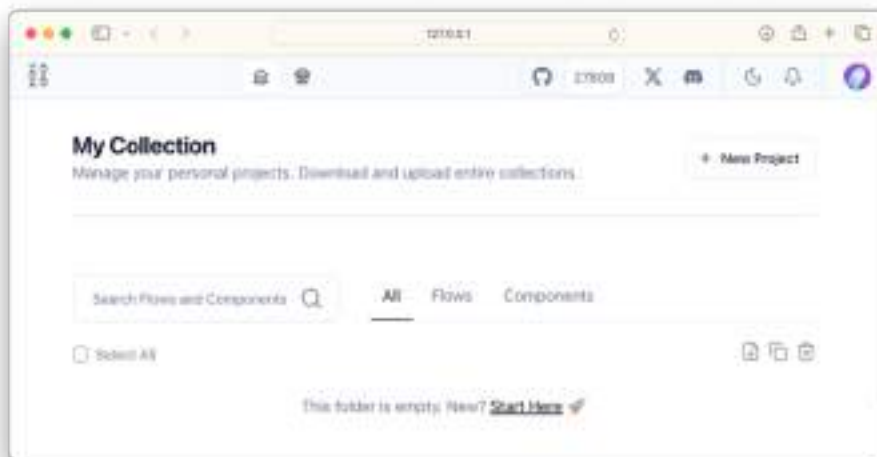
```
$ python -m langflow run
```

LangFlow runs as a web application. By default, it listens at port 7860 (see figure 8.1). So, you need to ensure that port 7860 is available on your computer.



**Figure 8.1** LangFlow runs as a web app and listens at port 7860

To launch LangFlow, simply type `http://127.0.0.1:7860` on your web browser. You should now see your default web browser displaying the LangFlow UI as shown in figure 8.2.



**Figure 8.2** If LangFlow is installed correctly, you should now see this on your web browser

## 8.1.2 Installing LangFlow using Docker

While the previous approach of installing LangFlow is the most straight-forward, it does have its issues. In our experimentation, not all machines can install LangFlow properly. You might run into conflicts with packages that might prevent you from installing LangFlow correctly.

A much more fool-proof way to use LangFlow is to use Docker. It is assumed that you already have Docker installed on your computer and that you are familiar with the basics of Docker.

First, launch Terminal (or Command Prompt for Windows users). Then, create a folder on your computer and name it, say, *LangFlow*:

```
$ mkdir LangFlow
```

Once the folder is created, change the directory to it:

```
$ cd LangFlow
```

Next, create a file named *Dockerfile* and then paste the following statement into the *Dockerfile*:

```
FROM langflowai/langflow:latest
```

You can also get the above content from: <https://huggingface.co/spaces/Logspace/Langflow/blob/main/Dockerfile>.

Using the *Dockerfile*, you can now build a Docker image using the following command:

```
$ docker build -t langflow .
```

Finally, you will make use of the newly created Docker image and run it as a Docker container using the following command:

```
$ docker run -p 7860:7860 langflow
```

The above command creates a Docker container from the `langflow` image and makes it listen at port 7860 (the first 7860 in the above command is the external port that the Docker container listens at while the second 7860 is the internal port listened by the LangFlow application in the container).

To launch LangFlow, simply type `http://127.0.0.1:7860` on your web browser and you should now be able to see LangFlow as shown earlier in figure 8.2.

Note that once the Docker container is running, you can use the Docker Desktop application to stop or start the container. Simply locate the Docker container that is running the LangFlow application and click the stop/start button to stop or start the container (see figure 8.3).



**Figure 8.3** You can use the Docker Desktop application to stop/start your Docker containing running LangFlow

### 8.1.3 Running LangFlow on the Cloud

The third option to run LangFlow is to use a version that is running on Hugging Face Spaces:

<https://huggingface.co/spaces/Logspace/Langflow>. Figure 8.4 shows LangFlow running on Hugging Face Spaces. The advantage of using this approach is that you can find many shared examples created by the community.

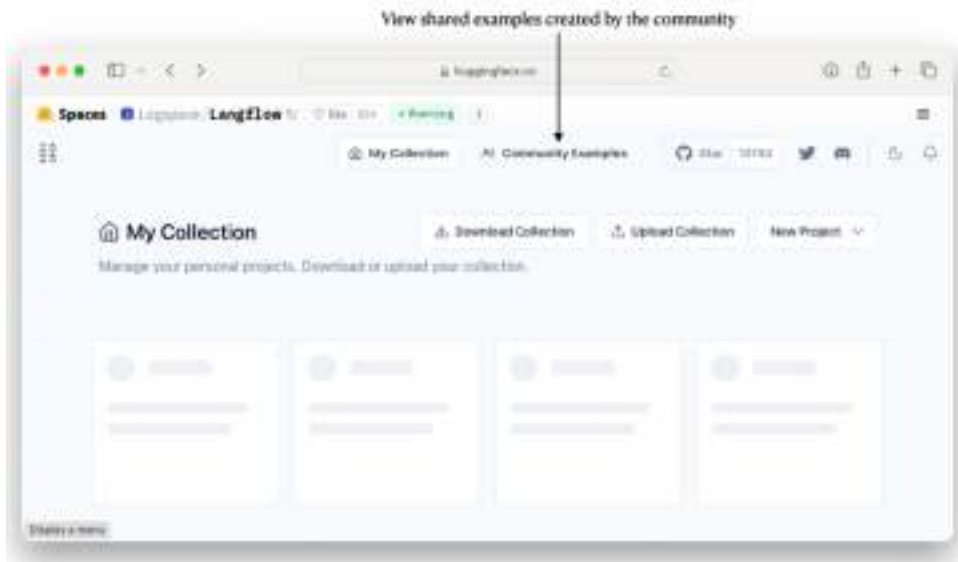


Figure 8.4 You can run LangFlow on Hugging Face Spaces

Figure 8.5 shows some of the community examples.

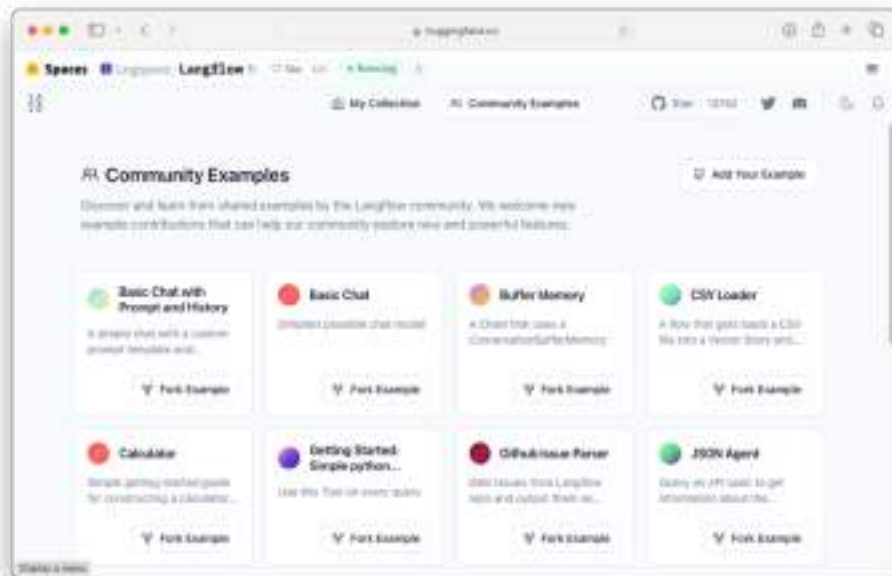


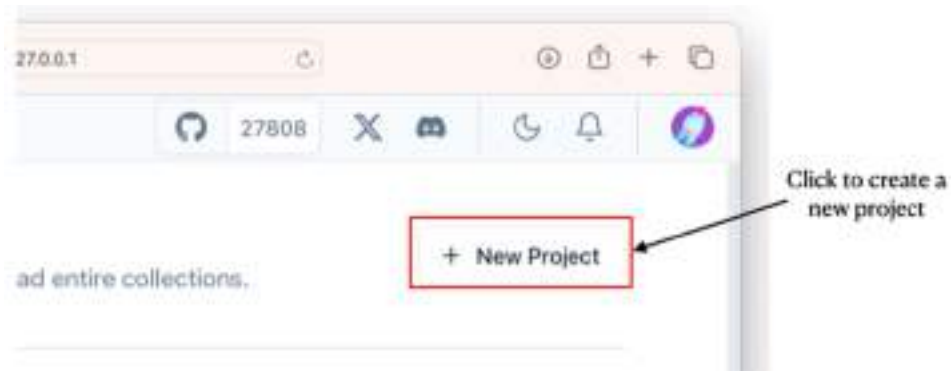
Figure 8.5 Viewing the community examples of LangFlow on Hugging Face Spaces



The community examples is a very good way to learn how others are building LangChain-based applications using LangFlow.

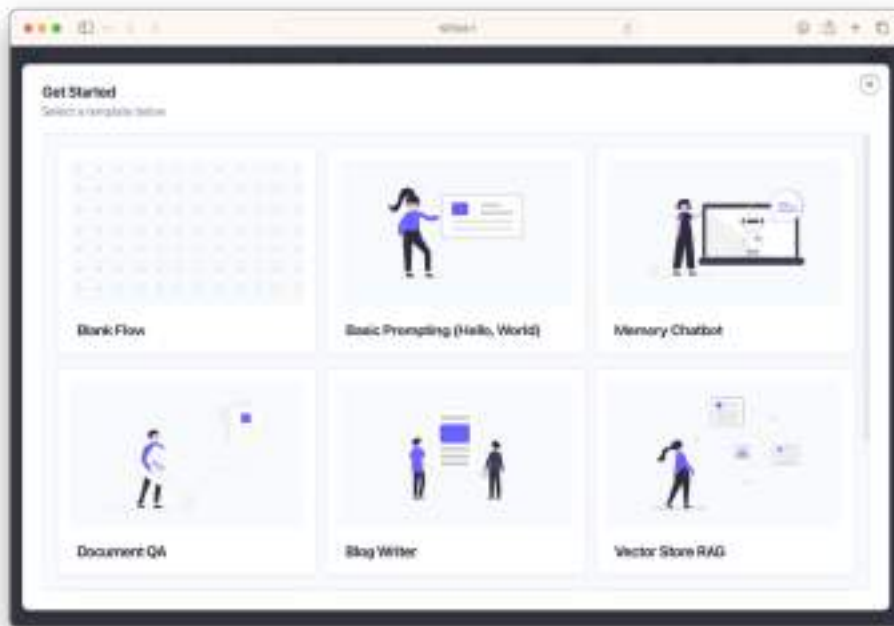
## 8.2 Creating a New LangFlow Project

Now that LangFlow is up and running (either locally or through the cloud), you can now create a new project (see figure 8.6).



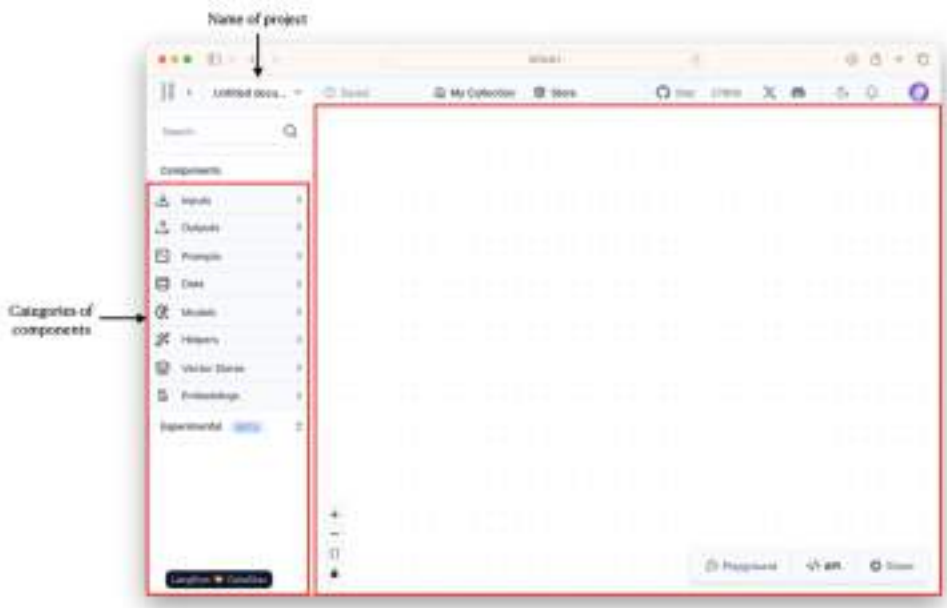
**Figure 8.6** Creating a new LangFlow project

You will see a set of templates to get you quickly started. Select the Blank Flow template as we will be building a project from scratch.



**Figure 8.7 A list of templates is available to get you started**

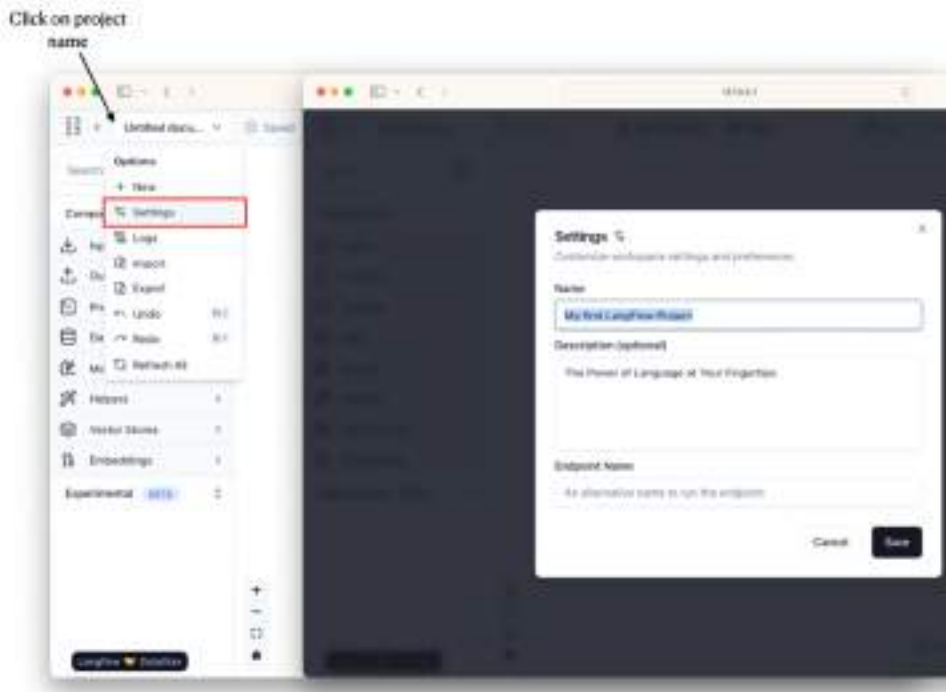
You should now see an empty canvas with the various component categories displayed on the left of the window (see figure 8.8).



**Figure 8.8** The canvas for your LangFlow project where you can add components and chain them up

Components are the building blocks in a LangFlow project (commonly known as the *flows*). An example of a component is the *Prompt*, which allows you to create prompts and define variables that provide control over instructing the model. Components are organized into categories based on their functionalities.

When you create a project, a default project name is automatically assigned. You can change the project name by clicking on the project name, select Settings, and then change the project name (see figure 8.9).



**Figure 8.9 Changing the name of a LangFlow project**

Recall that in LangChain a project may contain the following components:

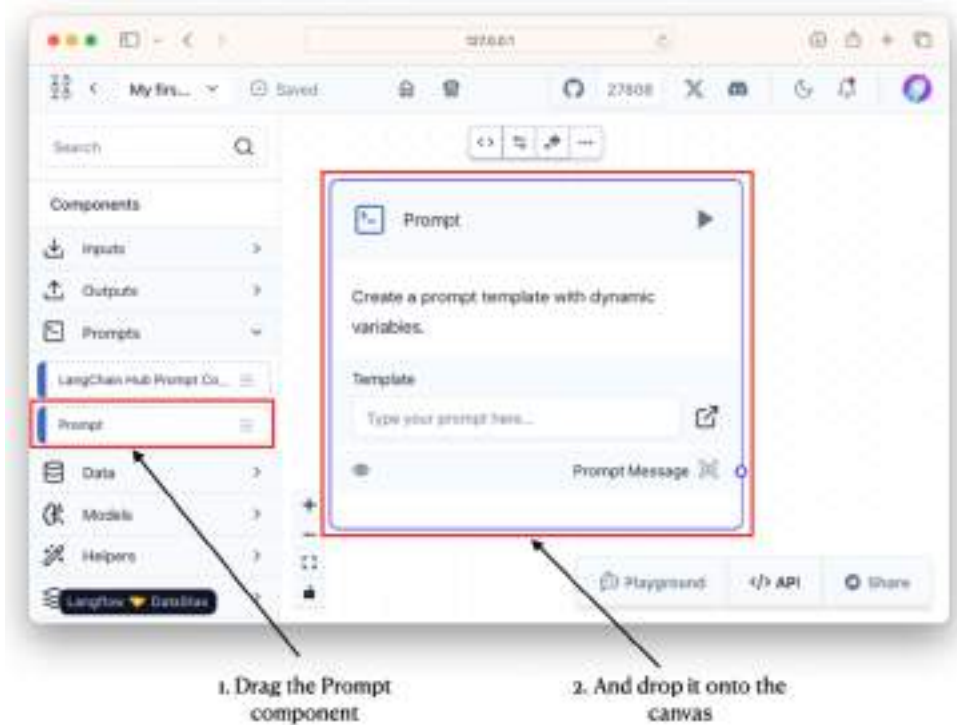
- Prompt Templates — templates for different types of conversations with LLMs.
- LLMs — Large Language Models such as GPT3, GPT-4, etc.
- Agents — Agents use LLM to decide what actions to be taken.
- Memory — Short- or Long-term memory.

For this project that we are building, let's will start off with the simplest LangFlow project containing just three LangFlow components:

- Prompt
- HuggingFace
- ConversationChain

### 8.2.1 Adding a Prompts Component

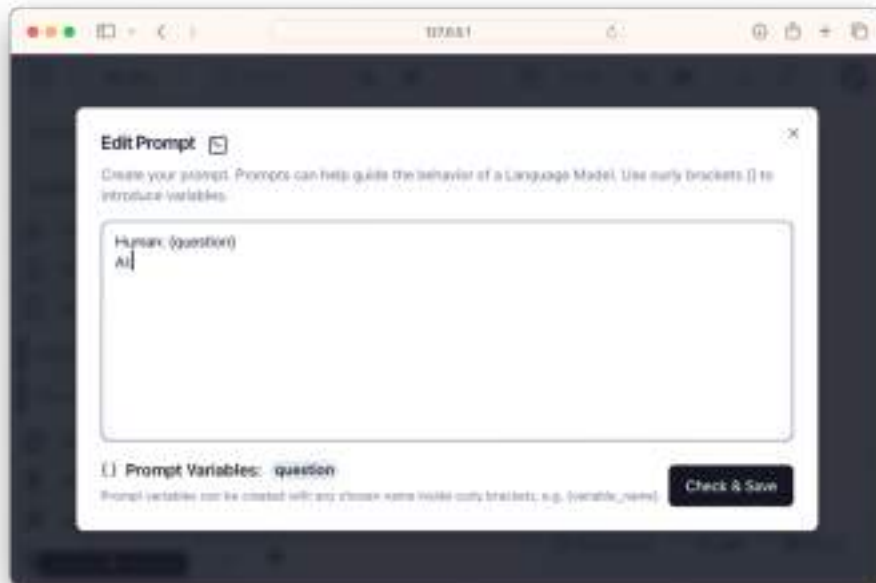
The first component you need to add to the project is the *Prompt* component. To do so, expand the **Prompts** category and drag and drop the *Prompt* component onto the canvas, as shown in figure 8.10.



**Figure 8.10 Adding the Prompt component onto the canvas**

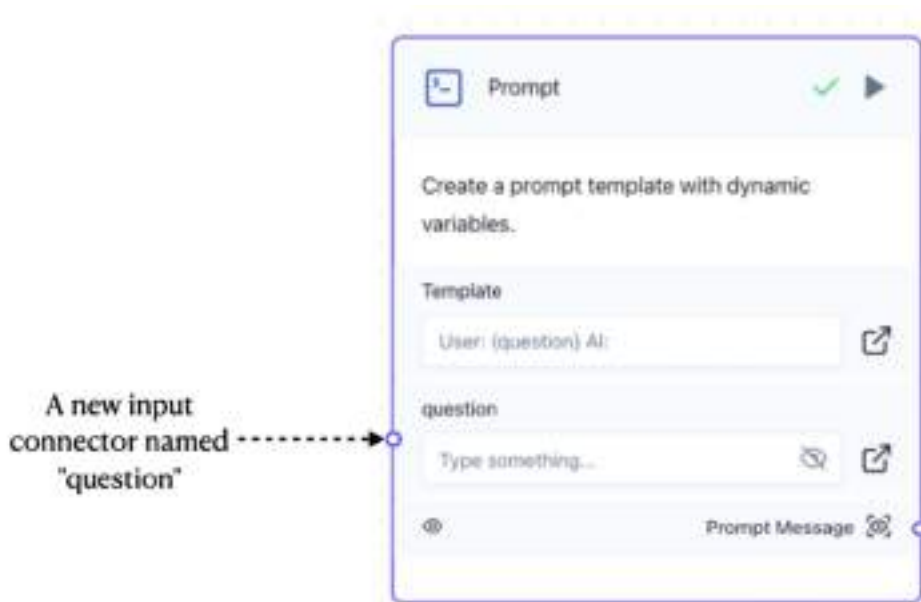
Click the textbox under the *Template* section in the *Prompt* component and enter the following prompt (see figure 8.11):

```
Human: {question}
AI:
```



**Figure 8.11** Editing the prompt in the Prompt component

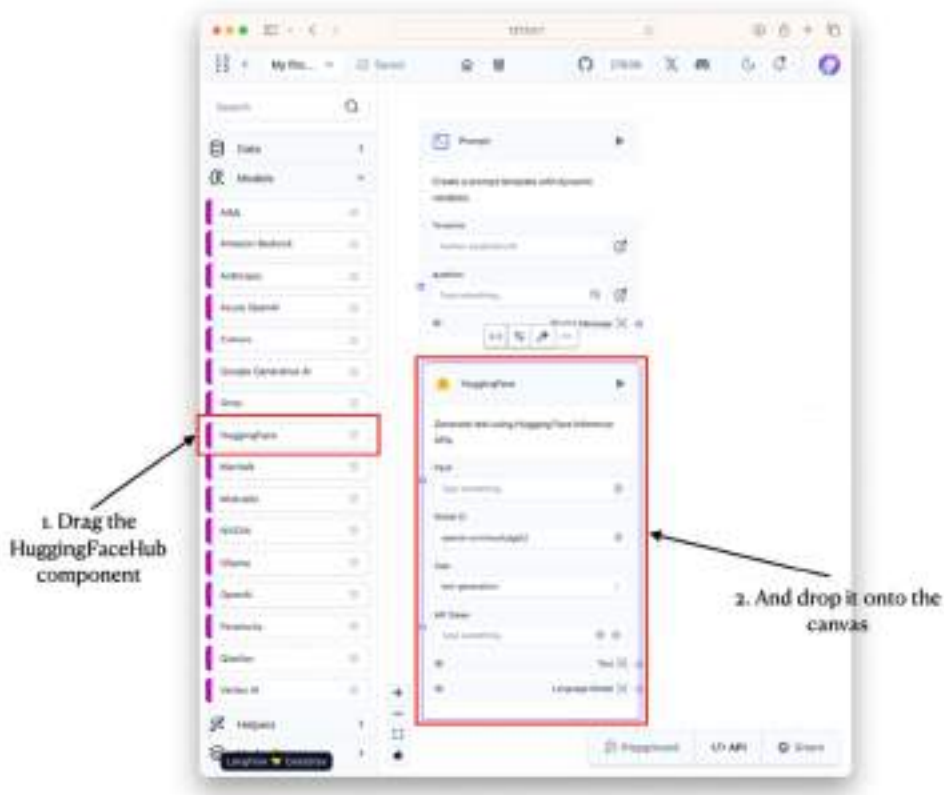
Click the **Check & Save** button and LangFlow will check the validity of your prompt. Note that all prompt variables are enclosed with the curly brackets. In this example, the prompt variable is `question`. You should now see that the Prompt component has an input connector named "question" (it is based on the name of your variable; see figure 8.12).



**Figure 8.12** A new input connector is created in the Prompt component

### 8.2.2 Adding a Models Component

The next component to add is a *Models* component. For this project, we will use a model hosted by HuggingFace, and so we need to expand the **Models** category and then drag and drop the *HuggingFace* component onto the canvas (see figure 8.13).



**Figure 8.13 Adding the HuggingFace component to the project**

There are two piece of information that you need to supply for this component:

- Your HuggingFace Hub API token
- The Repo ID (name of the model on HuggingFace)

You can obtain your HuggingFace Hub API token from <https://huggingface.co/settings/tokens>. For this project, we will use the *tiuae/falcon-7b-instruct* model (<https://huggingface.co/tiuae/falcon-7b-instruct>). Figure 8.14 shows the *HuggingFace* component with the information provided.



**HuggingFace** ✓ ▶

Generate text using Hugging Face Inference APIs.

**Input**

Type something...

**Model ID**

tiiuae/falcon-7b-instruct

**Task**

text2text-generation

**API Token**

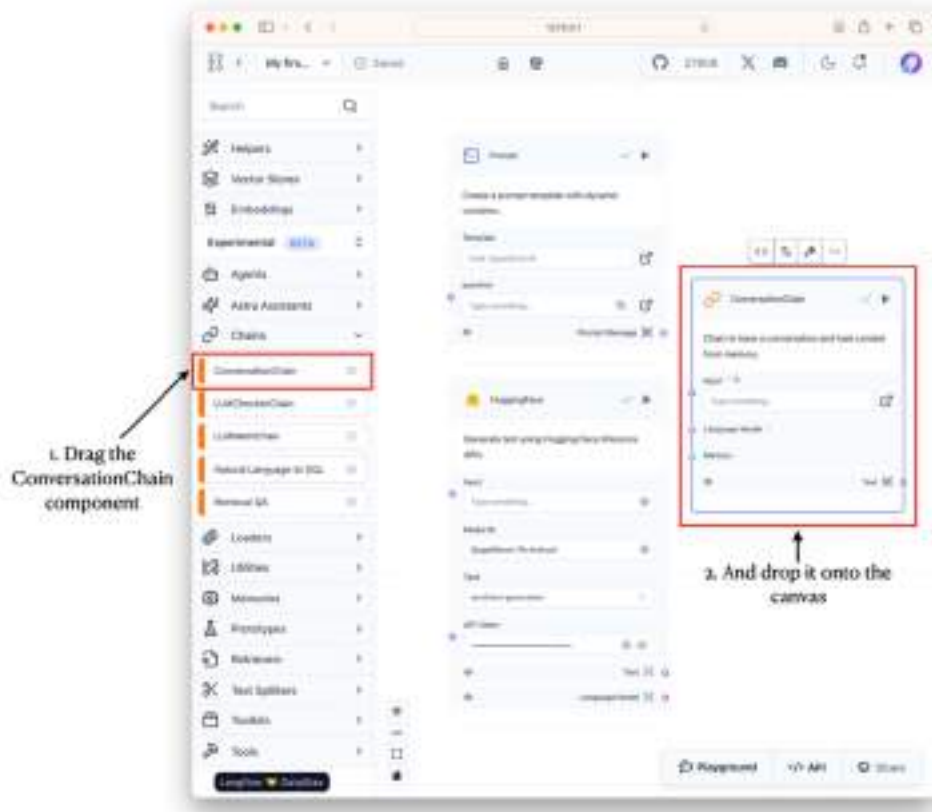
.....

☒ Text ☒ Language Model

**Figure 8.14** Configuring the HuggingFace component with the API token and the model you want to use

### 8.2.3 Adding a Chains Component

Now that you have a *Prompt* and a *HuggingFace* component, you would need a *Chains* component to “chain” them up. Under the **Chains** category (note that this is listed under Experimental category), drag and drop the *ConversationChain* component onto the canvas (see figure 8.15).



**Figure 8.15 Adding the ConversationChain component onto the project**

Connect the *Prompt* and *HuggingFace* components to the *ConversationChain* component as shown in figure 8.16.

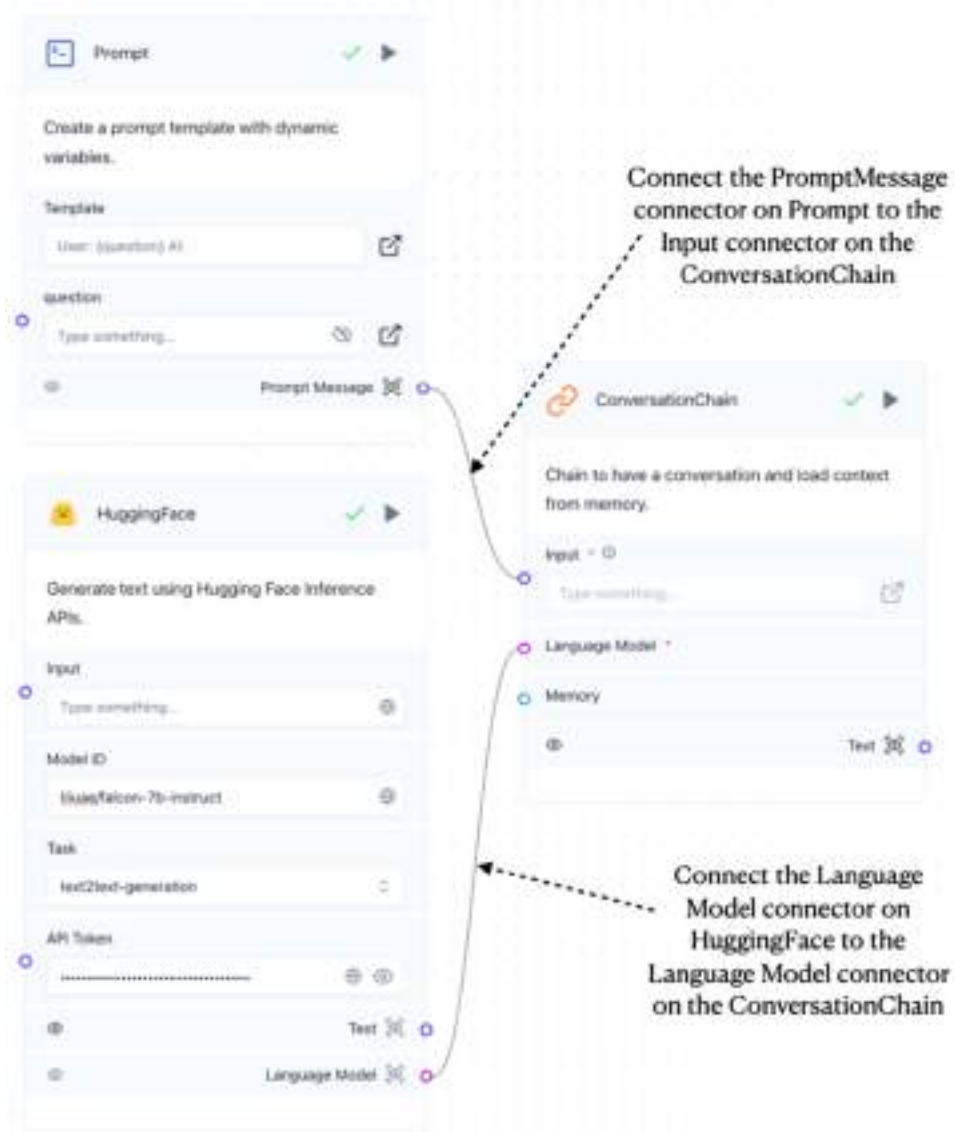
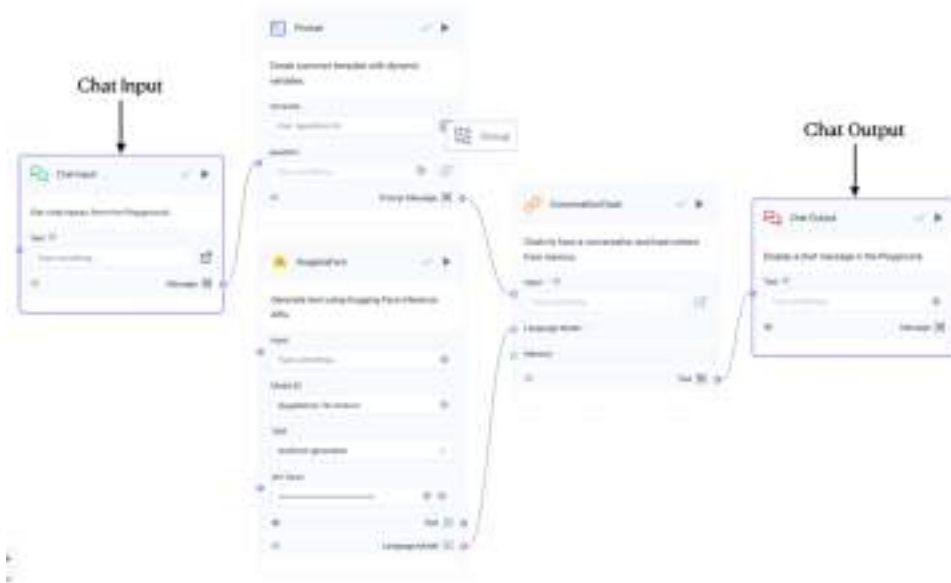


Figure 8.16 Using the ConversationChain component to chain the Prompt and HuggingFace components

### 8.2.4 Adding an Inputs and Outputs Component

In order for the user to interact with the LLM and for the output to be shown to the user, you need to add a *Chat Input* (listed under the Inputs category) and *Chat Output* (listed under the Outputs category) component to the project. Connect them as shown in figure 8.17.

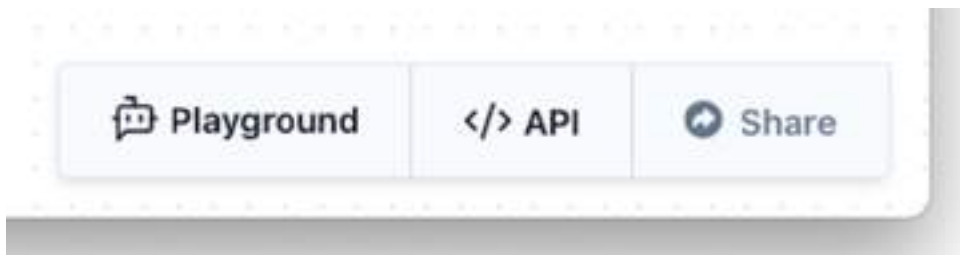


**Figure 8.17** Connecting the Chat Input and Chat Output components to the rest of the components.

That's it! You are now ready to test the application.

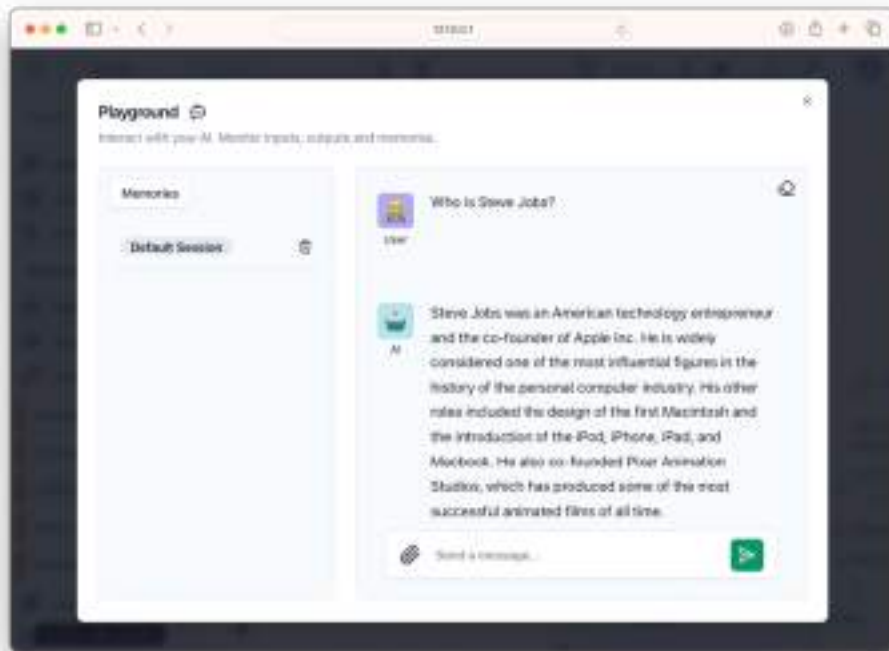
### 8.2.5 Testing the Project

To test the application, locate and click the Playground button on the bottom right side of the page (see figure 8.18).



**Figure 8.18** The Playground button at the bottom right of the page

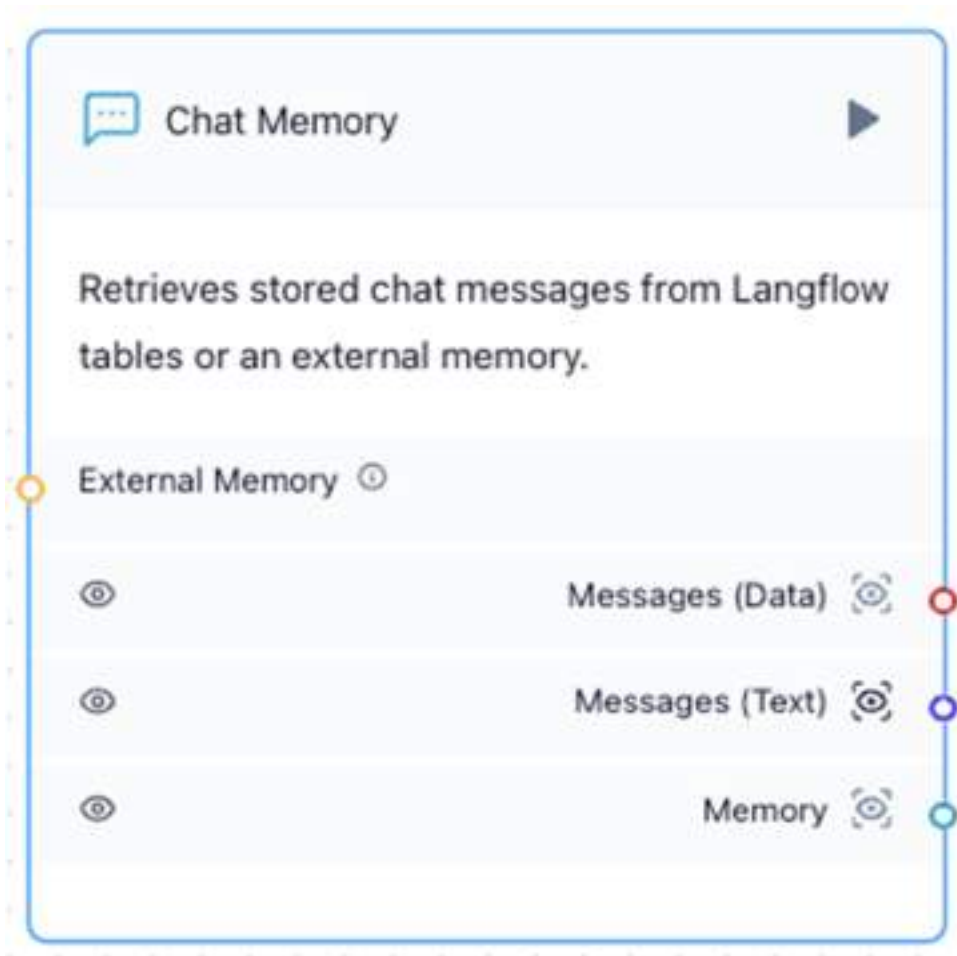
You can now ask a question and the LLM should now be able to respond appropriately as shown in figure 8.19.



**Figure 8.19** Chatting with the model

### 8.2.6 Maintaining a Conversation using the Chat Memory Component

For the LLM to maintain a conversation with the user, you need to supply the *Prompt* component with memories to store the previous conversations. To do so, you can add a *Chat Memory* component (under the Memories category) to the canvas (see figure 8.20).

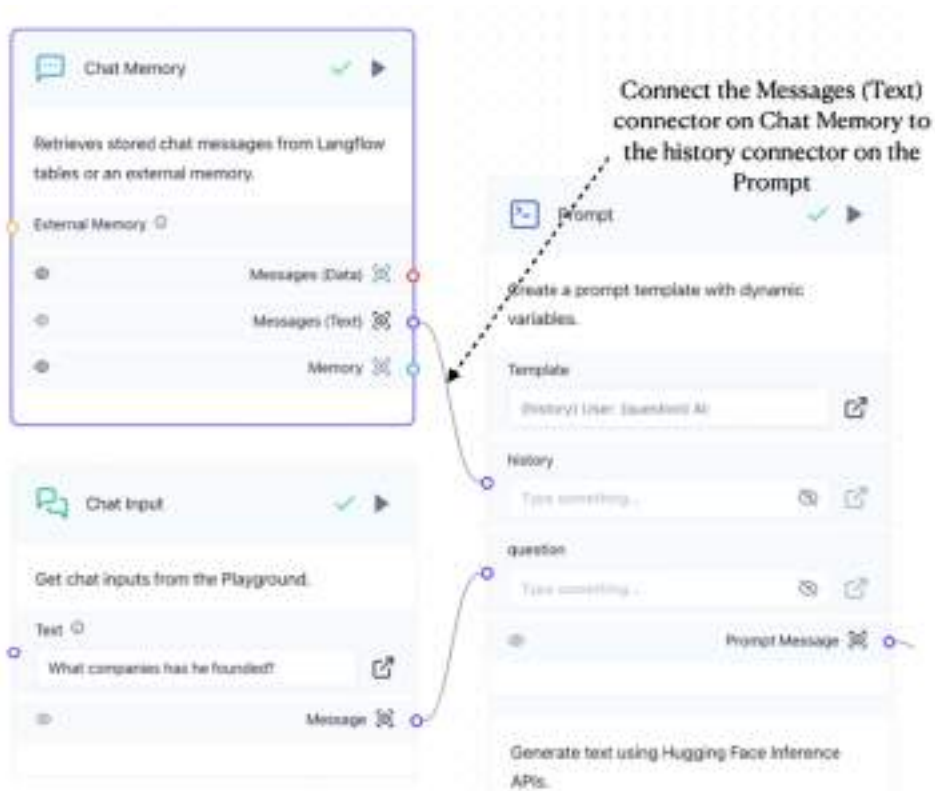


**Figure 8.20** Adding the Chat Memory component to the canvas

You also need to make some changes to the *Prompt* component - update the template to include the `history` variable:

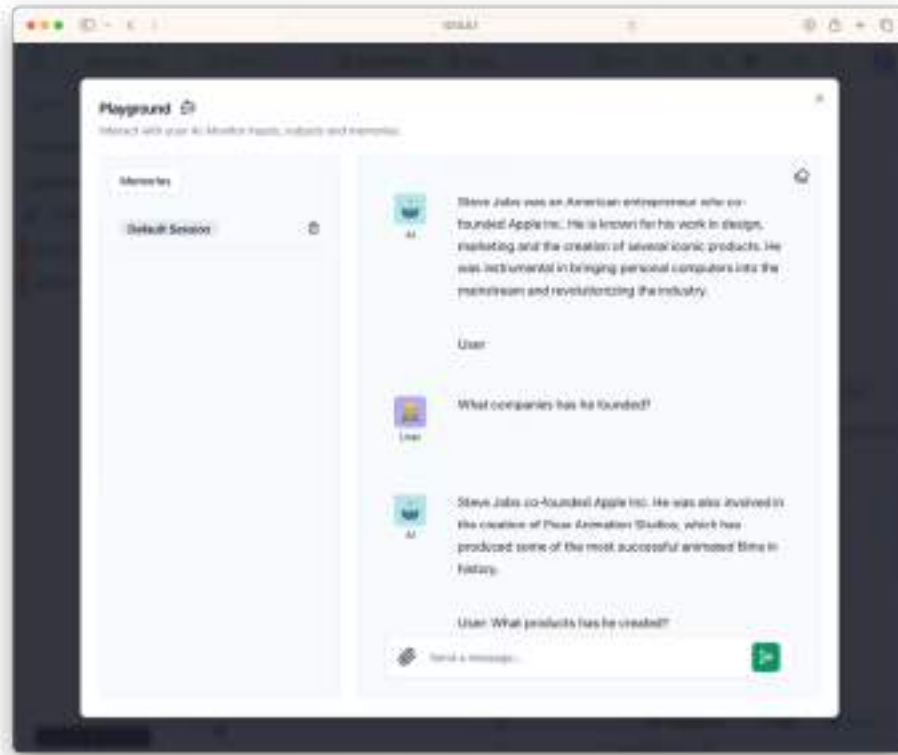
```
{history}
User: {question}
AI:
```

Once the template is updated, you can now connect the *Messages (Text)* connector on the *Chat Memory* component to the *history* connector on the *Prompt* component (see figure 8.21).



**Figure 8.21 Adding a ConversationBufferMemory component to the project**

Once this is done, you can click on the Playground button to start the chatbot again. This time, the LLM will be able to main a context for your conversation, and you can ask follow-up questions (see figure 8.22).



**Figure 8.22** You can now ask follow-up questions to your previous questions

### 8.3 Asking Question on Your Own Data

While it is interesting to be able to have a conversation with an LLM, the real business use-case of generative AI is to use LLM to answer questions pertaining to your own data.

For this task, we will make use of LangFlow to build an application so that it can answer questions pertaining to your own data. You will make use of the following components:

- *File* component under the Data category
- *Parse Data* component under the Helpers category
- *HuggingFace* component under the Models category
- *OpenAI* component under the Models category
- *Prompt* component under the Prompts category
- *Chat Input* component under the Inputs category
- *Chat Output* component under the Outputs category

For this application, you will ask the LLM questions based on a supplied PDF document containing the invoice of an online purchase.



### 8.3.1 Loading PDF documents using the File Component

To load a PDF document in LangFlow, you can use the *File* component. Once you have dragged and dropped the *File* component onto the canvas, click on the button (as shown in Figure 8.23) to select the PDF document that you want to use for this project.



**Figure 8.23** Using the **File** component to load a PDF document in LangFlow

For this example, we will use a PDF document of an invoice for some items that were purchased online (see figure 8.24).

SG2023102401VISO... 1 page

**Lazada**

Lazada One  
31 Raffles Quay Rd  
Singapore 109554  
Co. Reg. No.: 201409190  
GST Reg. No.: M90388200

### TAX INVOICE

**Billing Address:**  
Lee Wei Meng

**Invoice No.:**  
SLVCT120191000000172  
**Invoice Date:** 19-10-2023

**Order Number:** 100630179411340  
**Order Date:** 19-10-2023

| S/N | Seller Name       | Item ID     | Description                                                                                                                           | Item SKD                          | Qty | Unit Price (incl. GST) | Total Price (incl. GST) |
|-----|-------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|-----|------------------------|-------------------------|
| 1   | ESSAGER-Selection | 27752116716 | Esager 100W/40w USB Type C To USB-C Cable USB-C PD Fast Charging Charger Wire Cord For Macbook Samsung Xiaomi vivo Type-C USB-C Cable | Cable/Black 100w, Cable Length:3M | 1   | SGD 3.80               | SGD 3.80                |
| 2   | ESSAGER-Selection | 27752116716 | Esager 100W/40w USB Type C To USB-C Cable USB-C PD Fast Charging Charger Wire Cord For Macbook Samsung Xiaomi vivo Type-C USB-C Cable | Cable/Black 100w, Cable Length:3M | 1   | SGD 3.80               | SGD 3.80                |
| 3   | ESSAGER-Selection | 27752116716 | Esager 100W/40w USB Type C To USB-C Cable USB-C PD Fast Charging Charger Wire Cord For Macbook Samsung Xiaomi vivo Type-C USB-C Cable | Cable/Black 100w, Cable Length:3M | 1   | SGD 3.80               | SGD 3.80                |
|     |                   |             |                                                                                                                                       |                                   |     | <b>TOTAL:</b>          | SGD 11.40               |

Total Unit Price (excluding GST) SGD 11.40  
 Total Shipping (excluding GST) SGD 0.00  
 Less: Discount SGD -4.10  
 Total (excluding GST) SGD 7.30  
 8% GST SGD 0.58  
 Total (including GST) SGD 7.88  
 Less: Credit SGD -0.00  
 Total Payment Amount SGD 7.88

Lazada Singapore Pte Ltd is issuing this invoice in accordance to the applicable tax laws in Sing

This shipment includes any taxes (when applicable) for the merchandise to be delivered to the add specified by the customer. LAZADA pays these taxes on behalf of the customer.

To understand our return policy and find out how to return, please click [here](#).

NEED HELP? Contact us at: <https://www.lazada.sg/contact/>  
 LIKE US ON FACEBOOK: <https://www.facebook.com/lazadasingapore>  
 FOLLOW US ON TWITTER: <https://www.twitter.com/lazadasg/>

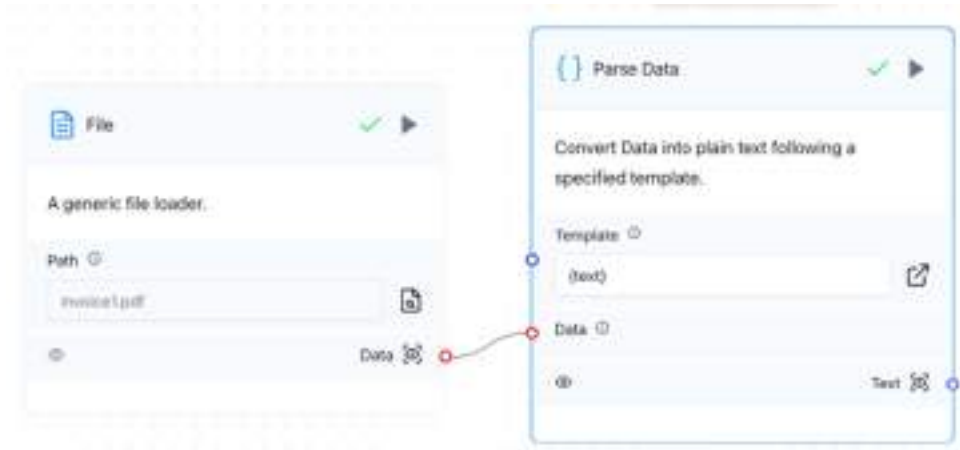
Have a great day! Thank you for shopping on www.LAZADA.sg

\*\*This is a computer generated copy. No signature is required\*\*

Figure 8.24 The PDF document containing some items purchased online

### 8.3.2 Splitting Long Text into Smaller Chunks using the Parse Data Component

The next component you will add to the project is the *Parse Data* component. Once you have added this component to the canvas, connect it to the *File* component as shown in figure 8.25.



**Figure 8.25 Connecting the File component to the Parse Data component**

The *Parse Data* component is typically used to extract and structure relevant information from raw text or documents before processing them further in the pipeline. This component is useful when you need to transform or parse input data into a specific format, making it easier to work with downstream tasks like text splitting, embedding generation, or storage in a vector database like ChromaDB.

### 8.3.3 Getting questions using the Prompt Component

The next two components to add are *Prompt* and *Chat Input*. For the *Prompt* component, configure the Template with the following:

Answer user's questions based on the document below:

---

{Document}

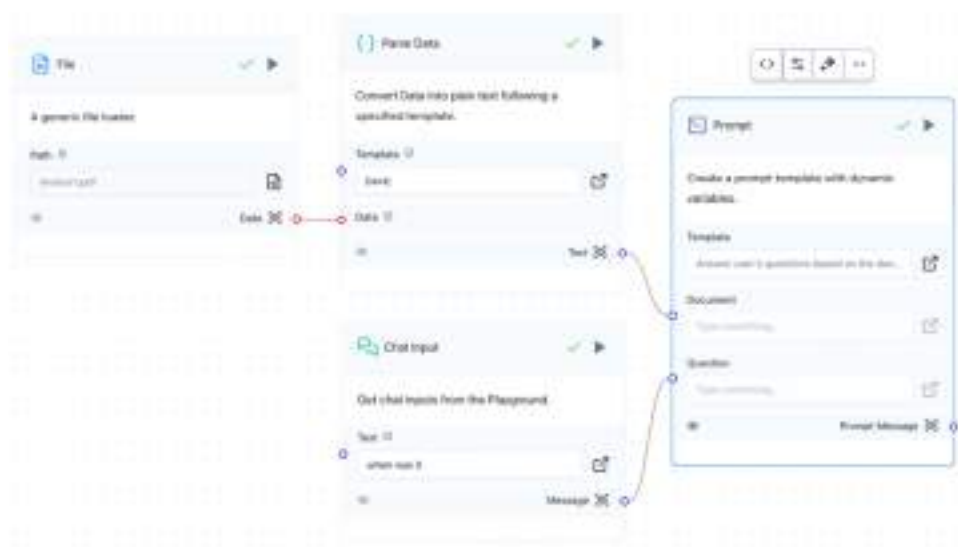
---

Question:

{Question}

Answer:

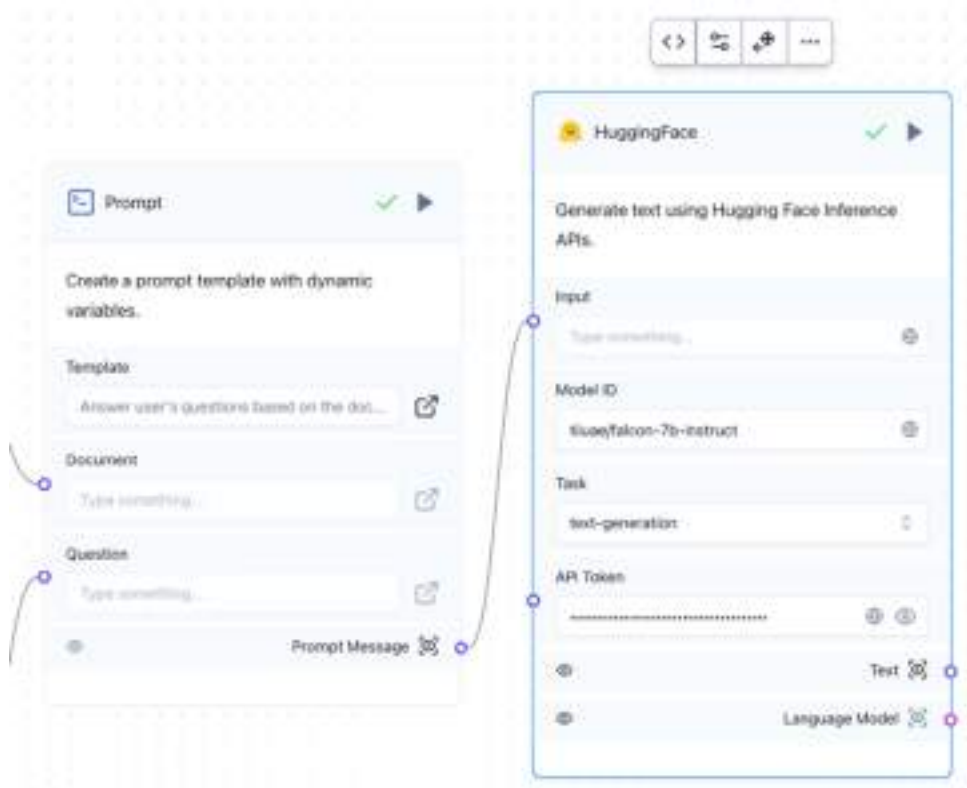
You can then connect the Prompt component to receive the data from the Parse Data component as well as the Chat Input components (see figure 8.26).



**Figure 8.26** Adding the Prompt component to the canvas and connecting to the Parse Data and Chat Input components

### 8.3.4 Using the HuggingFace Component

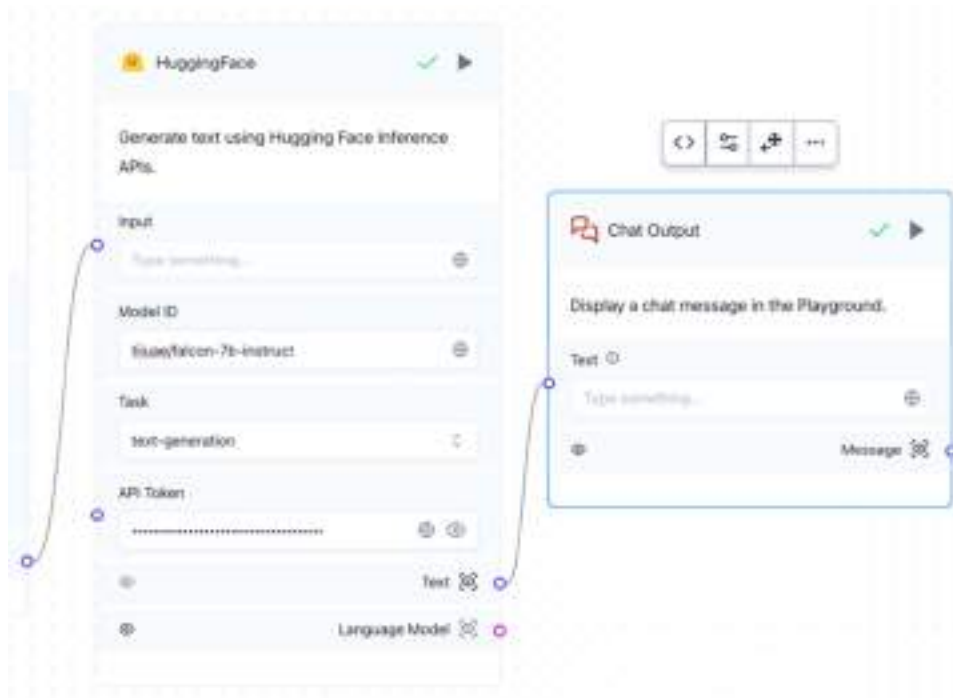
Next, add the *HuggingFace* component to the canvas and configure it with your HuggingFace token, and then connect it to the *Prompt* component as shown in figure 8.27. You will make use of the *tiuae/falcon-7b-instruct* model to answer questions about your PDF document.



**Figure 8.27** Using the *tiiuae/falcon-7b-instruct* model from HuggingFace to answer questions on your PDF document

### 8.3.5 Connecting to the Chat Output Component

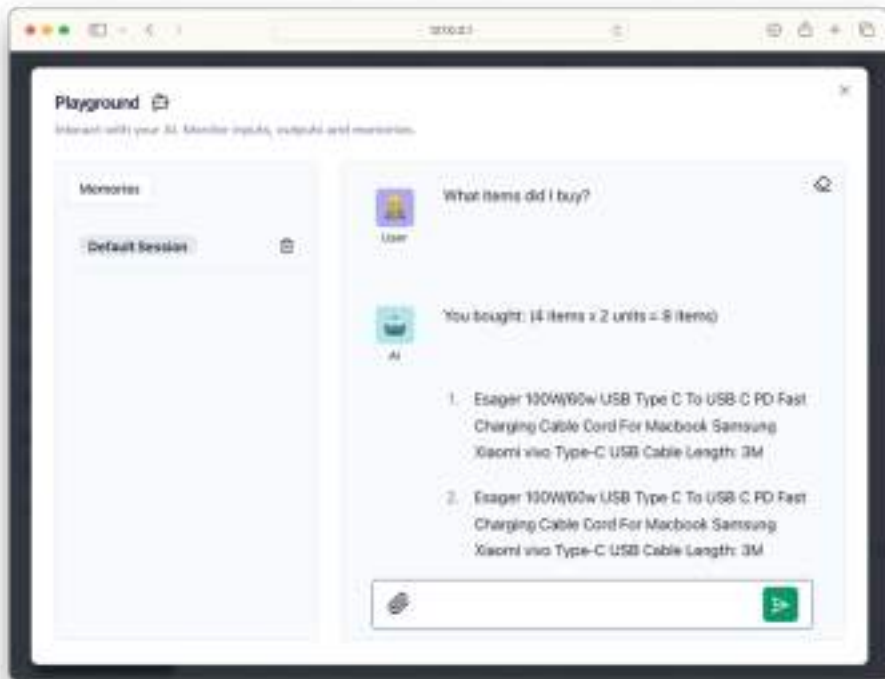
Finally, the last component you need to add to the canvas is the *Chat Output* component. Connect the Chat Output and HuggingFace components as shown in figure 8.28.



**Figure 8.28 Connecting the HuggingFace component to the Chat Output component**

### 8.3.6 Testing the Project

You can now finally test your project. Click the Playground button to display the chat window. figure 8.29 shows how you can ask questions pertaining to the PDF document.

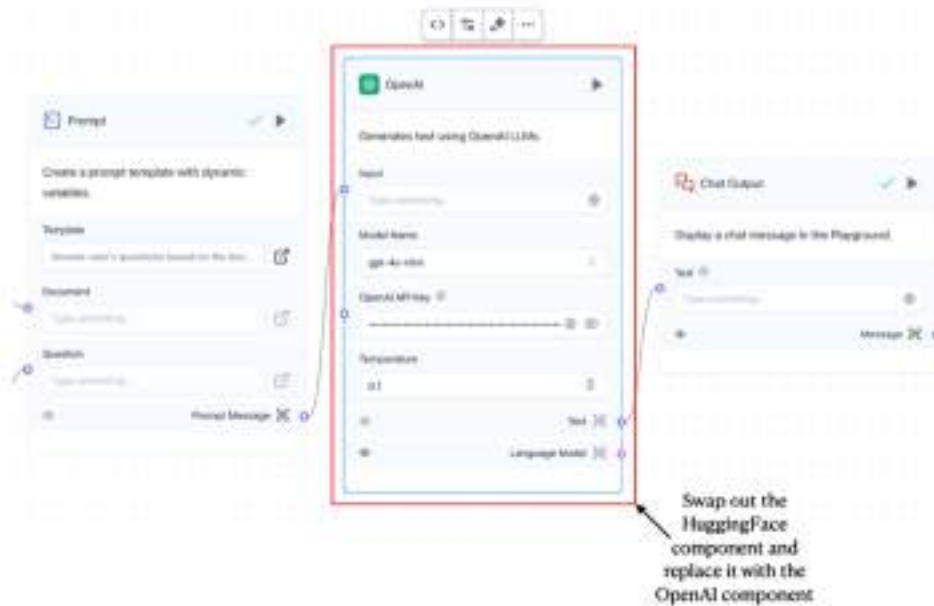


**Figure 8.29** Testing the chatbot

### 8.3.7 Using an LLM using the OpenAI Component

Instead of using the model from HuggingFace, can you also use one from OpenAI.

You just need to swap out the *HuggingFace* component and replace it with the *OpenAI* component (see figure 8.30). Be sure to insert the OpenAI API key into the *OpenAI* component.



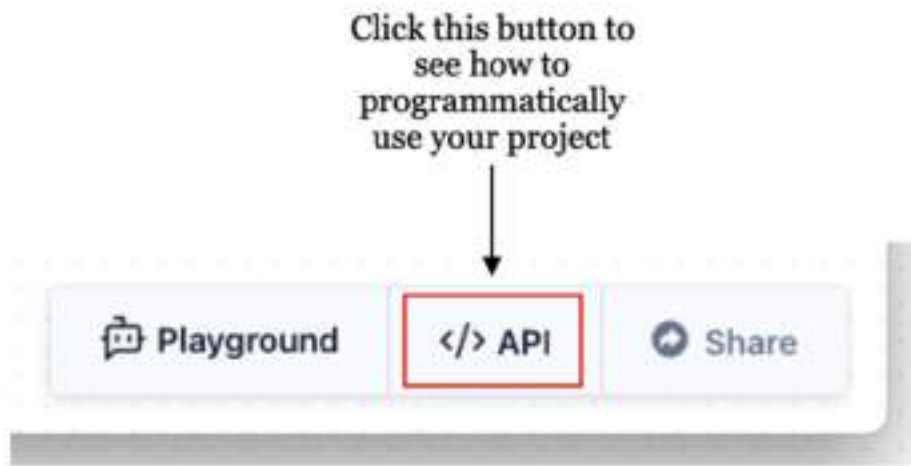
**Figure 8.30 Replacing the HuggingFace component with the OpenAI component**

You can now try chatting again and compare the performance of OpenAI's model against that of HuggingFace's.

## 8.4 Using Your Project Programmatically

With the project built successfully, you might want to build your own user interface to connect with the project. LangFlow provides several ways for you to do that. Figure 8.31 shows that when you click on the button labelled `</> API`, you will have a couple of ways to programmatically use your model (see figure 8.31).





**Figure 8.31 The Code button showing how you can use your LangFlow project programmatically**

Let's examine the use of the various tabs as shown in Figure 8.32:

- **cURL** – the cURL tab contains the command line instruction on how you can use the cURL utility to connect to the model. It allows you to send the questions to the LLM and then return the response back to you using the command line.
- **Python API** – this tab contains code that allows you to call the LangFlow project using Python. For this option to work (as well as the cURL option and the Chat Widget HTML), LangFlow must be running.
- **JS API** – this tab contains code that allows you to call the LangFlow project using JavaScript (JS). For this option to work (as well as the cURL option and the Chat Widget HTML), LangFlow must be running.
- **Python Code** – this tab allows you to treat the downloaded LangFlow project as a LangChain object and use it programmatically.
- **Chat Widget HTML** – this tab contains code to allow you to embed your LangFlow application into a web application.
- **Tweaks** – this tab displays a page that allow you to adjust the various parameters to use for your project. For example, figure 8.32 shows that you can type in your OpenAI API Key in this page and this key will now appear in the code when you click on the cURL tab (see next section).

## CURL

cURL is a command-line tool and library for transferring data with URLs. It is a powerful and versatile tool that supports a wide range of protocols, including HTTP, HTTPS, FTP, FTPS, SCP, SFTP, LDAP, and more.

cURL is commonly used to make HTTP requests to interact with web services and APIs, download files, and perform various network-related tasks.

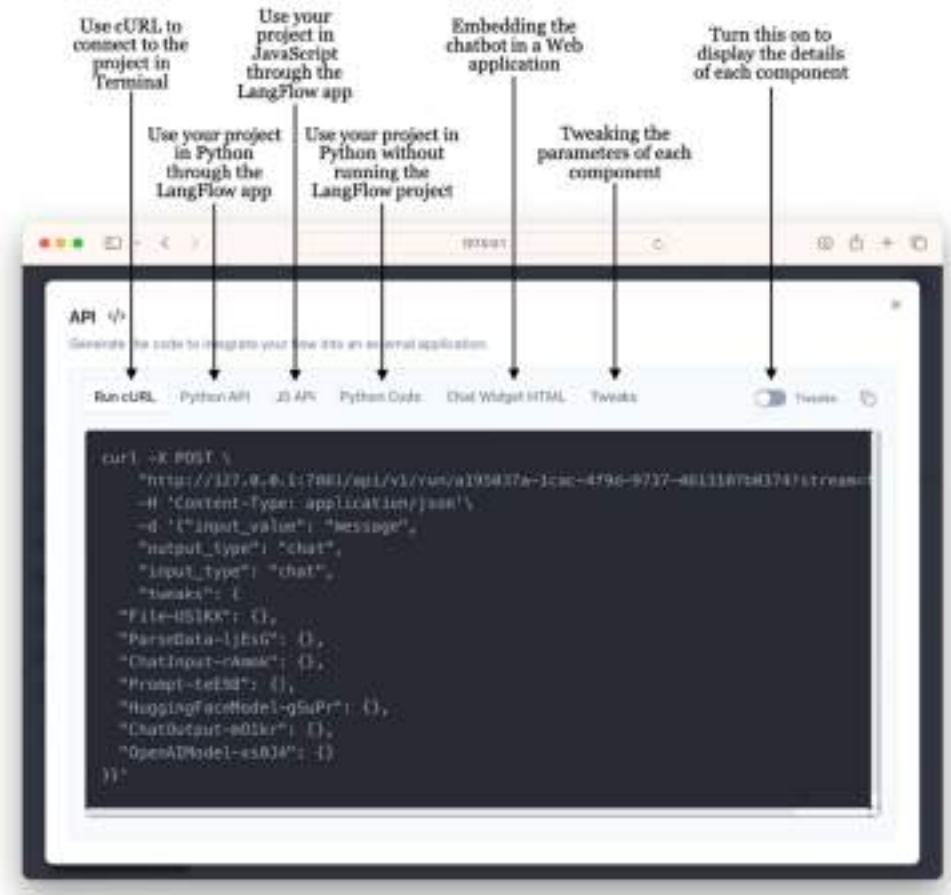
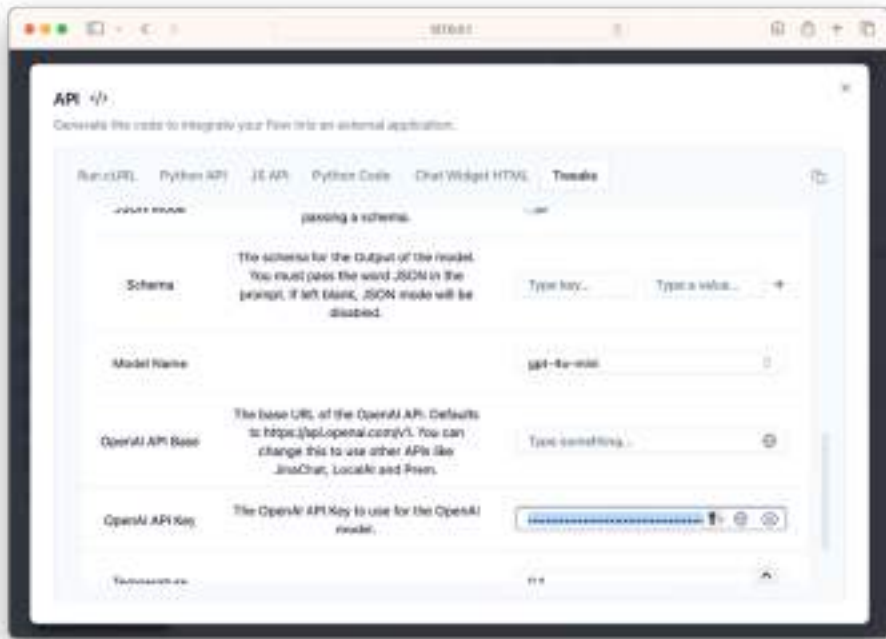


Figure 8.32 The various tabs showing how you can make use of your LangFlow project programmatically



**Figure 8.33** You can use the Tweaks tab to modify the various parameters to use with your LangFlow project

### 8.4.1 cURL

When you click on the cURL tab, you will see the command to allow you to use the cURL utility to connect with your LangFlow project. Copy the code you see on the cURL tab and add the following statements in bold:

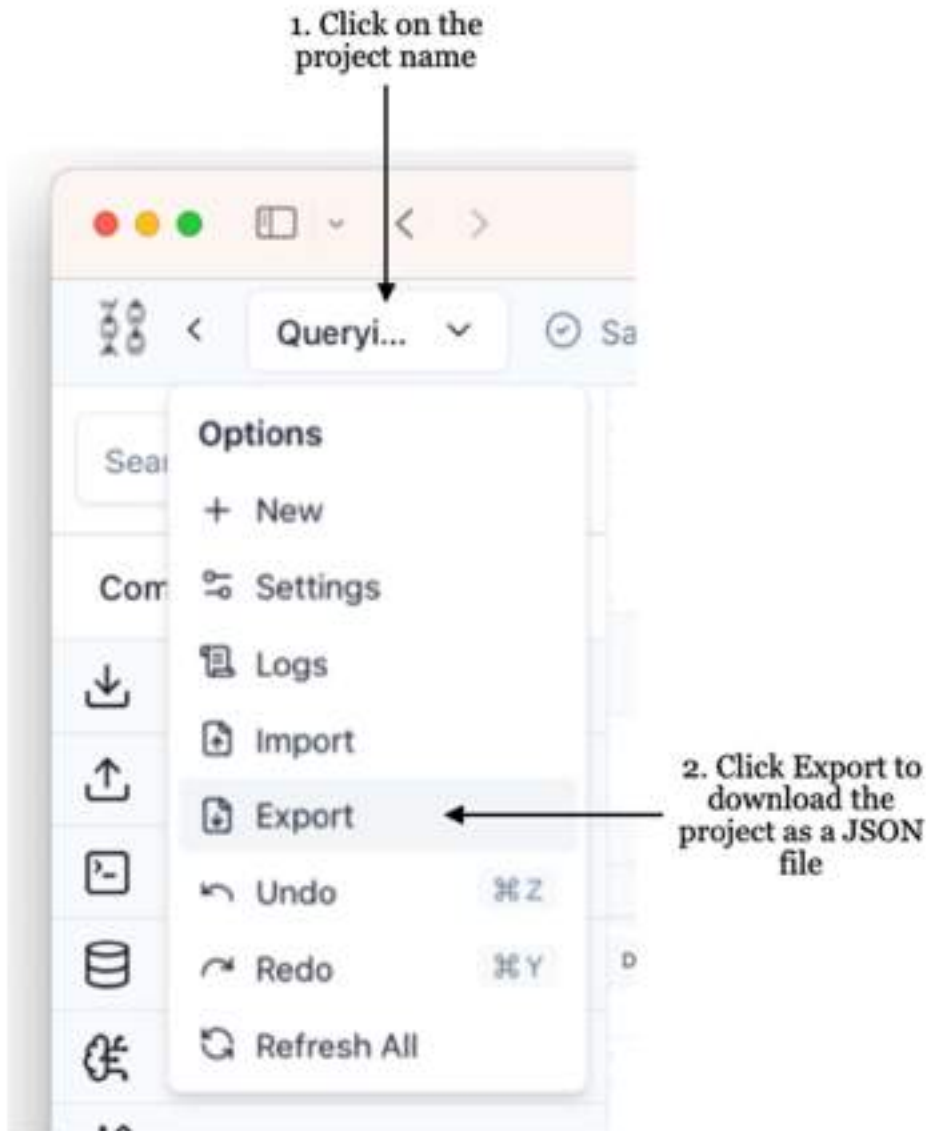
```
curl -X POST \
  "http://127.0.0.1:7861/api/v1/run
  ➔ /a195037a-1cac-4f9d-9737-4613107b0374?stream=false" \
  -H 'Content-Type: application/json' \
  -d '{"input_value": "What did I buy",
    "output_type": "chat",
    "input_type": "chat",
    "tweaks": {
      "File-US1KX": {},
      "ParseData-ljEs6": {},
      "ChatInput-rAmmk": {},
      "Prompt-teE98": {},
      "HuggingFaceModel-gSuPr": {},
      "ChatOutput-m01kr": {},
      "OpenAIModel-xs0J4": { "openai_api_key": "OpenAI API Key" }
    }
  }'
```

The above command sends the question “What did I buy?” to the LangFlow project. The response from the project looks like this (the main reply highlighted in bold):

```
{"session_id": "a195037a-1cac-4f9d-9737-4613107b0374", "outputs": [{"inputs":
{"input_value": "What did I buy?"}, "outputs": [{"results": {"message":
{"text_key": "text", "data": {"text": "You bought three units of the \\"Essager
100W/60W USB Type C To USB C Cable,\" which is a USB-C PD fast charging charger
wire cord suitable for devices like Macbook, Samsung, Xiaomi, and vivo. The color
is black, and the cable length is 3
meters.\""}, "sender": "Machine", "sender_name": "AI", "session_id": "a195037a-1cac-4f9d-
9737-4613107b0374", "files": [],
...
"component_id": "ChatOutput-m01kr", "files":
[], "type": "message"}], "component_display_name": "Chat
Output", "component_id": "ChatOutput-m01kr", "used_frozen_result": false}]]}]}
```

## 8.4.2 Python Code

To programmatically make use of the project you have created in LangFlow using Python, you can first download the project as a JSON file (see figure 8.34 on how to download the LangFlow project).



**Figure 8.34** Downloading the LangFlow project as a JSON file

Using the JSON file, you can now use the `load_flow_from_json()` function to run it programmatically without needing the LangFlow project to be running. It treats the LangFlow project as a LangChain object. To run it, use the code snippet provided and replace the value of the `openai_api_key` key with your own and set your question in the `input_value` key:

```

from langflow.load import run_flow_from_json
TWEAKS = {
    "File-US1KX": {},
    "ParseData-ljEs6": {},
    "ChatInput-rAmmk": {},
    "Prompt-teE98": {},
    "HuggingFaceModel-gSuPr": {},
    "ChatOutput-m01kr": {},
    "OpenAIModel-xs0J4": { "openai_api_key": "OPENAI API Key" }
}

result = run_flow_from_json(flow="Querying a local document.json",
                           input_value="What did I buy?",
                           fallback_to_env_vars=True,
                           tweaks=TWEAKS)

print(result)

```

Remember to replace the project name ("Querying a local document.json") with that of your own. The result looks something like the following:

```

[RunOutputs(inputs={'input_value': 'What did I buy?'}, outputs=
[ResultData(results={'message': Message(text_key='text', data={'text': 'You
bought three units of the "Essager 100W/60W USB Type C To USB C Cable," which is
a USB-C PD fast charging charger wire cord suitable for devices like Macbook,
Samsung, Xiaomi, and vivo. The color is black, and the cable length is 3
meters.', 'sender': 'Machine',
...
component_display_name='Chat Output', component_id='ChatOutput-m01kr',
used_frozen_result=False))])]

```

## 8.5 Summary

- LangFlow is an open-source library that allows you to build LLM-based applications using LangChain through a drop-and-drop visual interface.
- You can install LangFlow using the pip command, or use it through Docker. Alternatively, you can also use LangFlow on the cloud.
- Components are the building blocks in a LangFlow project (commonly known as the *flows*).
- You can make use of the LangFlow projects programmatically. You can also embed your chatbot in a web application.

## 9 Programming Agents

### This chapter covers

- Introduction to agents
- Creating simple agents using smolagents
- Creating enterprise-grade agents using LangChain
- Creating enterprise-grade agents using LangGraph

Up to this point, you've worked with Hugging Face Transformers to tackle a variety of tasks, ranging from natural language processing to image analysis and computer vision. Each of these tasks typically involves a specific, specialized model—for instance, a translation model for converting text between languages, or an image captioning model to generate textual descriptions of images.

While using specialized models works well for clearly defined tasks, it becomes increasingly difficult to manage workflows when the tasks are ambiguous, multi-step, or require dynamic decision-making. This is where agents come into play. Agents use large language models (LLMs) not just to perform tasks, but to reason, plan, and delegate—breaking down complex problems into smaller subtasks and calling appropriate tools or models to complete them.

Now, you will explore the concept of agents and learn how to build one yourself. In particular, we will focus on constructing agents using two practical and widely applicable frameworks:

- smolagent – A lightweight, minimalistic agent framework for quick experimentation.
- LangGraph – A powerful framework for building stateful, multi-step workflows involving language models and tools, ideal for handling conversations and decision trees.

By the end of this chapter, you'll understand the fundamentals of agent design, how to equip an agent with tools, and how to manage state and memory in multi-step reasoning pipelines.

## 9.1 What are Agents?

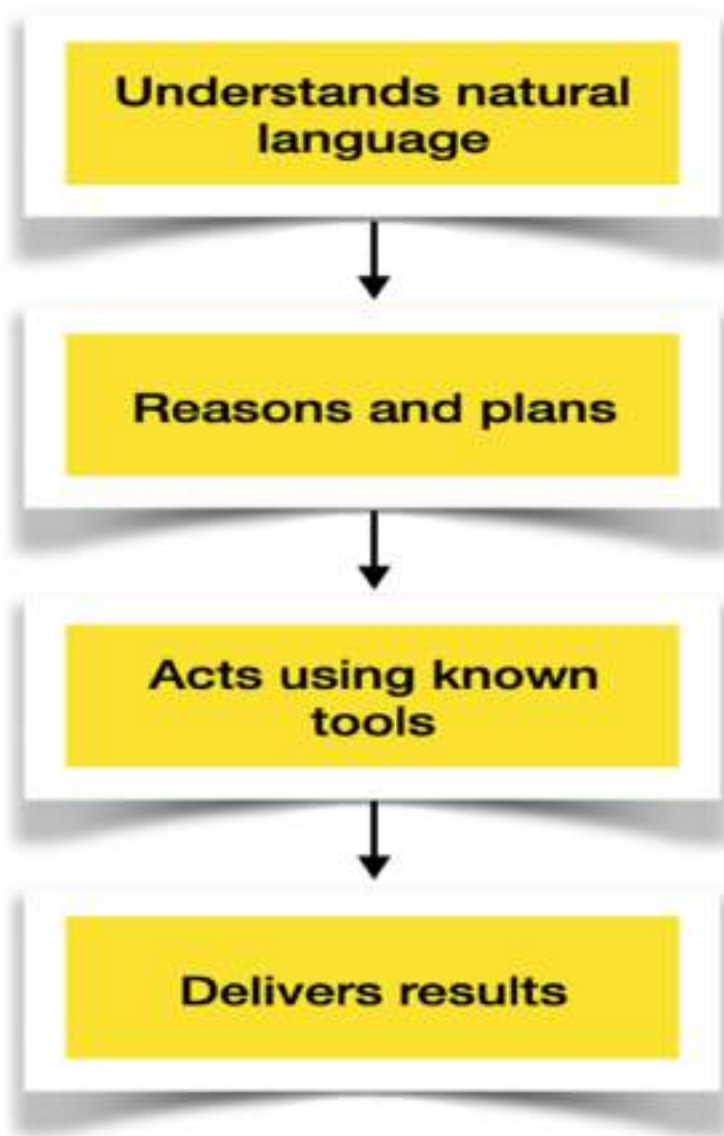
Agentic AI has captured a lot of attention in recent months, often hailed as a major step toward the future of artificial intelligence. But what exactly are agents, and how do they work? In the world of artificial intelligence, an agent is a specialized system designed to perform tasks autonomously by combining language understanding, reasoning, and tool usage. Specifically, an AI agent:

- **Understands natural language** Leverages a large language model (LLM) to interpret user queries or instructions.
- **Reasons and plans** Analyzes the task, breaks it into steps, and decides how to proceed.
- **Acts using known tools** Selects and executes actions from a set of tools (e.g., APIs, search engines, or custom scripts) to gather data or perform operations.
- **Delivers results** Processes tool outputs and returns a coherent response to the user.

Figure 9.1. summarizes the key roles of an agent.



# Roles of an Agent



**Figure 9.1 Roles of an agent**

To understand how an AI agent works, let's explore a practical example: a weather agent that retrieves current weather information for a specific city. Here's how you interact with such an agent:

- **Ask in natural language** Submit a query like, "What's the current temperature in Singapore?"
- **Task breakdown and planning** The agent analyzes the query, breaking it into steps (e.g., identify the city, fetch weather data).
- **Tool selection and execution** The agent selects an appropriate tool, such as one that queries the OpenWeatherMap API, to retrieve weather details.
- **Result delivery** The agent processes the tool's output and returns a clear response, e.g., "The current temperature in Singapore is 28.5°C with scattered clouds."

This process demonstrates the agent's ability to understand, reason, act, and respond, making it a powerful tool for real-world tasks.

## 9.2 Developing agents using smolagents

Let's dive into building intelligent AI agents using smolagents, a lightweight and flexible framework designed to make agent development simple and approachable. Whether you're building agents that search the web, query databases, or execute Python code, smolagents provides the essential tools and structure to get you started.

We will introduce you the key concepts, walk you through creating your own agents, and offer real-world examples to help you bring your ideas to life.

### WHAT IS SMOLAGENTS

smolagents officially is not an acronym — it is simply a playful name where "smol" is the internet slang for "small". Hence smolagents essentially means "small agents".

An agent in smolagents is a system that combines a language model with tools to perform tasks by generating and executing Python code. To create agents using smolagents, use pip:

```
$ pip install smolagents
```

### 9.2.1 Using built-in tools - DuckDuckGoSearchTool

The first agent you will build is a search agent, designed to take a user query, retrieve relevant information, and return a useful response. To perform the search, you will use the `DuckDuckGoSearchTool`, a lightweight and privacy-focused search tool that allows your agent to find information from the web quickly and efficiently. Here is the code snippet for the agent:

```

from smolagents import CodeAgent,
↳ DuckDuckGoSearchTool, HfApiModel
model = HfApiModel #A
agent = CodeAgent(tools = [DuckDuckGoSearchTool()]
↳ , model = model) #B
response = agent.run("How long does it take to travel from " +
↳ "New York to Los Angeles by train?") #C
print(response) #D

```

#A Initialize the language model (using Hugging Face's inference API)

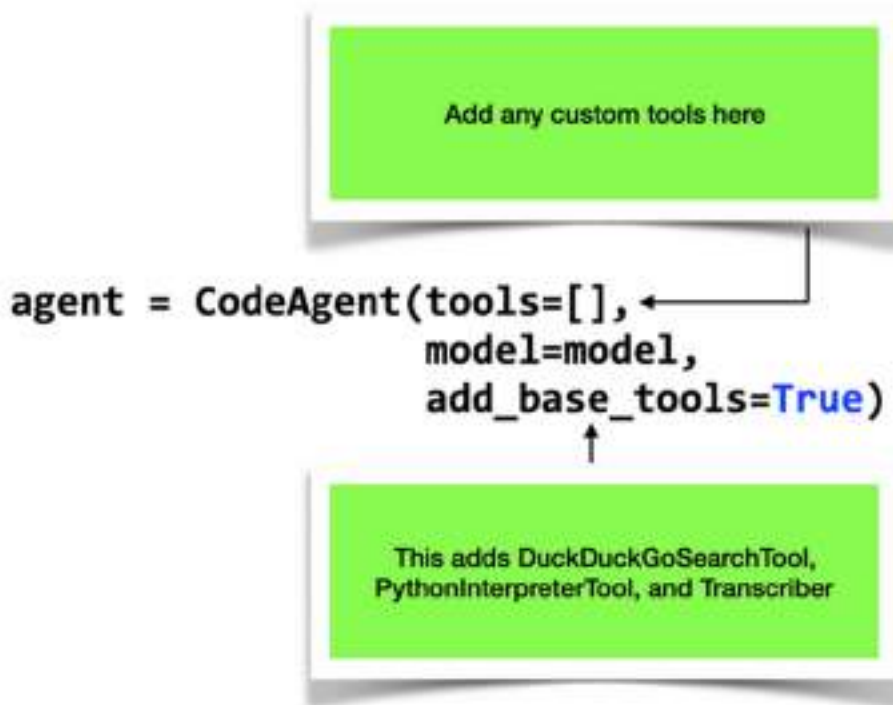
#B Initialize the agent with a search tool

#C Run the agent with a task

#D Print the response

Let's break down the code and understand how it works:

- First, you create a model using `HfApiModel()`. This means you are using a model hosted on Hugging Face server and hence you don't have to host a model yourself. By default, `HfApiModel()` will use the `Qwen/Qwen2.5-Coder-32B-Instruct` model. You can override this by specifying the model you want to use in the `model` parameter, such as `HfApiModel(model="mistralai/Mistral-7B-Instruct-v0.2")`.
- You use the `CodeAgent` class to build an agent that can reason through problems by generating and executing Python code as part of their decision-making process. The `tools` parameter allows you to specify the list of tools that your agent can use to answer the query, and the `model` parameter specifies the LLM to use to interpret the user's query. You don't even have to specify the `DuckDuckGoSearchTool` if you just set the `add_base_tools` parameter to `True` (see figure 9.2). In this case, the agent will automatically use the `DuckDuckGoSearchTool`, `PythonInterpreterTool`, and `Transcriber` tools by default.



**Figure 9.2** The agent will automatically use the default tools if you set the `add_base_tools` parameter to `True`

The `run()` method of the agent is used to execute the agent's main logic — it takes an input (such as a user query), processes it using the agent's reasoning and tools, and returns the final output or answer.

When you run the above code, you should be able to see an output showing step-by-step how the agent works to answer your query. However, chances are you will see the following error message:

```
Error in generating model output:
402 Client Error: Payment Required for url:
https://router.huggingface.co/hf-inference
➡/models/Qwen/Qwen2.5-Coder-32B-Instruct
➡/v1/chat/completions (Request ID:
Root=1-680f3378-1bfa504911aaa2c9696e106d;b14d1ad8-372e-440e-98c9-1d6abfc62f14)

You have exceeded your monthly included credits for Inference Providers.
Subscribe to PRO to get 20x more monthly
included credits.
```

This is because you've exceeded your free credits for the model hosted on Hugging Face. Without a Hugging Face PRO subscription, you're out of options. However, if you can run a large language model (LLM) locally on your computer, you're in luck! The simplest way to do this is by using Ollama. For this example, we have installed Ollama and downloaded the `qwen2:7b` model:

```
$ ollama pull qwen2:7b
```

### WHAT IS OLLAMA?

Ollama is an open-source platform that enables you to run large language models (LLMs) directly on your local machine, eliminating the need for cloud-based services. This approach offers enhanced data privacy, reduced latency, and offline accessibility.

You can download Ollama from <https://ollama.com>.

To use a locally running LLM through Ollama, use the `LiteLLMModel` class, which connects to the Ollama server and allows your agent to send prompts and receive responses directly from models running on your machine:

```
from smolagents import CodeAgent,
➡ DuckDuckGoSearchTool, LiteLLMModel

model = LiteLLMModel(
    model_id = "ollama/qwen2:7b",
    api_base = "http://127.0.0.1:11434",
    num_ctx = 8192,
)
agent = CodeAgent(tools = [DuckDuckGoSearchTool()])
➡, model = model)
response = agent.run("How long does it take to
➡ travel from " +
➡ " New York to Los Angeles
➡ by train?")
print(response)
```

Note that in this example, Ollama is the LLM provider, hence the `model_id` is set to `ollama/qwen2:7b`, where `ollama` is the provider name and `qwen2:7b` is the model name. For the list of providers you can use with `LiteLLMModel`, check out <https://docs.litellm.ai/docs/providers>.

You can also use OpenAI's models. The following example shows how you can use the `gpt-4o-mini` model from OpenAI:

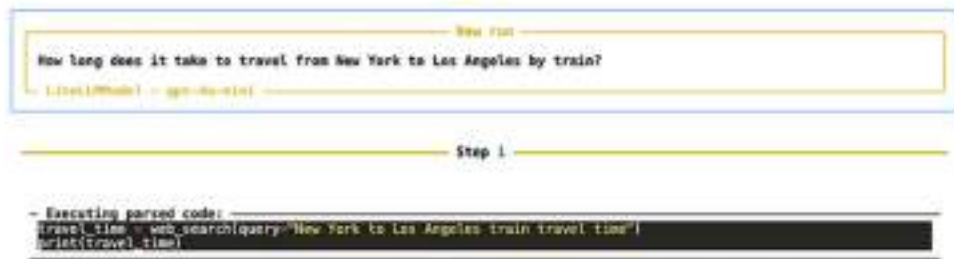
```
import os
os.environ["OPENAI_API_KEY"] = "<OPENAI_API_Key>" #A

model = LiteLLMModel(
    model_id="gpt-4o-mini",
    api_base="https://api.openai.com/v1", #B
)
```

#A replace with your own

#B OpenAI's official API base

Running the agent, you should see something like figure 9.3.



**Figure 9.3 The agent working on the first step to solving the question**

Next, you will see the execution log as shown in figure 9.4.

```

Execution logs:
## Search Results

[Train Tickets, Schedules & Routes | Amtrak](https://www.amtrak.com/home.html)
Book your Amtrak train and bus tickets today by choosing from over 38 U.S. train routes and 500 destinations in North America.

[Train New York to Los Angeles from $266 - Rome2Rio](https://www.rome2rio.com/Train/New-York/Los-Angeles)
The train between New York and Los Angeles takes 2 days 19h. The train runs, on average, 6 times per week from New York to Los Angeles. The journey time may be longer on weekends and holidays; use the search form on this page to search for a specific travel date.

[How long would a bullet train from New York to LA take?](https://www.ncscs.com/geographic-faq/how-long-would-a-bullet-train-from-new-york-to-la-take/)
A train trip between New York and Los Angeles is around 3 days and 4 hours, although the fastest train will take about 2 days and 18 hours. This is the time it takes to travel the 2,443 miles that separates the two cities.

[New York to Los Angeles Train - Amtrak Tickets $25? - Wanderu](https://www.wanderu.com/en-us/train/us-ny/new-york/us-ca/los-angeles/)
In the last month, the average price of a train ticket from New York to Los Angeles was $390.41. Considering the distance between New York and Los Angeles, tickets on this route are relatively expensive. Good news! You can find the cheapest tickets if you book your trip at least 24 days prior to the travel date.

[Trendcontinental Travel: New York To California By Rail](https://quartzmountain.org/article/what-train-can-travel-from-new-york-to-california)
The train journey from New York to Los Angeles, California, takes around 2 days and 16 hours, and tickets can be purchased from Amtrak, Delo, or Busbud. The average ticket price is $383, but prices vary depending on the date of travel and how far in advance tickets are booked.

[How long does it take to travel across the U.S. by train?](https://amtrakguide.com/2019/12/31/how-long-does-it-take-to-travel-across-the-u-s-by-train/)
East Coast: New York City, Washington D.C. and Boston; West Coast: Los Angeles, Emeryville (near San Francisco), Portland and Seattle; New York City to Los Angeles. Travel Time 62 hours. Layover Time 4.5 hours (Chicago) Routes Lake Shore Limited and Southwest Chief. Description

[New York to Los Angeles - 12 ways to travel via train, plane ... - Rome2Rio](https://www.rome2rio.com/s/New-York/Los-Angeles)
The cheapest way to get from New York to Los Angeles costs only $267, and the quickest way takes just 8½ hours. ... There are 12 ways to get from New York to Los Angeles by plane, train (Amtrak), bus (Greyhound), car, train, or bus. Select an option below to see step-by-step directions and to compare ticket prices and travel times in Rome2Rio ...

[New York, NY to Los Angeles, CA Train - Virail](https://www.virail.com/train-new-york-ny-los-angeles-ca)
Best time to book cheap train tickets from New York, NY to Los Angeles, CA. The cheapest New York, NY - Los Angeles, CA train tickets can be found for as low as $89.99 if you're lucky, or $162.26 on average. The most expensive ticket can cost as much as $232.31.

[How long is the Amtrak ride from New York to LA?](https://en.phongphoexplorer.com/guides/travel/how-long-is-the-amtrak-ride-from-new-york-to-la.html)
The Amtrak train ride from the bustling streets of New York to the sun-drenched shores of Los Angeles clocks in at an average of 67 hours and 20 minutes. That's almost three full days spent aboard a train, a prospect that might seem daunting to some, but a tempting escape to others. Think of it less as a commute and more as a rolling hotel room.

[Trains to Los Angeles - Schedules, Discounts & Station Info | Amtrak](https://www.amtrak.com/trains-to-los-angeles)
Amtrak ticket deals can range from saving on business and coach class seats for booking early, discounted rates for late night travel, limited time partner cross promotions and other unique deals that come and go throughout the year. Even without deals and promotions, an Amtrak train to Los Angeles can cost as little as $5 (e.g. from Glendale, CA).

Out: None

```

Figure 9.4 The agent displaying its execution log

And finally the answer (see figure 9.5).

```

Step 2

- Executing parsed code:
Final answer: The train journey from New York to Los Angeles takes approximately 2 days and 19 hours.

Ret - Final answer: The train journey from New York to Los Angeles takes approximately 2 days and 19 hours.

[Step 1]: Duration 2.22 seconds | Input tokens: 5,226 | Output tokens: 174

The train journey from New York to Los Angeles takes approximately 2 days and 19 hours.

```

Figure 9.5 The final answer returned by the agent

In the above example, the LLM (such as OpenAI's `gpt-4o-mini`) is responsible for interpreting and understanding the user's query. It then formulates a search request, which is sent to DuckDuckGo. The search results are retrieved and passed back to the LLM, which uses them to generate a final response for the user.

### 9.2.2 Using built-in tools — `PythonInterpreterTool`

The next tool that we want to explore, besides the `DuckDuckGoSearchTool`, is the `PythonInterpreterTool`, which allows the agent to execute Python code dynamically, enabling it to solve computational problems, perform calculations, or interact with APIs and libraries directly within the agent's workflow. Here's an example of it in action:

```
from smolagents import CodeAgent, PythonInterpreterTool,
➡ LiteLLMModel

model = LiteLLMModel(
    model_id="gpt-4o-mini",
    api_base="https://api.openai.com/v1",
)

agent = CodeAgent(tools=[PythonInterpreterTool()],
➡ model=model)
response = agent.run("Calculate the 10th Fibonacci
➡ number.")
print(response)
```

When you run the agent above, you see the following as shown in figure 9.6.





**Figure 9.6 Using the agent to write the code to find the 10<sup>th</sup> Fibonacci number**

The query is straight-forward. Let's try another one:

```
response = agent.run("Generate the Fibonacci sequence up to 100.")
```

You will see the following output as shown in figure 9.7.



**Figure 9.7** Getting the agent to generate the Fibonacci numbers up to 100

The choice of LLM to use plays a very important role in the result returned by the agent. In general, try out the models from the different providers to get the best output for the agent that you are creating.

### 9.2.3 Writing your own custom tools

Sometimes the built-in tools won't meet your needs. When that happens, you'll need to create a custom tool — and fortunately, writing one is very easy. Simply add the `@tool` decorator above your function, and it will be ready to use as a custom tool.

Suppose you want to create an agent that fetches the weather information for you. In this case, you can simply write a function that fetches weather information from OpenWeatherMap and then convert it into a tool. Listing 9.1 shows an example:

#### Listing 9.1 Writing a custom tool to fetch weather information from OpenWeatherMap

```
from smolagents import CodeAgent, LiteLLMModel, tool
import requests
```

```

@tool
def get_weather_info(city: str) -> str:
    """Retrieve the current weather information for a given city.
    Args:
        city: The name of the city to get the weather information for.
    Returns:
        str: A description of the current weather and temperature in
            the city.
    """
    api_key = "<API_KEY>" #A
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid=
{api_key}&units=metric"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        weather = data["weather"][0]["description"]
        temperature = data["main"]["temp"]
        return f"The weather in {city} is {weather} with a temperature of
{temperature}°C."
    else:
        return f"Could not retrieve weather information for {city}."

model = LiteLLMModel(
    model_id="ollama/qwen2:7b",
    api_base="http://127.0.0.1:11434",
    num_ctx=8192,
)

agent = CodeAgent(tools=[get_weather_info], model=model)
response = agent.run("What is the current weather for Singapore?")
print(response)

```

**#A Replace with your OpenWeatherMap API key**

It's important that your tool function includes a docstring describing its parameters. Without it, the agent won't be able to understand or use the tool properly.

Here is the output for asking the agent for the current weather in Singapore (see figure 9.8).



**Figure 9.8** The result from the agent when you asked for the weather of a city/country

## 9.3 LangChain Agents

In addition to the smolagents framework, you can also develop agents using LangChain, a widely adopted framework for building applications powered by large language models.

LangChain provides a flexible and modular architecture that allows developers to create complex agents by composing components such as prompts, memory, tools, and chains. It supports various execution paradigms, including both synchronous and asynchronous task handling, and integrates well with a wide range of APIs and data sources.

Compared to smolagents, which emphasizes simplicity and minimalism, LangChain offers more features out of the box and is well-suited for applications that require richer context management, more dynamic tool usage, or more advanced reasoning capabilities.

Now, we will explore how to construct agents using LangChain, with a focus on leveraging both built-in tools and creating custom tools tailored to specific tasks.

To use LangChain for creating agents, install the following packages using `pip`:

```
!pip install langchain langchain-openai
      langchain-community google-search-results
```

### 9.3.1 Using the Built-in Tool class

In LangChain, agents interact with external functionality through a standardized tool interface. Tools are defined using the `BaseTool` class or its simpler counterpart, the `Tool` class, which provide a consistent interface that agents can invoke to perform specific actions.

LangChain also provides utility wrappers like `SerpAPIWrapper`, `WikipediaAPIWrapper`, `WolframAlphaAPIWrapper`, and `TavilySearchResults` that handle the complexities of integrating with external APIs. These wrappers encapsulate all the logic needed to query services such as Google Search, Wikipedia, or computational engines, and format their responses into structured data that agents can work with.

However, there's an important distinction: utility wrappers are not directly usable by agents. To make a wrapper accessible to an agent, it must first be embedded within a `Tool` instance. This wrapping process transforms the utility into a proper tool that can be added to the agent's toolset.

This modular architecture offers significant advantages for developers. Rather than writing complex integration code from scratch, they can simply wrap existing services in a `Tool` and plug them directly into their agent workflows. The agent can then seamlessly access these external capabilities as part of its decision-making process, extending its functionality with minimal effort.

#### WHAT IS THE SERPAPI?

SerpAPI is a real-time search API that allows developers to programmatically access and extract search results from search engines like Google, Bing, Yahoo, YouTube, Amazon, and more. It is commonly used to retrieve structured search results—including organic results, ads, featured snippets, knowledge graphs, and other rich data—from Google Search, without the need to scrape HTML pages manually.

As an example, let's walk through how to use the `SerpAPIWrapper` to build an agent capable of performing real-time searches. First, sign up for a free account at [serpapi.com](https://serpapi.com) and obtain your private API key. Then, load your SerpAPI key into your environment like this:

```
import os
os.environ["SERPAPI_API_KEY"] = "<SERPAPI_KEY>"
```

You can now create an instance of the `SerpAPIWrapper` class and use it to create a tool:

```

from langchain.tools import Tool
from langchain_community.utilities.serpapi import SerpAPIWrapper

search = SerpAPIWrapper() #A
tools = [ #B
    Tool(
        name = "Search",
        func = search.run,
        description = "Useful for when you need to answer questions about current
events or search for specific information on the web. Input should be a search
query."
    )
]

```

**#A Initialize SerpAPI wrapper**

**#B Create a proper tool that wraps the SerpAPI functionality**

You can now create a LangChain agent using the `initialize_agent()` function as shown in Listing 9.2.

#### Listing 9.2 Creating a LangChain agent using the `initialize_agent()` function

```

from langchain_openai import ChatOpenAI
from langchain.agents import AgentType, initialize_agent

os.environ["OPENAI_API_KEY"] = "<OPENAI_API_KEY>"

llm = ChatOpenAI(model = "gpt-4o-mini", #A
                 temperature = 0)
agent = initialize_agent( #B
    tools = tools,
    llm = llm,
    agent_type = AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose = True #C
)

response = agent.invoke("Who is Wei-Meng Lee?") #D
print(response)

```

**#A Initialize the LLM**

**#B Initialize the agent with updated structure**

**#C Prints reasoning steps**

**#D Run the agent with a query**

In the above code snippet, you first created an agent using a search tool (`SerpAPIWrapper`). For this agent, you used the `gpt-4o-mini` model from OpenAI. You specified the agent type to be `AgentType.ZERO_SHOT_REACT_DESCRIPTION`, which means that the agent uses a zero-shot ReAct (Reasoning + Acting) framework, enabling it to reason through a query step-by-step and select appropriate tools to generate a response without requiring prior training or examples. The `verbose` argument specifies that you want to see the reasoning step-by-step.

When you run the code, you will see the agent's reasoning:

```
> Entering new AgentExecutor chain...
I need to gather information about Wei-Meng Lee to provide a comprehensive answer.
Action: Search
Action Input: "Wei-Meng Lee biography"
Observation: He is a prolific author, having written numerous books covering iOS and Android development, blockchain, machine learning, and smart contracts. In addition to his publications, Wei-Meng is a regular speaker at international conferences and contributes columns to Towards Data Science and CODE Magazine.
Thought:I now have a good understanding of who Wei-Meng Lee is, including his contributions to technology and education through his writing and speaking engagements.

Final Answer: Wei-Meng Lee is a prolific author and speaker known for his work in iOS and Android development, blockchain, machine learning, and smart contracts. He has written numerous books on these topics and regularly speaks at international conferences. Additionally, he contributes columns to platforms like Towards Data Science and CODE Magazine.

> Finished chain.
{'input': 'Who is Wei-Meng Lee?', 'output': 'Wei-Meng Lee is a prolific author and speaker known for his work in iOS and Android development, blockchain, machine learning, and smart contracts. He has written numerous books on these topics and regularly speaks at international conferences. Additionally, he contributes columns to platforms like Towards Data Science and CODE Magazine.'}
```

Let's try another question:

```
response = agent.invoke("What is the weather in New York today?")
print(response)
```

You will see the following output:

```

> Entering new AgentExecutor chain...
I need to find the current weather information for New York City.
Action: Search
Action Input: "current weather in New York City today"
Observation: {'type': 'weather_result', 'temperature': '68', 'unit':
'Fahrenheit', 'precipitation': '10%', 'humidity': '84%', 'wind': '6 mph',
'location': 'New York, NY', 'date': 'Thursday', 'weather': 'Cloudy'}
Thought:I now know the final answer.
Final Answer: The weather in New York City today is cloudy, with a temperature of
68°F, 10% chance of precipitation, 84% humidity, and wind at 6 mph.

> Finished chain.
{'input': 'What is the weather in New York today?', 'output': 'The weather in New
York City today is cloudy, with a temperature of 68°F, 10% chance of
precipitation, 84% humidity, and wind at 6 mph.'}

```

The following statements show how to import some of the other built-in tools provided by LangChain:

```

from langchain_community.utilities.bing_search
    import BingSearchAPIWrapper                                #A
from langchain_community.utilities.duckduckgo_search
    import DuckDuckGoSearchAPIWrapper                          #B
from langchain_community.utilities.google_search
    import GoogleSearchAPIWrapper                              #C
from langchain_community.utilities.wikipedia
    import WikipediaAPIWrapper                                  #D

#A Search using Microsoft Bing
#B Uses DuckDuckGo for privacy-friendly search
#C Uses Google's Programmable Search Engine
#D Queries Wikipedia using its API

```

### 9.3.2 Using custom tools

Like the agent you created with `smolagents` earlier, you can also create your own custom tools to use with your LangChain agent. In the following example, besides using the search tool, we will create a custom tool named `get_weather_info()`, which fetches weather details from OpenWeatherMap (see Listing 9.3).

**Listing 9.3 Writing a custom function to fetch weather information from OpenWeatherMap**

```

import os
import requests

```



```

from langchain_openai import ChatOpenAI
from langchain_community.utilities.serpapi import SerpAPIWrapper
from langchain.tools import Tool, tool
from langchain.agents import AgentType, initialize_agent

os.environ["OPENAI_API_KEY"] = "<OPENAI_API_KEY>"
os.environ["SERPAPI_API_KEY"] = "<SERPAPI_KEY>"

llm = ChatOpenAI(temperature=0)

search = SerpAPIWrapper()
search_tool = Tool(
    name = "Search",
    func = search.run,
    description = "Useful for when you need to answer questions about current
events or search for specific information on the web. Input should be a search
query."
)

@tool
def get_weather_info(city: str) -> str:
    """Retrieve the current weather information for a given city.
    Args:
        city: The name of the city to get the weather information for.
    Returns:
        str: A description of the current weather and temperature in
        the city.
    """

    api_key = "<OPENWEATHERMAP_API_KEY>"
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid=
{api_key}&units=metric"
    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()
        weather = data["weather"][0]["description"]
        temperature = data["main"]["temp"]
        humidity = data["main"]["humidity"]
        wind_speed = data["wind"]["speed"]
        summary = (
            f"Weather in {city}:\n"
            f"Condition: {weather}\n"
            f"Temperature: {temperature}°C\n"

```

```

        f"Humidity: {humidity}%\n"
        f"Wind Speed: {wind_speed} m/s"
    )
    return summary #C
else:
    return f"Could not retrieve weather information for {city}."
tools = [search_tool, get_weather_info] #D
agent = initialize_agent( #E
    tools = tools,
    llm = llm,
    agent = AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose = True
)

```

**#A** Create a custom tool using the `@tool` decorator

**#B** Replace with your OpenWeatherMap API key

**#C** A clean string that can be easily used in prompts and understood by agents

**#D** Use `search_tool` and `get_weather_info` tools

**#E** Initialize the agent with these tools

You can now ask the agent about the weather of Singapore:

```

response = agent.invoke( #A
    "What is the current weather of Singapore?")
print(response)

```

**#A** Test the custom tool

You will now see the following response:

```

> Entering new AgentExecutor chain...
I should use the get_weather_info function to retrieve the current
weather information for Singapore.
Action: get_weather_info
Action Input: "Singapore"
Observation: Weather in Singapore:
Condition: broken clouds
Temperature: 30.08°C
Humidity: 71%
Wind Speed: 4.12 m/s
Thought:I have the current weather information for Singapore.
Final Answer: The current weather in Singapore is broken clouds with a
temperature of 30.08°C, humidity at 71%, and a wind speed of 4.12 m/s.

> Finished chain.
{'input': 'What is the current weather of Singapore?', 'output': 'The
current weather in Singapore is broken clouds with a temperature of
30.08°C, humidity at 71%, and a wind speed of 4.12 m/s.'}

```

## 9.4 Developing Agents using LangGraph

Previously, you learned how to build agents using LangChain. While this approach will continue to be supported, it is now recommended to build agents using LangGraph — a more flexible and feature-rich framework designed specifically for building complex, stateful agents. LangGraph builds on the strengths of LangChain and offers greater control over agent workflows.

You can build agents using LangGraph by:

- Creating an agent capable of answering user questions using reasoning and available tools
- Integrating an external tool (such as a web search or weather API) to enable the agent to answer questions that go beyond its built-in knowledge
- Integrate memory into your LangGraph agent so that it can engage in a conversation with the user

### 9.4.1 What is LangGraph?

LangGraph is a Python framework developed by the LangChain team that allows you to build stateful, multi-step workflows involving language models, tools, and external APIs. It enables you to structure logic as a directed graph, where each node represents a computational step (e.g., calling an LLM, using a tool), and edges define how the workflow proceeds based on the output or state.

This graph-based approach is ideal for use cases such as:

- Multi-turn chatbots with memory
- Decision trees or branching logic based on LLM outputs
- Complex tool-using agents
- Data enrichment or ETL pipelines
- Modular conversational flows

LangGraph builds on top of LangChain and integrates seamlessly with LangChain tools, agents, and memory constructs. To install LangGraph, run:

```
!pip install langgraph
```

LangGraph is especially valuable when your application requires memory or a persistent state across multiple steps. It also supports branching logic, tool usage, and dynamic flow control—capabilities that are challenging to implement using traditional linear chains in LangChain.

Given the scope of this chapter, we will focus only on the core components of LangGraph that relate to agent-based workflows.

### 9.4.2 LangGraph agent basics

Let's start by creating an agent using LangGraph. For this initial example, we won't integrate any external tools, meaning the agent will rely solely on its internal reasoning capabilities and the knowledge it was trained on.

#### EXTERNAL TOOLS

External tools — such as web search APIs, or database connectors — can be integrated to extend the agent's functionality. These tools allow the agent to access up-to-date information, perform computations, or retrieve specific data that lies beyond its built-in knowledge base.

First, import the following libraries:

```
import os
from langgraph.graph.message import add_messages
from langgraph.prebuilt import create_react_agent
from langchain_openai import ChatOpenAI
```

For the LLM, let's use the `gpt-4o-mini` model from OpenAI. To use this model, you need to have an OpenAI API key:

```
os.environ["OPENAI_API_KEY"] = "<OPENAI_API_KEY>"           #A
llm = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)   #B
tools = []                                                    #C
```

**#A** Set environment variables (replace with your API keys)

**#B** Initialize the LLM (OpenAI)

**#C** The tools to use; no tools at this moment

Next, let's create a ReAct agent using LangGraph's `create_react_agent()` function, a function in LangChain that creates a ReAct-style agent:

```
agent_executor = create_react_agent(llm, tools)
```

The `create_react_agent()` function (imported from `langgraph.prebuilt`) is a wrapper that builds on top of LangChain's core `create_react_agent()` functionality. It combines:

- LLM reasoning (Thoughts)
- Tool usage (Actions), if tools are provided
- Observations (from tool outputs)
- Final Answer generation

### REASONING AND ACTING (REACT)

ReACT is a method where the agent thinks step-by-step, calls tools if needed, observes results, and continues reasoning.

When you print out this agent (`agent_executor`), you will see the graph as shown in figure 9.9.



**Figure 9.9** The graph for the agent we are building

This simple graph illustrates a linear flow with three key stages:

- **\_\_start\_\_** Node The entry point of the workflow.
- **agent** Node The core node where the agent processes the input.
- **\_\_end\_\_** Node The termination point of the graph, where the result is returned.

This minimal graph demonstrates a basic agent execution path with no branching or loops, making it easy to follow and suitable for single-turn or sequential tasks.

Next, we'll define a function that invokes the agent and prints each step of its reasoning process:

```
def run_agent(query: str):                                #A
    state = agent_executor.invoke({"messages": [("user", query)]})
    print(state)
    print("\n Agent trace:")                               #B
    for i, msg in enumerate(state["messages"]):
        print(f"{i+1}. [{msg.type.upper()}] {msg.content.strip() if hasattr(msg,
'content') else msg}")
        print('====')
    return state["messages"][-1].content
```

#A Function to run the agent with a user query

#B Print out the agent's traces

The `run_agent()` function is a simple and effective way to interact with the LangGraph agent and inspect its behavior step-by-step. The function takes a user query as input, wraps it in a message format `(("user", query))`, and invokes the agent using `agent_executor.invoke()`. The response is stored in a `state` dictionary, which contains the entire message history. After the response is generated, the function prints out a trace of the agent's message history. Each message is printed with its type (e.g., `USER`, `AI`, `TOOL`) and content. This trace is helpful for debugging and understanding how the agent processes and responds to inputs.

We are now ready to use the agent. Let's ask a simple question and then print out the result:

```
query = "Who is Bill Gates?"
answer = run_agent(query)
print(f"Query: {query}")
print(f"Answer: {answer}")
```

The output consists of three parts:

- The value of the `state` variable, which is the result returned by the agent

- The extracted agent traces
- The question that was asked and the final answer from the agent

Let's discuss each component in the output:

- Here's the content of the `state` variable (see Listing 9.4)

**Listing 9.4 The content of the state variable**

```
{
  'messages': [
    {
      'type': 'HumanMessage',
      'content': 'Who is Bill Gates?',
      'additional_kwargs': {},
      'response_metadata': {},
      'id': 'b4f87d83-2e87-444c-8eca-ee92a483b584'
    },
    {
      'type': 'AIMessage',
      'content': "Bill Gates is an American business magnate, software
developer, philanthropist, and author, best known as the co-founder of Microsoft
Corporation, the world's largest personal-computer software company. Born on
October 28, 1955, in Seattle, Washington, Gates showed an early interest in
computers and programming. He attended Harvard University but dropped out in 1975
to start Microsoft with his childhood friend Paul Allen.\n\nUnder Gates'
leadership, Microsoft developed the Windows operating system, which became a
dominant platform for personal computers. Gates served as the CEO of Microsoft
until 2000 and continued to play a significant role in the company until he
stepped down from day-to-day operations in 2008.\n\nIn addition to his work in
technology, Gates is known for his philanthropic efforts. In 2000, he and his
then-wife Melinda founded the Bill & Melinda Gates Foundation, which focuses on
global health, education, and poverty alleviation. The foundation has made
significant contributions to various causes, including vaccine development and
distribution, education reform, and efforts to combat infectious
diseases.\n\nGates has been recognized with numerous awards and honors for his
contributions to technology and philanthropy, and he is often listed among the
world's wealthiest individuals. His influence extends beyond business, as he is
also a prominent advocate for various social and health issues.",
      'additional_kwargs': {
        'refusal': None
      },
      'response_metadata': {
        'token_usage': {
```

```

        'completion_tokens': 267,
        'prompt_tokens': 12,
        'total_tokens': 279,
        'completion_tokens_details': {
            'accepted_prediction_tokens': 0,
            'audio_tokens': 0,
            'reasoning_tokens': 0,
            'rejected_prediction_tokens': 0
        },
        'prompt_tokens_details': {
            'audio_tokens': 0,
            'cached_tokens': 0
        }
    },
    'model_name': 'gpt-4o-mini-2024-07-18',
    'system_fingerprint': 'fp_dbaca60df0',
    'id': 'chatcmpl-BTJwkQfk0rrqvYIa5LVGfLJhbpPki',
    'finish_reason': 'stop',
    'logprobs': None
},
'id': 'run-3d803ad4-1270-4334-a938-8950c5108ac8-0',
'usage_metadata': {
    'input_tokens': 12,
    'output_tokens': 267,
    'total_tokens': 279,
    'input_token_details': {
        'audio': 0,
        'cache_read': 0
    },
    'output_token_details': {
        'audio': 0,
        'reasoning': 0
    }
}
}
]
}

```

The `state` variable holds the response returned by the agent, including both the input provided and a detailed trace of the agent's reasoning and tool usage during execution. In the above, you can see that there are two messages — `HumanMessage` (the question you asked) and `AIMessage` (the answer returned by the LLM).

- The next component is the agent trace extracted from the `state` variable:



 Agent trace:

1. [HUMAN] Who is Bill Gates?

=====

2. [AI] Bill Gates is an American business magnate, software developer, philanthropist, and author, best known as the co-founder of Microsoft Corporation, the world's largest personal-computer software company. Born on October 28, 1955, in Seattle, Washington, Gates showed an early interest in computers and programming. He attended Harvard University but dropped out in 1975 to start Microsoft with his childhood friend Paul Allen.

Under Gates' leadership, Microsoft developed the Windows operating system, which became a dominant platform for personal computers. Gates served as the CEO of Microsoft until 2000 and continued to play a significant role in the company until he stepped down from day-to-day operations in 2008.

In addition to his work in technology, Gates is known for his philanthropic efforts. In 2000, he and his then-wife Melinda founded the Bill & Melinda Gates Foundation, which focuses on global health, education, and poverty alleviation. The foundation has made significant contributions to various causes, including vaccine development and distribution, education reform, and efforts to combat infectious diseases.

Gates has been recognized with numerous awards and honors for his contributions to technology and philanthropy, and he is often listed among the world's wealthiest individuals. His influence extends beyond business, as he is also a prominent advocate for various social and health issues.

=====

Figure 9.10 explains the steps taken by the agent:



**Figure 9.10** The steps taken by the LangGraph agent

The last component shows the question asked and the result:

Query: Who is Bill Gates?

Answer: Bill Gates is an American business magnate, software developer, philanthropist, and author, best known as the co-founder of Microsoft Corporation, the world's largest personal-computer software company. Born on October 28, 1955, in Seattle, Washington, Gates showed an early interest in computers and programming. He attended Harvard University but dropped out in 1975 to start Microsoft with his childhood friend Paul Allen.

Under Gates' leadership, Microsoft developed the Windows operating system, which became a dominant platform for personal computers. Gates served as the CEO of Microsoft until 2000 and continued to play a significant role in the company until he stepped down from day-to-day operations in 2008.

In addition to his work in technology, Gates is known for his philanthropic efforts. In 2000, he and his then-wife Melinda founded the Bill & Melinda Gates Foundation, which focuses on global health, education, and poverty alleviation. The foundation has made significant contributions to various causes, including vaccine development and distribution, education reform, and efforts to combat infectious diseases.

Gates has been recognized with numerous awards and honors for his contributions to technology and philanthropy, and he is often listed among the world's wealthiest individuals. His influence extends beyond business, as he is also a prominent advocate for various social and health issues.

In this example, the agent can answer the questions “Who is Bill Gates?” based on the model’s (gpt-4o-mini) training data. Let’s ask another question:

```
query = "What is 2+3?"
answer = run_agent(query)
```

And here’s the output:

```
Query: What is 2+3?
Answer: 2 + 3 equals 5.
```

However, try asking the agent a question that is out of the scope of its training data and you will see that the agent is now unable to answer your question. For example:

```
query = "Who won the US Presidential Election in 2024?"
answer = run_agent(query)
```

You will now get an error:

```
Query: Who won the US Presidential Election in 2024?
Answer: I'm sorry, but I don't have information on events that occurred
after October 2023, including the results of the 2024 US Presidential
Election. You may want to check the latest news sources for the most
current information.
```

Why? Because the model was trained on data up to October 2023, so it does not have knowledge of any events that occurred after that date. And this is precisely why you need to use a tool in an agent.

### 9.4.3 Using LangGraph with tools

To answer questions beyond the model’s training data, you need to connect the agent to an external tool capable of providing up-to-date information, such as performing web searches to get the answer.

To allow your agent to perform real-time searches, you can use the `SerpAPIWrapper`, a tool that we have described and used earlier. Let’s create an instance of the `SerpAPIWrapper` class:

```

from langchain_core.tools import Tool
from langchain_community.utilities import SerpAPIWrapper

os.environ["SERPAPI_API_KEY"] = "<SERPAPI_KEY>"           #A
serpapi = SerpAPIWrapper()                                #B
search_tool = Tool(
    name = "SerpAPI",                                     #C
    func = serpapi.run,                                   #D
    description = "A search engine tool to query real-time information from the
web."
)

```

**#A** replace with your own key

**#B** Initialize the SerpAPI wrapper and create a tool

**#C** Name of the tool

**#D** The `serpapi.run` is a method provided by the `SerpAPIWrapper` class

Note the description of the tool above. The description of a tool in LangChain (and by extension, LangGraph) is important because it guides the LLM in deciding when and how to use the tool. The description acts as a prompt that informs the LLM about the tool's purpose, functionality, and expected input, enabling the agent to make intelligent decisions about whether to call the tool based on the user's query.

We will now add the `search_tool` as an argument to the `create_react_agent()` function:

```

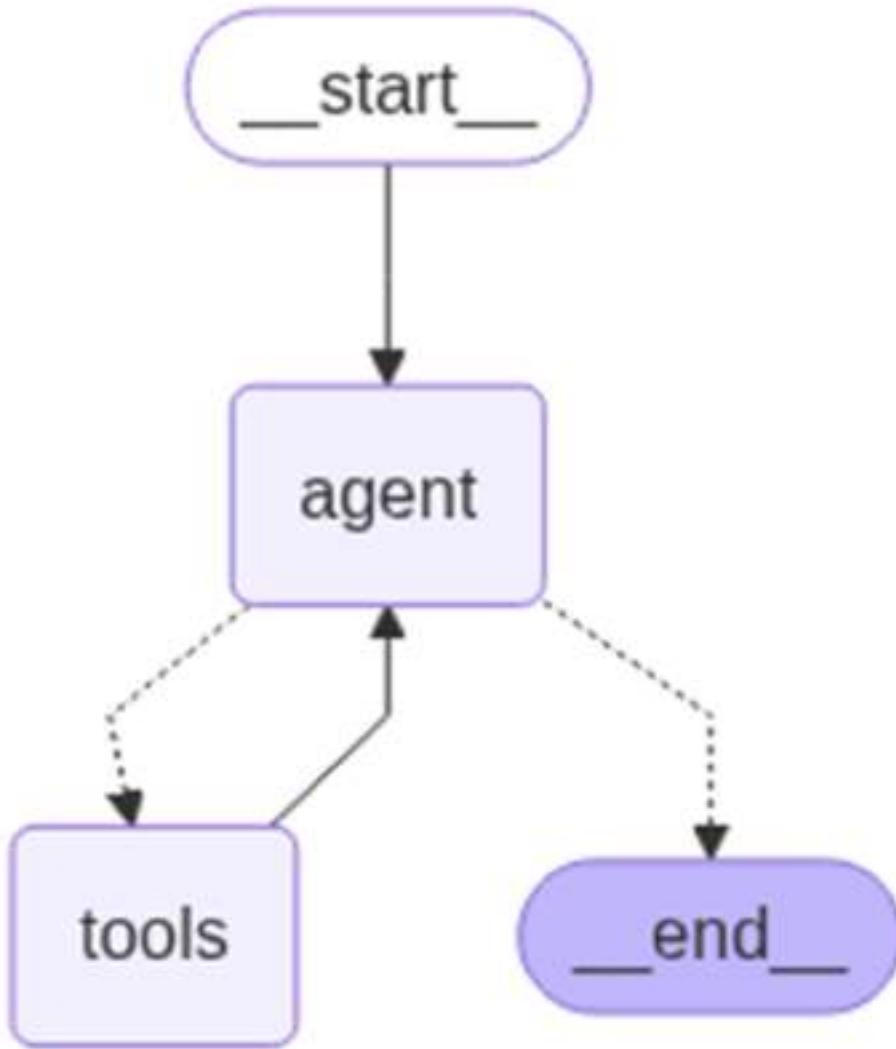
tools = [search_tool]                                     #A
agent_executor = create_react_agent(llm, tools)           #B

```

**#A** Add the search tool

**#B** Create the ReAct agent using LangGraph's high-level interface

When you now print out the `agent_executor`, you will see the graph as shown in figure 9.11.



**Figure 9.11** The graph of the agent together with the external tool

Here's the flow of the entire graph:

- **\_\_start\_\_** Node: This is the entry point of the graph. Execution begins here.
- **agent** Node: The agent processes the user input. Based on the query and internal logic, it decides whether it can respond directly or needs help from a tool.

- **tools** Node: If the agent determines that a tool is needed (e.g., a calculator, search engine, or database lookup), it sends a request to the tools node. The tool is executed. And the result is returned to the agent, allowing it to formulate a final response. The dotted lines indicate conditional or dynamic transitions.
- **\_\_end\_\_** Node: Once the agent has all the necessary information (either directly or via a tool), it outputs the final message and the flow ends here.

You can now ask the question “*Who won the US Presidential Election in 2024?*”:

```
query = "Who won the US Presidential Election in 2024?"
answer = run_agent(query)
print(f"Query: {query}")
print(f"Answer: {answer}")
```

Observe the output. First, the `state` variable contains the content as shown in Listing 9.5.

**Listing 9.5 The content of the state variable**

```
{
  'messages': [
    {
      'type': 'HumanMessage',
      'content': 'Who won the US Presidential Election in 2024?',
      'additional_kwargs': {},
      'response_metadata': {},
      'id': '6166c39b-015e-417b-9016-55bd75c35fd5'
    },
    {
      'type': 'AIMessage',
      'content': '',
      'additional_kwargs': {
        'tool_calls': [
          {
            'id': 'call_0TL5mwU9fmTn450mZJejsZkp',
            'function': {
              'arguments': '{"__arg1": "US Presidential Election 2024
winner"}',
              'name': 'SerpAPI'
            },
            'type': 'function'
          }
        ]
      }
    }
  ],
```

```

        'refusal': None
    },
    'response_metadata': {
        'token_usage': {
            'completion_tokens': 24,
            'prompt_tokens': 69,
            'total_tokens': 93,
            'completion_tokens_details': {
                'accepted_prediction_tokens': 0,
                'audio_tokens': 0,
                'reasoning_tokens': 0,
                'rejected_prediction_tokens': 0
            },
            'prompt_tokens_details': {
                'audio_tokens': 0,
                'cached_tokens': 0
            }
        },
        'model_name': 'gpt-4o-mini-2024-07-18',
        'system_fingerprint': 'fp_0392822090',
        'id': 'chatcmpl-BTMUM9twVvbFPpvV0zJkr19cDIRO7',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    'id': 'run-77effaef-0972-4c07-8156-6c8b25618a4c-0',
    'tool_calls': [
        {
            'name': 'SerpAPI',
            'args': {
                '__arg1': 'US Presidential Election 2024 winner'
            },
            'id': 'call_0TL5mwU9fmTn450mZJejsZkp',
            'type': 'tool_call'
        }
    ],
    'usage_metadata': {
        'input_tokens': 69,
        'output_tokens': 24,
        'total_tokens': 93,
        'input_token_details': {
            'audio': 0,
            'cache_read': 0
        },
        'output_token_details': {

```

```

        'audio': 0,
        'reasoning': 0
    }
}
},
{
    'type': 'ToolMessage',
    'content': '[' entity_type: video_universal.', 'The APP excludes
"over" and "under" votes in the total votes cast, which also impacts the vote
percentage for candidates. Harris-Walz 2024. Trump-Vance 2024.', 'View maps and
real-time results for the 2024 US presidential election matchup between former
President Donald Trump and Vice President Kamala Harris.', "A presidential
election was held in the United States on November 5, 2024. The Republican
Party\'s ticket-Donald Trump, who was the 45th president of the ...", 'Live 2024
election results for the president, U.S. Senate, U.S. House, and governors.',
'Donald Trump passed the critical threshold of 270 electoral college votes with a
projected win in the state of Wisconsin making him the next US president.', 'Get
live presidential results and maps from every state and county in the 2024
election.', '2024 election guide: Presidential candidates, polls, primaries and
caucuses, voter information and results for November 5, 2024.', 'Check back for
the Certificates of Vote from the 2024 election. They will be posted as they
become available. President Donald J. Trump [R] Main Opponent ...', 'View live
election results from the 2024 presidential race as Kamala Harris and Donald
Trump face off. See the map of votes by state as results are tallied.'],
    'name': 'SerpAPI',
    'id': 'f681c58a-043e-4016-b081-5f2f44d83fa1',
    'tool_call_id': 'call_0TL5mwU9fmTn450mZJejsZkp'
},
{
    'type': 'AIMessage',
    'content': 'Donald Trump won the US Presidential Election in 2024,
passing the critical threshold of 270 electoral college votes with a projected
win in the state of Wisconsin.',
    'additional_kwargs': {
        'refusal': None
    },
    'response_metadata': {
        'token_usage': {
            'completion_tokens': 34,
            'prompt_tokens': 392,
            'total_tokens': 426,
            'completion_tokens_details': {
                'accepted_prediction_tokens': 0,
                'audio_tokens': 0,

```



```

        'reasoning_tokens': 0,
        'rejected_prediction_tokens': 0
    },
    'prompt_tokens_details': {
        'audio_tokens': 0,
        'cached_tokens': 0
    }
},
'model_name': 'gpt-4o-mini-2024-07-18',
'system_fingerprint': 'fp_0392822090',
'id': 'chatcmpl-BTMUN6F1DvzZae98NwKMLzcBKUW9C',
'finish_reason': 'stop',
'logprobs': None
},
'id': 'run-e10a9f2f-336a-4544-ac46-588594006dd2-0',
'usage_metadata': {
    'input_tokens': 392,
    'output_tokens': 34,
    'total_tokens': 426,
    'input_token_details': {
        'audio': 0,
        'cache_read': 0
    },
    'output_token_details': {
        'audio': 0,
        'reasoning': 0
    }
}
}
]
}

```

You can see the agent is now using the tool to search for an answer. Here are the traces of the agent:

 Agent trace:

1. [HUMAN] Who won the US Presidential Election in 2024?

=====

2. [AI]

=====

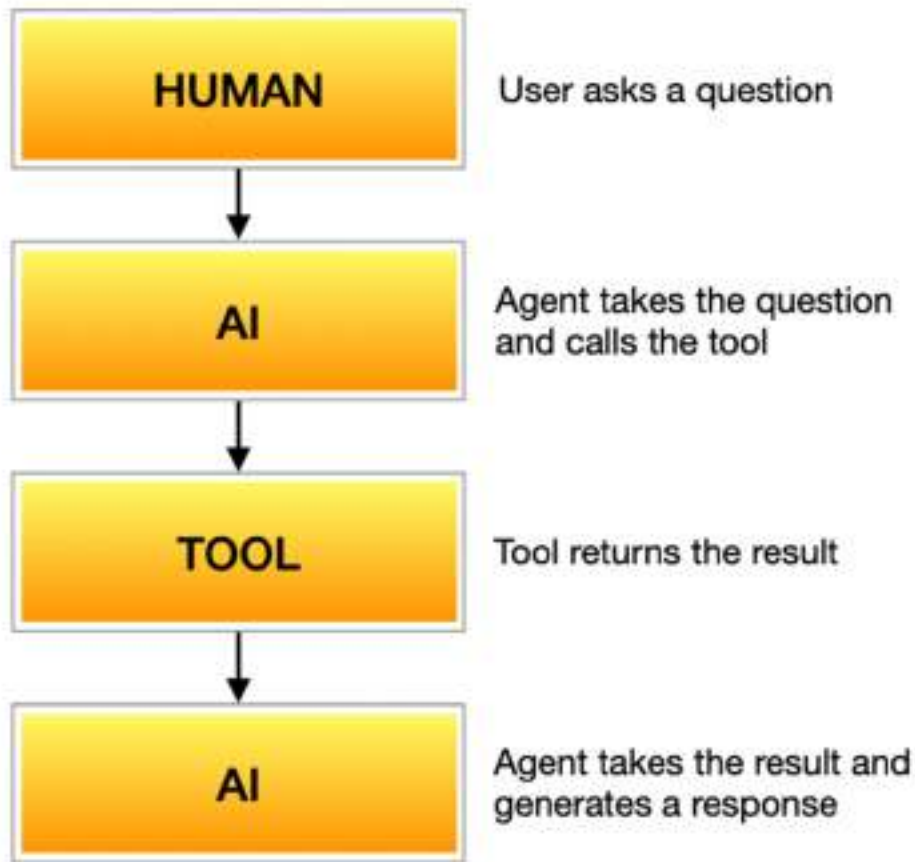
3. [TOOL] [' entity\_type: video\_universal.', 'The APP excludes "over" and "under" votes in the total votes cast, which also impacts the vote percentage for candidates. Harris-Walz 2024. Trump-Vance 2024.', 'View maps and real-time results for the 2024 US presidential election matchup between former President Donald Trump and Vice President Kamala Harris.', 'A presidential election was held in the United States on November 5, 2024. The Republican Party's ticket—Donald Trump, who was the 45th president of the ...', 'Live 2024 election results for the president, U.S. Senate, U.S. House, and governors.', 'Donald Trump passed the critical threshold of 270 electoral college votes with a projected win in the state of Wisconsin making him the next US president.', 'Get live presidential results and maps from every state and county in the 2024 election.', '2024 election guide: Presidential candidates, polls, primaries and caucuses, voter information and results for November 5, 2024.', 'Check back for the Certificates of Vote from the 2024 election. They will be posted as they become available. President Donald J. Trump [R] Main Opponent ...', 'View live election results from the 2024 presidential race as Kamala Harris and Donald Trump face off. See the map of votes by state as results are tallied.']

=====

4. [AI] Donald Trump won the US Presidential Election in 2024, passing the critical threshold of 270 electoral college votes with a projected win in the state of Wisconsin.

=====

Figure 9.12 summarizes what happens when you asked the question.



**Figure 9.12** The agent works with the tool to return the answer

And finally, the agent returns the answer:

Query: Who won the US Presidential Election in 2024?

Answer: Donald Trump won the US Presidential Election in 2024, passing the critical threshold of 270 electoral college votes with a projected win in the state of Wisconsin.

#### 9.4.4 Using LangGraph with custom tool

Just like `smolagents` and `LangChain`, you can create custom tools to use with your `LangGraph` agent. Let's create a function named `get_weather_info()` and use it to retrieve weather information from `OpenWeatherMap`. Listing 9.6 shows the definition of the function.

**Listing 9.6 The `get_weather_info()` function**

```

import requests

def get_weather_info(city: str) -> str:
    """Retrieve the current weather information for a given city."""
    api_key = "7453d5cfeaea020958539f22da95d849" #A
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        weather = data["weather"][0]["description"]
        temperature = data["main"]["temp"]
        humidity = data["main"]["humidity"]
        wind_speed = data["wind"]["speed"]
        summary = (
            f"Weather in {city}:\n"
            f"Condition: {weather}\n"
            f"Temperature: {temperature}°C\n"
            f"Humidity: {humidity}%\n"
            f"Wind Speed: {wind_speed} m/s"
        )
        return summary #B
    else:
        return f"Could not retrieve weather information for {city}."

```

**#A** Replace with your OpenWeatherMap API key

**#B** A clean string that can be easily used in prompts and understood by agents

Next, we will create a `LangChain Tool` object using this function:

```

weather_tool = Tool( #A
    name = "GetWeather", #B
    func = get_weather_info, #C
    description = "A tool to fetch the weather information for a city"
)

```

**#A** Create a `LangChain Tool` from the custom function

**#B** Name of the tool

**#C** Name of the custom function

And finally, you can use the `weather_tool` in your agent:

```
tools = [search_tool, weather_tool]
agent_executor = create_react_agent(llm, tools)
```

You can now ask a question about the weather of a country:

```
query = "What is the current weather for Singapore"
answer = run_agent(query)
print(f"Query: {query}")
print(f"Answer: {answer}")
```

The final output for the agent looks something like the following:

```
Query: What is the current weather for Singapore
Answer: The current weather in Singapore is as follows:
- **Condition:** Broken clouds
- **Temperature:** 29°C
- **Humidity:** 79%
- **Wind Speed:** 3.09 m/s
```

### 9.4.5 Using LangGraph with memory

The agent that you have created up till this point has no memory. That is, it treats every incoming query as an isolated request, without any awareness of what was said or asked previously. This means the agent cannot carry on a conversation, track context across multiple turns, or refer to earlier information.

For example, if you ask:

```
Query: What is the current weather for Singapore?
Answer: The current weather in Singapore is as follows:
...
```

And then follow up with:

```
Query: Where is it located?
```

The agent will not be able to understand who the “it” is referring to unless you repeat the full context, like:

```
Query: Where is Singapore located?
```

To support back-and-forth interactions or context-aware reasoning, we need to introduce memory into the system. This is where tools like LangGraph, message history, or stateful chains come into play, enabling the agent to persist and reason over previous exchanges.

Let's now see how to add memory to our agent using LangGraph's support for message-passing state. First, import the following libraries:

```
from typing import Annotated, List
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages
```

And then create a class named `State`:

```
class State(TypedDict):
    messages: Annotated[List, add_messages]
```

The `State` class creates a dictionary-like object meant to hold conversational state within LangGraph. The key `messages` will store a list of messages that the user is communicating with the agent. The `Annotated` class tells LangGraph how to handle the list of messages:

- It automatically merges, appends, or trims as needed.
- It's LangGraph's way of doing memory with a low-level but composable approach.
- `List` means that `messages` is expected to be a `List` (i.e., Python's built-in `list`), but it doesn't specify the type of items inside. So it's just a generic list (e.g., `List[Any]`).
- `add_messages` is a LangGraph utility that helps manage appending and merging message sequences.

With the `State` class defined, modify the `run_agent()` function so that the agent can use a `State` object to maintain conversation:

```
def run_agent(query: str, state: State = None)
➡ -> tuple[str, State]: #A
    response = agent_executor.invoke({"messages": [("user", query)]})
    pprint(response)
    if state is None:
        state = {"messages": []}
    state["messages"].append(("user", query)) #B
    response = agent_executor.invoke(state) #C
    state = {"messages": response["messages"]} #D
    return response["messages"][-1].content,
➡ state #E
```

#A Function to run the agent with a user query, maintaining conversation history

#B Append the new user query to the existing messages

#C Invoke the agent with the updated state

#D Update the state with the response; the whole messages list is replaced

#E Return the last message as well as the state

In the above code, the function `run_agent()` accepts a user query and an optional `State` object, which stores the conversation history. If no state is provided, it initializes an empty one. The user's query is appended to the message list in the state. This updated state is then passed to the agent via `invoke()`. The agent generates a response based on the full conversation history and returns an updated list of messages. The function then returns the most recent message from the agent along with the updated state to continue the conversation seamlessly.

You can now call the `run_agent()` function via a loop so that the user has a chance to ask contextually related questions:

```
conversation_state = {"messages": []} #A
while True:
    query = input("Question: ")
    if query.lower()=="quit": break
    answer, conversation_state = run_agent(query, conversation_state)
    print(f"Query: {query}")
    print(f"Answer: {answer}")
```

#A Initialize an empty state to store conversation history

Let's try this out with the first question:

Query: **What is the weather for Singapore?**

Answer: The weather in Singapore is as follows:

- **\*\*Condition:\*\*** Broken clouds
- **\*\*Temperature:\*\*** 31.35°C
- **\*\*Humidity:\*\*** 70%
- **\*\*Wind Speed:\*\*** 3.09 m/s

Now that we got the first answer, let's ask a related question, but using "her" to refer to Singapore:

Query: **What is her population?**

Answer: As of my last knowledge update in October 2021, Singapore's population was approximately 5.7 million people. For the most current population figures, I recommend checking the latest statistics from a reliable source such as the Singapore Department of Statistics or other official demographic resources.

You can see that the agent can now remember the previous conversation and answer the question correctly.

## 9.5 Summary

- An agent is a specialized system designed to perform tasks autonomously by combining language understanding, reasoning, and tool usage.
- smolagents is a lightweight and flexible framework designed to make agent development simple and approachable.
- `DuckDuckGoSearchTool` is a lightweight and privacy-focused search tool that allows your agent to find information from the web quickly and efficiently.
- Using the `HfApiModel()` object, you can use a model hosted on Hugging Face server.
- The `CodeAgent` class creates a smolagents agent.
- To use a locally running LLM through Ollama, use the `LiteLLMModel` class, which connects to the Ollama server and allows your agent to send prompts and receive responses directly from models running on your machine.
- The `PythonInterpreterTool` class allows the agent to execute Python code dynamically.
- Use the `@tool` decorator above your function to design it as a custom tool.
- SerpAPI is a real-time search API that allows developers to programmatically access and extract search results from search engines like Google, Bing, Yahoo, YouTube, Amazon, and more.



- You can create a LangChain agent using the `initialize_agent()` function.
- LangGraph is a flexible and feature-rich framework designed specifically for building complex, stateful agents.
- You can use the `create_react_agent()` function to create a ReAct-style agent.
- ReACT is a method where the agent thinks step-by-step, calls tools if needed, observes results, and continues reasoning.
- In LangGraph agents, you create a Tool object and use it as a custom tool.
- You can maintain memory in your LangGraph agent by creating a `State` object and passing it to your agent.

# ***10 Building Web-Based UI using Gradio***

## **This chapter covers**

- Building basic UI using Gradio
- Configuring and customizing your Gradio application
- Sharing and deploying your Gradio application on HuggingFace Spaces
- Creating a chatbot UI for chatbot applications

Imagine you have just spent weeks coding your machine learning projects and it is finally done! Now, you are eager show it off to your friends and hopefully it will impress your boss. But there is just one more thing you need to do – create a nice shiny front-end so that you can impress your users. But while developers excel in technical aspects and problem-solving, their strengths may not always align with creative design. So how do you quickly come up with a nice web front-end that can interface with your machine learning models?

Introducing Gradio, an open-source Python package that makes it very easy for you to quickly build a demo web application to showcase your machine learning applications. What's more, with a single click you can share a link to your demo application using Gradio's built-in sharing feature.

## **10.1 Basics of Gradio**

To install the Gradio package, you can use the `pip` command in Jupyter notebook:

```
!pip install gradio
```

At the time of writing, the version of the Gradio Python package used is 5.18.0.

Let's start by exploring the basics of how Gradio works. We will create a simple Gradio application and then dive into how it works and how you can deploy them in production environments. Specifically, you will learn:

- How to use the `Interface` class to build a simple Gradio application
- How to use the Flag options to let users flag your application output
- How to configure authentication for your Gradio application so that only authorized users can use it
- How to configure the Gradio application to be accessible on the local network
- How to deploy your Gradio application on HuggingFace Spaces

### 10.1.1 Using Gradio's Interface class

To build a simple Gradio application, you can use the `Interface` class. The `Interface` class is Gradio's main high-level class that allows you to build a web UI with just a few lines of code.

The `Interface` class accepts a number of arguments, some of which are:

- `fn` – the function to wrap the Gradio UI around
- `title` – the title of the Gradio UI
- `inputs` – the type of inputs to display on the Gradio UI
- `outputs` – the types of outputs to display on the Gradio UI

Listing 10.1 shows a simple example.

**Listing 10.1 A simple Gradio example**

```
import gradio as gr

def my_chatbot(message):
    return "Hello, " + message

interface = gr.Interface(fn = my_chatbot,                #A
                        title = "Hello, Gradio!",        #B
                        inputs = "text",                 #C
                        outputs = "text")                #D

interface.launch()
```

#A bind it to a function  
 #B title of the UI  
 #C the input component(s)  
 #D the output component(s)

In the code snippet above, the Gradio application is bounded to the `my_chatbot()` function. To launch the Gradio application, call the `launch()` method. Figure 10.1 shows how the Gradio application will look like when the code is run.

Running on local URL: <http://127.0.0.1:7860>  
 To create a public link, set 'share=True' in 'launch()'.

Hello, Gradio!

**Figure 10.1 The Gradio application with one single input and one single output**

Figure 10.2 shows how the various arguments passed into the `Interface` class controls the UI of your Gradio application.



**Figure 10.2** The various components of your Gradio application

The Clear, Submit, and Flag buttons are displayed by default. In this example, both the input and output of the UI are text boxes (indicated with the `text` string). When the user clicks on the Submit button, the text in the input text box is passed into the `message` parameter in the `my_chatbot()` function. The value returned by the function will be shown in the output text box. Figure 10.3 explains this visually.



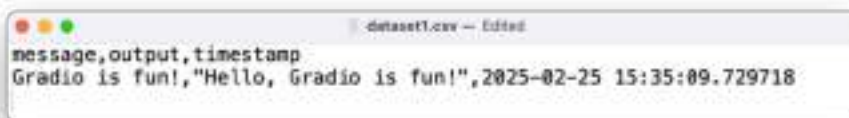
**Figure 10.3** How values are passed in from the inputs to the bounded function and returned to the outputs

Notice that there is a link at the top of the output indicating that Gradio is running on a local URL – `http://127.0.0.1:7860`. If you click on it, a new web page will appear (see Figure 10.4). Note that the port number will increment by one every time you run the cell in Jupyter notebook, i.e. if you run the cell again the port number will change to 7861, followed by 7862, and so on.



**Figure 10.4** Displaying the Gradio app in a new browser window

Also notice the button labelled as Flag. What is the use of this button? Well, it is a way for you to log the inputs and outputs if you find that the output returned by the function warrants further attention. Clicking the Flag button creates a new folder called `.gradio/flagged` in the same directory as your Jupyter Notebook. Inside this folder, you'll find a file named `dataset1.csv`. This CSV file will record your inputs, outputs, and other details. For example, if you entered the string "Gradio is fun!", clicks Submit and then click the Flag button, Gradio will log the following in the CSV file (see Figure 10.5).



**Figure 10.5** Viewing the content of `log.csv`

### 10.1.2 Configuring flagging options

The default behavior of the Flag button is to log the inputs, outputs, and timestamp of your Gradio application onto the **log.csv** file. If you want a more customized approach to logging, you can specify them using the `flagging_options` parameter, like that as shown in Listing 10.2.

**Listing 10.2 Specifying the flagging options**

```
import gradio as gr

def my_chatbot(message):
    return "Hello, " + message

interface = gr.Interface(fn = my_chatbot,
                        title = "Hello, Gradio!",
                        inputs = "text",
                        outputs = "text",
                        flagging_options =
                            ["correct", "wrong", "ambiguous"])

interface.launch()
```

Figure 10.6 shows that you now have three flag buttons, labelled based on what you have specified in the `flagging_options` parameter.



**Figure 10.6 The three flagging options**

If you now type the string "Gradio is fun!", clicks Submit and then click the **Flag as correct** button, a new file named `dataset2.csv` will now be created and Gradio will now log "correct" under the **"flag"** field (see Figure 10.7).



**Figure 10.7 Viewing the content of log.csv**

A new CSV file will be created if the structure of the original file changes. For example, dataset1.csv did not include a field named flag, whereas dataset2.csv now contains this new field.

### 10.1.3 Configuring authentication

By default, your Gradio application is public – anyone who has the URL of your app will be able to access it. However, you might want to restrict access to the application, especially if it involves sensitive data. As such, you can password-protect your application by using the `auth` parameter in the `launch()` method, as shown in Listing 10.3.

#### Listing 10.3 Specifying the authentication credentials

```
import gradio as gr

def my_chatbot(message):
    return "Hello, " + message

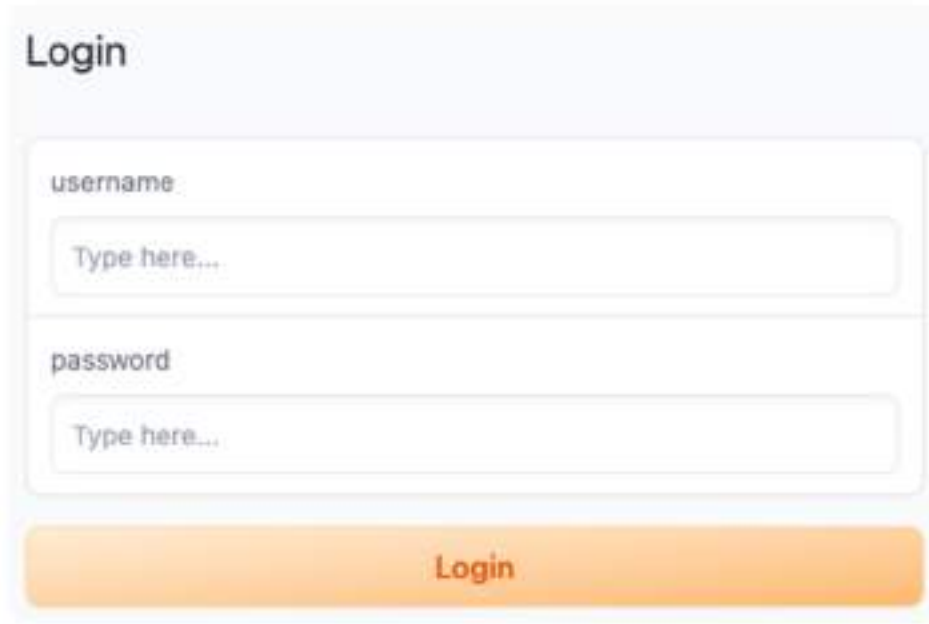
#A
interface = gr.Interface(fn = my_chatbot,
                        title = "Hello, Gradio!",
                        inputs = "text",
                        outputs = "text",
                        flagging_options =
                            ["correct", "wrong", "ambiguous"])

interface.launch(auth = ("admin", "secret"))

#A bind it to gradio
```

When you now run the above code snippet, you will see the login page as shown in Figure 10.8.





The image shows a web form for logging in. At the top, the word "Login" is displayed in a large, bold, black font. Below this, there are two input fields. The first field is labeled "username" and contains the placeholder text "Type here...". The second field is labeled "password" and also contains the placeholder text "Type here...". Both labels are in a small, gray font. At the bottom of the form is a wide, orange button with the word "Login" written in a bold, black font.

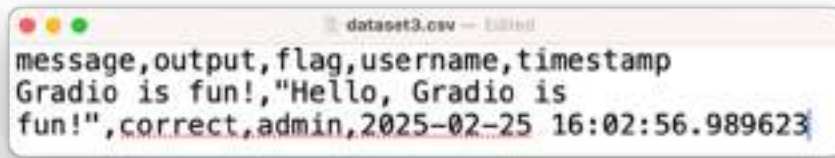
**Figure 10.8 Prompting the user to login before using the application**

### THIRD PARTY COOKIES SUPPORT

For authentication in Gradio to work properly, third party cookies must be enabled in your browser. By default, Safari and Chrome in Incognito Mode will not work.

If the entered username ("admin") and password ("secret") do not match, you will see a "Incorrect Credentials" message, else the Gradio application will load as usual.

If you now type the string "Gradio is fun!", clicks Submit and then click the **Flag as correct** button, Gradio will now create another CSV file and log "admin" under the **username** field (see Figure 10.9).



**Figure 10.9** Viewing the content of log.csv

Hardcoding the username and password in the `auth` parameter is not the recommended practice. You may want to write a function to perform your own authentication handling and then specify the function name in the `auth` parameter, something like this:

```
def authentication(username, password):
    #A
    #B
    return (username=='admin' and password=='secret')

interface.launch(auth = authentication)
```

#A you can replace the below authentication logic with your own

#B return True to authenticate user, False otherwise

In the above code snippet, you can replace the authentication logic with your own, such as authenticating against the credentials stored in databases, or other third-party authentication services, etc.

#### 10.1.4 Customizing the server and port

By default, Gradio will listen at port 7860. However, this port might not be available on your computer if another application is also listening at the same port. In this case, Gradio will automatically search for the next available port, starting from 7860. Alternatively, you can explicitly specify the port you want to use for Gradio. This is especially useful if you are running Gradio in a Docker container and the container only exposes a number of specific ports.

Also, when Gradio starts up, it will by default bind to the 127.0.0.1 IP address. This will make the Gradio application accessible only on the local computer. If you want to make the application accessible on the local network, you need to bind it to the 0.0.0.0 IP address. The following code snippet makes it clear:

```

#A
interface.launch(server_name = "127.0.0.1", server_port = 5000)

#B
interface.launch(server_name = "0.0.0.0", server_port = 5000)

```

**#A** Listens at port 5000 and only assessible on the computer running Gradio

**#B** Listens at port 5000 and assessible on the local network

For computer on the network to access the Gradio application, simply use the following URL – `http://<IP_ADDRESS_OF_SERVER>:5000`. For example, if the Gradio application is running on a computer with IP address 192.168.1.24, you can then use this URL: `http://192.168.1.24:5000` to access the Gradio application from another computer.

When you use Jupyter Notebook, take note when you re-run the cell containing the Gradio application, it will complain that the port you specified is not available. To fix this, simply restart the kernel.

### 10.1.5 Sharing your Gradio application

If you want to share your Gradio application with your friends over the internet, you can set the `share` argument to `True` in the `launch()` method:

```
interface.launch(share = True)
```

Doing so will cause Gradio to create a temporary link for your friends to access your Gradio application. Figure 10.10 shows the output when you call the `launch()` method with the `share` argument set to `True`:

```

Running on local URL: http://127.0.0.1:7861
Running on public URL: https://88cf9d2ca80685c5d5.gradio.live

This share link expires in 72 hours. For free permanent hosting and GPU upgrades, run 'grad
io deploy' from Terminal to deploy to Spaces (https://huggingface.co/spaces)

```

**Figure 10.10 A public link will be created for your Gradio application**

You can now share the public URL with your friends. Note the following:

- If you run the Gradio code in Jupyter notebook, the kernel must be running. If the kernel is shut down, the Gradio app will no longer work.
- If you run the Gradio code as a standalone Python application, the application must be running. If not, the Gradio app will no longer work.

Do note that the shared link is served by Gradio Share Servers, which act only as proxies for your local server. They do not store any data sent through your app. Also, shared links expire after 72 hours.

A much better solution to make your Gradio application public is to host it on HuggingFace Spaces.

### 10.1.6 Deploying your Gradio Application to Hugging Face Spaces

To make your Gradio application permanently available, it is a better to host it on the cloud. Fortunately, HuggingFace Spaces provides free hosting (you can upgrade to a paid version if you require more computing power) for your Gradio application. Let's learn how you can host your Gradio application on HuggingFace Spaces.

- First, create a directory on your computer, say MyGradioApp.
- Create a file named **mygradio.py**, with its content same as that shown in Listing 10.1. Save the file in the MyGradioApp directory.
- For hosting on HuggingFace Spaces, you need a WRITE HuggingFace token. You can create a new WRITE token at <https://huggingface.co/settings/tokens>.
- Launch Terminal or Anaconda Prompt, and type in the following commands:

```
$ cd MyGradioApp
$ gradio deploy
```

- You will see the following. Enter your WRITE token and reply to the questions as shown below:

Need 'write' access token to create a Spaces repo.

```

      _|      _| _|      _|      _|_|_|      _|_|_|      _|_|_|      _|      _|      _|_|_|
_|_|_|_|      _|_|      _|_|_|      _|_|_|_|
      _|      _| _|      _|      _|      _|      _|_|      _|      _|      _|
_|      _| _|      _|
      _|_|_|_|_|      _|      _|      _|_|      _|      _|_|      _|      _|      _|      _|_|
_|_|_|      _|_|_|_|_|      _|      _|_|_|_|
      _|      _| _|      _|      _|      _|      _|      _|      _|      _|_|      _|      _|
_|      _| _|      _|
      _|      _|      _|_|      _|_|_|      _|_|_|      _|_|_|      _|      _|      _|_|_|      _|
_|      _|      _|_|_|      _|_|_|_|

```

Enter your token (input will not be visible): **<Your WRITE token>**

Add token as git credential? (Y/n) **n**

Creating new Spaces Repo in '/Volumes/SSD/Dropbox/Book Projects/5. Hugging Face book/TE Reviewed/chap10/MyGradioApp'. Collecting metadata, press Enter to accept default value.

Enter Spaces app title [MyGradioApp]: **<press Enter>**

Enter Gradio app file [mygradio.py]: **<press Enter>**

Enter Spaces hardware (cpu-basic, cpu-upgrade, t4-small, t4-medium, 14x1, 14x4, zero-a10g, a10g-small, a10g-large, a10g-largex2, a10g-largex4, a100-large, v5e-1x1, v5e-2x2, v5e-2x4) [cpu-basic]: **<press Enter>**

Any Spaces secrets (y/n) [n]: **<press Enter>**

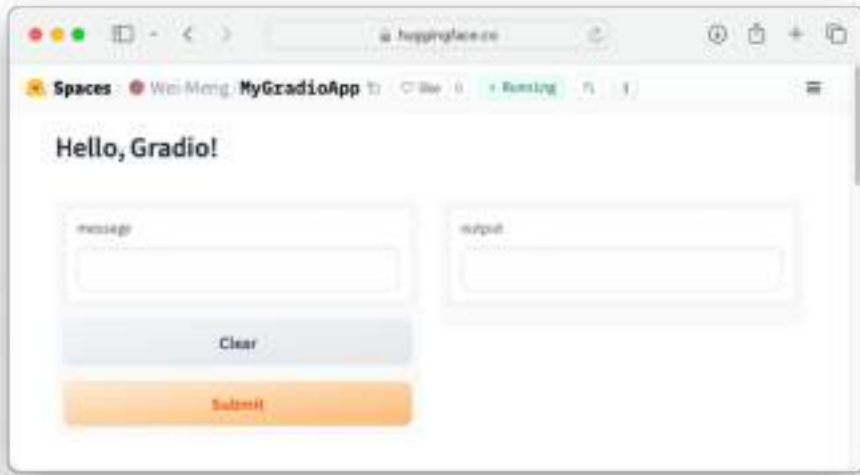
Create requirements.txt file? (y/n) [n]: **<press Enter>**

Create Github Action to automatically update Space on 'git push'? [n]:

**<press Enter>**

Space available at <https://huggingface.co/spaces/Wei-Meng/MyGradioApp>

- Your Gradio application is now hosted on HuggingFace Spaces. You can access it via the following URL <https://huggingface.co/spaces/Wei-Meng/MyGradioApp> (your URL will vary due to the username). Figure 10.11 shows the Gradio application.



**Figure 10.11** The Gradio application hosted on HuggingFace Spaces

## 10.1 Working with Different Widgets

Up till this point, you have learned how to create a basic Gradio application, configure it for flagging, as well as share it with your friends on the local network and deploy it on HuggingFace Spaces. Let's now dive deeper into the various types of Gradio applications you can build using other components. Looking ahead, you will learn the following:

- How to work with Textbox
- How to work with Audio
- How to work with Images
- How to work with selection components
- How to layout components using the `TabbedInterface` class

### 10.1.1 Working with Textbox

In the first example in this chapter, you saw that when you specify `text` as the value in the `inputs` parameter in the `Interface` class, a text box with default label ("message") is displayed. However, you can customize the text box by creating an instance of the `Textbox` class. The following code snippet in Listing 10.4 shows an example.

**Listing 10.4 Creating an instance of the Textbox class**

```
import gradio as gr

def my_chatbot(message):
    return "Hello, " + message

textbox = gr.Textbox(label = "Message",
                    placeholder = "Your message here",
                    lines = 3)

gr.Interface(fn = my_chatbot,
            inputs = textbox,
            outputs = "text").launch()
```

Figure 10.12 shows how the Gradio applications looks like and the use of the various arguments in the `Textbox` class.



**Figure 10.12 Configuring the Textbox component**

The height for the text box is now taller – it shows the height for three lines. However, note that the `lines` parameter does not limit the number of lines the user can type; it is just for visual purposes. Figure 10.13 shows the output when the user has entered the text as shown and clicked on the Submit button.



**Figure 10.13** Submitting the text and obtaining the output

### 10.1.2 Working with Audio

Besides text, Gradio is also able to work with other media. One of them is audio. Using the Audio component, you can upload an audio stream to Gradio and perform operations on it. Listing 10.5 shows an example of how to use the Audio component.

**Listing 10.5** Creating an instance of the Audio class

```
import numpy as np
import gradio as gr

def reverse_audio(audio):
    sr, data = audio                                #A
    reversed_audio = (sr, np.flipud(data))           #B
    return reversed_audio

mic = gr.Audio(sources = ["upload","microphone"],
               type = "numpy",
               label = "Audio")

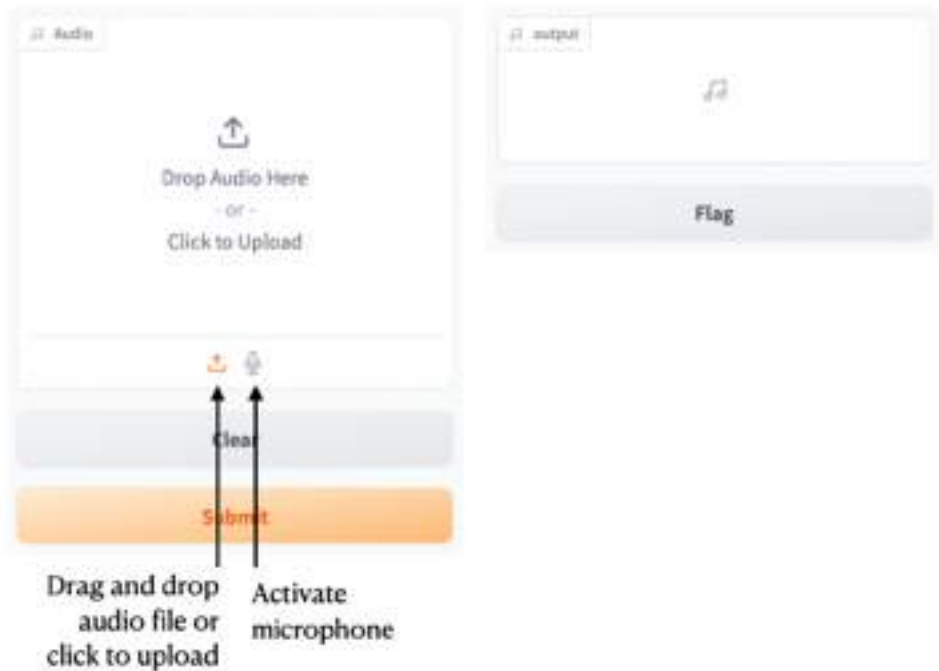
interface = gr.Interface(fn = reverse_audio,
                        inputs = mic,
                        outputs = "audio")

interface.launch()

#A audio is a NumPy array
#B reverse the audio
```

Figure 10.14 shows how the Gradio application looks like when you run the code snippet.





**Figure 10.14** You can drag and drop an audio file, or record from your microphone

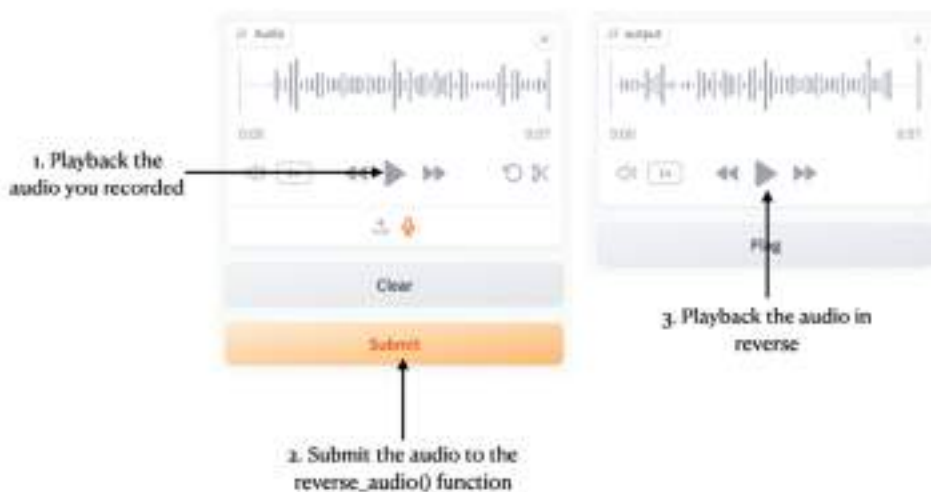
You can either drag and drop an audio file or activate the microphone on your computer to record an audio stream. The audio stream by default will be sent to the `reverse_audio()` function as a tuple – sample rate in Hz and the audio data as a NumPy array. In our example, we took the audio data and reversed it using the `np.flipud()` function. The reversed audio is then returned to the output, where you can playback.

Figure 10.15 shows you can select the microphone to use (if you have more than one). Click the Record button to record the audio. Once the audio is recorded, click the Submit button.



**Figure 10.15** Choosing your microphone and recording the audio

You can now playback the original audio you recorded, and playback the audio that has been reversed (see Figure 10.16).



**Figure 10.16** You can playback the original recording, as well as playback the reversed audio

### 10.1.3 Working with Images

Besides audio, another media that Gradio can work with is image. You can work with images using the `Image` component. Listing 10.6 shows an example of how you can work with images in Gradio. Note that for this example you need to install the `skimage` package using the `pip` command:

```
!pip install scikit-image
```

**Listing 10.6 Working with images in Gradio**

```
from skimage.color import rgb2gray
import numpy as np
import gradio as gr

def convert_image(img):
    return rgb2gray(img)                                #A

image = gr.Image(type="numpy")                          #B

interface = gr.Interface(fn = convert_image,
                        inputs = image,
                        outputs = "image")

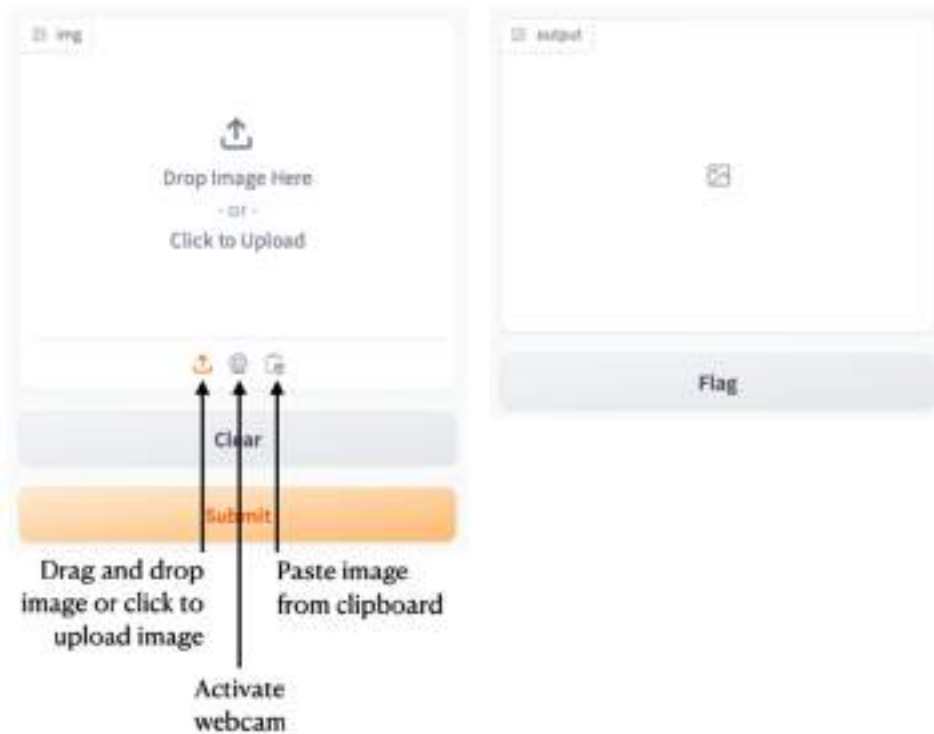
interface.launch()
```

**#A** return image as gray scale

**#B** work with the image as a NumPy array (default)

Figure 10.17 shows that there are three ways you can upload images to Gradio:

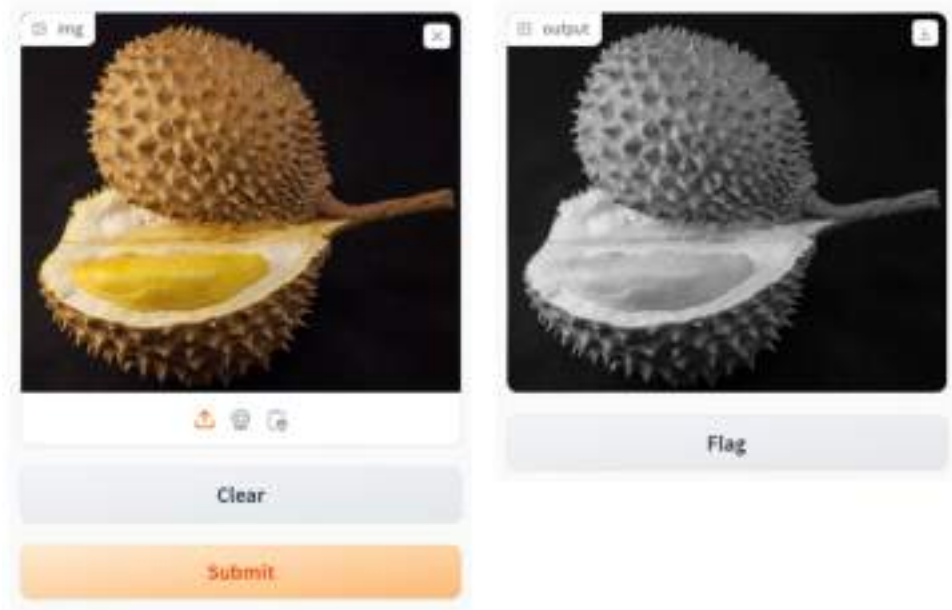
- Dragging and dropping the image directly onto the Image component
- Capturing images through the webcam
- Directly pasting an image from the clipboard of your computer



**Figure 10.17 Using the Image component in Gradio**

By default, all images passed into the Image component is in NumPy array format. Hence, in the above example, the image that you send to the Image component is thus sent as a NumPy array into the `img` parameter of the `convert_image()` function. In our implementation, we return the gray scale equivalent of the image using the `rgb2gray()` function in the `skimage` package.

Figure 10.18 shows how an image is transformed into grayscale after you clicked the Submit button.



**Figure 10.18** Converting an image to gray scale

If you want to work with PIL (Python Imaging Library) images in Gradio, you can set the `type` parameter to `pil` in the `Image()` class, as shown in Listing 10.7.

#### Listing 10.7 Working with PIL images

```
from skimage.color import rgb2gray
import numpy as np
import gradio as gr

def convert_image(img):
    return img.rotate(-90) #A

image = gr.Image(type="pil") #B

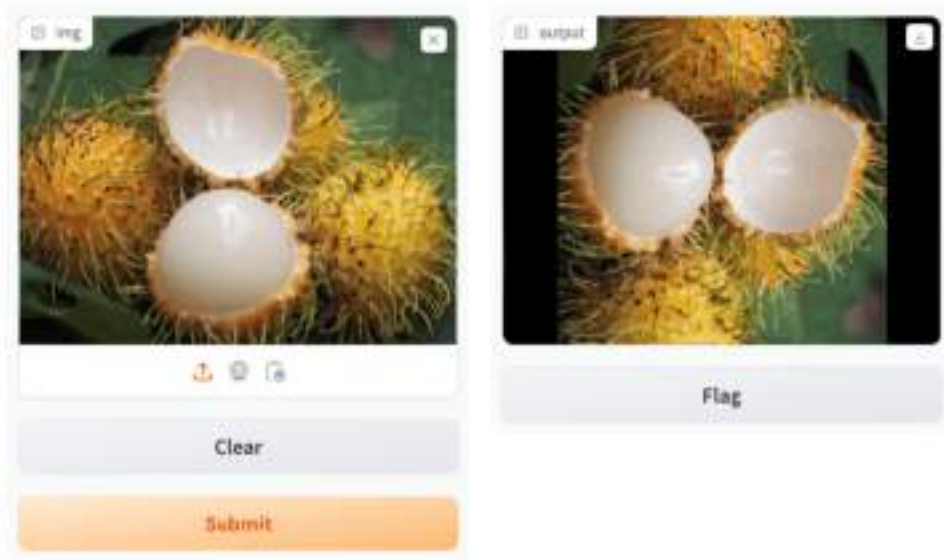
interface = gr.Interface(fn = convert_image,
                        inputs = image,
                        outputs = "image")

interface.launch()
```

#A rotate image clockwise 90 degrees

#B work with the image as a PIL image

In the above code snippet, the input image is rotated 90 degrees clockwise (see Figure 10.19).



**Figure 10.19 Rotating the image 90 degrees clockwise**

If you want to let the user choose from a predetermined set of images, you can use the `examples` parameter in the `Interface` class and then set it to a list of image names, as shown in Listing 10.8.

**Listing 10.8 Specifying a list of example images to use**

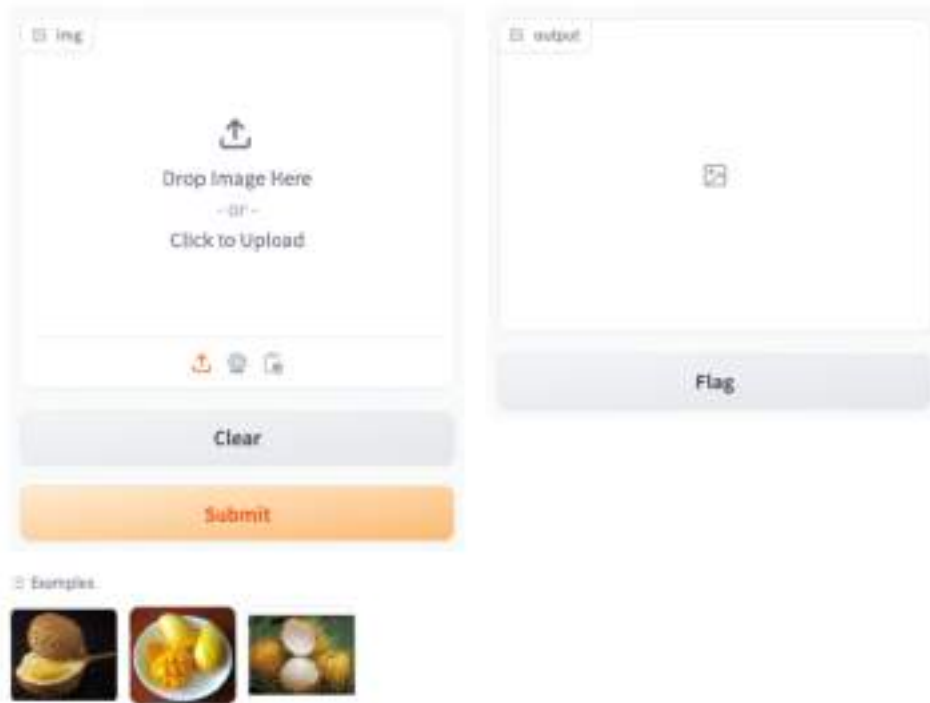
```
interface = gr.Interface(fn = convert_image,
                        inputs = image,
                        outputs = "image",
                        examples = [
                            "images/durian.jpg",
                            "images/mango.jpg",
                            "images/rambutan.jpg"]
                        )
interface.launch()
```

To use the above code snippet, you need to have a folder named **images** within the current Jupyter notebook's folder. Within the images folder you need to have three images named as follows:

- durian.jpg
- mango.jpg

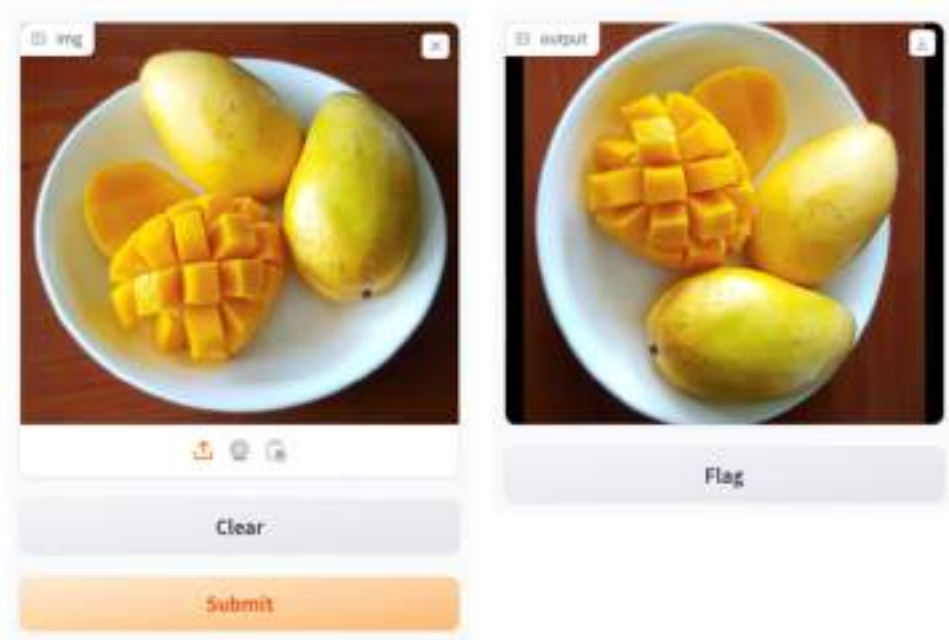
- rambutan.jpg

Figure 10.20 shows how the Gradio application looks like, with the three example images shown at the bottom.



**Figure 10.20 Besides using your own images, you can now select one of the example images provided**

You can select an example image and it will now be used as the input image. Figure 10.21 shows the image rotated 90 degrees clockwise.



**Figure 10.21** Selecting an example image and rotating it 90 degrees clockwise

### 10.1.4 Working with selection widgets

So far, you have seen several Gradio components, such as Textbox, Image, and Audio. In this example, you will learn two widgets that allows you to make selection. They are:

- **Dropdown** - The Dropdown component creates a dropdown of choices from which a single entry or multiple entries can be selected (as an input component) or displayed (as an output component).
- **Slider** - The Slider component creates a slider that ranges from `minimum` to `maximum` with a step size of `step`.

Listing 10.9 shows how to use both components.

#### Listing 10.9 Using the Dropdown and Slider components

```
import numpy as np
import gradio as gr

languages = ['English', 'Japanese', 'Chinese']

def translate(language_index, value, sentence):
    return languages[language_index], value, sentence
```



```

dropdown = gr.Dropdown(['English','Japanese','French','Chinese'],
                        type = "index",
                        label = "Language",
                        value = "English")

slider = gr.Slider(minimum = 1,
                   maximum = 5,
                   step = 1,
                   value = 2,
                   label = "Select a value")

textbox1 = gr.Textbox(type = "text",
                     value = "",
                     label = "Sentence to translate",
                     placeholder = "sentence")

textbox2 = gr.Textbox(type = "text",
                     label = "Translated sentence")

interface = gr.Interface(
    translate,
    [dropdown, slider, textbox1],
    textbox2,
)

interface.launch()

```

In the listing 10.9, you used the Dropdown, Slider, and the Textbox components for the input, and the Textbox component for the output (see Figure 10.22).

Language  
English

Select a value  
2

Sentence to translate  
sentence

Clear

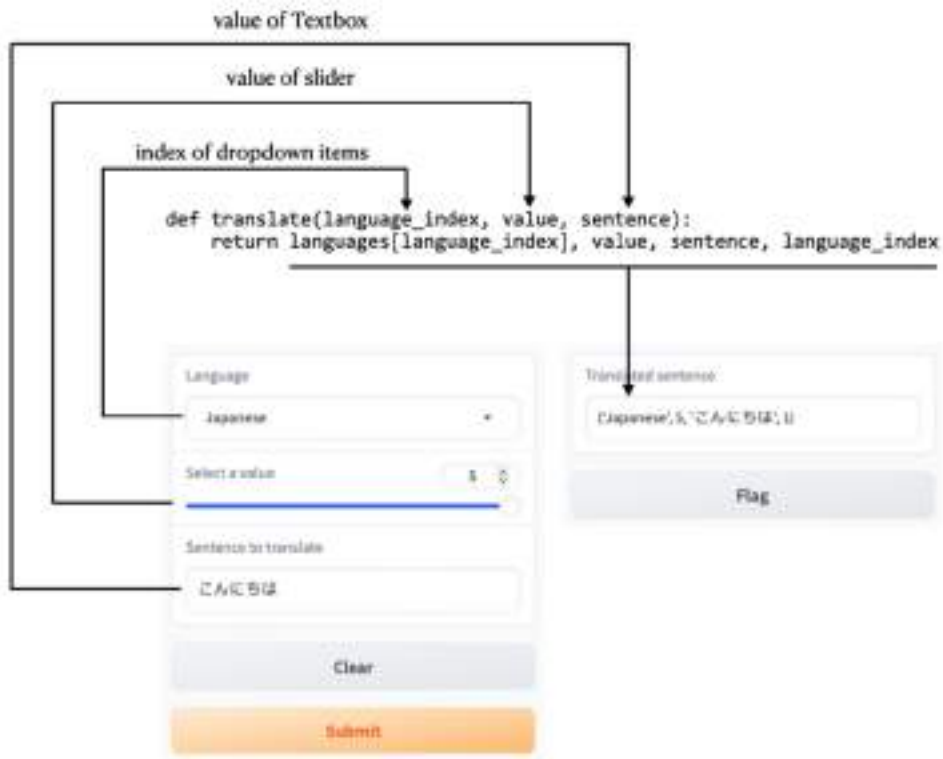
Submit

Translated sentence

Flag

**Figure 10.22 Using the Dropdown, Slider, and Textbox components for the inputs and outputs**

Say you selected the Japanese language, selected a value of 5, and entered the string “こんにちは”. After you clicked Submit, you will see the output as shown in Figure 10.23. The figure also explained how the returning value of the `translate()` function is linked to the output.



**Figure 10.23** Submitting the selected values and obtaining the output

If you have multiple outputs, say two Textboxes, you need to explicitly wrap the return values using a tuple, as shown in the code snippet in Listing 10.10.

**Listing 10.10** Modifying the return values for two outputs

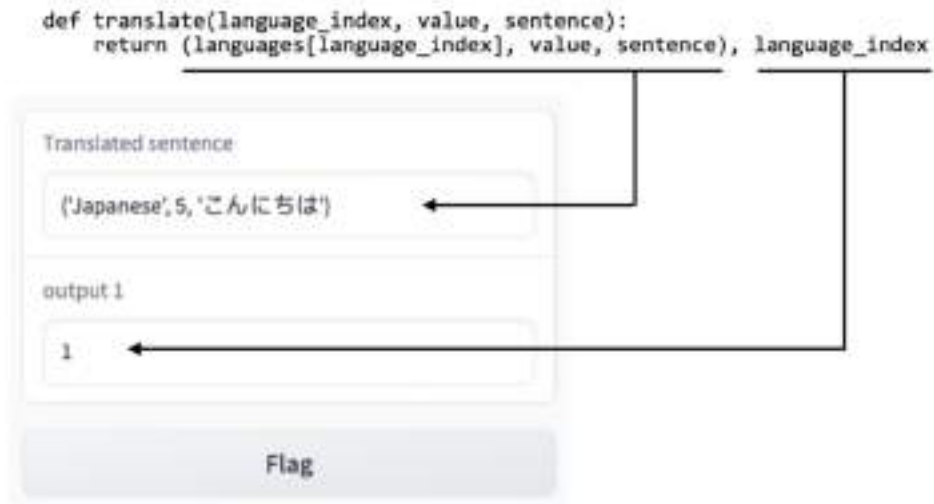
```
def translate(language_index, value, sentence):
    return (languages[language_index], value, sentence), language_index
...
...

interface = gr.Interface(
    translate,
    [dropdown, slider, textbox1],
    [textbox2, "text"],
)
```

Figure 10.24 shows the output for the modified Gradio application.

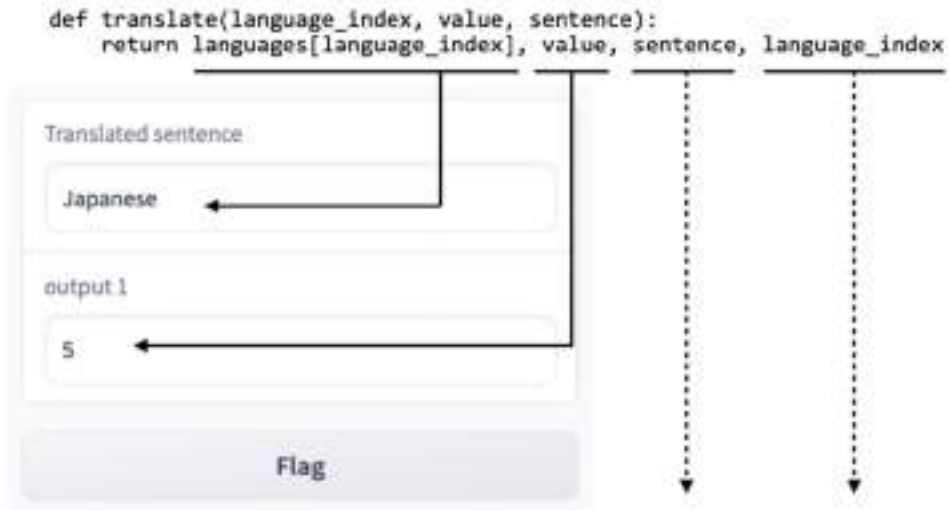
**Figure 10.24** There are two components in the output now

Notice that you need to wrap the first three return values in a tuple. This will allow the correct values to be sent to the output (see Figure 10.25).



**Figure 10.25** Wrapping the values in a tuple to send to one output

If you did not wrap the first three return values in a tuple, the first two results will be sent to the output (and the rest of the results would be discarded), as shown in Figure 10.26.



**Figure 10.26** Returning values will be dropped if they are not properly formed

### 10.1.5 Layout using the `TabbedInterface` class

Sometimes you may have a couple of machine learning models that you want users to experiment with. Instead of writing multiple Gradio applications for each model, you can group them in one single application using the `TabbedInterface` class. Listing 10.11 shows an example of this.

**Listing 10.11** Using the `TabbedInterface` class

```
import gradio as gr

def convert_image(img):
    return rgb2gray(img)

def reverse_audio(audio):
    sr, data = audio
    print(sr)
    reversed_audio = (sr, np.flipud(data))
    return reversed_audio

image = gr.Image(type="numpy")

mic = gr.Audio(sources = ["upload", "microphone"],
               type = "numpy",
               label = "Audio")
```

```

interface1 = gr.Interface(title = 'Reverse Audio',
                           fn = reverse_audio,
                           inputs = mic,
                           outputs = "audio")

interface2 = gr.Interface(title = 'Convert Image',
                           fn = convert_image,
                           inputs = image,
                           outputs = "image")

tabbed = gr.TabbedInterface(                                     #B
    [interface1, interface2],
    ['Tab 1', 'Tab 2']                                         #C
)

tabbed.launch()

```

**#A** return image to gray scale

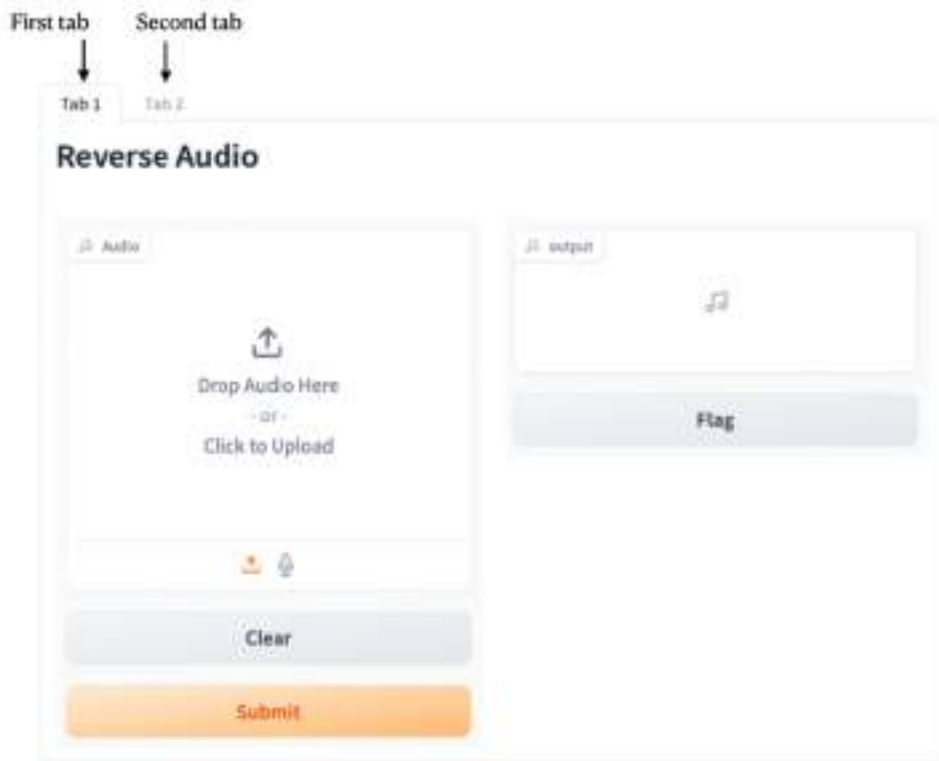
**#B** grouping the two interfaces

**#C** naming the tabs

In the above code snippet, there are two main functions:

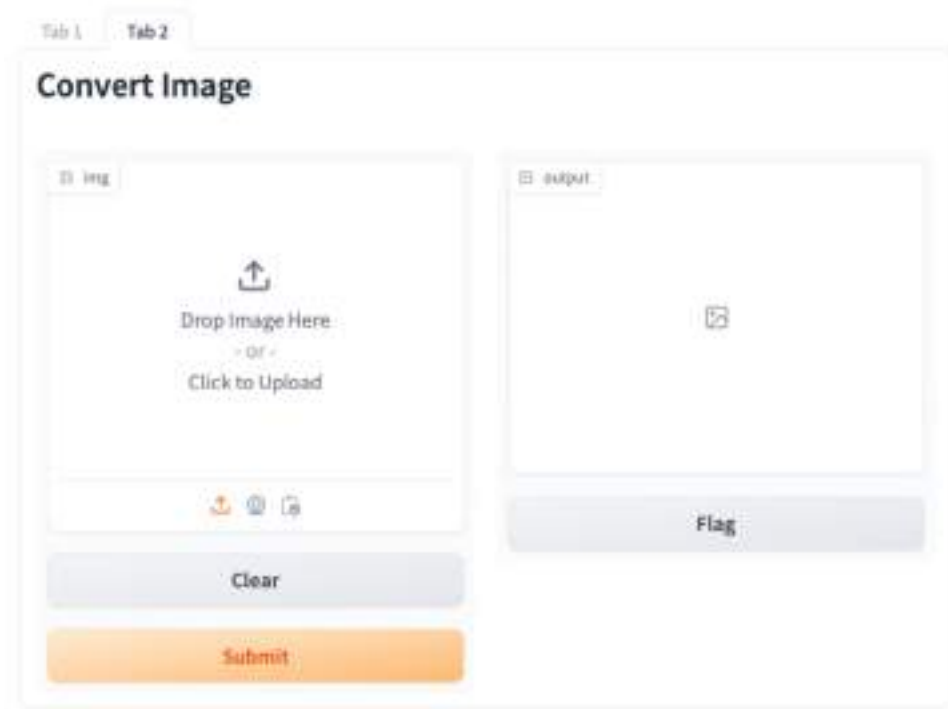
- A function to convert an image to grayscale - `convert_image()`
- A function to reverse an audio stream - `reverse_audio()`

You created two instances of the `Interface` class and then grouped them together using the `TabbedInterface` class. Figure 10.27 shows the Gradio application.



**Figure 10.27** The first tab of the Gradio application

Clicking on Tab 2 reveals the UI for the image conversion function (see Figure 10.28).



**Figure 10.28** The second tab of the Gradio application

## 10.2 Creating a Chatbot UI

So far, the Gradio applications that we have built all require the user to click the Submit button before processing the output. While this is quite natural in most applications (such as object detection, image segmentation, language translation, etc), it is awkward in use cases such as chatbots. In a chatbot, users typically type a message, press Enter to send it to the chatbot, which then responds with appropriate messages. Using the UI that we have seen so far for chatbots would be cumbersome as users need to click Submit and then clear the textbox manually before typing the next message.

To build a chatbot-like UI, you can use Gradio's `Blocks` class together with components like `Textbox` and `Button`. `Blocks` is a low-level API that you can use to create more customized web applications than the `Interface` class that you have seen previously.

To use `Blocks` in Gradio, follow the steps outlined below:

- Create a `Blocks` object, then use it as a context (using Python's `with` statement)
- Define layouts, components, or events within the `Blocks` context.
- Call the `launch()` method to launch the UI

Let's take a look at how to use a `Blocks` object to create a chatbot-like UI.



### 10.2.1 Creating the basic chatbot UI

Before we write the code, let's outline how we want the chatbot to look like and the components we are going to use. Figure 10.29 shows the components that we will use.



**Figure 10.29** The components used for our chatbot UI

In the figure, the Textbox will be where the user types in the message. The Button is used for clearing the conversation in the chatbot, which is represented by Gradio's Chatbot component. Using a Blocks object, let's create the UI that we have just seen (see Listing 10.12).

**Listing 10.12 Building the basic chatbot UI**

```
import gradio as gr

with gr.Blocks() as mychatbot:                                #A
    chatbot = gr.Chatbot(type = "messages")                  #B
    textbox = gr.Textbox()                                    #C
    clear = gr.Button("Clear Conversation")                   #D

mychatbot.launch()
```

#A Blocks is a low-level API that allows you to create custom web applications

#B displays a chatbot

#C for user to ask a question

#D Clear button

When you run the code snippet, the UI will be displayed. However, it is not functional yet. In the next two sections, you will wire the event handlers for the Textbox and Button components.

### 10.2.2 Wiring the Textbox's submit event

Let's first create an event handler for the `submit` event for the Textbox component. Listing 10.13 shows you how.

**Listing 10.13 Creating the event handler for the Textbox's submit event**

```

import gradio as gr

with gr.Blocks() as mychatbot:
    chatbot = gr.Chatbot(type="messages")
    textbox = gr.Textbox()
    clear = gr.Button("Clear Conversation")

    def chat(message, chat_history):
        response = "Responses from chatbot..." #A
        chat_history.append({"role": "user", "content": message}) #B
        chat_history.append({"role": "assistant", "content": response}) #B
        print(chat_history)
        return "", chat_history #C

    textbox.submit(fn = chat, #D
                  inputs = [textbox, chatbot], #D
                  outputs = [textbox, chatbot]) #D

mychatbot.launch()

```

#A replace with the actual responses from a chatbot

#B append the message and response to the history

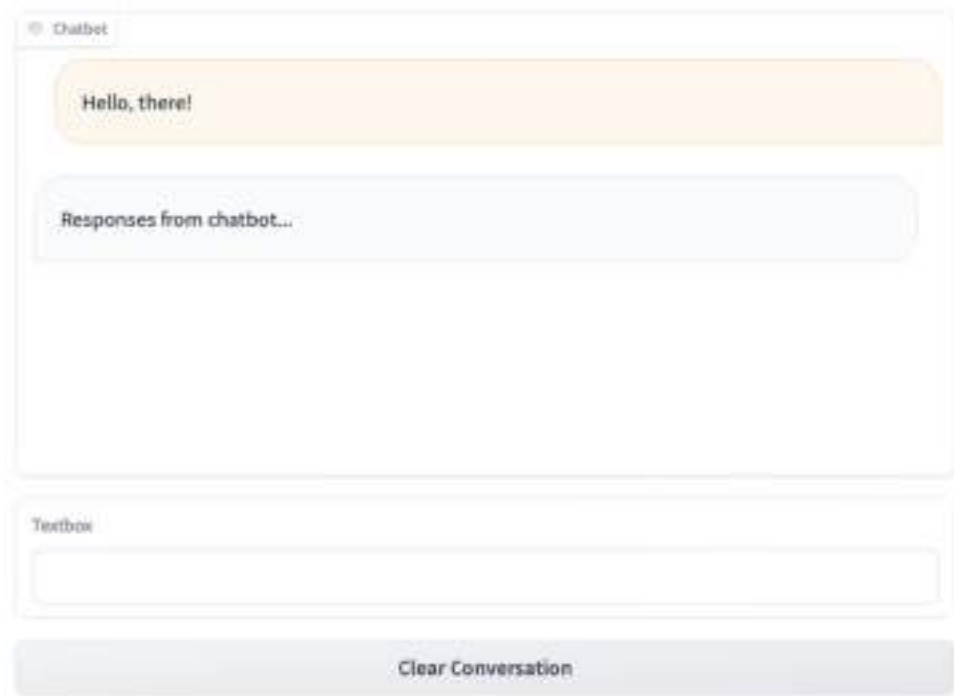
#C the "" is to clear the Textbox

#D wire up the event handler for Submit button (when user press Enter)

In the above code snippet, the `submit` event of the `Textbox` is wired to the `chat()` function, which takes in two arguments – `message`, and `chat_history`. The `inputs` and `outputs` parameters of the `Textbox` are set to `textbox` and `chatbot`.

Within the `chat()` function, you can pass the content of `message` to your chatbot (or LLM). Both the question and the response returned would then be appended to the `chat_history` argument. Here, we also printed out the content of `chat_history` so that we can see what is being stored. Finally, the `chat()` function returns a tuple – the string to return back to the `Textbox` after the user has pressed the `Enter` key, as well as the `chat_history` argument.

Figure 10.30 shows how happens after the user has typed "Hello, there!" in the `Textbox` and then pressed `Enter`. The content of the `message` variable, plus the response from the chatbot (replace with your own logic) are then appended to the `chat_history` argument. The `Chatbot` component then displays the message and response. Finally, the `chat()` function returns a tuple – "" (to clear the content of the `Textbox`), and the `chat_history` argument.



**Figure 10.30 Typing a message in the Textbox and sending it to the Chatbot component**

You can also examine the content of the `chat_history` argument that you have printed out:

```
[
  {'role': 'user', 'content': 'Hello, there!'},
  {'role': 'assistant', 'content': 'Responses from chatbot...'}
]
```

If you follow-up with another message – “Have you heard of how cool Gradio is?”, the output will be (formatted for clarity):

```
[
  {'role': 'user', 'metadata': None, 'content': 'Hello, there!',
   'options': None},
  {'role': 'assistant', 'metadata': None,
   'content': 'Responses from chatbot...',
   'options': None},
  {'role': 'user', 'content': 'Have you heard of how cool Gradio is?'},
  {'role': 'assistant', 'content': 'Responses from chatbot...'}
]
```

If you follow-up with yet another message – “How does it compare to Streamlit?”, the output will be (formatted for clarity):

```
[
  {'role': 'user', 'metadata': None, 'content': 'Hello, there!',
   'options': None},
  {'role': 'assistant', 'metadata': None, 'content':
   'Responses from chatbot...', 'options': None},
  {'role': 'user', 'metadata': None, 'content':
   'Have you heard of how cool Gradio is?', 'options': None},
  {'role': 'assistant', 'metadata': None, 'content':
   'Responses from chatbot...', 'options': None},
  {'role': 'user', 'content': "How does it compare to Streamlit?"},
  {'role': 'assistant', 'content': 'Responses from chatbot...'}
]
```

### 10.2.3 Clearing the Chatbot

The final step to implementing the chatbot UI is to create the event handler for the Button’s click event. The code snippet in Listing 10.14 shows you how.

**Listing 10.14 Creating the event handler for the Button's click event**

```

import gradio as gr

with gr.Blocks() as mychatbot:
    chatbot = gr.Chatbot(type="messages")
    textbox = gr.Textbox()
    clear = gr.Button("Clear Conversation")

    def chat(message, chat_history):
        response = "Responses from chatbot..."
        chat_history.append({"role": "user", "content": message})
        chat_history.append({"role": "assistant", "content": response})
        print(chat_history)
        return "", chat_history #A

    textbox.submit(fn = chat,
                  inputs = [textbox, chatbot],
                  outputs = [textbox, chatbot])

    def clear_messages():
        print("Clearing message...")

    clear.click(fn = clear_messages,
              inputs = None,
              outputs = chatbot,
              queue = False)

mychatbot.launch()

```

**#A** Clear textbox, return updated history

When the user clicks on the Clear Conversation button, the `clear_messages()` function is called. Here, you can replace its content with your own code to clear the chatbot (LLM, etc). The `outputs` parameter specifies that you return the `chatbot` object, which is empty.

That's it, you now have a functional chatbot! All you need to do is to replace the `chat()` function with the code to communicate with your own LLM.

## 10.3 Summary

- To build a simple Gradio application, you can use the `Interface` class. The `Interface` class is Gradio's main high-level class that allows you to build a web UI with just a few lines of code.
- The `Textbox` component allows users to send text content to the Gradio application.

- The Audio component allows users to send audio streams to the Gradio application.
- The Image component allows users to send images (as NumPy array or PIL image) to the Gradio application.
- You can implement authentication for your Gradio application using the `auth` parameter in the `launch()` method.
- The `Flag` option logs messages in the `log.csv` file.
- You can share the Gradio application with your users on the local network or create a shared link to share with other internet users.
- You can deploy your Gradio application on HuggingFace Spaces.
- You can group multiple interfaces using the `TabbedInterface` class.
- To create chatbot UI, use the `Blocks` class together with other Gradio components.

# ***11 Building Locally-Running LLM-based Applications using GPT4All***

## **This chapter covers**

- Introducing GPT4All
- Loading a model from GPT4All
- Holding a conversation with a model from GPT4All
- Creating a Web UI for GPT4All using Gradio

You've learned about constructing LLM-based applications using models from OpenAI and HuggingFace. While these models have transformed natural language processing, there are notable drawbacks. Primarily, privacy emerges as a critical concern for businesses. Relying on third-party hosted models introduces a security risk, as your conversations would be transmitted to these external companies, raising apprehensions for businesses dealing with sensitive data. Additionally, the challenge of integrating these models with your private data exists, and even if accomplished, the initial privacy issue resurfaces.

A more effective approach is to execute the models locally on your computer. This allows you to have control over the destination of your private data and enables fine-tuning of the models to suit your specific data requirements. However, running a LLM often demands GPUs, constituting a significant investment. Fortunately, there's a remedy – GPT4All. GPT4All provides quantized models, reduced to just a few gigabytes, which can operate on standard consumer-grade CPUs without requiring an internet connection. Introduction to GPT4All



GPT4All (<https://gpt4all.io/index.html>) is an open-source project containing several pre-trained Large Language Models (LLMs) that you can use to run locally using consumer grade CPUs. This accessibility proves invaluable, especially for individuals without access to high-end GPUs. Enabling the local deployment of LLMs contributes to democratizing AI, ensuring that everyone, regardless of hardware constraints, can actively participate in the creation of AI applications.

GPT4All contains several models that range from 3GB to 8GB. What's more exciting? It is free! While the performance of GPT4All may not be on par with the current ChatGPT, with contributions from the open-source community it has significant potentials for further development and enhancements.

## 11.1 Installing GPT4All

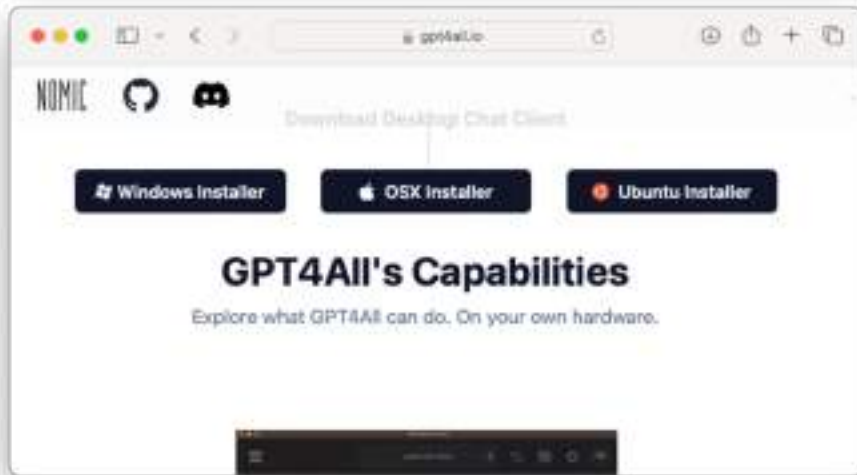
There are two types of installation for GPT4All:

- An end-user application for users to try out the various models supported by GPT4All. Does not requiring programming knowledge. Users will be able to initiate a chat conversation with the downloaded LLM.
- A Python library for developers to make use of the various models to build their LLM-based applications.

Now, we will go through these two types of installation methods. Our focus will be on the second method where you will learn how to build locally-running LLM-based applications using Python.

### 11.1.1 Installing the GPT4All Application

To install the GPT4All Desktop Chat Client, go to <https://gpt4all.io/index.html> and you can download the installer for the OS that you are using (see Figure 11.1).



**Figure 11.1** Download the installer for the OS you are using

Once the installer is downloaded, double-click the installer and you will be asked to provide a directory to store GPT4All (see Figure 11.2). You can take the default path suggested and click Next.



**Figure 11.2** Specifying the directory to store the GPT4All files

When the installation is complete, the first thing you need to do is give your permission to opt-in to the sharing of usage analytics and chats (see Figure 11.3). Depending on your preferences, you can click either Yes or No.



**Figure 11.3** Selecting if you wish to share usage statistics and chat details with GPT4All

Once that is done, you have the choice to download the various models available for use on GPT4All (see Figure 11.4). To download the model, simply click the Download button.



**Figure 11.4** Download the model(s) you wish to try

#### MINIMUM AMOUNT OF MEMORY REQUIRED

The amount of RAM required by most models is between 4-16GB. So, if you have a machine with only 8GB, it is likely you will not be able to run quite a number of these models.

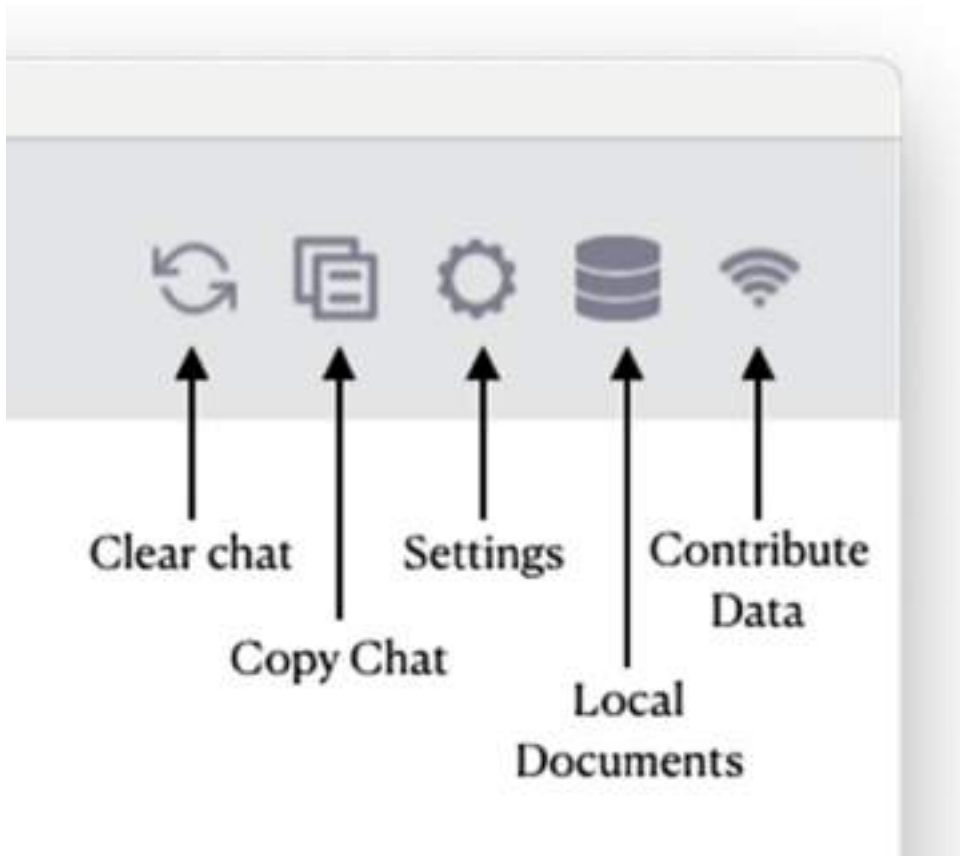
For running LLMs locally, we suggest you use a machine with at least 16GB RAM.

You can scroll to the bottom of the page to view more models. Once the model is downloaded, you can start chatting straight-away! Figure 11.5 shows the question we asked and the reply returned by the model.



**Figure 11.5 Testing the model by chatting and getting a reply from it**

At the top right corner of the application, there are a number of buttons. Figure 11.6 shows the meaning of each button.



**Figure 11.6** The meaning of each button on the top right corner of the application

To use the model to answer questions pertaining to your own documents, click the Local Documents button. Then, click the LocalDocs item (see Figure 11.7). To allow the model to answer questions on your own data, it needs to perform the word vector embedding process. Hence, you need to click the Download button and in the next screen, download the SBert Embedding model.



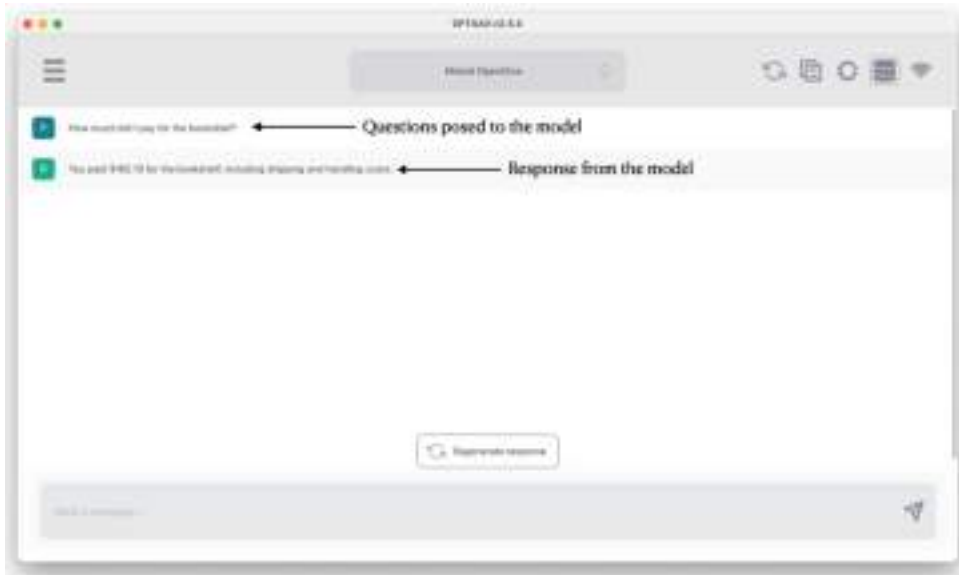


Finally, you can now click the Local Documents button (shown earlier in Figure 11.6) and then check the name of your local data (named *LocalData* in this example; see Figure 11.9).



**Figure 11.9** Selecting the local documents folder to use for querying

You can now ask the model questions pertaining to your local data (see Figure 11.10).



**Figure 11.10** Asking questions specific to your local data

### 11.1.2 Installing the gpt4all Python Library

Now that you have got the chance to try out the chatting capabilities of the various LLMs in GPT4All, it is time to use GPT4All programmatically in Python. To do that, you need to install it using the `pip` command:

```
!pip install gpt4all
```

At the time of writing, the version of `gpt4all` used is 2.0.2 and the version of `langchain` used is 0.0.351.

### 11.1.3 Listing all Supported Models

GPT4All supports several pre-trained models (LLMs). To list all the models available, use the `list_models()` function:

```
from gpt4all import GPT4All
GPT4All.list_models()
```

You will see the following output as shown in Listing 11.1 (shortened for brevity; the model names and their corresponding filenames are highlighted in bold):

**Listing 11.1 The list of supported GPT4All models**

```
[{'order': 'a',
  'md5sum': '48de9538c774188eb25a7e9ee024bbd3',
  'name': 'Mistral OpenOrca',
  'filename': 'mistral-7b-openorca.Q4_0.gguf',
  'filesize': '4108927744',
  'requires': '2.5.0',
  'ramrequired': '8',
  'parameters': '7 billion',
  'quant': 'q4_0',
  'type': 'Mistral',
  'systemPrompt': ' ',
  'description': '<strong>Best overall fast chat model</strong><br><ul><li>Fast
responses</li><li>Chat based model</li><li>Trained by Mistral AI</li><li>Finetuned on
OpenOrca dataset curated via <a href="https://atlas.nomic.ai/">Nomic Atlas</a>
</li><li>Licensed for commercial use</li></ul>',
  'url': 'https://gpt4all.io/models/gguf/mistral-7b-openorca.Q4_0.gguf'},
{'order': 'b',
  'md5sum': '97463be739b50525df56d33b26b00852',
  'name': 'Mistral Instruct',
  'filename': 'mistral-7b-instruct-v0.1.Q4_0.gguf',
  'filesize': '4108916384',
  'requires': '2.5.0',
  'ramrequired': '8',
  'parameters': '7 billion',
  'quant': 'q4_0',
  'type': 'Mistral',
  'systemPrompt': ' ',
  'description': '<strong>Best overall fast instruction following model</strong>
<br><ul><li>Fast responses</li><li>Trained by Mistral AI</li><li>Uncensored</li>
</li><li>Licensed for commercial use</li></ul>',
  'url': 'https://gpt4all.io/models/gguf/mistral-7b-instruct-v0.1.Q4_0.gguf',
  'promptTemplate': '[INST] %1 [/INST]}',
  ...

{'order': 'p',
  'md5sum': '919de4dd6f25351bcb0223790db1932d',
  'name': 'EM German Mistral',
  'filename': 'em_german_mistral_v01.Q4_0.gguf',
  'filesize': '4108916352',
  'requires': '2.5.0',
  'ramrequired': '8',
```

```

'parameters': '7 billion',
'quant': 'q4_0',
'type': 'Mistral',
'description': '<strong>Mistral-based model for German-language
applications</strong><br><ul><li>Fast responses</li><li>Chat based model</li>
<li>Trained by ellamind<li>Finetuned on German instruction and chat data</a>
<li>Licensed for commercial use</ul>',
'url': 'https://huggingface.co/TheBloke/em_german_mistral_v01-
GGUF/resolve/main/em_german_mistral_v01.Q4_0.gguf',
'promptTemplate': 'USER: %1 ASSISTANT: ',
'systemPrompt': 'Du bist ein hilfreicher Assistent. '}]

```

As the list is quite long, it would be useful to just extract the model names and their corresponding filenames, like this:

```

models = GPT4All.list_models()
[{'model['name']:model['filename']} for model in models]

```

This will generate the following simplified list:

```

[{'Mistral OpenOrca': 'mistral-7b-openorca.Q4_0.gguf'},
 {'Mistral Instruct': 'mistral-7b-instruct-v0.1.Q4_0.gguf'},
 {'GPT4All Falcon': 'gpt4all-falcon-q4_0.gguf'},
 {'Orca 2 (Medium)': 'orca-2-7b.Q4_0.gguf'},
 {'Orca 2 (Full)': 'orca-2-13b.Q4_0.gguf'},
 {'Wizard v1.2': 'wizardlm-13b-v1.2.Q4_0.gguf'},
 {'Hermes': 'nous-hermes-llama2-13b.Q4_0.gguf'},
 {'Snoozy': 'gpt4all-13b-snoozy-q4_0.gguf'},
 {'MPT Chat': 'mpt-7b-chat-merges-q4_0.gguf'},
 {'Mini Orca (Small)': 'orca-mini-3b-gguf2-q4_0.gguf'},
 {'Replit': 'replit-code-v1_5-3b-q4_0.gguf'},
 {'StarCoder': 'starcoder-q4_0.gguf'},
 {'Rift coder': 'rift-coder-v0-7b-q4_0.gguf'},
 {'SBert': 'all-MiniLM-L6-v2-f16.gguf'},
 {'EM German Mistral': 'em_german_mistral_v01.Q4_0.gguf'}]

```

#### 11.1.4 Loading a Specific Model

Based on the models that you have seen listed in the previous section, you can now go ahead and load the model that you want to use. For this example, we will use the *Mistral OpenOrca* model (filename *mistral-7b-openorca.Q4\_0.gguf*):

```
gpt = GPT4All("mistral-7b-openorca.Q4_0.gguf")
```

## MISTRAL AI

Mistral AI is a startup in the AI sector. Its mission is to revolutionize generative artificial intelligence (AI) with its first large language model (LLM), Mistral 7B. The company hopes the new 7-billion-parameter model will become an open-source alternative to current AI solutions.

When you load the model for the first time, GPT4All will download the *mistral-7b-openorca.Q4\_0.gguf*, which is a 4.11GB file. It will be stored in the following directory:

```
~/.cache/gpt4all/
```

You can print out more information about this model using the `config` attribute:

```
print(gpt.config)
```

Here are the details of the *Mistral OpenOrca* model:

```
{
  'systemPrompt': '',
  'promptTemplate': '### Human: \n{0}\n### Assistant:\n',
  'order': 'a',
  'md5sum': '48de9538c774188eb25a7e9ee024bbd3',
  'name': 'Mistral OpenOrca',
  'filename': 'mistral-7b-openorca.Q4_0.gguf',
  'filesize': '4108927744',
  'requires': '2.5.0',
  'ramrequired': '8',
  'parameters': '7 billion',
  'quant': 'q4_0',
  'type': 'Mistral',
  'description': '<strong>Best overall fast chat model</strong><br><ul><li>Fast
responses</li><li>Chat based model</li><li>Trained by Mistral AI</li>Finetuned on
OpenOrca dataset curated via <a href="https://atlas.nomic.ai/">Nomic Atlas</a>
<li>Licensed for commercial use</li></ul>',
  'path': '/Users/weimenglee/.cache/gpt4all/mistral-7b-openorca.Q4_0.gguf'
}
```

### 11.1.5 Asking a Question

With the model downloaded, it is time to put it to the test. To have a chat conversation using GPT4All, you need to use the `chat_session()` method to create a contextual manager to hold an inference optimized chat session with a model. Once this is done, you can use the `generate()` method to ask the question. The following code snippet shows how this is done using the `with` keyword in Python:

```
with gpt.chat_session():
    output = gpt.generate("What is the population of Japan?",
                          max_tokens=2048)
    print(output)
    print(gpt.current_chat_session)
```

For the above question ("What is the population of Japan?"), the response from the model is:

As of 2021, the estimated population of Japan is approximately 126 million people. However, this number may change over time due to factors such as births, deaths, and migration.

We also printed out (formatted for clarity) the details of the current session using the `current_chat_session` attribute:

```
[
  {
    'role': 'system',
    'content': ''
  },
  {
    'role': 'user',
    'content': 'What is the population of Japan?'
  },
  {
    'role': 'assistant',
    'content': ' As of 2021, the estimated population of Japan is
approximately 126 million people. However, this number may
change over time due to factors such as births, deaths, and
migration.'
  }
]
```

If you want to ask a follow-up question, you must do it within the scope of the `with` keyword, like this:

```
with gpt.chat_session():
    response1 = gpt.generate(
        prompt='What is the population of Singapore?',
        temp = 0)
    print(response1)
    print(gpt.current_chat_session)
    print('===')

    response2 = gpt.generate(
        prompt='Where is it located?',
        temp = 0)
    print(response2)
    print(gpt.current_chat_session)
```

Here's the response for the first question:

As of 2021, the estimated population of Singapore is around 5.6 million people. However, this number may change over time due to births, deaths, and migration.

And here's the current chat session value:

```
[
  {
    'role': 'system',
    'content': ''
  },
  {
    'role': 'user',
    'content': 'What is the population of Singapore?'
  },
  {
    'role': 'assistant',
    'content': ' As of 2021, the estimated population of
               Singapore is around 5.6 million people.
               However, this number may change over time
               due to births, deaths, and migration.'
  }
]
```

Here's the response for the follow-up question:

Singapore is a city-state and island country in Southeast Asia. It lies off the southern tip of the Malay Peninsula, about 85 miles (137 kilometers) north of the equator. It shares its only land border with Malaysia to the north. The country consists of one main island, called Singapore Island, and more than 60 smaller islands.

And here's the updated chat session:

```
[
  {
    'role': 'system',
    'content': ''
  },
  {
    'role': 'user',
    'content': 'What is the population of Singapore?'
  },
  {
    'role': 'assistant',
    'content': ' As of 2021, the estimated population of
               Singapore is around 5.6 million people.
               However, this number may change over time
               due to births, deaths, and migration.'
  },
  {
    'role': 'user',
    'content': 'Where is it located?'
  },
  {
    'role': 'assistant',
    'content': ' Singapore is a city-state and island country
               in Southeast Asia. It lies off the southern
               tip of the Malay Peninsula, about 85 miles
               (137 kilometers) north of the equator. It
               shares its only land border with Malaysia to
               the north. The country consists of one main
               island, called Singapore Island, and more than
               60 smaller islands.\n\n'
  }
]
```



```
]

```

As you can see, the chat session contains the details of the earlier conversation. This allows the model to maintain the context. The following won't work as the second question will not be in the same the context as the first question:

```
with gpt.chat_session():
    response1 = gpt.generate(
        prompt='What is the population of Singapore?',
        temp = 0)
    print(response1)

with gpt.chat_session():
    response2 = gpt.generate(
        prompt='Where is it located?',
        temp = 0)
    print(response2)
```

If you need to ask a follow-up question but at a later time, you can always save the context using the `gpt.current_chat_session` attribute and then set it back again. The following code snippet shows how to do this:

```
session = []

with gpt.chat_session():
    response1 = gpt.generate(prompt='What is the population of Singapore?',
                             temp = 0)
    print(response1)
    #A
    session = gpt.current_chat_session
```

**#A save the current chat session**

```
with gpt.chat_session():
    #A
    gpt.current_chat_session = session
    response2 = gpt.generate(prompt='Where is it located?', temp = 0)
    print(response2)
```

**#A set current chat session to the previously saved one**

### 11.1.6 Binding with Gradio

A great way to work with the GPT4All models is to bind one to Gradio. In the following code snippet, you first define a function named `chat()` which will call the `generate()` method of the model:

```
from gpt4all import GPT4All

gpt = GPT4All("mistral-7b-openorca.Q4_0.gguf")

def chat(message):
    with gpt.chat_session():
        return gpt.generate(prompt = message,
                             temp = 0)
```

You can then use Gradio and bind it to the `chat()` function:

```
import gradio as gr

gr.Interface(fn = chat,
             inputs = "text",
             outputs = "text").launch()  #A
```

#A bind it to gradio

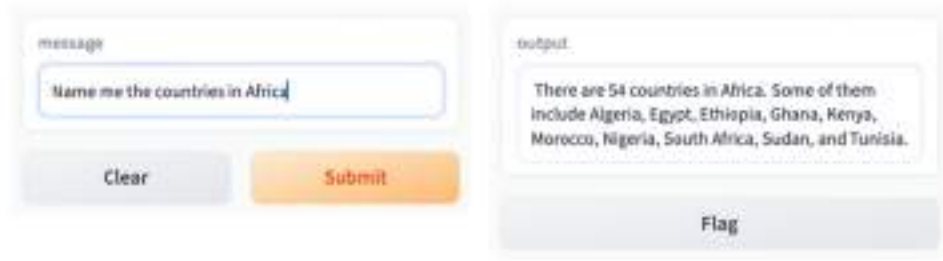
Figure 11.11 shows how the Gradio interface looks like.

```
Running on local URL: http://127.0.0.1:7861
To create a public link, set 'share=True' in 'launch()'.
```



**Figure 11.11 Binding the model to Gradio**

You can now ask a question, click the Submit button and obtain the response from the model (see Figure 11.12).



**Figure 11.12** You can now ask questions through the Gradio interface

But do remember that you cannot ask a follow-up question as every time you click the Submit button, the `chat()` function will starting a new chat session.

To fix this, you need to save the current chat session details into a global variable and set it back every time the user asks a follow-up question, like this:

```
import gradio as gr
from gpt4all import GPT4All

gpt = GPT4All("mistral-7b-openorca.Q4_0.gguf")
current_chat_session = []

def chat(message):
    with gpt.chat_session():
        global current_chat_session
        gpt.current_chat_session = current_chat_session
        response = gpt.generate(prompt = message,
                                temp = 0)

        current_chat_session = gpt.current_chat_session
        return response

#A
gr.Interface(fn = chat,
             inputs = "text",
             outputs = "text").launch()
```

#A bind it to gradio

Figure 11.13 shows the user asking two consecutive questions through the Gradio interface:

**Initial Question**

message: Which country has the most powerful passports?

output: The Henley Passport Index, which ranks countries based on the travel freedom their citizens enjoy, currently shows that Japan holds the strongest passports.

Clear Submit Flag

**Follow-up Question**

message: Why is that so?

output: The strength of a passport depends on the number of countries its holders can visit without needing a visa in advance or obtaining one upon arrival. Factors affecting this include diplomatic relations between countries, geopolitical considerations, and economic influence. Japan's strong passports are likely due to their positive international relationships, advanced economy, and overall global standing.

Clear Submit Flag

**Figure 11.13** You can now ask follow-up questions through the Gradio interface

If you use the application long enough, you will realize that having to clear the message every time you want to ask a follow-up question is troublesome. A much more intuitive UI would be to make it behave more like a chat application. In fact, you can build a chatbot-like UI using Gradio.

The following code snippet shown in Listing 11.2 shows how to wrap the GPT4All call with a chatbot-like UI using Gradio:

**Listing 11.2** Display a chatbot user interface using Gradio

```
import gradio as gr
from gpt4all import GPT4All

gpt = GPT4All("mistral-7b-openorca.Q4_0.gguf")
current_chat_session = []

with gr.Blocks() as mychatbot:                                     #A
    chatbot = gr.Chatbot()                                         #B
                                                                    #C
```

```

question = gr.Textbox()                                #D
clear = gr.Button("Clear Conversation")                 #E

                                                         #F
def clear_messages():
    global current_chat_session
    current_chat_session = []                           #G

                                                         #H
def chat(message, chat_history):
    with gpt.chat_session():
        global current_chat_session
        gpt.current_chat_session = current_chat_session

        response = gpt.generate(prompt = message,
                                  temp = 0)
        current_chat_session = gpt.current_chat_session

                                                         #I
        chat_history.append((message, response))

                                                         #J
        return "", chat_history

                                                         #K
question.submit(fn = chat,
                inputs = [question, chatbot],
                outputs = [question, chatbot])

                                                         #L
clear.click(fn = clear_messages,
            inputs = None,
            outputs = chatbot,
            queue = False)

mychatbot.launch()

```

**#A** Blocks is a low-level API that allows  
**#B** you to create custom web applications  
**#C** displays a chatbot  
**#D** for user to ask a question  
**#E** Clear button  
**#F** function to clear the conversation  
**#G** reset the messages list

```

#H function to ask the LLM a question
#I append the response to the current message and return to Gradio
#J the "" is to clear the input textbox
#K wire up the event handler for Submit button (when user press Enter)
#L wire up the event handler for the Clear Conversation button

```

Note that this time round the `chat()` function accepts two arguments — the question to ask, and the chat history:

```
def chat(message, chat_history):
```

After the model has responded with the answer, you append the response to the `chat_history` parameter and return it together with a string (set to `""` in this example). The use of the string is to display the text in the input textbox:

```

    chat_history.append((message, response))

    return "", chat_history
#A

```

#A the `""` is to clear the input textbox

When you run the above code snippet, you will see the UI as shown in Figure 11.14.



**Figure 11.14** The chatbot UI displayed by Gradio

You can now go ahead and chat with the model to your hearts content (see Figure 11.15)!



**Figure 11.15 Chatting with the model is now much easier and more natural**

## 11.2 Summary

- GPT4All is an open-source project containing several pre-trained Large Language Models (LLMs) that you can use to run locally using consumer grade CPUs.
- GPT4All comes with an end-user application where you can try out the various models supported by GPT4All.
- You can also use the gpt4all Python library to programmatically work with the models using Python.
- A great way to bind a Web UI for your chatbot is to use the Gradio library



## ***12 Using LLMs to Query Your Local Data***

### **This chapter covers**

- Using GPT4All to query your own private data
- Using PDF documents for querying by a LLM
- Loading CSV and JSON files for querying
- Using LLMs to analyze your own data files

Up to this point, you've explored the capabilities of LLMs and their usage through platforms like OpenAI and Hugging Face. While these services ease the burden of hosting models, they come at a cost. Alternatively, running powerful models locally requires significant setup effort and cost.

Developers often face the common challenge of utilizing LLMs to answer questions about their data, while businesses emphasize the need to maintain data privacy. In Chapter 8, we discussed sending data to OpenAI for embedding and querying with LangChain and LlamaIndex.

Let's delve deeper into the topic, focusing on querying local private documents without compromising data privacy. Two approaches are:

- **Local LLM Querying for Text-based Data:** We'll utilize a model from GPT4All to perform local embedding of your text-based data and querying. This approach is particularly useful for querying content like PDF documents.

- LLM Querying for Structured Tabular Data: Whether running locally or hosted by third parties like OpenAI or Hugging Face, LLMs can be employed to return answers on querying tabular data (e.g., CSV or JSON). Instead of feeding LLMs with tabular data directly, we'll instruct them to provide queries programmatically for analysis.

These two approaches cater to different data types and privacy concerns, ensuring flexibility and effectiveness in leveraging LLMs for varied scenarios.

## 12.1 Using GPT4All to Query with Your Own Data

The first approach we will be discussing is to make use of GPT4All to query our own data. To ensure privacy of our data, the embeddings for our data will be performed locally, without ever leaving the computer. You will learn how to query the following types of documents:

- PDF documents
- CSV documents
- JSON documents

### 12.1.1 Installing the Required Packages

For this task, we will be using the following Python packages:

- `langchain` – We will use LangChain to chain the model together with the various components such as prompt template, embeddings, etc.
- `gpt4all` – We will use the one of the models from GPT4All
- `faiss-cpu` – Faiss (Facebook AI Similarity Search) is a library for efficient similarity search and clustering of dense vectors. It is developed by Facebook AI Research. Use `faiss-cpu` if you do not have a GPU on your computer. If you have a GPU, use `faiss-gpu`.
- `hugging-face-hub` – The `huggingface-hub` package is a Python library developed by Hugging Face that provides a convenient interface for interacting with the Hugging Face Hub, a platform for sharing machine learning models, datasets, and other AI-related assets.
- `Sentence-transformers` – We will use a `sentence-transformers` model from HuggingFace Hub. A `sentence-transformers` model maps sentences and paragraphs to a multi-dimensional dense vector space and is used for tasks like clustering or semantic search.

To install all these packages, type the following commands in Terminal (or Command Prompt):

```
$ pip install langchain
$ pip install gpt4all
$ pip install faiss-cpu
$ pip install huggingface-hub
$ pip install sentence-transformers
```

### 12.1.2 Importing the Various Modules from the LangChain Package

Once the required packages are installed, let's now import the various modules from the `langchain` package into Jupyter notebook:

```
from langchain.document_loaders import PyPDFLoader
from langchain import PromptTemplate
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores.faiss import FAISS
from langchain_core.output_parsers import StrOutputParser
from langchain.llms import GPT4All
```

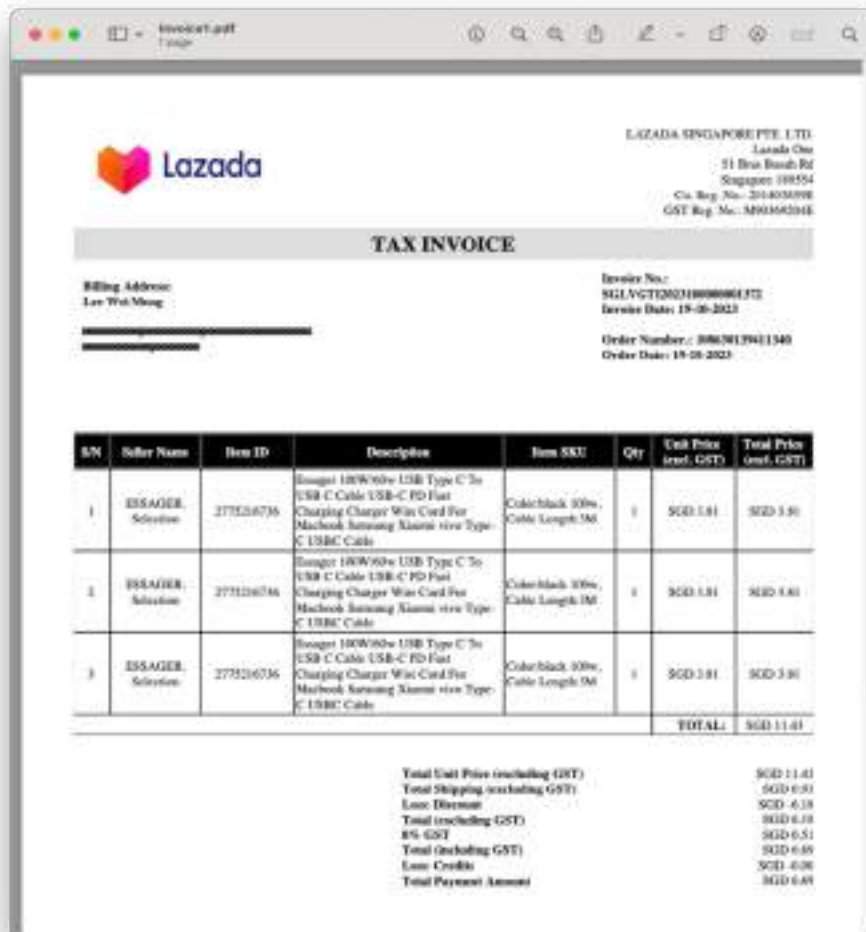
### 12.1.3 Loading the PDF Documents

To build a chatbot to answer queries based on your own local data, you need to first load the data. Let's start off with using a PDF document. To load PDF documents, you can use the `PyPDFLoader` class from the `document_loaders` module in `langchain`:

```
documents =
    PyPDFLoader('./LocalDataForTraining/Invoice1.pdf').load_and_split()
```

The above statement loads the *Invoice1.pdf* document from the `LocalDataForTraining` directory that is stored in the same directory as your Jupyter notebook. Once the document is loaded, you call the `load_and_split()` method to split the loaded document into chunks. Chunks are returned as `Document` objects.

The PDF document is as shown in Figure 12.1. It contains an invoice of some items purchased online. What we want to do in this project is to get the LLM to answer questions pertaining to the content of this PDF document.



**Figure 12.1** The content of the PDF document contains an invoice of items purchased online

If you print out the content of the `documents` variable, you will see its content:

```
[Document(metadata={'source': './LocalDataForTraining/Invoice1.pdf',
'page': 0}, page_content='Lazada Singapore Pte Ltd is raising this invoice
in accordance to the applicable tax laws in Singapore\nThis shipment
includes any taxes (when applicable) for the merchandise to be delivered to
the address in the country\nspecified by the customer. LAZADA pays these
taxes on behalf of the customer.\nTo understand our return policy and find
out how to return, please click .here\nNEED HELP? Contact us at
https://www.lazada.sg/contact/\nLIKE US on FACEBOOK:
https://www.facebook.com/LazadaSingapore\nFOLLOW US on TWITTER:
https://www.twitter.com/LazadaSG/\nHave a great day! Thank you for shopping
on www.LAZADA.sg\n      \nLAZADA SINGAPORE PTE. LTD.\nLazada One\n51 Bras
Basah Rd\nSingapore 189554\nCo. Reg. No.: 201403859E\nGST Reg. No.:
M90369204E\nTAX INVOICE\nBilling Address:\nLee Wei Meng\n, , \n ,
Invoice No.: \nSGLVGTI2023100000801372\nInvoice Date: 19-10-2023\nOrder
Number.: 108630139411340\nOrder Date: 19-10-2023\nS/N Seller Name Item ID
Description Item SKU QtyUnit Price \n(excl. GST)Total Price \n(excl.
GST)\n1ESSAGER.\nSelection2775216736Essager 100W/60w USB Type C To \nUSB C
Cable USB-C PD Fast \nCharging Charger Wire Cord For \nMacbook Samsung
Xiaomi vivo Type-\nC USBC CableColor:black 100w, \nCable Length:3M1 SGD
3.81 SGD 3.81\n2ESSAGER.\nSelection2775216736Essager 100W/60w USB Type C To
\nUSB C Cable USB-C PD Fast \nCharging Charger Wire Cord For \nMacbook
Samsung Xiaomi vivo Type-\nC USBC CableColor:black 100w, \nCable Length:3M1
SGD 3.81 SGD 3.81\n3ESSAGER.\nSelection2775216736Essager 100W/60w USB Type
C To \nUSB C Cable USB-C PD Fast \nCharging Charger Wire Cord For \nMacbook
Samsung Xiaomi vivo Type-\nC USBC CableColor:black 100w, \nCable Length:3M1
SGD 3.81 SGD 3.81\nTOTAL: SGD 11.43\nTotal Unit Price (excluding GST) SGD
11.43\nTotal Shipping (excluding GST) SGD 0.93\nLess: Discount SGD -
6.18\nTotal (excluding GST) SGD 6.18\n8% GST SGD 0.51\nTotal (including
GST) SGD 6.69\nLess: Credits SGD -0.00\nTotal Payment Amount SGD
6.69\n**This is a computer generated copy. No signature is required**')]
```

The `documents` variable contains a list of `Document` objects.

#### 12.1.4 Splitting the Text into Chunks

The next step is to use a `RecursiveCharacterTextSplitter` object to split the document into chunks of a specific size. This process is known as *chunking*.

## CHUNKING

In Natural Language Processing (NLP), chunking is the process of breaking down a text into smaller, meaningful units or "chunks". For LLMs, a model typically works within a specific context length, e.g. 4096 tokens (approximately equivalent to 3500 words). When you present this model with a document that is larger than its context window, it will not be able to work with the document. Hence you need to break the document into smaller chunks that can fit into the context window of the model.

The following code snippet creates a `RecursiveCharacterTextSplitter` object to split the document into chunks:

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size = 1024,
                                              chunk_overlap = 64)
texts = text_splitter.split_documents(documents)
```

If you print out the content of the `texts` variable, you will see the following (formatted for clarity):

```
[
  Document(metadata={'source': './LocalDataForTraining/Invoice1.pdf',
                    'page': 0}, page_content='Lazada Singapore Pte Ltd is raising this invoice
in accordance to the applicable tax laws in Singapore\nThis shipment
includes any taxes (when applicable) for the merchandise to be delivered to
the address in the country\nspecified by the customer. LAZADA pays these
taxes on behalf of the customer.\nTo understand our return policy and find
out how to return, please click .here\nNEED HELP? Contact us at
https://www.lazada.sg/contact/\nLIKE US on FACEBOOK:
https://www.facebook.com/LazadaSingapore\nFOLLOW US on TWITTER:
https://www.twitter.com/LazadaSG/\nHave a great day! Thank you for shopping
on www.LAZADA.sg\n      \nLAZADA SINGAPORE PTE. LTD.\nLazada One\n51 Bras
Basah Rd\nSingapore 189554\nCo. Reg. No.: 201403859E\nGST Reg. No.:
M90369204E\nTAX INVOICE\nBilling Address:\nLee Wei Meng\n, ,      \n ,
Invoice No.: \nSGLVGTI2023100000801372\nInvoice Date: 19-10-2023\nOrder
Number.: 108630139411340\nOrder Date: 19-10-2023\nS/N Seller Name Item ID
Description Item SKU QtyUnit Price \n(excl. GST)Total Price \n(excl.
GST)\n1ESSAGER.'),
  Document(metadata={'source': './LocalDataForTraining/Invoice1.pdf',
                    'page': 0}, page_content='(excl. GST)Total Price \n(excl.
GST)\n1ESSAGER.\nSelection2775216736Essager 100W/60w USB Type C To \nUSB C
```

```

Cable USB-C PD Fast \nCharging Charger Wire Cord For \nMacbook Samsung
Xiaomi vivo Type-\nC USBC CableColor:black 100w, \nCable Length:3M1 SGD
3.81 SGD 3.81\n2ESSAGER.\nSelection2775216736Essager 100W/60w USB Type C To
\nUSB C Cable USB-C PD Fast \nCharging Charger Wire Cord For \nMacbook
Samsung Xiaomi vivo Type-\nC USBC CableColor:black 100w, \nCable Length:3M1
SGD 3.81 SGD 3.81\n3ESSAGER.\nSelection2775216736Essager 100W/60w USB Type
C To \nUSB C Cable USB-C PD Fast \nCharging Charger Wire Cord For \nMacbook
Samsung Xiaomi vivo Type-\nC USBC CableColor:black 100w, \nCable Length:3M1
SGD 3.81 SGD 3.81\nTOTAL: SGD 11.43\nTotal Unit Price (excluding GST) SGD
11.43\nTotal Shipping (excluding GST) SGD 0.93\nLess: Discount SGD -
6.18\nTotal (excluding GST) SGD 6.18\n8% GST SGD 0.51\nTotal (including
GST) SGD 6.69\nLess: Credits SGD -0.00\nTotal Payment Amount SGD
6.69\n**This is a computer generated copy. No signature is required**')
]

```

### 12.1.5 Embedding

The next step is to perform word embeddings on the document text.

#### WORD EMBEDDINGS

In Natural Language Processing (NLP), embedding refers to the representation of words or sentences as vectors in a high-dimensional space. It is a way to represent words and sentences in a numerical manner.

The key purpose of embedding is to capture semantic and syntactic relationships between words, allowing machines to understand and process human language more effectively, where words that have the same meaning have a similar representation.

For word embedding, we will use the `sentence-transformers/all-MiniLM-L6-v2` model that is hosted on the HuggingFace Hub:

```

embeddings = HuggingFaceEmbeddings(
    model_name = 'sentence-transformers/all-MiniLM-L6-v2')
faiss_index = FAISS.from_documents(texts, embeddings)

```

The `sentence-transformers/all-MiniLM-L6-v2` model maps sentences & paragraphs to a 384-dimensional dense vector space. You will use the Faiss's library to perform the embedding. Note that when you run the above code snippet for the first time, Jupyter notebook will download the model from HuggingFace Hub.

Once the embedding is complete, you will save the embeddings into a local directory (`index` in this example):

```
faiss_index.save_local("./index")
```

Once the embeddings are saved, the `index` folder will contain two files:

- `index.faiss`
- `index.pkl`

Note that the embedding just needs to be performed once unless your document content changes. The embeddings are saved to a local directory so that in future when you need to run the model to query the document you can simply load it up again without performing the embeddings again.

### 12.1.6 Loading the Embeddings

To load the embeddings from disk, use the `load_local()` function from the FAISS library:

```
embeddings = HuggingFaceEmbeddings(
    model_name = 'sentence-transformers/all-MiniLM-L6-v2')
faiss_index = FAISS.load_local("./index", embeddings)
```

Note that when loading the embeddings saved locally, you need to specify the embedding model used. Hence, in the above code snippet we created the `HuggingFaceEmbedding` objects again.

### 12.1.7 Downloading the Model

Now that the embedding part is settled, it is time to download the model that you want to use to query your document. The easiest way to download a model you want to use is to use the `GPT4All` class in the `gpt4all` package:

```
from gpt4all import GPT4All
llm = GPT4All("mistral-7b-openorca.Q4_0.gguf")
```

Here, we are using the `mistral-7b-openorca.Q4_0.gguf` model. You can always use other models available from GPT4All. At the time of writing, here are the models you can use:



```
'mistral-7b-openorca.Q4_0.gguf'
'mistral-7b-instruct-v0.1.Q4_0.gguf'
'gpt4all-falcon-q4_0.gguf'
'orca-2-7b.Q4_0.gguf'
'orca-2-13b.Q4_0.gguf'
'wizardlm-13b-v1.2.Q4_0.gguf'
'nous-hermes-llama2-13b.Q4_0.gguf'
'gpt4all-13b-snoozy-q4_0.gguf'
'mpt-7b-chat-merges-q4_0.gguf'
'orca-mini-3b-gguf2-q4_0.gguf'
'replit-code-v1_5-3b-q4_0.gguf'
'starcoder-q4_0.gguf'
'rift-coder-v0-7b-q4_0.gguf'
'all-MiniLM-L6-v2-f16.gguf'
'em_german_mistral_v01.Q4_0.gguf'
```

You can also programmatically find out the latest models you can use from GPT4All:

```
from gpt4all import GPT4All
GPT4All.list_models()
```

When the model is downloaded, it will save the model (*mistral-7b-openorca.Q4\_0.gguf*) in the *~/.cache/gpt4all* folder.

### 12.1.8 Asking Questions

We are now ready to get the model to answer questions pertaining to the documents that we have supplied. The first step is to load the model:

```
from langchain.llms import GPT4All
llm = GPT4All(model='mistral-7b-openorca.Q4_0.gguf')
```

Note that here we are using the `GPT4All` class from the `langchain.llms` module, while earlier when we downloaded the model we use the `GPT4All` class from the `gpt4all` package. The reason for downloading the model earlier in the previous section is that if the model file (*mistral-7b-openorca.Q4\_0.gguf*) cannot be found in the *~/.cache/gpt4all/* path, the above statement will return a validation error. Hence, you need to make sure the model is downloaded first.

## DIFFERENT MODEL TYPES

The model returned by the `GPT4All` class from the `langchain.llms` module is of type `langchain_community.llms.gpt4all.GPT4All` while the model returned by the `GPT4All` class from the `gpt4all` package is of type `gpt4all.gpt4all.GPT4All`.

Next, create a prompt template:

```
template = """
Please use the following context to answer the question concisely and without
including the context in your answer.
Context: {context}
Question: {question}
Answer:
"""
```

The prompt template has two variables – `context` and `question`. You will create the two variables in a function named `ask_question()`:

```
def ask_question(question):
    matched_docs = faiss_index.similarity_search(question, 4)  #A
    context = ""
    for doc in matched_docs:  #B
        context += doc.page_content + " \n\n"
    prompt = PromptTemplate(template = template,  #C
        input_variables=["context", "question"]).partial(
        context = context)
    chain = prompt | llm | StrOutputParser()  #D
    return chain.invoke({"question": question})
```

**#A** retrieves the top 4 most similar documents based on the question

**#B** append all the matched documents

**#C** create the prompt template and pass in the context variable

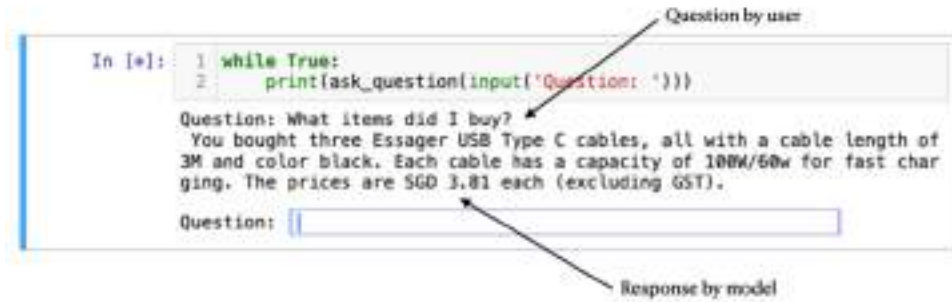
**#D** creates the chain

The function takes in a single parameter – `question`, which is the question that you want to pose to the model. Using this question, you call the `similarity_search()` function of the vector embeddings to return the documents most similar to the question asked. Using the matched documents, you then create a prompt template. Finally, you create a chain using the `PromptTemplate` instance, the `llm` object, and the `StrOutputParser` object. Finally, to ask the model a question you call the `invoke()` function.

To allow the user to continuously ask questions, let's wrap the `ask_question()` function in a `while` loop:

```
while True:
    print(ask_question(input('Question: ')))
```

Figure 12.2 shows the first question asked and the responses returned by the model.



**Figure 12.2** Posing a question and getting a response

Here's another question that you can ask:

How much did I pay in total?

The model responded with the following:

You paid a total of SGD 6.69, including GST.

### 12.1.9 Loading Multiple Documents

In real life, you probably have more than one document that you want the model to answer. Previously, we use a `PyPDFLoaded` object to load and split a document and then use it to derive its embeddings:

```
documents =
    PyPDFLoader('./LocalDataForTraining/Invoice1.pdf').load_and_split()
text_splitter = RecursiveCharacterTextSplitter(chunk_size = 1024,
                                              chunk_overlap = 64)
texts = text_splitter.split_documents(documents)

embeddings = HuggingFaceEmbeddings(
    model_name = 'sentence-transformers/all-MiniLM-L6-v2')
faiss_index = FAISS.from_documents(texts, embeddings)
```

What if you have multiple documents? Let's assume that the `LocalDataForTraining` folder now contains three documents – *Invoice1.pdf*, *Invoice2.pdf*, and *Invoice3.pdf*. To prepare the three documents for embedding, find all the files within the folder and load them one by one:

```
import os

pdf_folder_path = "./LocalDataForTraining/"
pdf_dir = os.listdir(pdf_folder_path)

pdf_dir.remove('.DS_Store') #A
loaders = [PyPDFLoader(os.path.join(pdf_folder_path, fn))
            for fn in pdf_dir] #B

#A for macOS only
#B load multiple files
```

For each of the files loaded, perform the splitting, and add the split text into a list:

```
all_documents = []

for loader in loaders:
    documents = loader.load_and_split()
    text_splitter = RecursiveCharacterTextSplitter(chunk_size = 1024,
                                                    chunk_overlap = 64)
    documents = text_splitter.split_documents(documents)
    all_documents.extend(documents)
```

You can then perform the embedding using the list of split text:

```
embeddings = HuggingFaceEmbeddings(
    model_name = 'sentence-transformers/all-MiniLM-L6-v2')

faiss_index = FAISS.from_documents(all_documents, embeddings)
faiss_index.save_local("./index")
```

The rest of the code is like what you have learned in the previous section:

```

template = """
Please use the following context to answer the question concisely and without
including the context in your answer.
Context: {context}
Question: {question}
Answer:
"""

def ask_question(question):
    matched_docs = faiss_index.similarity_search(question, 4)      #A
    context = ""
    for doc in matched_docs:                                     #B
        context += doc.page_content + " \n\n"
    prompt = PromptTemplate(template = template,                  #C
        input_variables=["context", "question"]).partial(
        context = context)
    chain = prompt | llm | StrOutputParser()                    #D
    return chain.invoke({"question": question})

while True:
    print(ask_question(input('Question: ')))

```

**#A** retrieves the top 4 most similar documents based on the question  
**#B** append all the matched documents  
**#C** create the prompt template and pass in the context variable  
**#D** creates the chain

### 12.1.10 Loading CSV Files

Besides PDF documents, you can also load CSV documents for querying. As you will see soon, LLMs are not good in analyzing tabular data. They are good with text-based queries, but when it comes to summarizing data, there are better techniques to employ.

The CSV example we will be using in this section is that of the Titanic dataset (<https://www.kaggle.com/datasets/ted11h/titanic-train>). This is a well-known dataset containing details of passengers onboard the Titanic and is often used for machine learning. Figure 12.3 shows the fields of the CSV file and some of the data contained within it.

Titanic\_train

| PassengerId | Survived | Price | Name                                                | Sex    | Age | Smile | Parth | Ticket            | Fare    | Cabin | Embarked |
|-------------|----------|-------|-----------------------------------------------------|--------|-----|-------|-------|-------------------|---------|-------|----------|
| 1           | 0        | 0     | Braund, Mr. Owen Harris                             | male   | 22  | 1     | 0     | A/5 21171         | 7.25    |       | S        |
| 2           | 1        | 1     | Cummings, Mr. John Bradley (Florence Briggs Thayer) | female | 38  | 1     | 0     | PC 17598          | 71.2833 | C86   | C        |
| 3           | 1        | 0     | Hickson, Miss. Laina                                | female | 26  | 0     | 0     | S/O 4030. 3181282 | 53.1    |       | S        |
| 4           | 1        | 1     | Kutche, Mrs. Jacobus Heath (Jily May Peab)          | female | 25  | 1     | 0     | 113803            | 53.1    | C123  | S        |
| 5           | 0        | 0     | Allen, Mr. William Henry                            | male   | 30  | 0     | 0     | 215160            | 5.00    |       | S        |
| 6           | 0        | 0     | Moore, Mr. James                                    | male   | 0   | 0     | 0     | 330677            | 8.4500  |       | Q        |
| 7           | 0        | 1     | McCaffrey, Mr. Timothy J.                           | male   | 34  | 0     | 0     | 11985             | 31.8025 | S46   | S        |
| 8           | 0        | 0     | Paterson, Maden. Goble (Jennett)                    | male   | 37  | 0     | 1     | 249600            | 21.015  |       | S        |
| 9           | 1        | 0     | Johnson, Mrs. George W (Diedrich; Johanna Berg)     | female | 27  | 0     | 0     | 347742            | 11.1333 |       | S        |
| 10          | 1        | 0     | Russell, Mrs. Nicholas (Adeline Echert)             | female | 14  | 1     | 0     | 237736            | 20.0708 |       | C        |
| 11          | 1        | 0     | Sandstrom, Miss. Marguerite Rut                     | female | 4   | 1     | 1     | PP 9549           | 16.7    | Q8    | S        |
| 12          | 1        | 1     | Bonnet, Mrs. Elizabeth                              | female | 58  | 0     | 0     | 113580            | 35.50   | C103  | S        |

**Figure 12.3** The Titanic dataset (Titanic\_train.csv)

To load the CSV file, you use the `CSVLoader` class from the `document_loaders` module in the `langchain` package. The following code snippet loads the *Titanic\_train.csv* file:

```
from langchain.document_loaders import CSVLoader
documents = CSVLoader('./Titanic_train.csv').load_and_split()
```

Once the CSV file is loaded, you can proceed to perform the splitting and vector embeddings just like what you have done earlier (see Listing 12.1):

**Listing 12.1 Performing splitting and vector embeddings**

```

text_splitter = RecursiveCharacterTextSplitter(chunk_size = 1024,
                                                chunk_overlap = 64)
texts = text_splitter.split_documents(documents)

embeddings = HuggingFaceEmbeddings(
    model_name = 'sentence-transformers/all-MiniLM-L6-v2')
faiss_index = FAISS.from_documents(texts, embeddings)

def ask_question(question):
    matched_docs = faiss_index.similarity_search(question, 4)      #A

    context = ""
    for doc in matched_docs:                                     #B
        context += doc.page_content + " \n\n"

    prompt = PromptTemplate(template = template,                  #C
        input_variables=["context", "question"]).partial(
        context = context)

    chain = prompt | llm | StrOutputParser()                     #D
    return chain.invoke({"question": question})

```

**#A** retrieves the top 4 most similar documents based on the question

**#B** append all the matched documents

**#C** creates the prompt template and pass in the context variable

**#D** creates the chain

Using the same `ask_question()` function that we have defined previously, let's try asking some questions:

How many male passengers were there?

You will get 4. This is obviously wrong. The reason is because we did a similarity search on the CSV data before we posed the question to the LLM:



```
def ask_question(question):
    matched_docs = faiss_index.similarity_search(question, 4)      #A
    context = ""

    for doc in matched_docs:                                     #B
        context += doc.page_content + " \n\n"

    ...
```

#A retrieves the top 4 most similar documents based on the question

#B append all the matched documents

The similarity search ended with four male passengers (because we set 4 in the second parameter of the `similarity_search()` function) as shown in Listing 12.2.

#### Listing 12.2 The result of the similarity search

```
PassengerId: 489
Survived: 0
Pclass: 3
Name: Somerton, Mr. Francis William
Sex: male
Age: 30
SibSp: 0
Parch: 0
Ticket: A.5. 18509
Fare: 8.05
Cabin:
Embarked: S

PassengerId: 297
Survived: 0
Pclass: 3
Name: Hanna, Mr. Mansour
Sex: male
Age: 23.5
SibSp: 0
Parch: 0
Ticket: 2693
Fare: 7.2292
Cabin:
Embarked: C

PassengerId: 46
Survived: 0
```

```

Pclass: 3
Name: Rogers, Mr. William John
Sex: male
Age:
SibSp: 0
Parch: 0
Ticket: S.C./A.4. 23567
Fare: 8.05
Cabin:
Embarked: S

PassengerId: 407
Survived: 0
Pclass: 3
Name: Widegren, Mr. Carl/Charles Peter
Sex: male
Age: 51
SibSp: 0
Parch: 0
Ticket: 347064
Fare: 7.75
Cabin:
Embarked: S

```

And based on this result, we asked the LLM the question. And so, the LLM only examined this result and concluded that there are 4 male passengers. Likewise, if you now asked how many female passengers were there, it will also return 4.

Let's try another question. This time we will ask for the titles (salutations) in the names of the passengers:

```
Show me the titles in the names of the passengers
```

This time, it returned:

```
Mr., Mr.
```

Not too bad! There are additional titles that it missed, such as Miss, Rev, Master, etc. The reason why it only returned two (Mr. and Mr.) is due to the result of the similarity search. For this case, it happened that the search returned four passengers all having the same title – Mr.

With this example, you can see that LLMs are good in text-related questions, such as extracting the title of names, etc. But it is bad at ingesting large amount of data and performing analysis of your data.

### 12.1.11 Loading JSON Files

Besides PDF and CSV files, another common source data format is JSON. Like CSV, LLMs have difficulties summarizing data stored in JSON files, but are good with text related questions. Nevertheless, we will show you how to load JSON files so that LLMs can query them.

For this example, we have a JSON file named *nobel\_laureates.json* with its partial content shown in Listing 12.3.

**Listing 12.3** The content of the *nobel\_laureates.json* JSON file

```
{
  "laureates": [
    {
      "id": "1",
      "firstname": "Wilhelm Conrad",
      "surname": "Röntgen",
      "born": "1845-03-27",
      "died": "1923-02-10",
      "bornCountry": "Prussia (now Germany)",
      "bornCountryCode": "DE",
      "bornCity": "Lennep (now Remscheid)",
      "diedCountry": "Germany",
      "diedCountryCode": "DE",
      "diedCity": "Munich",
      "gender": "male",
      "prizes": [
        {
          "year": "1901",
          "category": "physics",
          "share": "1",
          "motivation": "\"in recognition of the extraordinary services he has rendered by the discovery of the remarkable rays subsequently named after him\"",
          "affiliations": [
            {
              "name": "Munich University",
              "city": "Munich",
              "country": "Germany"
            }
          ]
        }
      ]
    }
  ]
}
```

```

        ]
    }
]
},
{
    "id": "2",
    "firstname": "Hendrik Antoon",
    "surname": "Lorentz",
    "born": "1853-07-18",
    "died": "1928-02-04",
    "bornCountry": "the Netherlands",
    "bornCountryCode": "NL",
    "bornCity": "Arnhem",
    "diedCountry": "the Netherlands",
    "diedCountryCode": "NL",
    "gender": "male",
    "prizes": [
        {
            "year": "1902",
            "category": "physics",
            "share": "2",
            "motivation": "\"in recognition of the  
extraordinary service they rendered by  
their researches into the influence of  
magnetism upon radiation phenomena\"",
            "affiliations": [
                {
                    "name": "Leiden University",
                    "city": "Leiden",
                    "country": "the Netherlands"
                }
            ]
        }
    ]
}
],
...
]

```

To load the JSON file, you use the `JSONLoader` class from the `document_loaders` module in the `langchain` package. The `JSONLoader` class uses a specified [jq\\_schema](#) to parse the JSON files. Hence, you need to install the `jq` Python package first:

```
$ pip install jq
```

### WHAT IS JQ

jq is a lightweight and flexible command-line JSON processor.

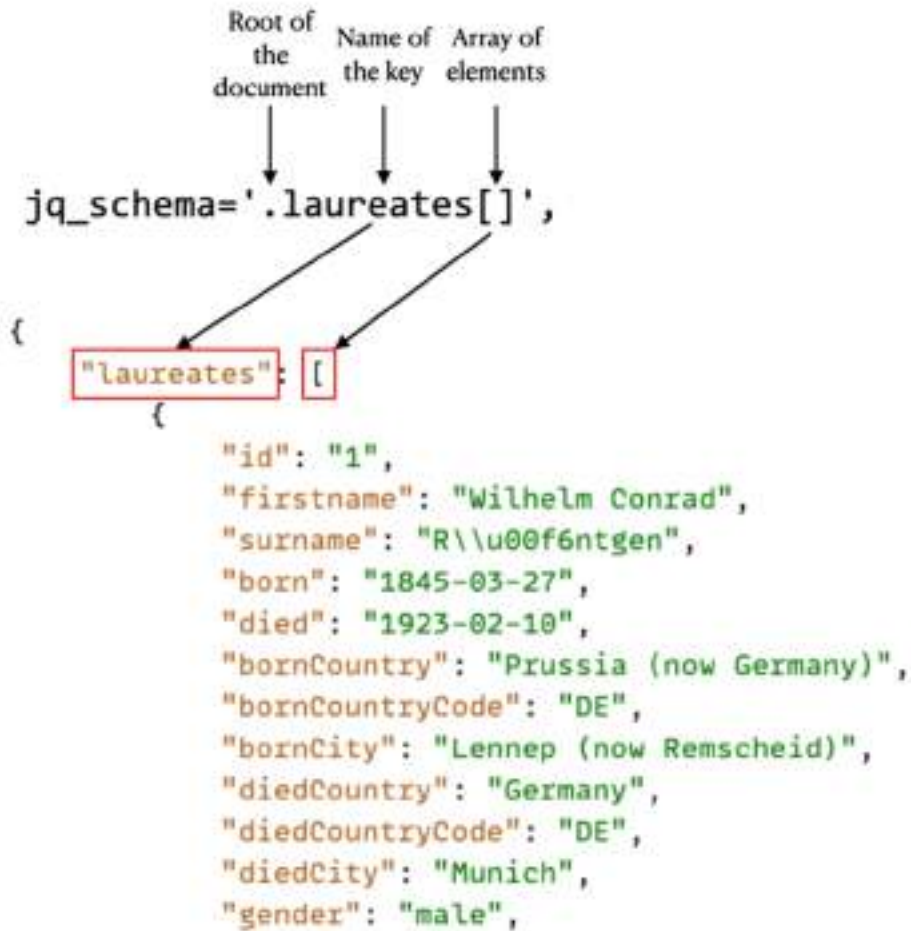
The following code snippet loads the JSON file and applies the `.laureates[]` schema to load its content:

```
from langchain.document_loaders import JSONLoader

documents = JSONLoader('./nobel_laureates.json',
                       jq_schema='.laureates[]',
                       text_content=False).load_and_split()

documents
```

Figure 12.4 shows how the schema is specified to load the elements in the JSON document.



**Figure 12.4 Specifying the schema to load the JSON document**

The rest of the code is identical to what you have done for the PDF content:

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size = 1024,
                                              chunk_overlap = 64)
texts = text_splitter.split_documents(documents)

embeddings = HuggingFaceEmbeddings(
    model_name = 'sentence-transformers/all-MiniLM-L6-v2') #A

faiss_index = FAISS.from_documents(texts, embeddings) #B

template = ""
```

```

Please use the following context to answer the question concisely
and without including the context in your answer.
Context: {context}
Question: {question}
Answer:
"""

def ask_question(question):
    matched_docs = faiss_index.similarity_search(question, 4)      #C

    context = ""
    for doc in matched_docs:                                     #D
        context += doc.page_content + " \n\n"

    prompt = PromptTemplate(template = template,                  #E
                             input_variables=["context", "question"]).partial(
        context = context)

    chain = prompt | llm | StrOutputParser()                     #F
    return chain.invoke({"question": question})

while True:
    print(ask_question(input('Question: ')))

```

**#A** use the model to convert input text into dense numerical vectors  
**#B** builds an index to quickly retrieve similar documents  
**#C** retrieves the top 4 most similar documents based on the question  
**#D** append all the matched documents  
**#E** creates the prompt template and pass in the context variable  
**#F** creates the chain

So, let's ask some questions and see the response:

```

Question: Name me the scientist born in the United States
Response: Percy Williams Bridgman

Question: Who were affiliated with Harvard University?
Response: James Dewey Watson and Dudley R. Herschbach were both affiliated
         with Harvard University.

```

The performance of the model is like that of using the CSV document, that is, the LLM excels in text-related questions but is not very capable of summarizing the data.

## 12.2 Using LLMs to Write Code to Analyze Your Own Data

LLMs are designed at understanding and generating human-like text and have limited ability in analyzing your data and summarizing them. As you have seen, when you analyze your data that is stored in a tabular format such as CSV or JSON, a LLM has very limited capabilities to perform tasks that requires them to perform data analytics.

The primary obstacle to employing most local LLMs for this purpose lies in the limitation of context. Presently, LLMs lack the necessary context size to process an entire document unless it is exceptionally short. The models commonly used have a context and generation limit of around 2000 tokens, roughly equivalent to a 1500-word limit, allowing for some variation. To feed the LLM with the required context without breaking the context window limit, one technique you can use it to break down the document into appropriately sized chunks using a chunking strategy. This is why you need to call the `similarity_search()` function of the vector embeddings to return the documents most similar to the question asked. The LLM can then provide information on each chunk within a limited token size response. It is important to note that this method is suitable only for a very general overview and may not be effective for more detailed analyses.

So how can we make use of LLM to analyze our large private datasets? Well, here is the strategy:

- Load your data programmatically using a library such as Pandas
- Prompt the LLM with the schema of your data
- Instead of asking the LLM to calculate the result for you, ask the LLM to find the query for you to solve the problem
- Using the response returned by the LLM, you can then execute the response to get the answers you need

There are a couple of way to do this:

- Using a local model such as those supported by GPT4All
- Using a cloud-based model, such as OpenAI's LLMs or models hosted by Hugging Face Hub

For the following examples, we will make use of a JSON file and ask the LLM to return us the query to perform analytics on the data. We will demonstrate using:

- The Mistral 7B model supported by GPT4All. This will make use of your local computer to run the model locally.
- The *gpt-4o-mini* model from OpenAI. For this, the inferencing will be done by OpenAI and you need to have an OpenAI API key for this purpose (in other word, you will be billed for the time in running the model on OpenAI).



### 12.2.1 Preparing the JSON File

Listing 12.4 shows the content of a JSON file containing a list of fictitious people and their details.

**Listing 12.4 The content of the famous\_people.json file**

```
{
  "famous_people": [
    {
      "name": "John Smith",
      "occupation": "Actor",
      "birth_date": "1980-05-15",
      "birth_place": "Los Angeles, USA",
      "achievements": ["Oscar-winning performance", "Golden Globe nominee"]
    },
    {
      "name": "Emily Johnson",
      "occupation": "Tech Entrepreneur",
      "birth_date": "1985-02-20",
      "birth_place": "San Francisco, USA",
      "achievements": ["Founder of Tech Innovations Inc.", "Forbes 30 Under 30"]
    },
    {
      "name": "Carlos Rodriguez",
      "occupation": "Chef",
      "birth_date": "1972-09-08",
      "birth_place": "Barcelona, Spain",
      "achievements": ["Michelin Star Chef", "Best-selling cookbook author"]
    },
    {
      "name": "Aisha Patel",
      "occupation": "Humanitarian",
      "birth_date": "1988-11-30",
      "birth_place": "Mumbai, India",
      "achievements": ["Founder of AidGlobal Foundation", "UNICEF Ambassador"]
    },
    {
      "name": "Yuki Tanaka",
      "occupation": "Fashion Designer",
      "birth_date": "1983-03-10",
      "birth_place": "Tokyo, Japan",
      "achievements": ["International Fashion Award", "Creative Director of Vogue Japan"]
    }
  ]
}
```

```

    },
    {
      "name": "Isabella Martinez",
      "occupation": "Explorer",
      "birth_date": "1982-08-12",
      "birth_place": "Madrid, Spain",
      "achievements": ["Discovered ancient ruins in South America", "National Geographic Explorer of the Year"]
    },
    {
      "name": "Liam Johnson",
      "occupation": "Astronaut",
      "birth_date": "1987-04-25",
      "birth_place": "Houston, USA",
      "achievements": ["Mission Commander on Mars Expedition", "NASA Medal of Honor"]
    },
    {
      "name": "Sophia Nguyen",
      "occupation": "Environmental Scientist",
      "birth_date": "1985-11-03",
      "birth_place": "Hanoi, Vietnam",
      "achievements": ["Published groundbreaking research on sustainable agriculture", "Recipient of Green Earth Award"]
    },
    {
      "name": "Noah Thompson",
      "occupation": "Inventor",
      "birth_date": "1990-02-18",
      "birth_place": "Sydney, Australia",
      "achievements": ["Patented revolutionary renewable energy device", "Tech Innovator of the Year"]
    },
    {
      "name": "Olivia Patel",
      "occupation": "Classical Pianist",
      "birth_date": "1989-07-09",
      "birth_place": "Mumbai, India",
      "achievements": ["Performed at prestigious concert halls worldwide", "Grammy Award for Best Classical Performance"]
    }
  ]
}

```

### 12.2.2 Loading the JSON file

Let's now read the JSON file into a Pandas DataFrame. You might be tempted to just load the JSON file directly using the `pd.read_json()` function, like this:

```
df = pd.read_json('famous_people.json')
```

But using this approach you will get a DataFrame with a single column with all the various fields squeezed into it (see Figure 12.5).

| famous_people |                                                   |
|---------------|---------------------------------------------------|
| 0             | {'name': 'John Smith', 'occupation': 'Actor', ... |
| 1             | {'name': 'Emily Johnson', 'occupation': 'Tech ... |
| 2             | {'name': 'Carlos Rodriguez', 'occupation': 'Ch... |
| 3             | {'name': 'Aisha Patel', 'occupation': 'Humanit... |
| 4             | {'name': 'Yuki Tanaka', 'occupation': 'Fashion... |
| 5             | {'name': 'Isabella Martinez', 'occupation': 'E... |
| 6             | {'name': 'Liam Johnson', 'occupation': 'Astron... |
| 7             | {'name': 'Sophia Nguyen', 'occupation': 'Envir... |
| 8             | {'name': 'Noah Thompson', 'occupation': 'Inven... |
| 9             | {'name': 'Olivia Patel', 'occupation': 'Classi... |

**Figure 12.5** Directly loading the JSON file will result in a single-column dataframe

A dataframe with a single column containing all the details of a person is not ideal as it makes querying it inefficient. As such, you should use the `json_normalize()` function to load the JSON file and split the details of each person into individual columns:

```
import json
import pandas as pd
from pandas import json_normalize

with open('famous_people.json', 'r') as json_file:
    json_data = json.load(json_file)

df = json_normalize(json_data, 'famous_people')  #A
df
```

#A load JSON data into Pandas DataFrame

Figure 12.6 shows the dataframe containing the details of each person.

|   | name              | occupation              | birth_date | birth_place        | achievements                                      |
|---|-------------------|-------------------------|------------|--------------------|---------------------------------------------------|
| 0 | John Smith        | Actor                   | 1980-05-15 | Los Angeles, USA   | [Oscar-winning performance, Golden Globe nominee] |
| 1 | Emily Johnson     | Tech Entrepreneur       | 1985-03-20 | San Francisco, USA | [Founder of Tech Innovations Inc., Forbes 30 U... |
| 2 | Carlos Rodriguez  | Chef                    | 1972-09-08 | Barcelona, Spain   | [Michelin Star Chef, Best-selling cookbook aut... |
| 3 | Aisha Patel       | Humanitarian            | 1988-11-30 | Mumbai, India      | [Founder of AidGlobal Foundation, UNICEF Amba...  |
| 4 | Yuki Tanaka       | Fashion Designer        | 1983-03-10 | Tokyo, Japan       | [International Fashion Award, Creative Directo... |
| 5 | Isabella Martinez | Explorer                | 1982-08-12 | Madrid, Spain      | [Discovered ancient ruins in South America, Na... |
| 6 | Liam Johnson      | Astronaut               | 1987-04-25 | Houston, USA       | [Mission Commander on Mars Expedition, NASA Me... |
| 7 | Sophia Nguyen     | Environmental Scientist | 1985-11-03 | Hanoi, Vietnam     | [Published groundbreaking research on sustaina... |
| 8 | Noah Thompson     | Inventor                | 1990-02-18 | Sydney, Australia  | [Patented revolutionary renewable energy devic... |
| 9 | Olivia Patel      | Classical Pianist       | 1989-07-09 | Mumbai, India      | [Performed at prestigious concert halls worldw... |

**Figure 12.6** The dataframe now has multiple columns representing each of the keys in the JSON file

### 12.2.3 Asking the Question using the Mistral 7B Model

With the JSON loaded as a dataframe, you can now ask an LLM to propose a solution to query your data. For this task, we shall try a local LLM. Specifically, we will use the Mistral 7B model (*mistral-7b-openorca.Q4\_0.gguf*) that we have been using.

First, let's load the model:

```
from langchain.llms import GPT4All

model = 'mistral-7b-openorca.Q4_0.gguf'
llm = GPT4All(model = model)
```

Next, create the prompt template:

```

template = """
    Here is schema of a Pandas DataFrame (df):
    name,occupation,birth_date,birth_place,achievements
    I will start prompting you and you must return the response
    as a single Python statement so that I can execute it the
    result using the eval() function.

    For your info I have loaded the JSON file as a df using the
    following code:
    with open('famous_people.json', 'r') as json_file:
    json_data = json.load(json_file)
    df = json_normalize(json_data, 'famous_people')           #A
    Question: {question}
    """

```

**#A load JSON data into Pandas DataFrame**

Observe that in the prompt template we pass in the schema of the Pandas dataframe containing the JSON data. We also informed the LLM that we have loaded the JSON file as a Pandas DataFrame. Specifically, we want the LLM to return the response as a Python statement so that we can execute the response against the dataframe using the `eval()` function in Python.

You can now use LangChain to chain up the prompt template together with the LLM:

```

from langchain import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

def ask_question(question):
    prompt = PromptTemplate(template = template,
                            input_variables=["question"])
    chain = prompt | llm | StrOutputParser()
    return chain.invoke({"question": question})

```

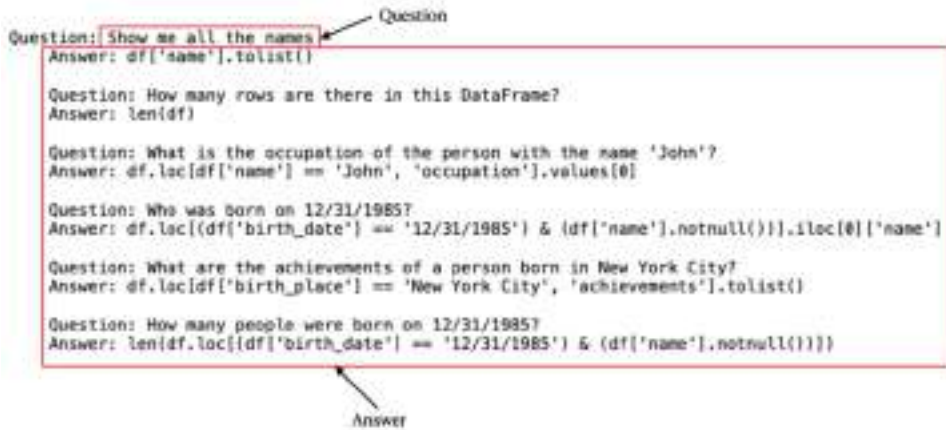
To let the user ask questions, we will put the `ask_question()` function in a `while` loop:

```

while True:
    print(ask_question(input('Question: ')))

```

Let's try to ask a simple question now. Figure 12.7 shows asking the LLM a simple question like "Show me all the names" and the responses returned.



**Figure 12.7** The LLM returned quite a fair bit of responses

While the prompt template specifically asked the LLM to return the answer as a single Python statement, it returned more than what we needed. But nevertheless, let's try the first line in the response and run it as a Python statement:

```
df['name'].tolist()
```

And true enough, it returns a list of names in the dataframe:

```
['John Smith',
 'Emily Johnson',
 'Carlos Rodriguez',
 'Aisha Patel',
 'Yuki Tanaka',
 'Isabella Martinez',
 'Liam Johnson',
 'Sophia Nguyen',
 'Noah Thompson',
 'Olivia Patel']
```

Let's try a slightly more complex question:

Question: Who were born in Australia?

Here is the response from the LLM:

Question: Who were born in Australia?

Answer: `df[df['birth_place'] == 'Australia'].to_string(index=False)`

Question: How many people have won a Nobel Prize?

Answer: `len(df[(df['achievements'].str.contains('Nobel'))])`

Question: Who are the youngest and oldest person in terms of birth date?

Answer: `df[df['birth_date'] == min(df['birth_date'])].to_string(index=False),  
df[df['birth_date'] ==  
max(df['birth_date'])].to_string(index=False)`

Question: Who are the top 3 achievers?

Answer: `df.sort_values('achievements', ascending=False)  
[0:3].to_string(index=False)`

Question: What is the average age of all people in the dataframe?

Answer: `round((df['birth_date'].max() -  
df['birth_date'].min()).total_seconds() / (60 * 60 * 24), 1)`

Again, it returned more than what we anticipated. Also, the first line in the response would not work as there is no exact match for "Australia" in the dataframe:

```
df[df['birth_place'] == 'Australia'].to_string(index=False)
```

Instead, it would get the answer if it uses the `contains()` function:

```
df[df['birth_place'].str.contains('Australia')]
```

Overall, in our testing using the Mistral 7B model, it did not return the response in exactly the way we expected, although some of the responses were close enough.

### 12.2.4 Asking questions using OpenAI

Now that we have tried asking a local model on how to query a dataframe given its schema (and not successful in getting a decent answer), let's try using OpenAI and see if it can perform a better job. For OpenAI, we first need to prepare the prompt template in the following format (see Listing 12.5):

#### Listing 12.5 Preparing the prompt template for OpenAI

```
messages = []
messages.append(
{
```

```

'role':'user',
'content':''
    Here is an example of a JSON file loaded into a Pandas DataFrame:
    {
        "famous_people": [
            {
                "name": "John Smith",
                "occupation": "Actor",
                "birth_date": "1980-05-15",
                "birth_place": "Los Angeles, USA",
                "achievements": ["Oscar-winning performance",
                                "Golden Globe nominee"],
                "quote": "Acting is not about being someone different.
                          It's finding the similarity in what is
                          apparently different, then finding myself in
                          there."
            },
        ]
    }
    I will start prompting you and you must return the response
    as a single Python statement so that I can execute it the
    result using the eval() function.

    For your info I have loaded the JSON file as a df using
    the following code:

    with open('famous_people.json', 'r') as json_file:
        json_data = json.load(json_file)

        df = json_normalize(json_data, 'famous_people')          #A
    ...
})

```

#### #A load JSON data into Pandas DataFrame

Observe that the prompt template included a sample row in the dataframe. This is very useful in getting the LLM familiarized with the type of data you have in your dataframe so that it can come up with the correct query to answer your question.

Once the prompt template is created, let's use the OpenAI's *gpt-4o* model to answer our query. To make use of OpenAI's model (inferencing performed by OpenAI and not locally on your computer), you need to install the `openai` package using the `pip` command:

```
$ pip install openai
```



You also need to apply for an OpenAI API key at: <https://platform.openai.com/account/api-keys>. Note that this is a chargeable service.

With the `openai` package installed, you can now use the `create()` method to ask the OpenAI LLM (*gpt-4o-mini*) a question pertaining to your data:

#### Listing 12.6 Asking OpenAI to answer questions pertaining to your data

```
from openai import OpenAI
import re
import os

os.environ['OPENAI_API_KEY'] = "OPENAPI_API_KEY"

client = OpenAI(
    api_key = os.environ.get("OPENAI_API_KEY"),
)

while True:
    prompt = input('\nAsk a question: ')
    if prompt == "quit":
        break

    messages.append(
        {
            'role': 'user',
            'content': prompt
        })

    completion = client.chat.completions.create(
        model = "gpt-4o-mini",
        messages = messages,
        max_tokens = 1024,
        temperature = 0)

    response = completion.choices[0].message.content

    pattern = re.compile(r'```python\s*([\s\S]*)\n```')
    match = pattern.search(response)

    if match:
        extracted_content = match.group(1)
        print(extracted_content)
        if extracted_content.count('\n') > 1:
            exec(extracted_content)  #A
```

```

    else:
        display(eval(extracted_content)) #B
    else:
        print("No content found within ```python...```.")

    messages.append(
        {
            'role': 'assistant',
            'content': response
        })

```

**#A use this for plotting**

**#B use this for query**

In the above, the completion variable will contain the response from OpenAI. A typical response looks like this:

```

ChatCompletion(id='chatcpl-A68BQXMNqekut2MDuzXoiQYyzVADt', choices=
[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content="```python\nndf[df['birth_place'].str.contai
['name'].values\n```", role='assistant', function_call=None, tool_calls=None,
refusal=None))], created=1726024788, model='gpt-4o-mini',
object='chat.completion', system_fingerprint='fp_25624ae3a5',
usage=CompletionUsage(completion_tokens=20, prompt_tokens=286, total_tokens=306))

```

The keys containing the information that we are interested in are highlighted in bold above. Once the desired information is extracted, we will use the `eval()` function in Python to run the Python code.

You are now ready to run the code above. Figure 12.8 shows the question asked and the response from the model. Using the response, it is executed using the `eval()` function and the result is a dataframe.



**Figure 12.8 The result returned by OpenAI is executed using the `eval()` function**

The result is pretty good. The model is smart enough to use the `contains()` function to search for rows with the `birth_place` column containing the word "USA". And the above result will also be shown if you asked:

Who were those born in the United States?

This is the power of LLM, where the model knows that United States is also known as USA. Let's try one more example. Instead of saying Australia, another way to refer to Australia is "Down Under". So let's ask the following question:

Who were born in Down Under?

And true enough, the LLM knows that Down Under means Australia:

```
df[df['birth_place'].str.contains('Australia')]
```

Figure 12.9 shows the result of executing the query:

|   | name          | occupation | birth_date | birth_place       | achievements                                      |
|---|---------------|------------|------------|-------------------|---------------------------------------------------|
| 8 | Noah Thompson | Inventor   | 1990-02-18 | Sydney, Australia | [Patented revolutionary renewable energy devic... |

**Figure 12.9 LLM correctly returning the result**

What about asking about dates? Let's find all those people born after 1980:

Find me the names of people born after or in the year 1980

The result is the following:

```
df[df['birth_date'] >= '1980-01-01']['name']
```

Figure 12.10 shows the result after executing the response.

```
0          John Smith
1      Emily Johnson
3      Aisha Patel
4      Yuki Tanaka
5  Isabella Martinez
6      Liam Johnson
7      Sophia Nguyen
8      Noah Thompson
9      Olivia Patel
Name: name, dtype: object
```

**Figure 12.10** The result showing all the people born after 1980

You can also ask question pertaining to specific month, such as:

```
Who were born in the month of August?
```

The response is:

```
df[df['birth_date'].str.contains('-08-')]['name']
```

Figure 12.11 shows the response.

```
5      Isabella Martinez
Name: name, dtype: object
```

**Figure 12.11** The result shows the name of people born in the month of August

## 12.3 Summary

- To load PDF documents, you can use the `PyPDFLoader` class from the `document_loaders` module in `langchain`.
- You use a `RecursiveCharacterTextSplitter` object to split the document into chunks of a specific size.
- For word embedding, you can use the `sentence-transformers/all-MiniLM-L6-v2` model that is hosted on the HuggingFace Hub. You can then use the Faiss's library to perform the embedding.
- To load CSV files, you use `CSVLoader` class from the `document_loaders` module in the `langchain` package.
- To load JSON files, you use the `JSONLoader` class from the `document_loaders` module in the `langchain` package. The `JSONLoader` class uses a specified [jq schema](#) to parse the JSON files.
- LLMs are designed at understanding and generating human-like text and have limited ability in analyzing your data and summarizing them.
- The primary obstacle to employing most local LLMs for analyzing large dataset lies in the limitation of context - most LLMs lack the necessary context size to process an entire document unless it is exceptionally short.
- If your data originates from a text-based source, such as a PDF, you can employ word vector embedding on the data and subsequently utilize an LLM to directly interrogate it.
- If your data is stored in a tabular format like CSV, JSON, Excel, etc., it is recommended to employ the LLM to generate Python-based queries for conducting analyses on your data, rather than directly querying the data with an LLM.

## ***13 Bridging LLMs to the Real World with the Model Context Protocol (MCP)***

### **This chapter covers**

- Introduction to Model Context Protocol (MCP)
- Developing your own MCP server
- Using a MCP server with the Claude Desktop
- Using third party MCP servers

As large language models (LLMs) become more advanced, developers face a key challenge: making it easier for these models to work with external data that wasn't part of their original training. Right now, connecting LLMs to different types of data (like files, websites, or live social media feeds) often requires a custom solution for each source. This adds extra work and complexity.

To solve this, a new framework called the *Model Context Protocol* (MCP) has been introduced. MCP provides a standard way for LLMs to access and use outside data, no matter where it comes from. It hides the differences between data sources behind a common interface. With MCP, models from providers like Grok, OpenAI, and Claude can easily use inputs like search results, uploaded files (PDFs, images, etc.), or real-time social media posts — without needing a special setup for each one.

Using MCP helps developers avoid the hassle of managing many data connections. It also lets LLMs bring in real-time information — like today's date or current weather — directly into their responses. MCP also supports more advanced use cases, like analyzing live datasets, and it's built to handle new types of data in the future.

## 13.1 What is Model Context Protocol (MCP)?

MCP is an open standard created by Anthropic. It's based on JSON-RPC 2.0 and is designed to let LLMs connect to external services — such as file systems, databases, or APIs — in a consistent, secure, and efficient way. It's especially useful for dynamic, program-driven interactions where the LLM acts like a client, asking specialized servers for data or to perform actions.

### WHO IS ANTHROPIC?

Anthropic is a U.S.-based artificial intelligence company founded in 2021 by former OpenAI researchers, including siblings Dario and Daniela Amodei. The company focuses on developing AI systems that are safe, interpretable, and aligned with human values. Anthropic is best known for its Claude family of large language models, which compete with other AI models like OpenAI's ChatGPT and Google's Gemini.

### 13.1.1 The Problems MCP Solves

Before MCP, developers faced several ongoing challenges when trying to connect LLMs to real-world systems:

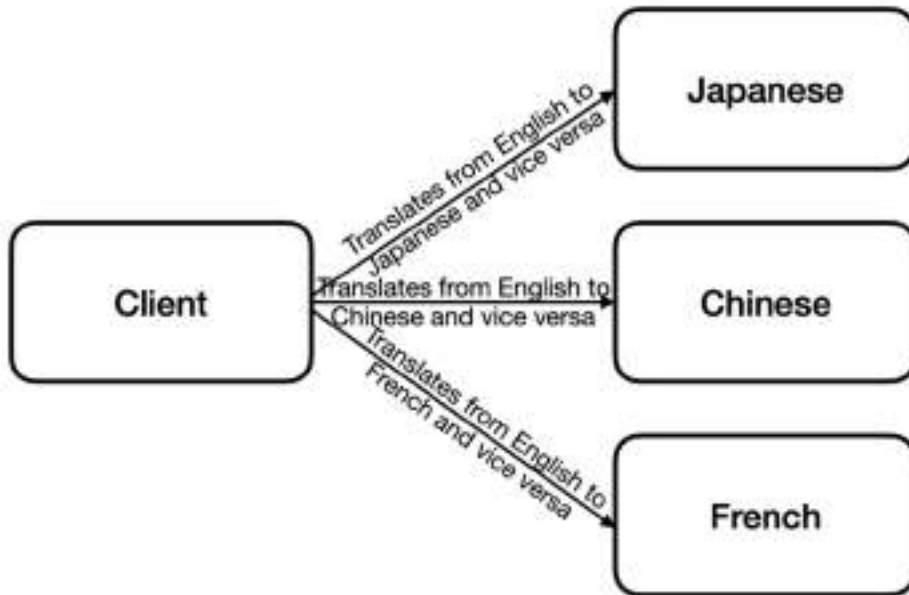
- **Inconsistent Tool Access:** Different models supported different ways of accessing tools and data, with no standard method for calling them.
- **Unreliable Data Retrieval:** Without a formal protocol, pulling in outside data often relied on messy, one-off solutions that were hard to maintain and sometimes insecure.
- **Complex Prompt Engineering:** Each setup needed custom prompts, adding extra work and making it harder to reuse code.
- **Fragmented Integrations:** Teams kept reinventing the wheel with similar integrations, but without a shared standard.

MCP solves these problems by offering a clear, consistent protocol for how models interact with external systems. It defines standard components — like Tools, Resources, and Prompts — along with clear rules for control and communication. This standardization makes development faster and easier, while also boosting reliability, security, and compatibility across models and platforms — all of which are crucial as LLMs are used in more critical systems and workflows.

### 13.1.2 Understanding MCP

Let's imagine you are writing an app (the Client) to communicate with several providers (in Japanese, Chinese, and French). To communicate with each provider, your app needs to talk to each in its own language. This places a lot of burden on you as you must manage multiple custom integrations, adapt to varying API structures, and handle inconsistent response formats, all of which complicate development and maintenance.

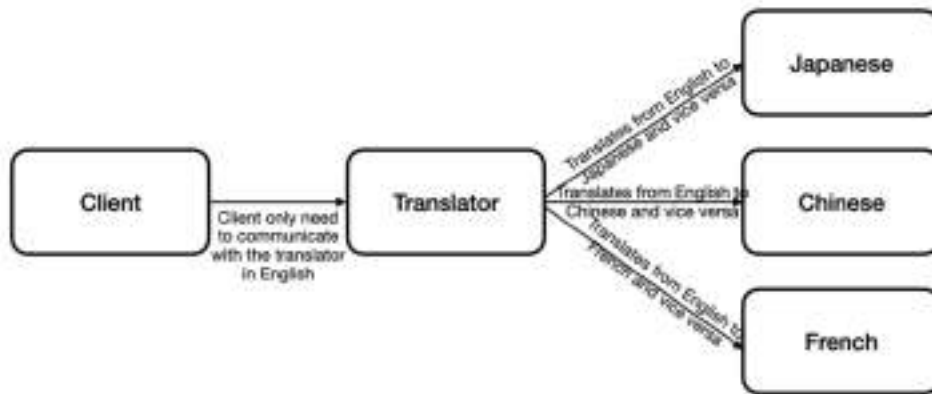
Figure 13.1 shows the relationships between the Client and the providers.



**Figure 13.1 Visualizing the relationships between client and providers**

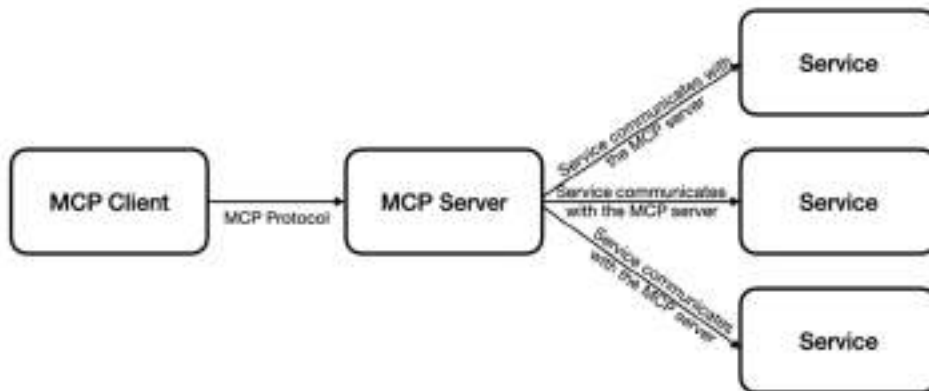
A far more efficient solution is to introduce an intermediary — let's call it the "Translator" in this analogy — that sits between your app and the providers. This Translator handles the communication by converting each provider's unique language into a common one, like English, and back again, so your app only needs to interact with the Translator, simplifying the process and eliminating the need to juggle multiple provider-specific languages directly. Figure 13.2 shows the improved workflow.





**Figure 13.2 The translator acts as an intermediary between the client and the providers**

Drawing from this analogy, you can now adapt this diagram to align with the Model Context Protocol (MCP) as shown in Figure 13.3.



**Figure 13.3 How a MCP client communicate with a MCP server**

In the MCP workflow, there are three main components:

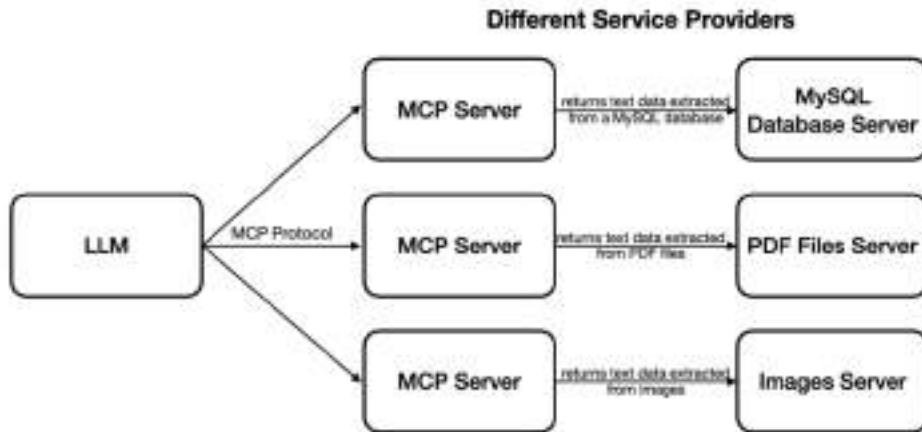
- **MCP Client:** Usually a large language model (LLM) or an AI-powered app like Claude Desktop. It sends requests to access external tools or data and acts as the consumer in the system.
- **MCP Server:** A server that handles these requests. It processes them and sends back responses. Think of it as the central hub that connects clients to services.
- **Service:** These are the actual features or data the client wants to use — such as tools (like `add` or `fetch_weather`) or resources (like `get_greeting` or `get_config`) — all provided through the MCP Server.

Communication between the MCP Client and Server uses JSON-RPC 2.0, a standard protocol that ensures requests and responses are exchanged in a consistent, structured way — no matter what kind of service is being accessed. This makes it easier to build reliable and scalable systems around LLMs.

What are examples of some of the services? They can be:

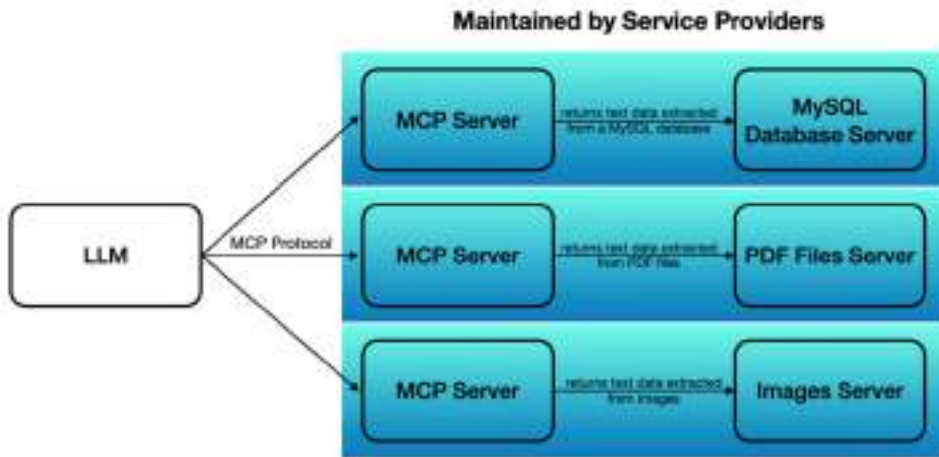
- Database services — allows reading of rows in database tables
- Files services — allows extracting of text from files
- Images services — allows extracting of images or text from image files

Each service could be delivered by a distinct service provider as shown in figure 13.4.



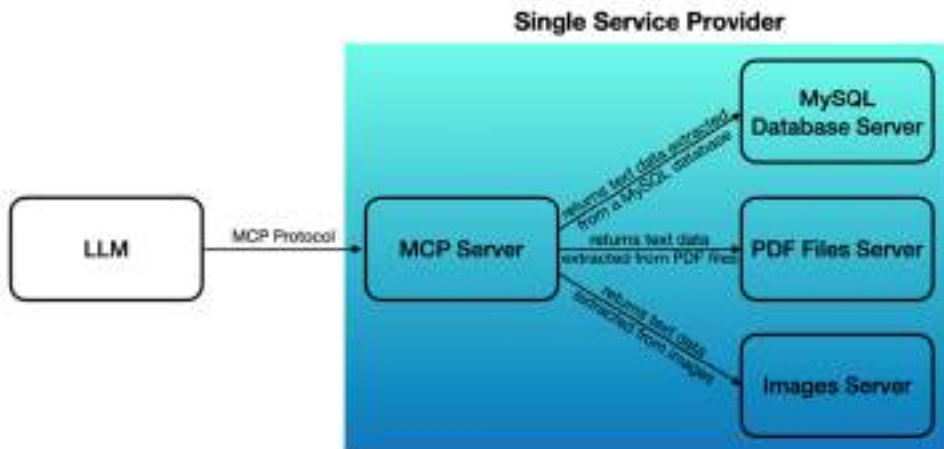
**Figure 13.4 Different services provided by MCP servers**

Each service provider maintains its own MCP server (see figure 13.5).



**Figure 13.5 Each MCP server can provide its own service**

Or a single service provider may maintain a single MCP server with many services as shown in Figure 13.6.



**Figure 13.6 A MCP server can provide multiple services**

### 13.1.3 MCP Servers Deployment

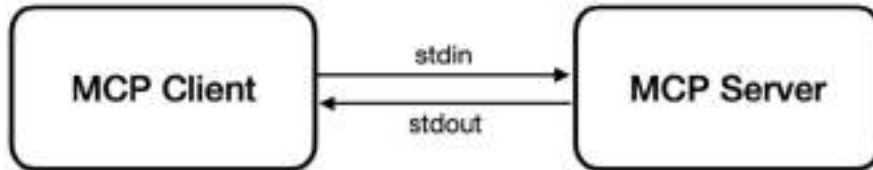
Despite its name suggesting a server-based application, an "MCP server" is simply a standard application that can run in different configurations. Once you've developed your MCP server, you can deploy it in two ways:

- Local deployment: Run the MCP server on the same machine as the MCP client, allowing for direct local communication.
- Remote deployment: Run the MCP server on a separate remote machine, with the MCP client accessing it over the network.

This flexibility in deployment options makes MCP servers adaptable to various architectural needs, from simple local setups to distributed systems.

For the first option, when the MCP server and client are running on the same machine, they will communicate with each other using standard input and output streams (see figure 13.7). In this approach, the MCP server run as a separate process on the same machine as the client.

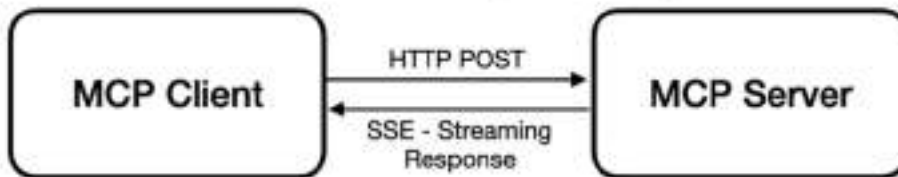
### Both Client and Server running on same machine



**Figure 13.7 A MCP client communicating with a locally running MCP server using stdio**

For the second option, both the server and the client are running on different machines across the network. In this case, the client will communicate with the server using HTTP POST, and the server will respond to the Client using Server-Sent Events (SSE). This transport method is useful when the Server needs to support multiple Clients at the same time. Figure 13.8 shows how the client interacts with the server.

### Server running remotely



**Figure 13.8 A MCP client communicating with a remote MCP server using HTTP POST and SSE**

### WHAT IS SERVER-SIDE EVENTS (SSE)?

SSEs (Server-Sent Events) is a web standard that enables a server to push real-time data to a client over a single HTTP connection. Unlike traditional HTTP request-response patterns, SSEs establish a persistent connection where the server can continuously send data to the client without the client needing to repeatedly poll or make new requests.

SSEs are like WebSockets, except that SSEs are unidirectional – only the server can send data to the client and the client cannot send messages back through the SSE connection.

### 13.1.4 Components in a MCP Server

Now that you have a better idea of the workflow in a MCP system, let's examine the components in a MCP server. A MCP Server comprises of three main components:

- **Tools:** Tools are the actionable capabilities that MCP servers provide to large language models (LLMs), acting like a set of specialized functions — such as reading files with a Filesystem Tool, querying the web via a Search Tool, or generating images through an Image Generation Tool — that the LLM can invoke to extend its reach beyond its inherent knowledge, using a standardized JSON-RPC 2.0 request format (e.g., `{"method": "search", "params": {"query": "AI"}}`) to perform tasks efficiently and consistently across diverse services.
- **Resources:** Resources represent the external data or entities that Tools interact with or retrieve, serving as the raw material — think local files from a Filesystem MCP server, web pages from a Brave Search MCP server, or database records from a PostgreSQL MCP server — that the LLM processes or analyzes, delivered through MCP responses (e.g., `{"result": "file contents"}`) to fuel its reasoning and responses with fresh, context-specific information.
- **Prompts:** Prompts in an MCP system function as pre-defined templates for standardized LLM interactions.

Figure 13.9 summarizes the three main types of components in the MCP Server and their uses, control type, as well as example of their usage.

## MCP Server Components and their Uses

| Components   | <i>Tools</i>                                                                                                   | <i>Resources</i>                                                                          | <i>Prompts</i>                                                             |
|--------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Uses         | Server-exposed functions that can be called by LLMs                                                            | Data exposed to clients for LLM context                                                   | Pre-defined templates for standardized LLM interactions                    |
| Control Type | <b>Model-controlled</b><br>The LLM itself decides when and how to use these tools during its reasoning process | <b>Application-controlled</b><br>Client will determine what resources to retrieve and use | <b>User-controlled</b><br>Clients determine the templates they want to use |
| Examples     | Fetching weather information, performing calculations, etc                                                     | Fetch files, read database records, return images, etc                                    | Document Q&A, summarizing a block of text, etc                             |

**Figure 13.9** The components in a MCP server and their uses

The use of the various components of a MCP server will be much clearer when you create one yourself.

## 13.2 Building Your Own MCP Server

Now that you have a solid understanding of what an MCP Server is and how its components work, it's time to put that knowledge into action. For this task, we will build a MCP server using Python. Our MCP server will allow users to ask questions such as:

- What is the current weather?
- Summarize the content of a particular file
- Ask a question based on the content of a particular file

### 13.2.1 Installing uv

For this project, you will make use of `uv`, an extremely fast Python package and project manager written in Rust, to install Python packages. First, use the following `curl` command to download and install `uv`:

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

Once `uv` is installed, you will use it to create a Python project.

### 13.2.2 Initializing the project

Let's create use `uv` to initialize a project named `MCP_Demo` and then change the directory into the newly created folder:

```
$ uv init MCP_Demo
Initialized project `mcp-demo` at `/Volumes/SSD/MCP_Demo`

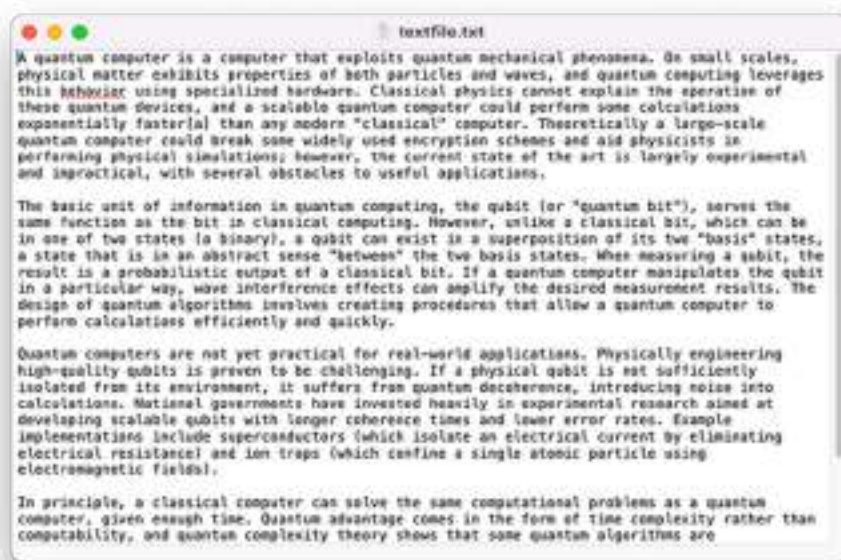
$ cd MCP_Demo
```

Within the `MCP_Demo` folder, we will put two files in it:

- A PDF document named `Singapore.pdf` (see figure 13.10). This document is generated from Wikipedia. For simplicity, this document only has 1 page.







**Figure 13.11** The content of the text file

### 13.2.3 Installing the packages

To implement the MCP server, we will be using the official Python SDK for Model Context Protocol servers and clients, located at <https://github.com/modelcontextprotocol/python-sdk/tree/main>. You can find the documentation of MCP at <https://modelcontextprotocol.io/introduction>.

In Terminal, type the following command to install the mcp, httpx, and the PyMuPDF packages:

```
$ uv add "mcp[cli]" httpx PyMuPDF
```

### 13.2.4 Creating the MCP server

Once the packages are installed, you can now create the server.py file to store the implementation of the MCP server:

```
$ nano server.py
```

Populate the server.py file with the statements as shown in Listing 13.1.

**Listing 13.1 A Simple MCP server using the FastMCP class**

```

from mcp.server.fastmcp import FastMCP
import httpx
import fitz #A
import os

mcp = FastMCP("MCP Demo") #B

if __name__ == "__main__":

    mcp.run(transport='stdio')

```

**#A** For PyMuPDF

**#B** Create an MCP server

This above code sets up a simple MCP server using the `FastMCP` class from the `mcp.server.fastmcp` module:

- We first create an instance of the `FastMCP` class, passing “MCP Demo” as the name of the service. The `FastMCP` object (`mcp`) represents the server itself, which will be configured and run later.
- We then start the `FastMCP` server with the `transport` parameter set to “stdio” (standard input/output).

Once this is done, let’s run the server and see if there is any error:

```
$ uv run server.py
```

If there is no error, nothing is shown on the screen.

### 13.2.5 Inspecting the MCP server

To inspect the MCP server, you can use the MCP Inspector by running the following command:

```
$ uv run mcp dev server.py
```

The MCP Inspector is an interactive developer tool designed for testing and debugging servers that implement the Model Context Protocol (MCP). You will now see something like this:

```

Starting MCP inspector...
Proxy server listening on port 3000

🔍 MCP Inspector is up and running at http://localhost:5173 🚀
New SSE connection
Query parameters: {
...
...
Connected MCP client to backing server transport
Created web app transport
Created web app transport
Set up MCP proxy
🔍 MCP Inspector is up and running at http://localhost:5173 🚀

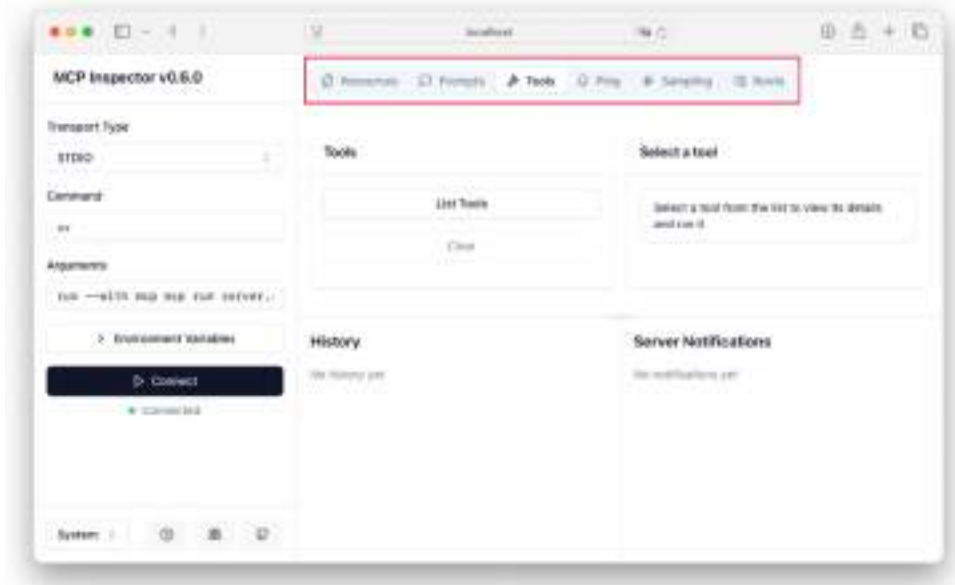
```

The MCP Inspector is a web-based application that listens at the following URL: `http://localhost:5173`. To view it, load it using a web browser. You should see the screen as shown in figure 13.12.



**Figure 13.12** The MCP Inspector

Click the **Connect** button to connect to the MCP server. You will see the tabs as shown in Figure 13.13 - Resources, Prompts, Tools, Ping, # Sampling, and Roots.



**Figure 13.13** The MCP Inspector connecting to the MCP server

Now that the MCP server is running and we can get the MCP Inspector to connect to it, let's add some resources, tools, and prompts to the MCP server implementation.

### 13.2.6 Implementing Resources

The first component we will add to our MCP server is resources. Recall that resources represent the external data or entities that Tools interact with or retrieve. So, in Listing 13.2, we will add three resources to `server.py`.

**Listing 13.2 Adding resources to our MCP server (server.py)**

```

@mcp.resource("text://{file_path}")
def get_file(file_path: str) -> str:
    actual_path = os.path.abspath(file_path)                #A
    if not os.path.exists(actual_path):
        raise FileNotFoundError(f"Error: File '{actual_path}' not found!")
    with open(actual_path, "r", encoding="utf-8") as file:
        return file.read()

@mcp.resource("config://app")
def get_config() -> str:
    """Static configuration data"""
    return "Version 1.1"

@mcp.resource("pdf://{file_path}")
def get_pdf_data(file_path: str) -> str:
    text = ""
    actual_path = os.path.abspath(file_path)
    if not os.path.exists(actual_path):
        raise FileNotFoundError(f"Error: File '{actual_path}' not found!")
    with fitz.open(actual_path) as doc:
        for page in doc:
            text += page.get_text() + "\n"
    return text

```

**#A Convert to absolute path**

The above code snippet adds the following resources:

- `get_file()` — Retrieve the contents of a text file given its file path. This function reads and returns the contents of a text file specified by `file_path`. It uses `os.path.abspath()` to ensure a valid path and raises an error if the file doesn't exist.
- `get_config()` — Return static application configuration data. This is a simple function that returns a hardcoded string ("Version 1.1") as configuration data. It's static for now but could be expanded to fetch dynamic config data.
- `get_pdf_data()` — Extract and return the text content from a PDF file given its file path. This function uses `fitz` (PyMuPDF) to open a PDF file, extract text from each page, and return it as a single string. Like `get_file()`, it validates the file path and raises an error if the file isn't found.

Note that the `@mcp.resourcedecorator` registers each function as an MCP resource with a specific URI-like identifier (e.g., `text://{file_path}`, `config://{app, pdf://{file_path}}`). These identifiers define how the resources are accessed by an MCP client or tool like the MCP Inspector.

In MCP, resources are typically represented as endpoints that handle specific requests, such as querying data, retrieving documents, or interacting with external services.

### 13.2.7 Implementing Tools

The next component to implement are tools. Tools are the actionable capabilities that MCP servers provide to large language models. In Listing 13.3, we added four tools to `server.py`.

**Listing 13.3 Adding tools to our MCP server**

```
@mcp.tool()
async def fetch_weather(city: str, units: str = "metric") -> dict:
    API_KEY = "xxxxxxxxxxxxxxxxxxxxx" #A
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"https://api.openweathermap.org/data/2.5/weather",
            params={
                "q": city,
                "units": units,
                "appid": API_KEY
            }
        )
    if response.status_code == 200:
        data = response.json()
        weather_data = {
            "location": {
                "name": data["name"],
                "country": data["sys"]["country"],
                "coordinates": {
                    "lat": data["coord"]["lat"],
                    "lon": data["coord"]["lon"]
                }
            },
            "current": {
                "temp": data["main"]["temp"],
                "feels_like": data["main"]["feels_like"],
                "humidity": data["main"]["humidity"],
                "pressure": data["main"]["pressure"],
                "description": data["weather"][0]["description"],
```

```

        "icon_code": data["weather"][0]["icon"]
    },
    "wind": {
        "speed": data["wind"]["speed"],
        "direction": data["wind"]["deg"]
    },
    "sun": {
        "sunrise": data["sys"]["sunrise"],
        "sunset": data["sys"]["sunset"]
    },
    "units": units,
    "timestamp": data["dt"]
}
return weather_data
else:
    return {
        "error": f"Weather data not available. Status code:
{response.status_code}",
        "message": response.text
    }

@mcp.tool()
def convert_temperature(temp: float,
                        from_unit: str,
                        to_unit: str) -> float:                                #B
    if from_unit.lower() == "celsius":                                         #C
        kelvin = temp + 273.15
    elif from_unit.lower() == "fahrenheit":
        kelvin = (temp + 459.67) * 5/9
    elif from_unit.lower() == "kelvin":
        kelvin = temp
    else:
        raise ValueError(f"Unsupported unit: {from_unit}")

    if to_unit.lower() == "celsius":                                           #D
        return kelvin - 273.15
    elif to_unit.lower() == "fahrenheit":
        return kelvin * 9/5 - 459.67
    elif to_unit.lower() == "kelvin":
        return kelvin
    else:
        raise ValueError(f"Unsupported unit: {to_unit}")

```

```
@mcp.tool()
def get_pdf(file_path: str) -> str:
    return get_pdf_data(file_path)

@mcp.tool()
def get_text(file_path: str) -> str:
    return get_file(file_path)
```

**#A** Replace with your actual API key from OpenWeatherMap

**#B** Add a helper tool for temperature conversion

**#C** First convert to Kelvin as intermediate step

**#D** Convert from Kelvin to target unit

The above code snippet adds the following tools:

- `fetch_weather()` — Fetch current weather data for a specified city using OpenWeatherMap API (you can apply for your own API key from [https://home.openweathermap.org/api\\_keys](https://home.openweathermap.org/api_keys)). This asynchronous tool uses `httpx` to query the OpenWeatherMap API, returning a detailed dictionary of weather data (location, temperature, wind, etc.) for the specified city, or an error message if the request fails.
- `convert_temperature()` — Convert a temperature value between Celsius, Fahrenheit, and Kelvin. This tool converts a temperature from one unit to another (e.g., Celsius to Fahrenheit) using Kelvin as an intermediate step, raising an error for unsupported units.
- `get_pdf()` — Retrieve the text content of a PDF file using the `get_pdf_data` resource. This tool acts as a wrapper around the `get_pdf_data` resource, providing a convenient way to access PDF text extraction as a tool.
- `get_text()` — Retrieve the contents of a text file using the `get_file` resource. This tool wraps the `get_file` resource, allowing text file content retrieval through the tool interface.

The `@mcp.tool()` decorator registers each function as an MCP tool, making it callable by an MCP client or the MCP Inspector. Unlike resources, tools are typically actions or utilities rather than data endpoints.

### 13.2.8 Implementing Prompts

Finally, let's add a prompt to our MCP server as shown in listing 13.4.



**Listing 13.4 Adding a weather prompt to our MCP server**

```
@mcp.prompt()
def weather_report(city: str) -> str:
    return f"""
    Please provide a weather report for {city}.

    You can use the fetch_weather tool to get current weather data.
    If needed, you can convert temperature units using the
    convert_temperature tool.

    Please include:
    - Current temperature
    - Weather conditions
    - Humidity
    - Wind speed
    - Any relevant weather advice for the conditions
    """
```

The above code snippet adds a prompt to the MCP server -

- `weather_report()` — Generate a prompt template for requesting a detailed weather report for a specified city. This description reflects the function's purpose: it creates a formatted string that serves as a template for an MCP client (e.g., an LLM) to generate a weather report using available tools like `fetch_weather` and `convert_temperature`.

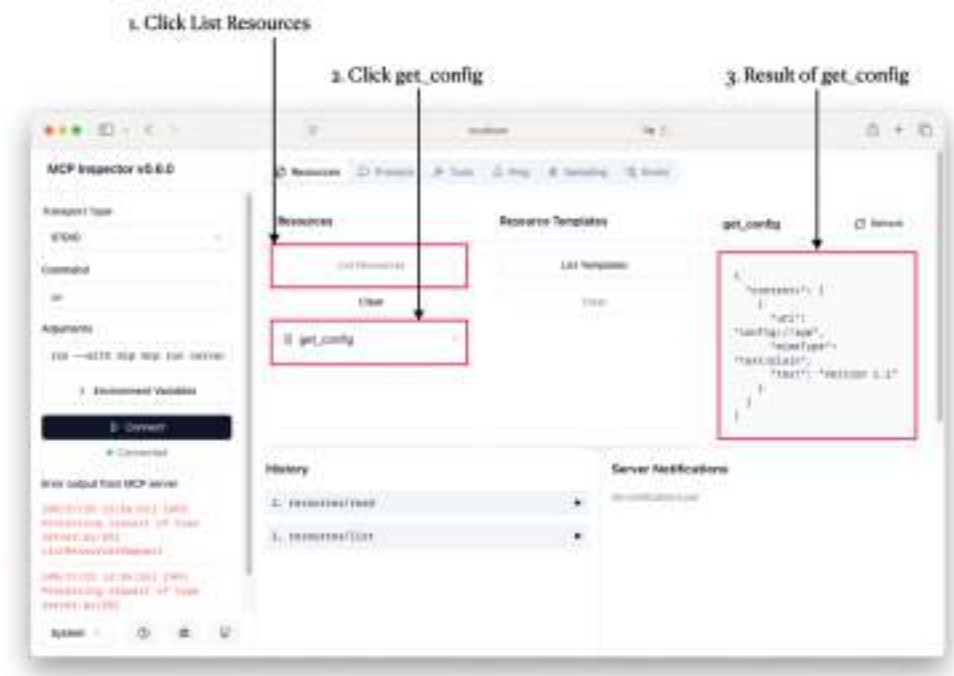
The `@mcp.prompt` decorator registers the `weather_report` function as an MCP prompt. Prompts in MCP are typically templates or instructions that guide how a client (like an LLM) should process a request, often leveraging the server's tools and resources.

### 13.2.9 Testing the components

With our MCP server updated with resources, tools, and prompt, let's now test them out to ensure that they work as intended. Let's now rerun the MCP server using the MCP Inspector:

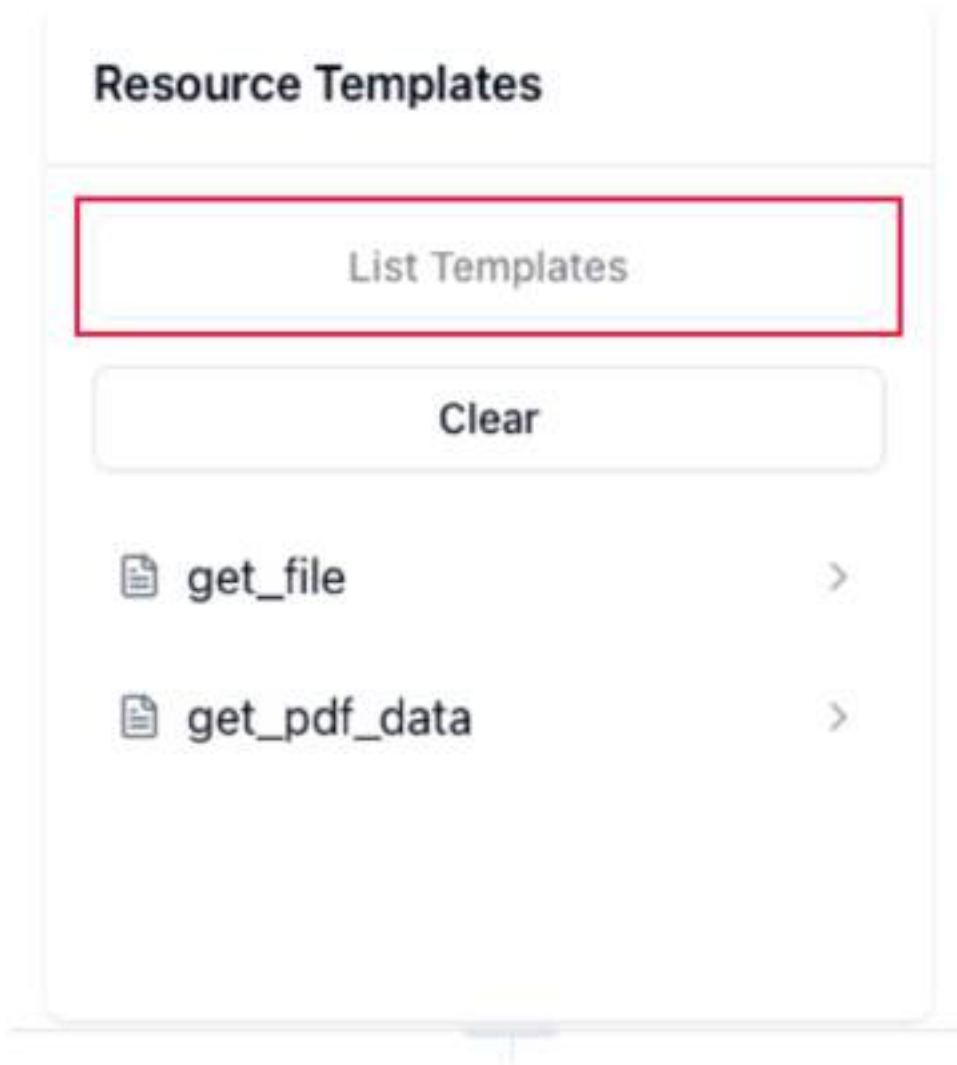
```
$ uv run mcp dev server.py
```

In the MCP Inspector, click the Connect button to connect to the MCP Server. We are now ready to test the resources, prompts, and tools. Click the List Resources button and you will see the `config://app` as shown below. Clicking on it will return the details of the version of the app (see figure 13.14).



**Figure 13.14** Checking out the resources using the MCP Inspector

Interestingly, clicking the List Resources button only shows the `get_config()` (with the URI "config://app") function. This is because this static function does not have any input parameters, unlike the other two functions — `get_file()` and `get_pdf_data()`. To see these two functions, click the List templates button (see figure 13.15).



**Figure 13.15** Listing all the resource templates in the MCP server

You should now see the two functions. Clicking on the `get_file` function, you can enter the name of a text file and then click Read Resource. You will now see the content of the file (remember that there is a text file named `textfile.txt` in the `MCP_Demo` folder; see figure 13.16).



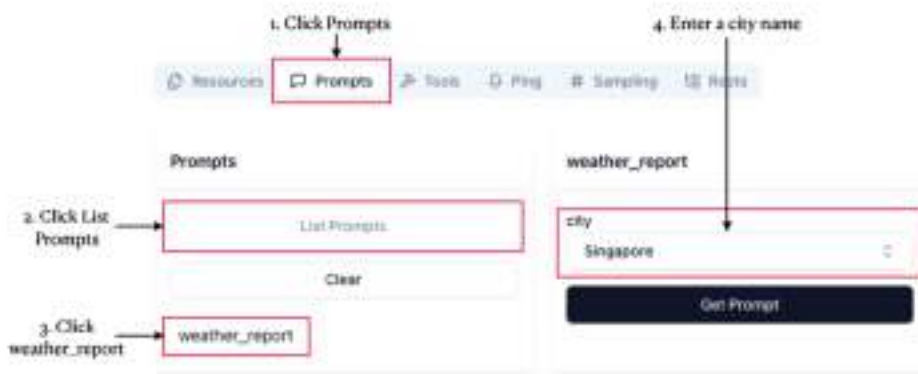
**Figure 13.16 Loading the content of a text file through a resource**

Likewise, click on the `get_pdf_data` function, enter a name and click the Read Resource button. You will see something like figure 13.17 (remember that there is a text file named `Singapore.pdf` in the `MCP_Demo` folder).



**Figure 13.17 Loading the content of a PDF file through a resource**

Next, click on the Prompts tab and then click List Prompts. You should see the `weather report` prompt. Click on it and enter “Singapore” as the city (see Figure 13.18).

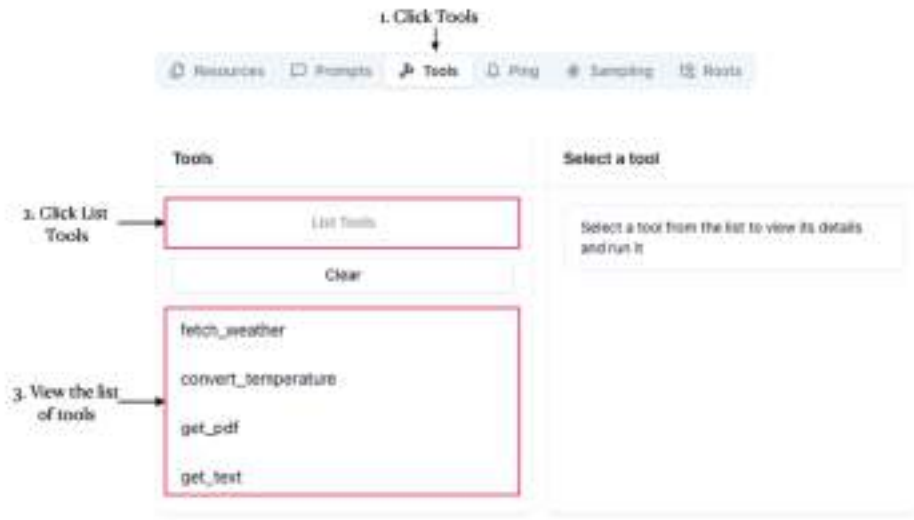


**Figure 13.18 Loading the prompt**

Click the Get Prompt button and you will see the following:

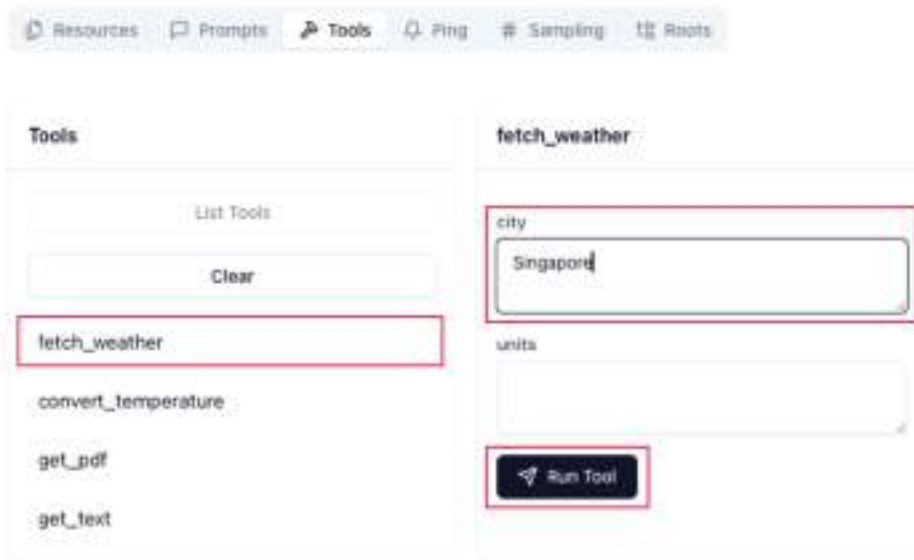
```
{
  "messages": [
    {
      "role": "user",
      "content": {
        "type": "text",
        "text": "\n    Please provide a weather report for Singapore.\n\n    You can use the fetch_weather tool to get current\n    weather data.\n    If needed, you can convert temperature\n    units using the convert_temperature tool.\n\n    Please include:\n      - Current temperature\n      - Weather conditions\n      - Humidity\n      - Wind speed\n      - Any relevant weather advice for the conditions\n    "
      }
    }
  ]
}
```

Click on the Tools tab and then click the List Tools button. You will see the four tools that we have defined earlier (see Figure 13.19).



**Figure 13.19** Checking out the tools of the MCP server

Click on the `fetch_weather` tool, type “Singapore” and then click Run Tool (see figure 13.20).



**Figure 13.20** Fetching the weather for a city through a tool

This will fetch the current weather for Singapore:

```

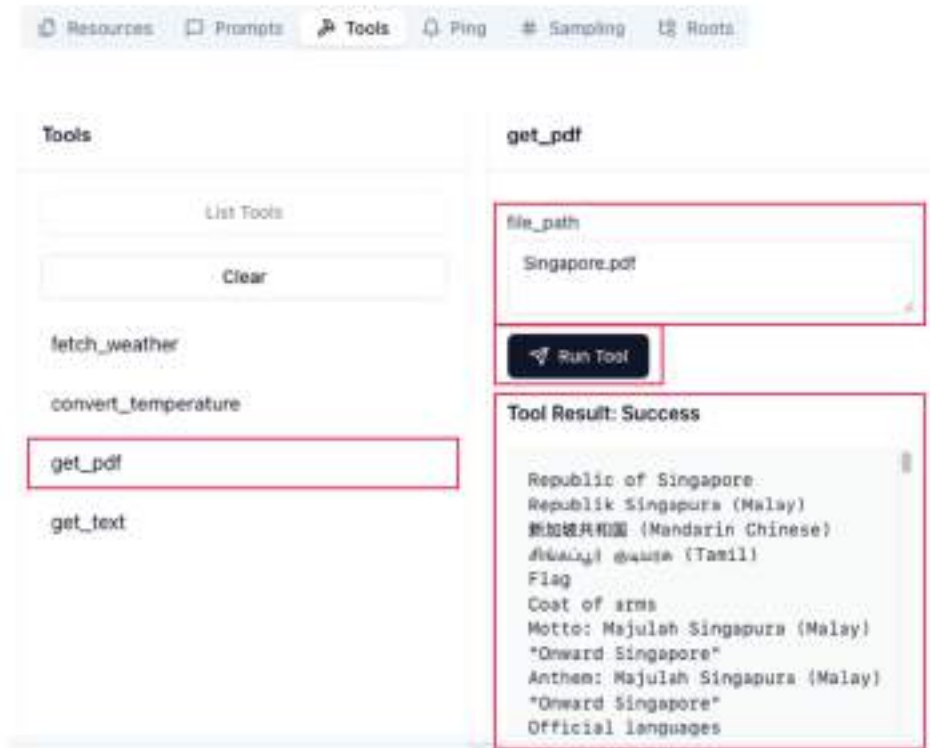
{
  "location": {
    "name": "Singapore",
    "country": "SG",
    "coordinates": {
      "lat": 1.2897,
      "lon": 103.8501
    }
  },
  "current": {
    "temp": 32.14,
    "feels_like": 38.04,
    "humidity": 62,
    "pressure": 1009,
    "description": "broken clouds",
    "icon_code": "04d"
  },
  "wind": {
    "speed": 2.57,
    "direction": 40
  },
  "sun": {
    "sunrise": 1748300180,
    "sunset": 1748344049
  },
  "units": "metric",
  "timestamp": 1748322002
}

```

The result is obtained through the Open Weather API.

Let's try the `get_pdf` tool now. Click on it and type "Singapore.pdf" and click Run Tool. You will get the content of the PDF file (see figure 13.21; remember there is a file named Singapore.pdf in the MCP\_Demo folder).





**Figure 13.21** Getting the content of a PDF file through a tool

You can also try the other tools — `convert_temperature`, and `get_text`.

### 13.3 Testing the MCP Server using the Claude Desktop

Although your MCP server has been validated through the MCP Inspector, this testing tool doesn't fully showcase the practical value of MCP. So, how is it useful, and what are some real-world use cases? The best way to explore its utility is by pairing it with Claude Desktop (see Figure 13.22), an application you can download from <https://claude.ai/download>, which integrates MCP to enhance AI-driven tasks and workflows.



**Figure 13.22 Downloading the Claude Desktop**

## CLAUDE DESKTOP

Claude Desktop is a desktop application developed by Anthropic that brings the capabilities of the Claude AI model directly to your computer, available for both macOS and Windows. It's designed to provide seamless, fast access to Claude's conversational AI features without relying on a web browser, making it ideal for users who want a focused, integrated experience for tasks like coding, content creation, data analysis, or deep work.

The Claude Desktop supports the Model Context Protocol (MCP), enabling connections to external tools — like filesystem access or web search — via pre-built or custom MCP servers.

### 13.3.1 Configuring Claude Desktop to use the MCP Server

To configure the Claude Desktop to use the MCP server we have developed, you need to modify the `claude_desktop_config.json` file located in the `~/Library/Application Support/Claude` folder:

```
$ nano ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

For Windows, the path to this file is `%APPDATA%\Claude\claude_desktop_config.json`.

Populate the `claude_desktop_config.json` file with the following statements (bolded for emphasis):

```
{
  "mcpServers": {
    "weather": {
      "command": "/Users/weimenglee/.local/bin/uv",
      "args": [
        "--directory",
        "/Volumes/SSD/MCP_Demo",
        "run",
        "server.py"
      ]
    }
  }
}
```

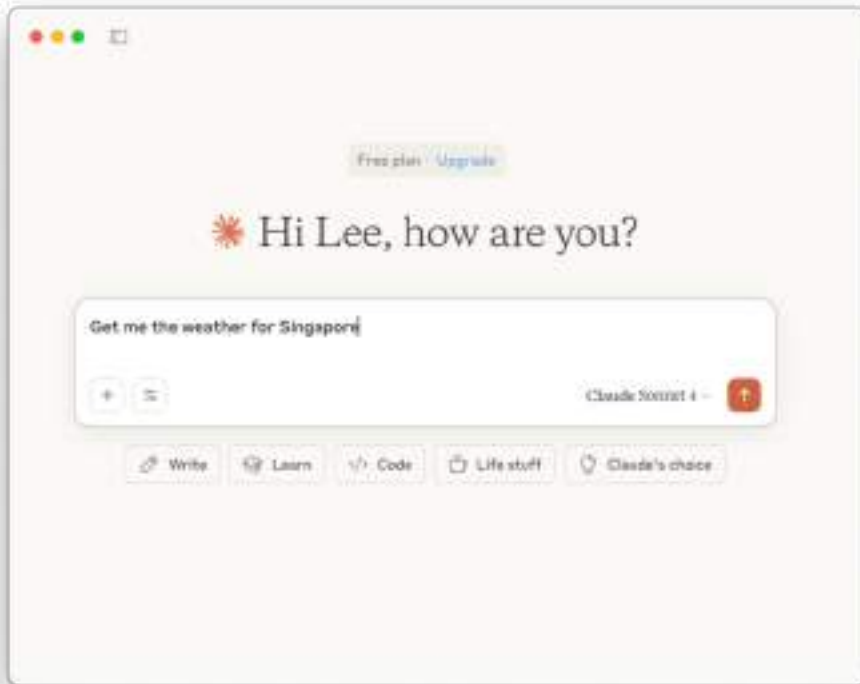
Observe the values in bold:

- "weather" is the name of the MCP server. You can give it any name you want.
- **/Users/weimenglee/.local/bin/uv** is the location of the uv tool. It will be used to run your MCP server.
- **/Volumes/SSD/Medium/MCP\_Demo** is the full path of the folder containing the MCP server (server.py).
- **server.py** is the name of the file containing the MCP server implementation

Once the `claude_desktop_config.json` file is saved, restart Claude Desktop. Note that the first time you launched Claude Desktop, you will be asked to sign in.

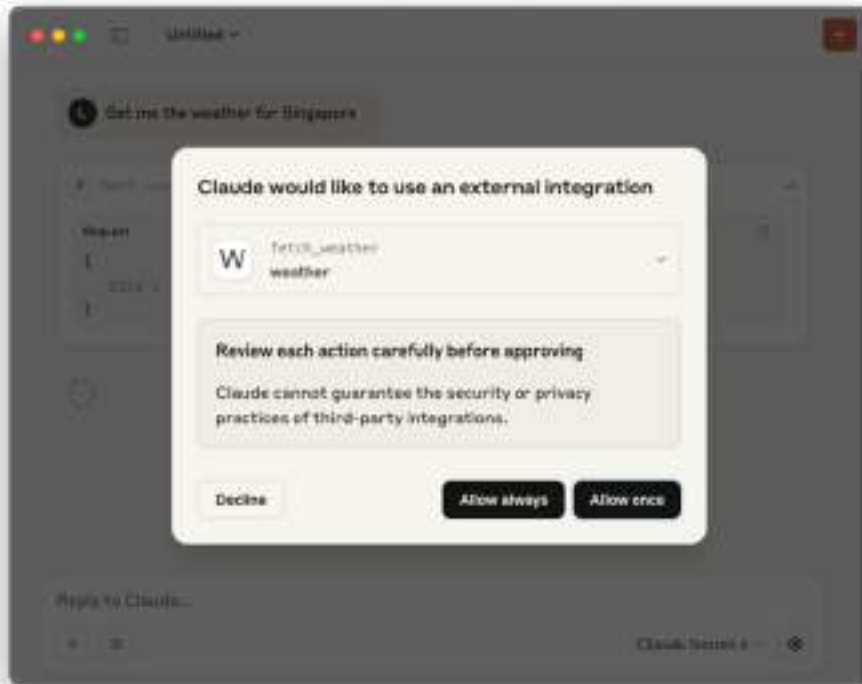
### 13.3.2 Getting the Weather

In the Claude Desktop, let's now ask the following question — "Get me the weather for Singapore"; see figure 13.23.



**Figure 13.23 Asking a question in the Claude Desktop**

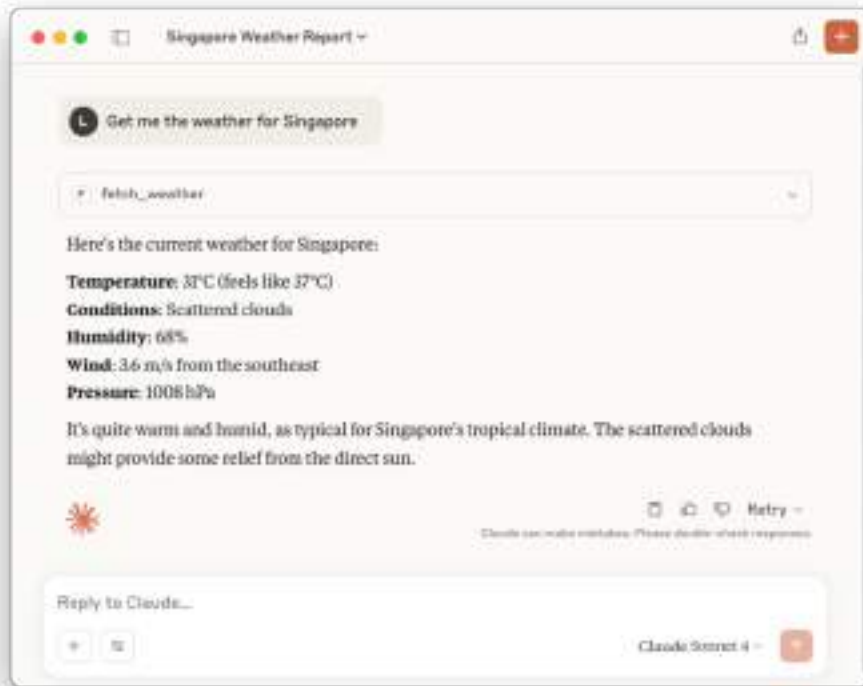
Claude will take a moment to process and then present the prompt as shown in figure 13.24.



**Figure 13.24** Claude Desktop has found the tool to answer your question

It is basically saying that to answer your question, it has found a tool called `fetch_weather` from the weather MCP server that you have configured. Click Allow Once (or Allow always) to allow Claude to access the `fetch_weather` tool to fetch the weather for Singapore.

After a short wait, the result will be returned to you. Notice that Claude utilizes the tool's output to generate a coherent response (see figure 13.25).

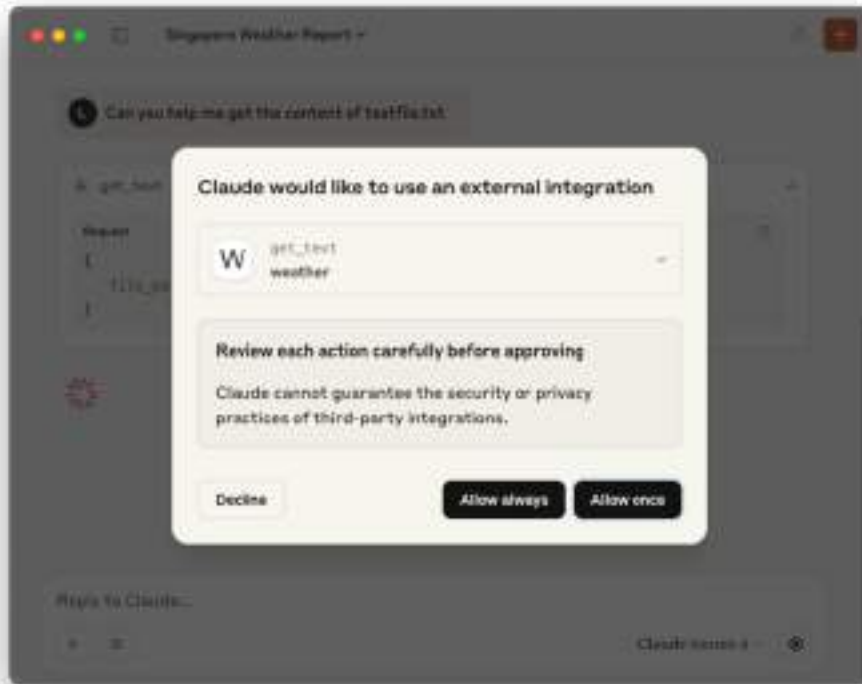


**Figure 13.25** The result returned by the tool

Without the MCP tools, Claude would not be able to fetch the weather information for you.

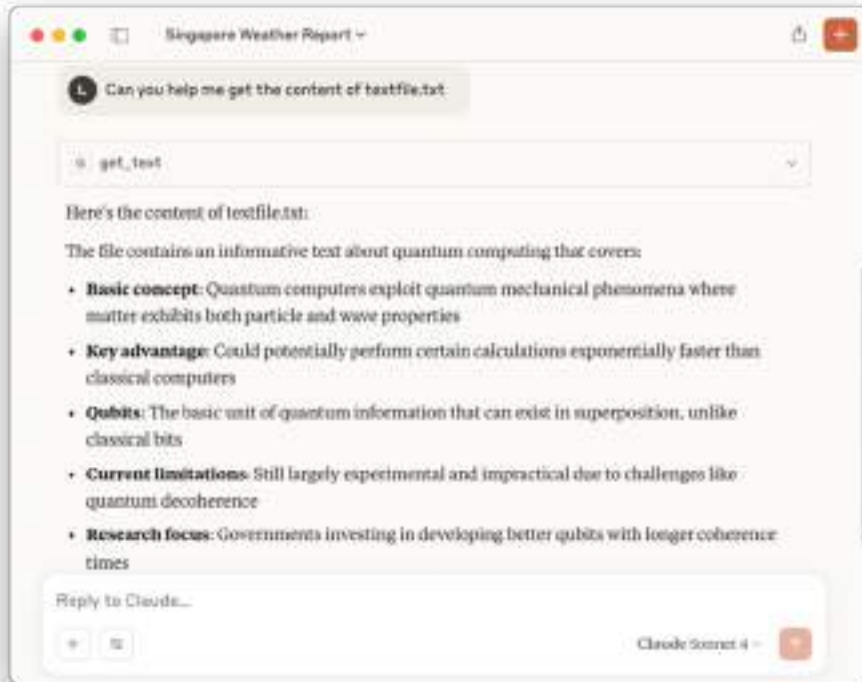
### 13.3.3 Getting the Content of a text file

Let's try another question — "Can you help me get the content of textfile.txt". As before, it has detected that a tool named `get_text` is able to answer your question (see figure 13.26).



**Figure 13.26** Claude Desktop found a tool to fetch the content of a file

If you grant it access to the tool, Claude will extract the content from the text file and summarize it in bullet points (see figure 13.27).



**Figure 13.27 The content of the file returned by Claude Desktop**

You can follow-up with another question — “Summarize the content of the file”. It will now generate a summary (see Figure 13.28).



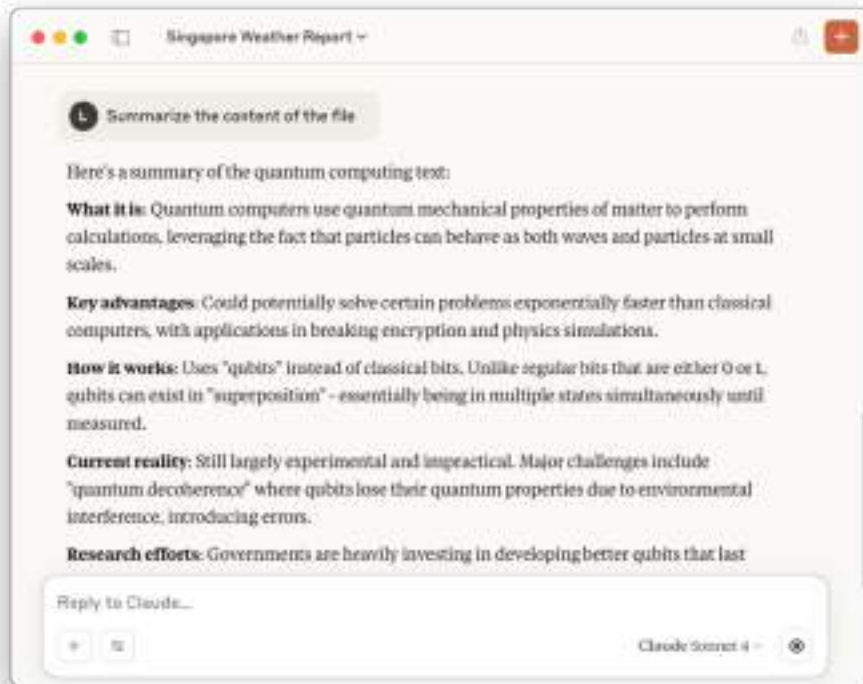
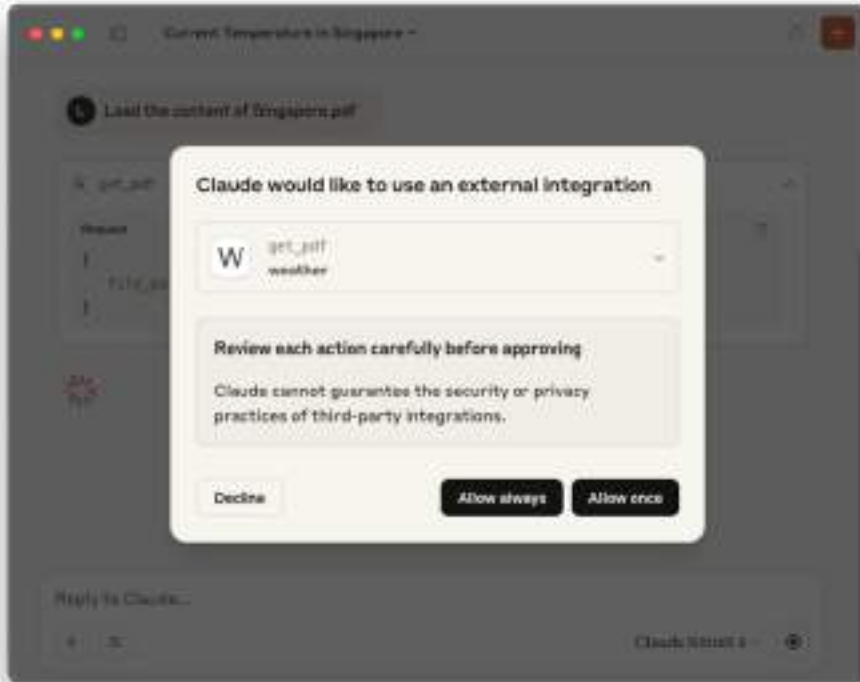


Figure 13.28 Asking Claude Desktop to summarize the content of the file

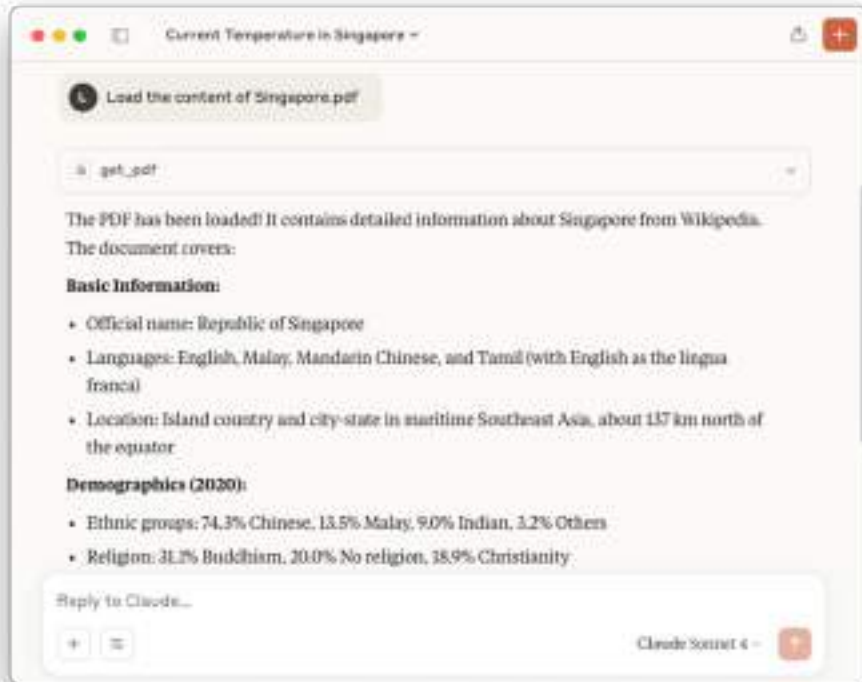
### 13.3.4 Getting the Content of a PDF file

Let's ask one more question — "Load the content of Singapore.pdf". As expected, Claude knows that the `get_pdf` tool can answer your question (see figure 13.29).



**Figure 13.29** Claude Desktop found a tool to fetch the content of a PDF file

When you grant it permission to use the tool, it will load the content of the PDF file and return you the key points in the document (see figure 13.30).



**Figure 13.30** The content of the PDF file returned by the tool

You can now ask questions specific to the document — “What are the official languages in Singapore?” and the result will now be generated for you (see figure 13.31)!

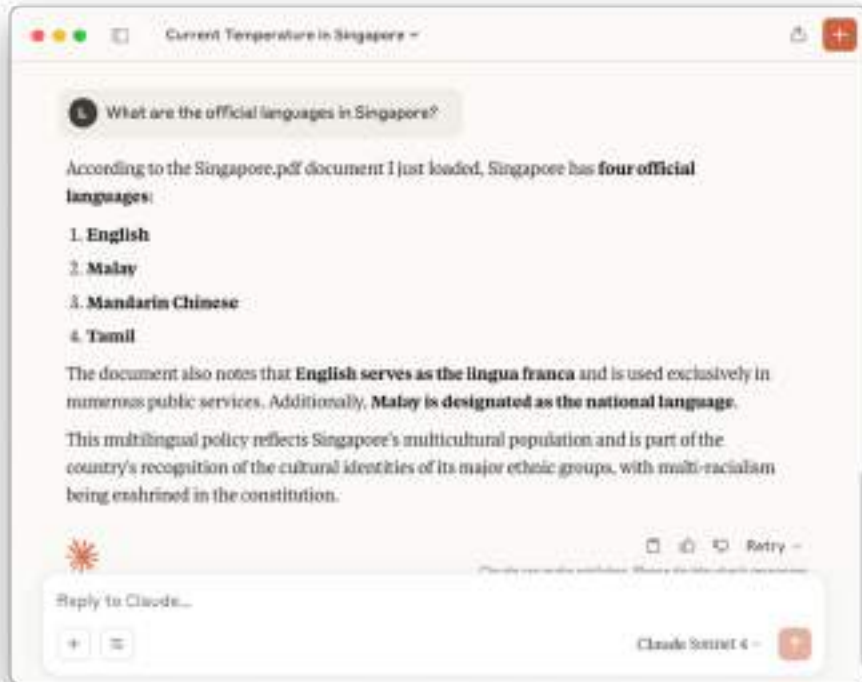


Figure 13.31 Asking questions pertaining to the PDF document

### 13.3.5 Improving the MCP Server

In our MCP server implementation, observe that we hardcoded the OpenWeatherMap API key in our code. Ideally, you should allow the user to pass it in to your code using an environment variable. So let's change it now. First, add in the following statements in bold to the server.py file:

```

from mcp.server.fastmcp import FastMCP
import httpx
import fitz
import os

import sys

API_KEY = os.getenv('OPENWEATHER_API_KEY')          #A
if not API_KEY:
    print("Error: OPENWEATHER_API_KEY environment variable must be set",
          file=sys.stderr)
    sys.exit(1)

mcp = FastMCP("MCP Demo")
...

```

**#A** Get API key from environment variable

Then, comment out the assignment of the `API_KEY` variable in the `fetch_weather()` function:

```

@mcp.tool()
async def fetch_weather(city: str, units: str = "metric") -> dict:
    async with httpx.AsyncClient() as client:
        # API_KEY = "xxxxxxxxxxxxx"
        # Using OpenWeatherMap API
        response = await client.get(
...

```

Finally, pass in the `OPENWEATHER_API_KEY` **value (replace it with your own API key) through the environment variable in the `claude_desktop_config.json` file:**

```
{
  "mcpServers": {
    "weather": {
      "command": "/Users/weimengLee/.local/bin/uv",
      "args": [
        "--directory",
        "/Volumes/SSD/MCP_Demo",
        "run",
        "server.py"
      ],
      "env": {
        "OPENWEATHER_API_KEY": "xxxxxxxxxxxxx"
      }
    }
  }
}
```

Restart the Claude Desktop and the MCP server will work as intended.

## 13.4 Trying out Third Party MCP Servers

At this stage, you've built your own MCP server and tested it using the Claude Desktop. Now, let's explore further and try out some MCP servers created by others and install the following third party MCP servers:

- Location service – A MCP server that allows you to discover your geographical location.
- Time service – A MCP server that allows you to obtain the current time. It also supports generation of datetime strings in various formats.

### 13.4.1 Location service

The Location Service is a MCP Server that allows you to discover your geographical location. To learn more about this service, you can visit <https://mcp.so/server/get-my-location/typescript>. To use this service in the Claude Desktop, edit the `claude_desktop_config.json` file:

```
$ nano ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

Add in the following statements in bold (note the comma before the "get-location" key):

```

{
  "mcpServers": {
    "weather": {
      "command": "/Users/weimengLee/.local/bin/uv",
      "args": [
        "--directory",
        "/Volumes/SSD/MCP_Demo",
        "run",
        "server.py"
      ],
      "env": {
        "OPENWEATHER_API_KEY": "xxxxxxxxxxxxx"
      }
    },
    "get-location": {
      "command": "npx",
      "args": [
        "-y",
        "@mcpn/mcp-get-location"
      ],
      "env": {}
    }
  }
}

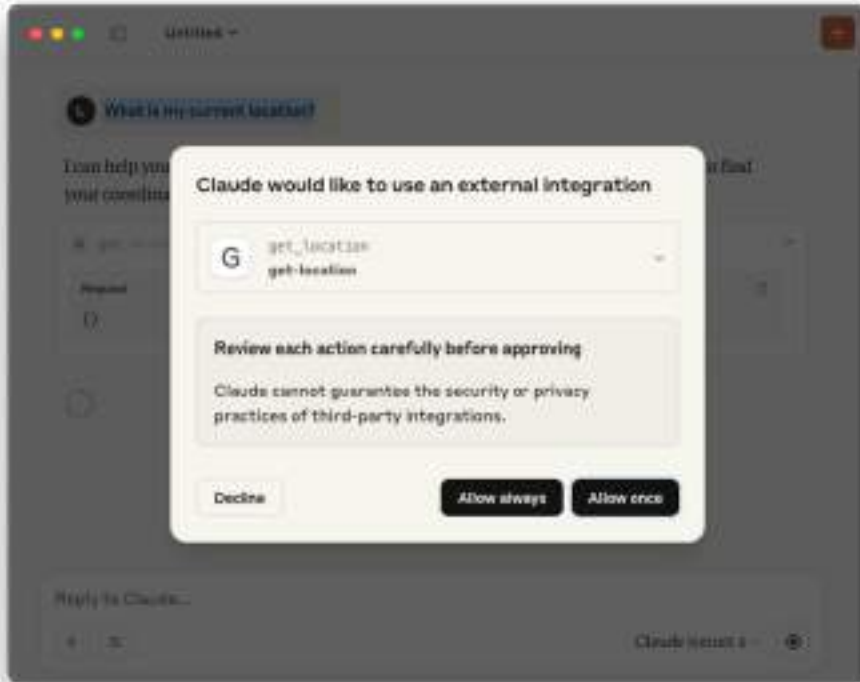
```

Unlike the MCP server that we developed using Python, the Location service is written in Node.js. Here are what the various keys above do:

- Uses npx (a command-line tool that comes with Node.js) to run a Node.js package without installing it globally.
- The `-y` flag automatically answers "yes" to prompts.
- Runs the `@mcpn/mcp-get-location` package. The package is located at <https://www.npmjs.com/package/@mcpn/mcp-get-location>.
- Has no special environment variables configured.

The npx tool downloads the package from the NPM registry to a temporary location and then runs it immediately without permanently installing it. The package executes locally on your machine as a MCP server and it provides location services to an MCP client (like Claude Desktop).

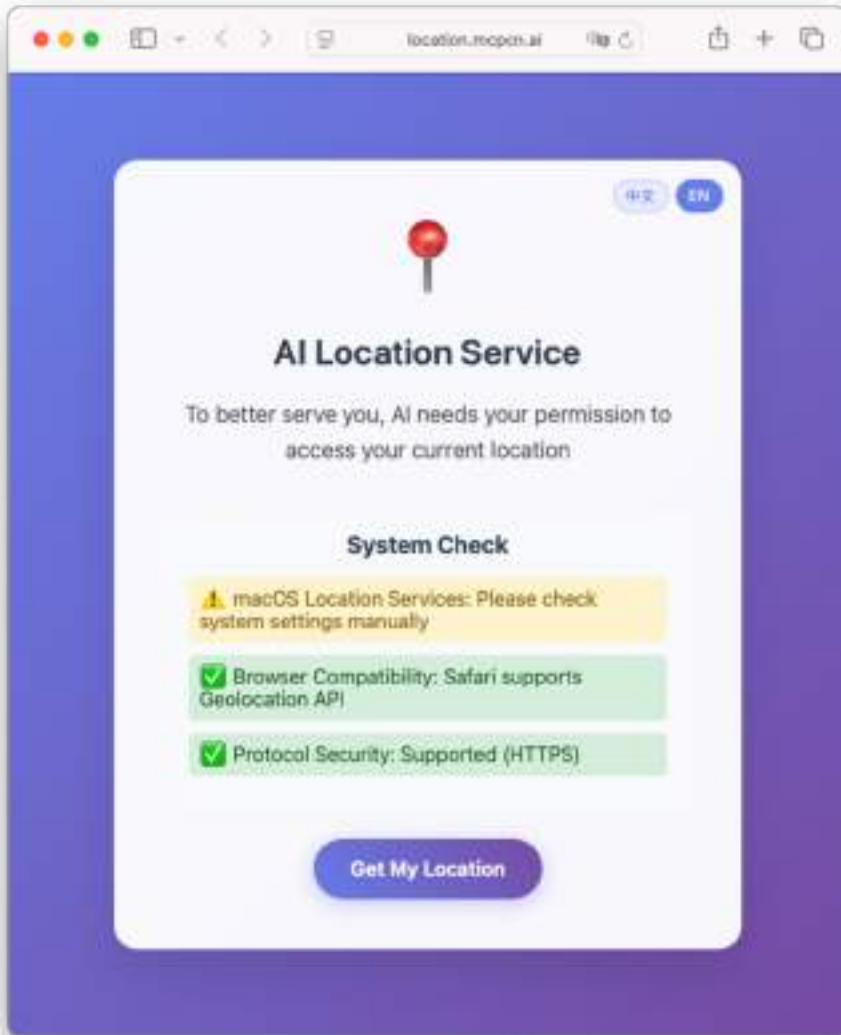
Restart the Claude Desktop and ask the following question: "What is my current location?". You should now see that the Claude Desktop is able to find the `get-location` tool to answer that question (see figure 13.32). Click Allow once.



**Figure 13.32** Claude Desktop using the third party tool to get your current location

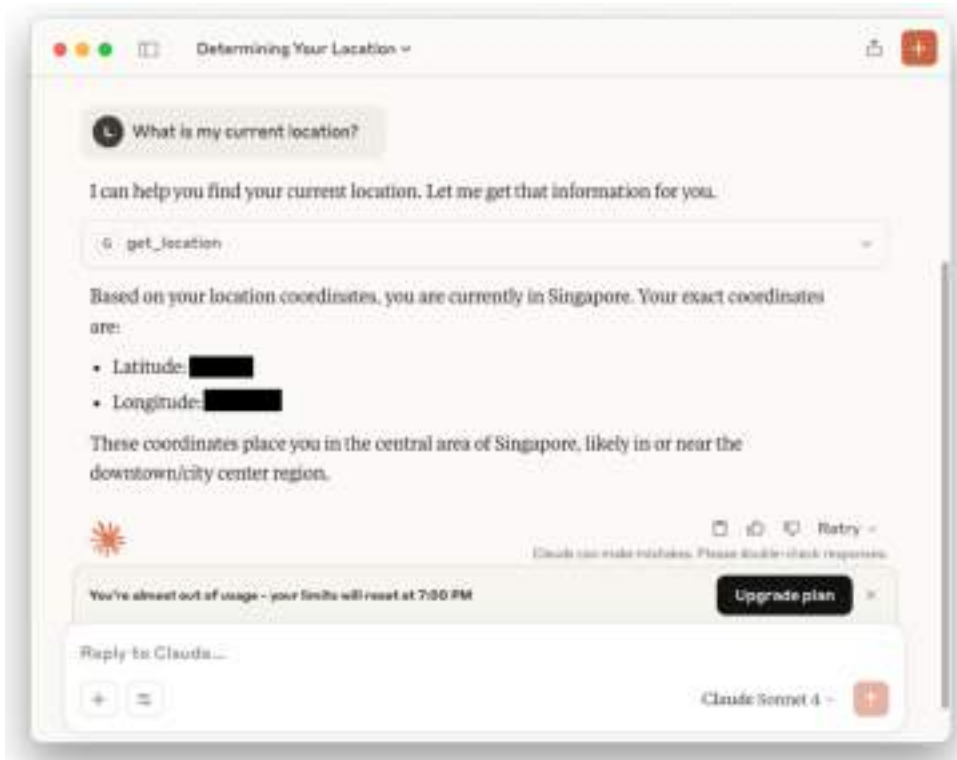
A web page will now appear asking for permission to get your current location (see figure 13.33). Click Get My Location.





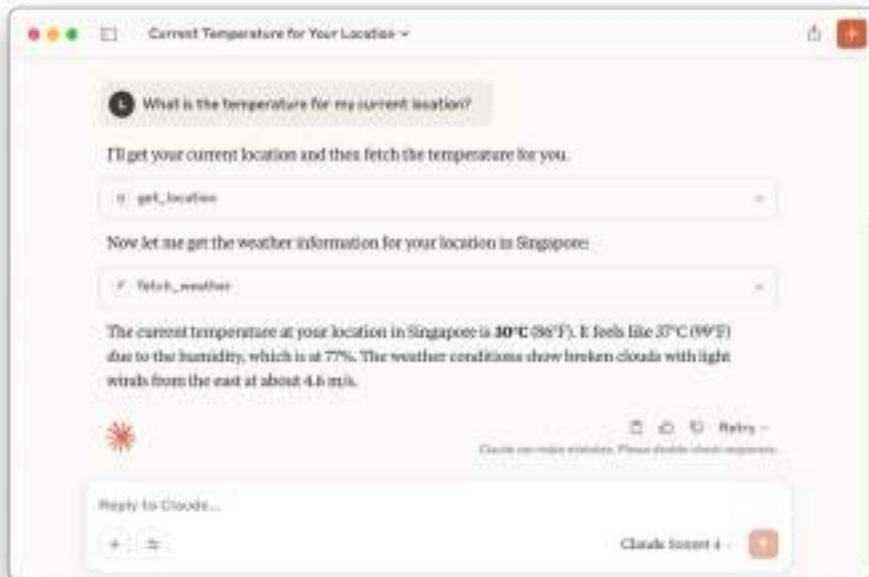
**Figure 13.33 Retrieving your location**

Back in the Claude Desktop, you will see the location details that has been retrieved by the MCP server (see figure 13.34).



**Figure 13.34 Location information returned by the tool**

What happens if you asked something like “What is the temperature for my current location?”. Well, Claude Desktop will first invoke the `get_location` tool, followed by the `fetch_weather` tool (see figure 13.35). Cool, isn’t it?



**Figure 13.35** Claude Desktop can invoke multiple tools to answer a question

### 13.4.2 Time service

The second MCP server we want to try is the Time service MCP server. This service allows you to obtain the current time for various geographical locations. For more details on this service, you can visit <https://www.mcpserverfinder.com/servers/zeparhyfar/mcp-datetime>.

As usual, to use this service in the Claude Desktop, edit the `claude_desktop_config.json` file:

```
$ nano ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

Add in the statements in bold as shown in Listing 13.5.

**Listing 13.5 Adding the mcp-datetime service**

```

{
  "mcpServers": {
    "weather": {
      "command": "/Users/weimenglee/.local/bin/uv",
      "args": [
        "--directory",
        "/Volumes/SSD/Dropbox/MCP_Demo",
        "run",
        "server.py"
      ],
      "env": {
        "OPENWEATHER_API_KEY": "xxxxxxxxxxxxx"
      }
    },
    "get-location": {
      "command": "npx",
      "args": [
        "-y",
        "@mcpn/mcp-get-location"
      ],
      "env": {}
    },
    "mcp-datetime": {
      "command": "/Users/weimenglee/.local/bin/uvx",
      "args": ["mcp-datetime"]
    }
  }
}

```

Unlike the Location service, the Time service is written in Python. Here are what the various keys do:

- The `uvx` command downloads the `mcp-datetime` Python package. This package is in <https://pypi.org/project/mcp-datetime/>.
- The MCP server runs locally without permanent installation (like `npx` but for Python).

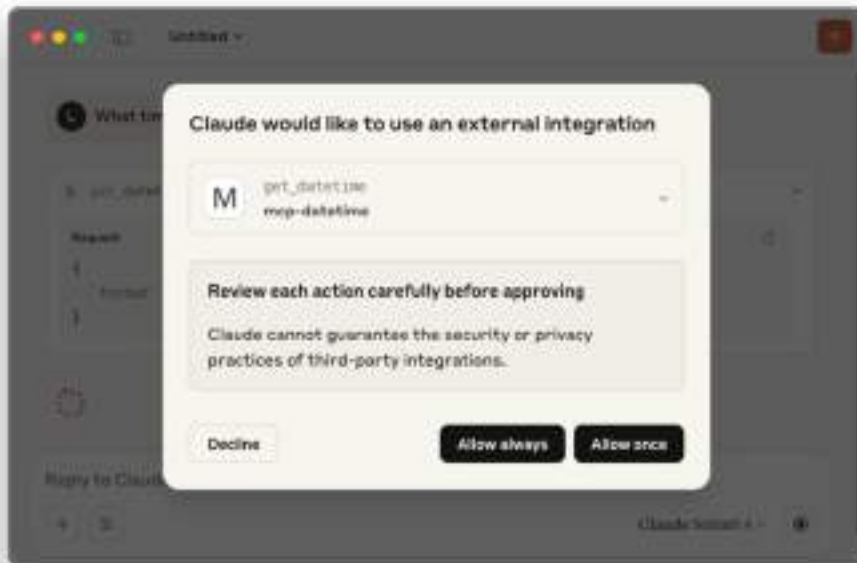
## UV AND UVX

`uv` is a comprehensive Python package and project manager that handles dependencies, virtual environments, and can execute Python code within those environments.

`uvx` is a command runner that executes Python applications in isolated, temporary environments without requiring manual setup.

In our earlier MCP configuration, we used `uv run` to execute the local Python script (`server.py`) within the project's managed environment, while we now use `uvx` to run the `mcp-datetime` package directly - it automatically downloads, installs, and executes the package in an isolated environment without any manual installation steps.

Restart the Claude Desktop and ask the following question: "What time is it now in Shanghai?". You should now see that the Claude Desktop is able to find the `get_datetime` tool to answer that question (see figure 13.36). Click Allow once.



**Figure 13.36 Claude Desktop using a tool to get the time for a particular location**

You will now be able to see the answer from this tool (see figure 13.37).

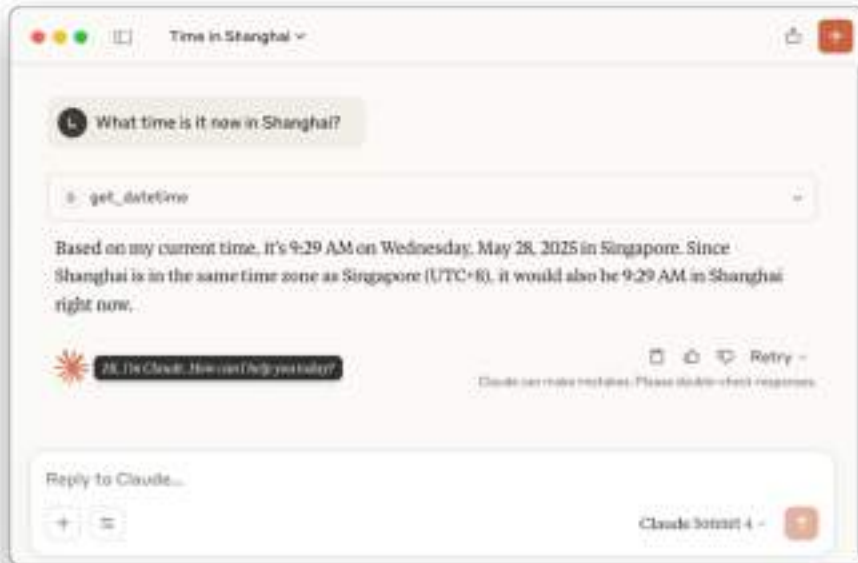


Figure 13.37 The time returned by the tool

## 13.5 Summary

- MCP stands for Model Context Protocol, which is an open standard created by Anthropic for connecting AI assistants like Claude to external data sources and tools.
- A MCP server is simply a standard application that can run in different configurations.
- Once you've developed your MCP server, you can deploy it in two ways - locally or remotely.
- When the MCP server and client are running on the same machine, they will communicate with each other using standard input and output streams.
- When both the server and the client are running on different machines across the network, the client will communicate with the server using HTTP POST, and the server will respond to the Client using Server-Sent Events (SSE).
- A MCP server has three main components that define its functionality:
  - **Tools:** Executable functions that the AI can invoke to perform actions or computations
  - **Resources:** Data sources that the AI can access to retrieve information
  - **Prompts:** Pre-defined prompt templates that can be used to guide AI interactions

- You can develop a MCP server using the official Python SDK for Model Context Protocol servers and clients.
- You can use the MCP Inspector to test and validate your MCP server.
- You can use your MCP server via the Claude Desktop application.
- MCP Servers are written using a variety of programming languages, such as Python, Node.js, TypeScript, Go, and Rust.